Master Your Phone V1.8

v1.8

ClassTechStar

目录

Part I: Foundations (Chapters 0-7) **2**

Chapter 0: Hi, Before We Begin **2**

What This Book Can Do for You (and What It Can't) 2

Our "Secret Code" Guide 2

Android and iOS—We Cover Both 2

Phone Storage: Why Did You Buy 128GB but Only Have Less Than 110GB Available? 3

Your Phone Is a "Digital Asset Vault" 3

Chapter 1: Hello, My Personal Computer **5**

The "Innards" of Phone Hardware: A Deep Dive 5

Operating Systems: Android vs iOS vs HarmonyOS 13

Phone Ports Explained 14

Phone Product Lines and Naming Conventions 15

Baseband and Antenna: The Unsung Heroes of Phone Communication 17

How Many Roles Does Your Phone Actually Play? 18

Appendix: Android Phone Memory Expansion Explained 19

Chapter 1 Appendix: HarmonyOS Operating System Special Topic 22

Chapter 2: Organize Your Digital Room—File Management **26**

I. Three-Tier Storage System: Your Digital Warehouse 26

II. Understanding the "File Manager" 26

III. File System Structure: A Deeper Look 27

IV. Data Backup and Recovery: Insure Your Data 27

V. File Security: Protect Your Privacy 28

VI. Out of Space? Time for a Deep Clean 28

VII. Complete Guide to Common File Formats 28

VIII. Image and File System Management: Keep Your Assets in Order 30

Chapter 3: Plugging Your Phone into Its “Lifeline” 33

Don’t Get Lost Online—Telling Wi-Fi and Mobile Data Apart via “Roads” and “Cars” . . . 33

I. Mobile Networks (4G/5G) and Data . . . 35

II. Wi-Fi: Connection, Sharing, and Security . . . 35

III. Bluetooth: The All-Rounder of Short-Range Wireless Communication . . . 37

IV. Infrared Remote Control: Phone Becomes a Universal Remote . . . 38

V. NFC: The Magic of a Tap . . . 39

VI. Personal Hotspot: Turn Your Phone into a Wi-Fi Router . . . 40

Chapter 4: Installing and Managing Apps 41

I. Official App Stores: The Safest “App Supermarket” . . . 41

II. Pure/Protection Mode: Turn On Your Phone’s “Antivirus Wall” . . . 41

III. Third-Party Sources and “Electronic Parasites”: Extremely High Risk! . . . 42

IV. Install, Update, Uninstall . . . 43

V. Permission Management: You Are the App’s “Boss” . . . 44

VI. Components of an App Installation Package . . . 44

Chapter 5: Make Your Phone Understand You Better 47

I. Sound and Vibration . . . 47

II. Display and Brightness . . . 47

III. Wallpaper, Themes, and Home Screen . . . 48

IV. Efficiency Settings: Make Your Phone Your “External Brain” . . . 48

V. Accessibility: The Power of Tech for Good . . . 50

Chapter 6: Guarding Your Digital Life 55

I. Lock Screen: The First Physical Line of Defense . . . 55

II. “Find My Phone”: The Ultimate Anti-Theft Tool . . . 55

III. System Updates: Patch Holes, Plug Vulnerabilities . . . 56

IV. Recognizing and Preventing Telecom Fraud . . . 56

V. Resist “Doxing” and Social Engineering Attacks: Cut the Privacy Leak Chain . . . 58

VI. Privacy Protection: Your Data Is Your Property . . . 59

VII. “First Aid” Checklist After Loss / Theft . . . 60

VIII. SIM Card and eSIM Security . . . 60

Chapter 7: Comprehensive Analysis of Cutting-edge Features 62

Apple . . . 62

Outdoor activities 63
If you don't respond before the countdown ends, automatically dial emergency numbers. 64
High speed 64

II. Software (Chapters 8-14) 67

Chapter 8: Communication —Texts, Calls, and Social 67

Advanced features of the Phone app 67
SMS and iMessage/RCS 69
WeChat/QQ: Managing China's national super-apps 70
Video call comparison 72
Email: Set up and use it on your phone 73

Chapter 9: Mobile Office and Learning 75

: If your company uses genuine Office, this is the best choice. Format compatibility with the desktop version is nearly perfect, avoiding the slight layout shifts that WPS may have with some complex layouts. Deeply integrated with OneDrive, the document you were halfway through editing on your computer opens as the latest version on your phone. 75

: WeChat's "Float" feature is actually a hidden gem. When you see a good article but don't have time to read it carefully, tap "Float" in the upper-right corner, and it shrinks into a small dot attached to the edge of the screen, ready to be tapped open to continue reading anytime. It's much more convenient than forwarding to "File Transfer Assistant". In addition, read-later tools like Cubox or Pocket can collect good content you come across anywhere, across platforms. 76

: Many brands' cameras have a built-in "Document Scan" mode. If you can't find it, download Microsoft Lens from the app store (free, and integrated with OneDrive). 77

: If you use a Windows PC + Office 365, OneDrive is a natural match. Save a document in Word, and it's automatically stored in the cloud. The 1TB space bundled with the Office subscription is a great deal. The mobile app's auto photo backup feature is also very handy. 77

The client raised a second round of comments, so I used WPS's brush tool to directly circle the revision positions on the PDF, with voice annotations explaining the revision approach. 78

Chapter 10: Mobile Payment and Life Services 80

This payment password, your bank card password, and your lock screen password should ideally be three different passwords —don't set them the same for convenience. 80

. This code automatically refreshes once per minute precisely to prevent fraudulent use. 80

Password-Free Payment Check 81

: Search for “Ride Code” in the app, and activate the bus or subway ride code for your city. The advantage of this solution is: no need to top up —it deducts directly from your balance or linked bank card. But it requires a network connection, and you need to open the app and find the code before swiping. 82

Ride-Hailing Safety 82

: Download the “Palm 12333” app or use “Citizen Center” in Alipay to check social security contribution records, pension account balances, unemployment benefit claims, and more. 83

Chapter 11: Photography and Image Management **84**

Have You Made These Mistakes Too? 84

A Few Small Tips to Immediately Boost Your Hit Rate 85

Portrait Mode and Pro Mode 85

Photo Editing: Don’t Overdo It —Just Right Is Enough 86

Too Many Photos? Organization Strategies 86

Screenshots and Screen Recording 87

Chapter 11 Appendix: Professional Imaging Techniques 87

Chapter 12: Audio-Visual Entertainment and Reading **97**

: Has the most complete licensing —popular artists like Jay Chou and Mayday are basically only on QQ Music. The sound quality options are also very rich, and members can enable lossless quality. If you care about “being able to find every song you want to listen to”, QQ Music is the most reliable choice. 97

: Built into iPhone, no need to download an additional app. Best integration with CarPlay, very convenient when driving. 98

: If you want a more refined experience, this paid app is worth buying. Nice interface, smooth gesture controls —swipe up and down on the left to adjust brightness, on the right to adjust volume, swipe left and right to fast-forward and rewind. Supports DTS and Dolby audio, giving a much stronger immersive feel when watching blockbusters. 98

: If you have a Kindle device, the phone Kindle app can sync your reading progress. Genuine Kindle e-books have much better layout quality than pirated PDFs, but the Kindle China store has closed, so you need to switch to an overseas region. 99

: Don't just max everything out. First ensure frame rate (smoothness), then pursue image quality. 60 fps is much smoother than 30 fps, but 120 fps puts a lot of pressure on the phone —mid-range phones shouldn't force it. If you play competitive games (like Honor of Kings or PUBG Mobile), frame rate priority is far more important than image quality —smooth control feel is a hundred times more important than pretty grass textures. 100

Chapter 13: Recommended Utility Apps 102

Productivity: Manage your time and attention 102

Health and fitness 105

Travel and getting around 106

Automation for beginners 108

Browsers and search engines: Your gateway to information 108

Useful system tools: Cleanup, spam blocking, and safety 111

Weather alerts, disaster warnings, and earthquake early warnings 114

Cross-device teamwork: Remote control, fast transfer, account safety 117

Chapter 14: Password Managers 122

I. Why Do You Need a Password Manager? 122

How Password Managers Work 122

III. Introduction to Encryption Principles 122

IV. Mainstream Password Manager Recommendations 124

V. Real Password Strength Assessment 125

VI. Passkey: The Future of Passwordless Login 126

VII. Hardware Security Keys 129

VIII. Family/Team Password Sharing 131

IX. Password Manager Best Practices 133

III. Advanced (Chapters 15-20) 136

Chapter 15: Battery Science and Battery Life Optimization 136

What's the Deal with Batteries? Cycles and Health 136

Charging Mysticism? Fast Charging, Wireless Charging, and the Golden Rule 137

Handshake phase 138

IV. New Battery Technology: Silicon-Carbon Anode Batteries 141

V. The Real Battery-Saving Tips 142

Chapter 16: Account Systems and Cloud Services 144

: From your child's birth to graduation ceremony, from first date to wedding trip. . . 144

Confirm "Find My Phone" is enabled 145

Upgrade 145

is one-way: The phone takes a "snapshot" of its current state and stores it in the cloud. After that, if you delete something on the phone, the copy in the cloud backup is still there. iCloud Backup, local computer backup—these are backups. The benefit is safer data; the downside is they take up space and can't update in real time. 147

: If you use Gmail, Google Photos, Google Contacts, these are all in your Google account. Google gives 15GB for free, more generous than Apple, but it's also shared with Gmail and Drive. 147

Hands-on: Give Your Account a Comprehensive Health Check 148

Chapter 17: Interconnection Between Phone and Other Devices 149

: Microsoft has an app called "Link to Windows", built into Samsung phones, and other Android phones can download it from the app store. After pairing, you can directly view phone notifications, reply to texts, browse photos, and even make and receive calls on your computer. I've used it for two years—during meetings I don't have to keep looking down at my phone; a notification popping up on the computer screen is enough. 149

: Each brand calls it differently—Huawei calls it "Wireless Projection", Xiaomi calls it "Cast", OPPO/vivo calls it "Multi-Screen Interaction". Usually findable in the quick settings panel; if not, search "screen cast" in Settings. 150

: The watch monitors heart rate, blood oxygen, sleep, and steps 24 hours a day, and this data is automatically aggregated into the health app on your phone. Over time, you can see your trends—have you been walking too little this month, has your sleep quality improved this week. 150

Lights 151

: AirPods with iPhone will auto-pop up to show battery, and can seamlessly switch between multiple Apple devices. Huawei FreeBuds with Huawei phones, Xiaomi earbuds with Xiaomi phones, also have similar pop-ups and optimizations. . . . 152

: Navigation (much better than built-in car navigation), phone calls (answer from steering wheel), music (music apps on your phone directly cast to the car), voice WeChat (Siri or Xiaoai helps read messages and reply). 153

Try It: Cast Your Phone Screen to TV and Set Up a Smart Scene 154

Chapter 18: Data Backup and Migration 155

: WeChat friend list, Taobao orders, transaction records in banking apps. These are stored on the corresponding company's servers, not on your phone. 155

Android 155

Free plan 156

Android same brand 157

: Use two different storage media for backups (e.g., one in the cloud, one on an external hard drive). 158

Chapter 19: Troubleshooting and Self-Rescue 159

Phone Lagging and Overheating—The Most Frequently Asked Question 159

App Crashes and Unresponsiveness—When Apps Misbehave 160

Won't Charge or Charges Slowly—It Might Not Be the Battery's Fault 160

Can't Connect to Network, Wi-Fi Keeps Dropping 161

Screen Issues: Unresponsive Touch, Flickering, Image Retention 161

Won't Turn On, Stuck at Logo—Don't Panic First 162

Sound Issues: Speaker Not Working, Low Call Volume 162

Finding Official Repair, Avoiding Pitfalls 163

Chapter 20: Developer Mode and Advanced Permissions 164

Chapter 20 Appendix: Termux—A Portable Linux Terminal Environment on Android Devices 172

IV. Beyond (Chapters 21-26) 178

Chapter 21: In-depth Exploration of Mobile Networks and Information Security178

: Your phone VPN encrypted tunnel VPN server target website. The data is encrypted and encapsulated, so "peeping toms" can only see a bunch of garbled text. Additionally, the target website sees the IP address of the VPN server, not your real IP. 178

cannot directly read your chat content or payment password 179

After verification passes, both parties use public key encryption to negotiate a "session key," and all subsequent communication is encrypted with this key. 180

! This means your ISP, Wi-Fi provider, or any intermediate node can see which websites you've visited. 180

The website you visit 181

: The "Send 'Do Not Track' request" option in browsers is essentially "asking" websites not to track you. But whether websites listen is another matter entirely; most websites simply ignore it. Don't count on it. 182

: Android directly set Private DNS to “dns.google”; iPhone use an App or profile to set up 1.1.1.1's DoH. 182

Chapter 22: How AI Works in Phones 184

Understanding (natural language understanding) 184

Execution and response 184

AI LLMs (Large Language Models) are AI systems trained on massive amounts of text data that can understand and generate natural language. You can imagine it as a “super scholar” who has read almost all the text on the internet, capable of answering questions, writing articles, translating, programming, and analyzing data. 184

Conversational assistants 185

Underlying principle 186

Many people use AI like using Baidu—typing in a few keywords. But AI is not a search engine; it's a language model that “reads, understands, and then answers.” . 187

: Most AI's training data has a cutoff date (such as early 2024); it doesn't know what happened after that. 192

Chapter 23: Programming Thinking and Automation Enlightenment 195

Automation is essentially the “if... then...” logic chain, the if-else in programming. Once you understand this way of thinking, even if you never write code in your life, your problem-solving organization will be much stronger. Core elements: . 195

: Instructions that run automatically based on trigger conditions, without needing you to manually trigger them. For example, “when the phone connects to car Bluetooth, automatically open the Maps App.” Automations are set up under the “Automation” tab, separate from “Shortcuts.” 196

Tasker is universally recognized as the most powerful automation tool on the Android platform, consistently topping the charts. Honestly, its learning curve is a bit steep and the interface isn't pretty, but once you get the hang of it, you'll feel your phone suddenly becomes omnipotent. Tasker's core is also the “trigger task” pattern: 197

Every morning, automatically read today's schedule from the calendar and generate a text message sent to family via WeChat. 199

: In iOS, most triggers under the “Automation” tab require manual confirmation to run (iOS's security mechanism). However, Wi-Fi connection, Bluetooth connection, and alarm stopped triggers can run fully automatically without confirmation.199

Chapter 23 Appendix: GitHub Beginner's Guide—From Code Hosting to Collaboration Platform 201

What Exactly Is GitHub? . . . 201
The Difference Between Git and GitHub (Key Concepts!) . . . 201
GitHub's Core Concepts and Features . . . 202
Registration and Getting Started: Create Your First Repository in Five Minutes . . 203
Basic Workflow: From Local to Cloud . . . 204
Team Collaboration Process: How Pull Request Works . . . 205
GitHub's Typical Application Scenarios . . . 205
Advice for Beginners . . . 206
Summary: Why Everyone Should Understand GitHub . . . 207

Chapter 24: Root Access and the Bootloader Lock Explained 209
Deep Understanding of Bootloader . . . 209
Detailed Process for Unlocking BL Lock (Using Xiaomi as an Example) . . . 212
Multiple Root Solutions . . . 212
Flashing Introduction . . . 212
V. Samsung Knox: The Irreversible Cost of Unlocking the BL . . . 212
VI. FAQ (Q& A) . . . 213

Chapter 25: Second-hand Phone Disposal and Digital Legacy 215
: Modern phones' "factory reset" usually includes encrypted erasure. . . . 215
Manual backup . . . 217
Give it a second life . . . 222

Chapter 26: Current Smartphone Technology Frontier 224
I. AI Phones: Marketing Gimmick or Real Revolution? . . . 224
II. Manufacturer AI Strategy Comparison: Who's Serious, Who's Labeling? . . . 225
III. Satellite Communication: From Flagship "Gimmick" to Mid-Range Standard . 225
IV. Foldable Screen Ecosystem Maturity: Ready to Be a Primary Phone? . . . 226
V. 6G and Wi-Fi 7: Speed and Boundaries of Next-Gen Connectivity . . . 227
VI. Silicon-Carbon Anode Batteries: Battery Life Finally Gets a Material-Level Breakthrough . . . 227
VII. Chapter Overview: Remember These Frontiers in One Sentence . . . 228

V. Practical (Chapters 27-32) 231

Chapter 27: Making Phones Easy for Parents —Complete Senior-Friendly Guide 231

First, Get the Mindset Right: It's Not "Seniors Can't Use Phones" —It's "Phones Weren't Designed for Them" . . . 231
I. First Step: Enable "Easy Mode" —The Phone Becomes Clean and Simple 232
II. If You Don't Use Easy Mode, Manual Adjustment Works Too — "Big Text + Big Icons" in Five Steps . . . 233
III. Anti-Fraud —Please Read This Section Together with Your Elderly Relatives . . 233
IV. Remote Assistance —Fix Your Parents' Phone from Anywhere . . . 235
V. WeChat "Senior-Friendly Makeover" —Just Tell Your Parents These Things . . 236
VI. What If the Phone Is Lost? Make an "Emergency Plan" for Elderly Relatives . 236
VII. Wireless Earphones and Hearing Aid Compatibility —An Overlooked Feature . 237
VIII. Health-Related Features —A Phone Is More Than a Communication Tool . . . 237

Chapter 28: Spend Wisely on Your Dream Phone —The Ultimate Smartphone Buying Guide 239

Three-Step Phone Selection: Needs Budget Model . . . 239
I. How to Read Core Specs —You Don't Need to Understand Everything, Just These Five . . . 239
II. Budget-Tier Recommendations (Based on Mid-2026 Market) . . . 241
III. Choose by Scenario —What Are Your Needs? . . . 242
IV. Five Common Pitfalls —Check Before You Buy . . . 243
V. When Is the Best Time to Buy? . . . 244
VI. Where to Buy —Which Channels Are Reliable? . . . 244

Chapter 29: Digital Wellbeing —Building a Healthy Relationship with Your Phone 246

I. Check Your "Screen Time" —Numbers Are a Wake-Up Call . . . 246
II. Set App Time Limits —Install a Speed Limiter for Yourself . . . 246
III. The Right Way to Use "Do Not Disturb"—Not Muting Notifications, But Muting Anxiety . . . 247
IV. Eye Protection Relies on Habits, Not Apps —The 20-20-20 Rule . . . 247
V. Neck Rescue Guide for "Phone Benders" . . . 248
VI. Sleep and Phone —The Bedroom Habit Most Worth Changing . . . 248
VII. Information Anxiety —Learn "Selective Blindness" . . . 249

Chapter 30: Screen Mirroring, Presentations, and Car Connectivity 251

I. Three Wireless Casting Camps —Don't Mix Them Up . . . 251
II. Wired Casting —More Stable, Zero Latency . . . 252

III. Phone Becomes Computer —Desktop Mode 252
IV. Car Connectivity —The Right Way to Use Your Phone While Driving 252
V. Meeting Room Casting in Practice —No More Panic 253

Chapter 31: Getting Started with Short Video Shooting and Content Creation 255
I. Think Before You Shoot: Who's Your Audience? 255
II. Phone Camera Basic Settings —5 Things to Do Before Shooting 255
III. Five Camera Movement Techniques —Make Your Footage Come Alive 256
IV. Editing Basics —CapCut Is Enough 256
V. The Importance of Thumbnails —70% of People Decide Whether to Watch Based on the Cover 257
VI. Three Content Creation Principles 257

Chapter 32: Phone Safety Emergency Handbook 259
Scenario 0: Earthquake / extreme weather —how your phone helps you stay alive . 259
Scenario 1: Phone lost or stolen —the golden 30 minutes 260
Scenario 2: You got scammed —the 72-hour window 261
Scenario 3: Phone infected with malware —what now? 261
Scenario 4: Account hijacked —take back control fast 262
Scenario 5: Blackmail message —don't panic, don't pay 262
Scenario 6: Deepfake video / voice cloning scams —seeing is no longer believing . . 263
Scenario 7: SIM swap —your number gets "stolen" 264

Part VI: Outlook and Car Connectivity (Chapters 33-34) 268

Chapter 33: Future Outlook —The Next Journey of Smartphones 268
I. AR/VR and Spatial Computing: The "Second Desktop" Beyond Phones 268
II. Brain-Computer Interface: Thought as Interaction Medium 269
III. The Phone's "Disappearance" and New Identity Foundation 269
IV. Ethical Reflection: Are We Still the Masters of Our Phones? 269
V. Stay Connected, Stay Independent —Your Digital Life Philosophy 270

Chapter 34: In-Depth Guide to Car-Phone Connectivity —The Fusion of Phone and Automobile 272
I. Three Routes for Car-Phone Connectivity 272
II. CarPlay / Android Auto / HiCar Comparison 273
III. HarmonyOS Cockpit: Seamless Flow Between Phone and Car 273

IV. Car Systems from New Automakers 274
V. Phone as Car System —Can It Work? 274
VI. Practical Tips 275

Children and Phones Special Topic (Chapter 35) 278

Chapter 35: Children and Phones —An Educational Guide for the Digital Age 278
I. At What Age Should Kids Get a Phone? 278
II. Complete Parental Control Guide 279
III. Youth Mode 280
IV. Digital Literacy Education 281
V. Healthy Usage Habits 281
VI. Digital Legacy and Children 282

Chapter 36: Phones and Smart Home —Make Your House Think 284
Smart home protocols: Pick the right one 284
The big three ecosystems 285
Your phone as the home hub 285
Starter kit picks 286
Safety basics 286

Part VII: AI Coding in Practice (Chapters 37-47) 289

Chapter 37: The Paradigm Shift —From Writing Code to Directing AI 289
The birth of Vibe Coding: A movement sparked by one sentence 289
2026 status: The Agentic Engineering era 290
The core mindset shift: From Coder to Conductor 291
Why methodology beats tools 291

Chapter 38: Five Thinking Principles of AI Coding 293
Principle 1: Think Before Coding 293
Principle 2: Simplicity First 294
Principle 3: Surgical Changes 295
Principle 4: Goal-Driven Execution 295
Principle 5: Iterate and Verify 296
Harness Engineering: Agent = Model + Harness 297

Chapter 39: Context Engineering —Make AI Understand You 299

Project memory files: AI's long-term memory . . . 299

Spec-Driven Development (SDD) . . . 300

Context window management . . . 301

Practical tips for context engineering . . . 302

Chapter 40: Requirements Engineering —Turn Ideas into AI-Ready Instructions 303

From fuzzy idea to structured requirements . . . 303

The art of layered requirement breakdown . . . 304

PRD writing template and practice . . . 304

Hands-on: full walkthrough from idea to plan . . . 305

Chapter 41: The 2026 AI Coding Tools Landscape 307

Three schools and a choice framework . . . 307

IDE-native AI school . . . 308

Terminal agent school . . . 308

Cloud builder school . . . 309

Tools change; methodology doesn't . . . 309

Chapter 42: LLM Landscape and API Practice 311

Global major LLMs: the big three . . . 311

China's Major Large Models: A Hundred Flowers in Bloom . . . 313

A Decision Framework for Choosing a Model . . . 317

Getting Started with API Calls: Connecting to a Large Model from Scratch . . . 318

A Buyer's Guide to Token Plans . . . 321

Hands-On: Wiring a Large Model API into a Project . . . 325

Chapter 43: Conversation Engineering —How to Talk with AI Effectively 329

Prompt Layering . . . 329

Debug Conversations: Describing Errors So the AI Can Pinpoint Them Fast . . . 330

Iteration Strategy: Small Quick Steps vs. Big Refactors . . . 331

Context Management for Multi-Turn Conversations . . . 332

Chapter 44: From 0 to 1 —A Complete Project, End to End 334

Project 1: A Minimal Personal Tools Site (Beginner) . . . 334

Project 2: A Full-Stack Expense Tracker (Intermediate) . . . 335

Project 3: An AI-Powered App (Advanced) . . . 337

The Universal Hands-On Checklist 337

Chapter 45: Quality Assurance —Making AI-Written Code Reliable 340

Test-Driven: Have the AI Write the Tests 340

Code Review Techniques 341

III. Version Control: Git's Role in AI Coding 341

IV. Common Pitfalls and How to Dodge Them 342

Chapter 46: The Road Ahead —From User to Architect 344

I. The MCP Protocol and Tool Extensions 344

II. Custom Workflows and Skills 344

III. Multi-Agent Collaboration Patterns 345

IV. The Continuous Learning Path 345

Chapter 47: Quick Reference and Resources 347

I. Mental Model Cheat Sheet 347

II. The Tool-Choice Decision Tree 348

III. FAQ: Common Questions 348

IV. Recommended Learning Resources 349

V. Closing Thoughts: Tools in Hand, Methodology in Mind 350

References 350

Part VIII: Android Development in Practice (Chapters 48-58) 353

Chapter 48: Getting to Know Android —From Operating System to Development Ecosystem 353

I. Android's Origin Story: From "Little Robot" to "The World's Biggest System" 353

II. Which Language to Write In: Java or Kotlin? 354

III. How Is an App Born? 355

IV. The Android Tech Stack at a Glance: Your "Weapon List" 356

V. Android vs. iOS: A Developer's First Lesson in Choosing Sides 356

Chapter 49: Setting Up Your Development Environment —Android Studio and Your First Project 359

I. Installing Android Studio 359

II. Creating Your First Project 360

III. Running It: Emulator and Real Device 360

IV. Project Structure Explained: What Each File Does 361
V. A First Try: Change “Hello” to Your Name 362
VI. First Aid for Common Errors: The Beginner’s “ER” for Your First Two Weeks 363

Chapter 50: Kotlin Quick Start —Android’s First Language 365

I. Variables and Types: Giving Data a Name 365
II. Functions: Packaging Logic into “Buttons” 366
III. Conditions and Loops: Teaching the Program to “Think” 366
IV. Null Safety: Kotlin’s “Life Saver” 367
V. Classes and Objects: Molds for Making “Parts” 367
VI. Lambdas and Higher-Order Functions: Kotlin’s “Syntax Candy” 368
VII. A Syntax Cheat Sheet for Absolute Beginners 368

Chapter 51: Building the UI —Getting Started with Jetpack Compose 370

I. Compose’s Big Idea: The UI Is a Function 370
II. Basic Components: Text, Button, Input Field, Image 371
III. The Layout Trio: Column, Row, Box 371
IV. State: Bringing the UI to Life 372
V. Theming and Dark Mode: Configure Once, Apply Everywhere 373
VI. Preview, Animation, and Live Edit: Compose’s “Cheat Codes” 374

Chapter 52: The Four Components and the Activity Lifecycle 376

The Four Components: Android’s Pillars 376
The Activity Lifecycle: Your First Android “Treasure Map” 377
Intents: The “Tickets” for Traveling Inside and Outside the App 378
Multiple Screens and Navigation 378
A Quick Look: What Is a Fragment (and Why This Book Doesn’t Teach It) 379

Chapter 53: Data Storage and Network Requests 380

Choosing a Storage Option: Look at This Table First 380
DataStore: Saving Small Settings 380
Room: Giving Your Data a “Real Home” 381
Retrofit: Talking to Servers 382
JSON Parsing: Translating Network Data 383
Coroutines: The “Common Language” of Android Concurrency 383
Coroutines: The “Common Language” of Android Concurrency 383

Chapter 54: Project 1 —My Expense Tracker (Local Version) 385

Thinking Through Requirements: Ask Five Questions First 385

The Data Layer: Giving Each Transaction a Home 386

MVVM: UI and Logic “Live Apart” 387

The UI: List Screen + Entry Screen 387

Edit and Delete: Let Data “Come and Go” 388

Acceptance: a three-layer check + two advanced challenges 389

Acceptance: a three-layer check + two advanced challenges 389

Chapter 55: Project 2 —Weather App (Networked Version) 391

Apply for a Free Weather API 391

The Network Layer: Retrofit + JSON Parsing 391

Three UI States: Loading / Success / Failure 392

Real UI Code: Drawing the Weather 393

UI and Interaction Details 394

UI and Interaction Details 394

Chapter 56: Polish and Release —Testing, Debugging, and Publishing on Google Play 395

The Three Debugging Tools: Logcat, Breakpoints, and the UI Inspector 395

Testing: Insuring Your Code 396

Signing and Packaging: Stamping Your App 396

The Full Google Play Publishing Process 397

Making Store Assets: First Impressions Are Downloads 398

A quick checklist for performance and crash reporting 398

Chapter 57: The Road Ahead —Permissions, Notifications, and Modern Architecture 400

Runtime Permissions: Knock First, Then Come In 400

Notifications: Saying Useful Things at the Right Time 401

Architecture, Next Level: Dependency Injection and Modularity 402

The 2026 Way: AI-Assisted Android Development 402

Compose Performance Tuning and Debugging Tools 403

Chapter 58: Quick Reference and Resources 404

Common Code Cheat Sheet 404

FAQ 404

Recommended Learning Resources 406
Common Gradle Commands Cheat Sheet 406
Closing: From Your First App to Your Nth App 407
Closing: From Your First App to Your Nth App 407
References 407

Part IX: iOS Development in Practice (Chapters 59-69) 410

Chapter 59: Getting to Know iOS —Apple's Ecosystem and Developer Ecosystem 410

iOS's Origin Story: One Phone Changed an Industry 410
The Walled Garden: The Apple Ecosystem's "Rules" and "Rewards" 411
Which Language to Write In: Objective-C or Swift? 412
How Is an App Born? 412
The Full iOS Tech Stack: Your "Weapon List" 413

Chapter 60: Setting Up Your Development Environment—Xcode and Your First Project 415

Getting a Mac Ready 415
Installing Xcode 416
Creating Your First Project 416
Project Structure, Dissected: What Each File Does 417
Testing on a Real Device: Install the App on Your iPhone 417
Xcode Efficiency Tricks: Five "Shortcut Power-Ups" for Beginners 417
Testing on a Real Device: Install the App on Your iPhone 418
Try It Yourself + First Aid for Common Errors 419
Try It Yourself + First Aid for Common Errors 419

Chapter 61: Swift Quick Start—Apple's Top Development Language 420

Variables and Types: Giving Data a Name 420
Optionals: Swift's "Anti-Crash Charm" 421
Functions and Closures: Packing Logic into "Buttons" 421
Structs and Protocols: Swift's "LEGO" System 422
A Syntax Cheat Sheet for Absolute Beginners 423

Chapter 62: Building Interfaces—Getting Started with SwiftUI 424

The SwiftUI Idea: Declarative, Not Imperative 424

The Basic Views: Text, Buttons, Text Fields, Images 425
The Layout Trio: VStack, HStack, ZStack 425
State and Binding: Bringing the Interface to Life 426
Previews, Theming, and Animation: SwiftUI's "Power-Ups" 427
Common Modifiers Cheat Sheet: One Page in Hand, No Interface Worries 428

Chapter 63: Views, State, and Navigation 430
The Big Picture of State Management and Data Flow 430
NavigationStack: The "Stack" Behind Screen Jumps 431
Pop-Ups and Bottom Tabs: sheet and TabView 432
List: iOS's Go-To List 432
The App Lifecycle: Foreground and Background 433

Chapter 64: Data Storage and Network Requests 435
Choosing a Storage Option: Look at This Table First 435
UserDefaults: Storing a Few Small Settings 436
SwiftData: Giving Data a "Real Home" 436
Codable: A "Translator" for JSON 437
URLSession + async/await: Talking to Servers 438
"Grade" Your Network Errors Too: Don't Throw Every Error at the User 439

Chapter 65: Project One—My Expense Tracker (Local Version) 441
Thinking About Requirements: Ask Five Questions First 441
The Data Layer: Giving Transactions a Home 441
MVVM: When Interface and Logic "Move Apart" 442
The Interface: List Screen + Entry Sheet 443
Editing, Deleting, and Summaries 444
Acceptance: A Three-Layer Check + Two Advanced Challenges 445

Chapter 66: Project 2—Weather App (the Networked Version) 446
Apply for a free weather API 446
The network layer: URLSession + Codable 446
Three UI states: loading / success / failure 447
Hands-on UI code: drawing the weather 448
Interaction details and favorite cities 449
How to debug network requests: printing, packet capture, and simulating a slow network 449

Chapter 67: Polish and Ship—Testing, Debugging, and the App Store 451

The three debugging essentials: print, breakpoints, and previews 451

Testing: insurance for your code 452

Signing and packaging: stamping your app 453

The full App Store submission process 453

Store assets: first impressions drive downloads 454

A mini checklist for crash reporting and performance 454

Chapter 68: The Road Ahead—Permissions, Notifications, and Modern Architecture 456

Permissions: knock before you enter 456

Notifications: say something useful at the right time 457

Architecture, going deeper: MVVM and testability 458

The 2026 twist: AI-assisted iOS development 458

Swift concurrency essentials: @MainActor and Task 459

Chapter 69: Quick Reference and Resources 461

Common code cheat sheet 461

Frequently asked questions (FAQ) 462

High-frequency Xcode shortcuts 463

Recommended learning resources 464

Recommended learning resources 464

Closing: from your first app to your Nth app 464

References 465

Appendix A: New Phone Unboxing "Step-by-Step" Guide 467

Unboxing Inspection "Three-Step Process": 467

Activation & Initial Setup "Key Reminders": 467

Installing Apps "Safety First": 467

Appendix B: Hands-On Practice Reference 469

Appendix C: Minimalist Tech Jargon Glossary 471

Appendix D: Where to Find Reliable Resources? 472

Official Support Is Your First Choice: 472

Useful Websites & Bloggers: 472

Security Self-Check One-Sentence Checklist: 472

Appendix E: Thanks and Notes **474**

What's new in version 1.6 474

What's new in version 1.7 475

What's new in version 1.8 476

Content additions: practical apps and harassment blocking 477

Content additions: weather alerts / natural disaster alerts / earthquake early warning 477

Content additions: cross-device teamwork and productivity tools 478

Appendix F: Foldable Phone Purchasing and Usage Special Topic **479**

I. How Far Have Foldable Screens Developed? 479

II. Who Should Buy a Foldable Phone? 480

III. Buying Guide: Check These 6 Metrics 480

IV. Unique Foldable Usage Tips 481

V. Maintenance Tips: Help Your Foldable Last Longer 482

VI. Foldable "Pitfalls" & Realistic Expectations 483

VII. How to Choose Among Mainstream Brands (2026) 484

Conclusion: Stay Curious, Stay Clear-Headed **486**

Appendix G: Quick-Reference Index for the Whole Book **487**

Appendix H: Version Roadmap **491**

Part I: Foundations (Chapters 0-7)

Chapter 0: Hi, Before We Begin

Before we officially start exploring, let's cover some basic rules to make sure you understand everything that follows. Don't rush—sharpening the axe won't delay the woodcutting.

What This Book Can Do for You (and What It Can't)

It can help you systematically understand how to use your phone, grasp the reasoning behind features, and—whether you're an Android fan or an Apple devotee—most of the thinking applies universally. It will guide you through hands-on practice, taking you from "able to use" to "truly understanding how to use."

It is not a manual for any specific phone model, nor is it tech news chasing the latest gadgets, and it's certainly not a guide to becoming addicted to your phone. Honestly, if you're expecting "how to enjoy scrolling videos more," you may be disappointed—we care far more about making your phone truly serve you, not the other way around.

Our "Secret Code" Guide

To be clearer, the book uses some small markers—these markers are like tacit codes between us:

- **[Button Name]**: Refers to a specific button or App name on the screen, such as [Settings], [OK].
- **>**: Indicates the order of operations. For example, "Tap [Settings] > [Sound]" means first tap Settings, then tap Sound.
- **Bold**: Very important! It may be a key concept or a safety warning—please pay close attention.
- *Italics*: Supplementary thoughts or analogies to help you understand better.

Android and iOS—We Cover Both

Guess what—some things have different names but do the same thing. For example: Application (Android) and App (iOS), Settings (called Settings on both), File Manager (An-

droid) and Files (iOS). Knowing they're the same thing is enough; don't get tripped up by the terminology.

Additionally, within the Android camp there's Huawei's HarmonyOS, which has its own characteristics—we'll cover it specifically later. In short, no matter what phone you use, this book can help you.

Phone Storage: Why Did You Buy 128GB but Only Have Less Than 110GB Available?

Everyone knows the storage units: 1TB = 1024GB, 1GB = 1024MB, 1MB = 1024KB... But here's the catch: hard drive manufacturers usually calculate in base 1000, while the system calculates in base 1024, so the "128G" drive you bought will "shrink" in actual capacity.

But there are three main reasons your phone's storage shrinks:

1. **The system itself takes up space**: The operating system is like a house's load-bearing walls—it must exist. Modern systems routinely occupy 10-20GB, which is completely normal.
2. **Manufacturer pre-installed apps**: The apps that come with a new phone also take up space; some can be uninstalled, some cannot.
3. **Partitioning and formatting losses**: For technical reasons—just like the lining of a backpack also takes up volume.

Tips for Buying a Phone

Nowadays 128GB phones are basically not enough; 256GB is a safer starting point. If you like taking photos or playing games, consider going straight to 512GB or larger. Remember—when it comes to storage, it's better to have surplus than to come up short.

Hands-on: Check Your Phone's "ID Card"

Open [Settings], find About Phone (or a similar option), and note down: phone model, system version, total storage, and available storage. This is the first step to understanding it. Also check battery health so you have a clear picture.

Your Phone Is a "Digital Asset Vault"

Before diving into specific knowledge, let's establish a core understanding:**your phone has long been more than a communication tool—it is your most important digital asset vault**. Contacts, chat history, family photos, work documents, banking apps, payment codes, health data, electronic IDs... Together, this data is often worth more than the phone itself.

Why Does This Matter?

If your phone is lost or broken, you lose not only a few thousand yuan of hardware,**but more likely years of irreplaceable data**—photos of your child growing up, voice messages from departed loved ones, important contracts that were never backed up. Scientific organization and protection habits can safeguard you across three dimensions:

- **Improve operating efficiency**: Standardized file classification makes system lookups faster and keeps the phone from lagging over the long term.
- **Extend service life**: Reasonable storage management reduces the read/write burden on the chip, making both battery and storage last longer.
- **Ensure data security**: Layered management + regular backups + tightened permissions keep ”data disasters” away from you.

Our Core Philosophy: “Prevention First, Combined with Treatment”

Just as you can’t wait until you’re sick to see a doctor for wellness,**phone management must start with daily habits**. What this book repeatedly emphasizes is not some advanced trick, but these simple principles:

- **Prevention first**: Install apps only from official stores, strip metadata from photos before sharing, never reuse passwords—these ”small actions” cost very little but block the vast majority of risks.
- **Regular check-ups**: Organize the download folder once a month, audit permissions once a quarter, do a full backup every six months—a fixed rhythm is a hundred times more effective than ad-hoc firefighting.
- **Always have a fallback**: Enable cloud sync, keep account passwords recorded, write a ”digital estate checklist”—so that when accidents happen, you and your family can respond calmly.

Chapter 1: Hello, My Personal Computer

Your phone is actually the most personal supercomputer in your pocket. It is countless times more powerful than the computers that once sent humans to the Moon, yet it rests quietly in your palm. Today, let's get to know this "silent giant."

The "Innards" of Phone Hardware: A Deep Dive

Small as it is, the phone has all the vital organs. Let's get to know these precision components one by one, like in an anatomy class.

SoC (System on Chip): The Phone's "Brain + Heart"

SoC stands for System on Chip, which translates to "cramming the entire system onto a single chip." It's not just the CPU (Central Processing Unit)—it integrates the CPU, GPU (Graphics Processing Unit), NPU (Neural Processing Unit), ISP (Image Signal Processor), baseband (responsible for communication), and more all together.

It's like building a city's government building, power plant, communication center, and police station all in one tower—extremely efficient.

CPU: The Core of General-Purpose Computing

The CPU is the most central component of the SoC, responsible for handling the vast majority of computing tasks. Phone CPUs use the ARM architecture, different from the x86 architecture of computers. ARM features a Reduced Instruction Set (RISC), with lower power consumption, making it more suitable for mobile devices.

CPU Big-LITTLE Core Design: Modern phone CPUs generally adopt a "big.LITTLE" core architecture (ARM's big.LITTLE technology), dividing cores into different performance tiers:

- **Super core / Big core**: Strongest performance, handles heavy-duty tasks. Such as game loading, app launching, complex computations. Highest frequency (3GHz+), but

also highest power consumption.

- **Medium core**: Balances performance and power, handles everyday medium-load tasks. Scrolling Weibo, watching videos, and multitasking mainly rely on it.
- **Small core / Efficiency core**: Lowest power consumption, handles lightweight background tasks. Standby, listening to music, receiving messages—almost no power draw.

The three tiers of cores work together: the system scheduler automatically assigns tasks based on load—lightweight tasks go to small cores to save power, heavy work goes to big cores to crank through it, then switches back to small cores when done. This is why a phone can both run games smoothly and stay on standby all day.

Brand Architecture Characteristics:

- **Qualcomm Snapdragon 8 Elite**: Uses the self-developed Oryon CPU architecture (derived from the Nuvia acquisition), 2 super cores (4.32GHz) + 6 performance cores (3.53GHz), all based on TSMC's second-generation 3nm process. Completely departs from ARM's public architecture, with leapfrog improvements in both single-core and multi-core performance. GPU is Adreno 830 with a slice architecture, graphics performance up 40%; Hexagon NPU has about 45 TOPS of computing power, AI inference performance up 45%. (Note: Samsung Galaxy custom version has higher frequencies, with super cores up to 4.47GHz.)
- **MediaTek Dimensity 9400**: 1 super core (Cortex-X925, 3.62GHz) + 3 performance cores (Cortex-X4, 3.3GHz) + 4 efficiency cores (Cortex-A720, 2.4GHz), continuing the all-big-core architecture, TSMC's second-generation 3nm process. GPU is Immortalis-G925 (12-core), peak performance up 41%, ray tracing performance up 40%. AI processor is the 8th-generation NPU 890, supporting on-device LLM operation, on-device LoRA training, and on-device video generation.
- **Apple A18 Pro**: 2 performance cores (~4.04GHz) + 4 efficiency cores (~2.4GHz), TSMC's second-generation 3nm process. 6-core GPU supports hardware ray tracing; 16-core NPU (~35 TOPS) provides powerful computing for on-device AI and Apple Intelligence. Apple's self-developed architecture still leads in single-core performance and energy efficiency, and the unified memory architecture provides high-bandwidth advantages for AI inference.
- **Huawei Kirin 9020**: Equipped on the Mate 70 series, uses the self-developed Taishan CPU architecture, with continuously enhanced NPU and ISP capabilities, supports 5.5G and satellite communication, representing the latest breakthrough in domestic chips.

GPU: The Graphics Rendering Engine

The GPU (Graphics Processing Unit) is specifically responsible for graphics and parallel computing tasks. It has far more cores than the CPU (often hundreds or thousands), but each

core's functionality is relatively simple, suitable for simultaneously processing large amounts of similar computations.

Graphics Rendering Pipeline:

1. **Vertex processing**: Calculates the position coordinates of each vertex in a 3D model.
2. **Rasterization**: Converts 3D graphics into pixels on a 2D screen.
3. **Pixel shading**: Calculates color, texture, and lighting effects for each pixel.
4. **Output compositing**: Composites all layers into the final image and sends it to the screen for display.

GPU Impact on Daily Use: Not just for games! Interface sliding, animation effects, video playback, photo filters, and AR apps all rely on the GPU. A phone with a weak GPU may drop frames when sliding the interface, and games will stutter directly.

GPUs by Brand: Qualcomm uses its self-developed Adreno GPU; MediaTek uses ARM Mali GPU; Apple uses its self-developed GPU, with performance and energy efficiency both being industry benchmarks.

NPU: The AI Acceleration Engine

The NPU (Neural Processing Unit) is a processor specifically designed for neural network operations. Compared to the CPU and GPU, the NPU is more efficient and consumes less power when handling AI tasks.

Neural Network Computing Methods: The core operations of an NPU are matrix multiplication and convolution—precisely the most fundamental and frequent computations in neural networks. Through hardware-level optimization, the NPU can process large amounts of matrix operations in parallel, dozens of times faster than a CPU, with much lower power consumption.

AI Processing Pipeline:

1. **Data input**: Camera images, microphone audio, sensor data, etc.
2. **Preprocessing**: Adjusts data format and size to fit neural network input requirements.
3. **Inference computation**: The NPU loads the trained model, performs forward computation on the input data, and outputs the result.
4. **Post-processing**: Converts AI output into usable results, such as face coordinates, text content, scene labels.

Practical Application Scenarios:

- **Computational Photography**: Night scene synthesis, portrait bokeh, HDR processing, AI beauty.

- **Voice assistants**: Speech recognition, natural language understanding, speech synthesis.
- **Real-time translation**: AI subtitles, conversation translation.
- **Smart recognition**: Text recognition (OCR), object recognition, scene recognition.
- **On-device LLMs**: The latest flagship chips already support running small AI LLMs (7B parameter class) on the phone, enabling offline AI conversations, document summarization, smart writing, and more. The Snapdragon 8 Elite's Hexagon NPU has about 45 TOPS of computing power, the Dimensity 9400's NPU 890 also supports on-device LLMs (LLM inference speed up 80%), and the Apple A18 Pro's 16-core NPU provides about 35 TOPS of computing power.
- **AI image processing**: AI erasure (one-tap removal of passersby from photos), AI image expansion (extending image boundaries), AI style transfer, text-to-image generation, and more.

ISP: Image Signal Processor

The ISP processes the raw data from the camera, turning light signals into beautiful photos.

How to judge performance?Look at the model! Apple A18 Pro, Qualcomm Snapdragon 8 Elite, MediaTek Dimensity 9400, and Huawei Kirin 9020 are all flagship-level. Mid-range options include Snapdragon 8s Gen3/7+ Gen3, Dimensity 8400. Don't just look at core count—architecture and process (nanometer number) are more important. The 3nm process is currently the most advanced, with energy efficiency far superior to 5nm and 4nm.

RAM: The Temporary Workbench

RAM is like your desk—the bigger it is, the more files you can spread out at once. 12GB is the current mainstream, 16GB suits heavy users, and 24GB+ is for gamers and geeks.

RAM type also matters: LPDDR5 is faster and more power-efficient than LPDDR4X. Although Apple's RAM isn't large, system optimization is good, so it appears "sufficient."

What happens when RAM is insufficient?Apps get killed by the system in the background, requiring reload when switching. This is why sometimes you switch to WeChat and come back to find your game is gone.

Storage (ROM/Flash): The Long-Term Warehouse

Storage is like your bookcase—the place that holds all your photos, videos, and apps. The mainstream now is UFS 3.1 or UFS 4.0, several times faster than the older eMMC.

Why does storage speed matter?It determines app opening speed, game loading speed, and photo saving speed. After long use, slow storage makes the phone increasingly laggy.

Screen: Your Eyes

The screen is the main window for interaction between you and your phone; its quality directly affects the experience.

- **Material**: OLED has vivid colors, high contrast, and is more power-efficient (pixels don't emit light when displaying black), but may have flicker at low brightness that strains the eyes; LCD is more eye-friendly and lower cost, but contrast and color aren't as good as OLED.
- **Resolution**: 1080P is mainstream, 2K is sharper but more power-hungry. Apple's "Retina display" concept means: at a certain distance, the human eye cannot distinguish individual pixels.
- **Refresh rate**: 60Hz is standard, 90Hz/120Hz is smoother, silky-smooth when scrolling. High refresh rates consume more power, so many phones support adaptive refresh rate (LTPO), dynamically adjusting based on content.
- **Touch sampling rate**: Affects touch response speed; gamers should pay attention—the higher, the more responsive.
- **Dimming method**: DC dimming is more eye-friendly than PWM dimming, less likely to tire the eyes at low brightness. Many phones now support high-frequency PWM dimming, also for eye protection.

Screen HDR Technology

HDR (High Dynamic Range) lets the screen simultaneously display brighter highlights and darker shadows, greatly improving image layering.

- **Implementation principle**: Ordinary screens have a limited brightness range (SDR), with insufficient gap between the brightest and darkest. HDR screens have higher peak brightness (usually above 1000 nits), greater contrast, and wider color gamut, able to display more detail.
- **Mainstream standards**: HDR10 (most basic, free and open source), HDR10+ (promoted by Samsung, dynamic metadata), Dolby Vision (best image quality but requires licensing). Which standard a phone supports depends on screen hardware and system licensing.
- **Practical effect**: When watching HDR video, sunlight is more glaring, shadows deeper, colors richer. The prerequisite is that the video itself is HDR-produced and the screen supports HDR display.

Photography HDR Technology

HDR in photography and screen HDR are two different things. Photography HDR uses algorithms to solve the problem of "overexposed highlights, dead-black shadows."

- **Algorithm principle**: When you press the shutter, the phone actually takes multiple photos at different exposures (underexposed to preserve highlight detail, overexposed to preserve shadow detail, normal exposure to preserve midtones), then synthesizes them into a single high dynamic range photo.
- **Processing pipeline**: Multi-frame capture alignment compensation (anti-shake) exposure fusion / tone mapping color correction output final photo.
- **Effect optimization**: Modern phones can already achieve "real-time HDR"—what you see in the viewfinder is the HDR-processed result, WYSIWYG. Apple's Smart HDR and Android's auto HDR are both very mature.

Battery and Charging: The Source of Energy

The battery is the phone's most "short-lived" component, visibly aging after two or three years. Mainstream capacity now is 5500-7500mAh.

- **Charging power**: The Android camp has already raced to 100W or even 200W, fully charging in a dozen minutes; Apple is still hovering at 20-30W. Does fast charging damage the battery? Using original accessories is generally fine—the phone has temperature protection.
- **Wireless charging**: Convenient but inefficient and generates heat. Suitable for casual charging at the office, not suitable when you're rushing out.
- **Reverse charging**: The phone becomes a power bank, charging earphones and watches —useful for emergencies.

Camera: The Magic of Light and Shadow

Modern phones often have three or four cameras, each with its own role:

- **Main camera**: The most commonly used, with the largest sensor and best image quality.
- **Ultra-wide angle**: For landscapes and architecture, with a wider field of view.
- **Telephoto**: For distant subjects, with both optical zoom and digital zoom.
- **Macro**: For shooting very small things at close range.

Pixel count isn't everything!Sensor size (bigger sensor beats smaller), aperture size (f/1.8 is larger than f/2.4), and stabilization (OIS optical stabilization is important) all affect image quality. Computational Photography (AI algorithms) is becoming increasingly important, able to combine multiple photos into one good photo.

Periscope Telephoto Lens

Traditional telephoto lenses require a long optical path, which would make the phone very thick. The periscope telephoto uses a 90-degree prism to refract light, allowing the optical path to unfold horizontally inside the phone, thereby achieving 5x or even 10x optical zoom while keeping the phone slim.

- **Structural principle**: Light enters through the lens prism refracts 90 degrees passes through the lens group along the phone's width reaches the sensor. It's like laying a vertical telescope on its side.
- **Optical characteristics**: High optical zoom multiplier (5x-10x), image quality far exceeding digital zoom. But the sensor is usually smaller than the main camera, with average low-light performance.
- **Imaging advantages**: Shooting the moon, distant buildings, and stage close-ups—beyond the reach of ordinary phones.

OIS Optical Stabilization vs EIS Electronic Stabilization

Stabilization is a key technology affecting photo and video quality; the two methods have completely different principles:

- **OIS (Optical Image Stabilization)**: Counteracts handshake by physically moving the lens or sensor. There's a micro-gyroscope inside the lens to detect the direction and magnitude of shake, then drives a motor to move the lens group in the opposite direction. The advantage is no image quality loss; the disadvantage is high cost and space usage.
- **EIS (Electronic Image Stabilization)**: Counteracts shake through software algorithms that crop and offset the image. The phone records a slightly larger frame, then crops a stable area based on gyroscope data. The advantage is no extra hardware needed; the disadvantage is that it crops the image (smaller field of view) and may blur in low light.
- **Practical choice**: For photos, prioritize OIS; for video, combining both works best. Many flagship phones now support ”Sensor-Shift” sensor-shift stabilization, which works better than traditional lens stabilization.

Optical Zoom vs Digital Zoom

- **Optical zoom**: Changes focal length by physically moving the lens group, with no image quality loss. Optical zoom on phones usually has only fixed multiples (such as 2x, 3x, 5x, 10x), and cannot zoom continuously like a camera.
- **Digital zoom**: Enlarges the center area of the image, essentially cropping + interpolation upscaling. The more you zoom, the worse the image quality. 100x digital zoom looks cool, but the actual image quality is already very blurry.
- **Hybrid zoom**: Combines optical and digital, using multi-frame synthesis algorithms to make up for image quality loss between optical zoom multiples. Better than pure digital, but still not as good as optical.

Audio: Speakers and Display Chip

Single speaker vs Dual speakers:

- **Single speaker**: Only one sound unit at the bottom, sound biased to one side, no sense of stereo. Common in mid-to-low-end phones.
- **Dual speakers (Stereo)**: One speaker each at the top and bottom, forming a stereo sound field. When watching videos or listening to music, left and right channels are distinct, with much stronger immersion. Using the earpiece as the top speaker is a common solution.
- **Sound quality difference**: Dual speakers aren't just "louder"—the key is spatial sense and layering. Watching blockbusters with a single speaker always feels like something's missing.

Gaming phone dedicated display chip:

Some gaming phones and performance phones are equipped with an independent display processing chip, whose functions are:

- **Frame insertion**: Interpolates 30fps game footage to 60fps or 120fps through algorithms, making the picture smoother. This process is done on the dedicated display chip, not occupying the SoC's GPU resources.
- **Super-resolution**: Uses AI algorithms to upscale low-resolution footage to high resolution, making the picture clearer.
- **Power reduction**: Because frame insertion and super-resolution are handled by the dedicated display chip, the SoC's load is reduced, lowering heat and power consumption, making games more stable.

GPS Positioning: Single-Frequency and Dual-Frequency

Phone positioning mainly relies on satellite navigation systems such as GPS.

- **Single-frequency GPS**: Only receives L1 band signals, with positioning accuracy of about 3-5 meters. Sufficient in open areas, but in dense urban canyons with tall buildings, signal reflection causes significant positioning deviation.
- **Dual-frequency GPS**: Simultaneously receives both L1 and L5 band signals. The L5 band has a longer wavelength and stronger anti-interference capability; dual-frequency joint computation can eliminate most ionospheric errors, achieving positioning accuracy of 1-3 meters. The advantage is obvious for city navigation and ride-hailing positioning.
- **Multi-constellation support**: Modern phones support not only GPS but also BeiDou (China), GLONASS (Russia), Galileo (Europe), and other satellite systems. The more satellites connected simultaneously, the faster and more accurate the positioning.

Sensors: The Phone's Little Tentacles

There are over a dozen sensors hidden inside the phone, working silently:

- **Accelerometer + Gyroscope**: Senses phone orientation, for game steering and auto-rotate.
- **Ambient light sensor**: Automatically adjusts screen brightness.
- **Proximity sensor**: Turns off the screen when the phone is close to your ear during a call.
- **Fingerprint sensor**: Mainstream now is under-display optical or ultrasonic fingerprint.
- **Face recognition**: Apple's Face ID uses 3D structured light, with high security; many Android phones use 2D photo recognition, with average security.
- **That "System Update" button you tap in [Settings]—that's it.**: Near Field Communication, for transit cards, access cards, bank cards.
- **Barometer**: Measures altitude, can even detect when you climb stairs.
- **Hall sensor**: Detects the opening and closing of magnetic cases.

Operating Systems: Android vs iOS vs HarmonyOS

Hardware is the body; the operating system is the soul. The three major systems each have their own characteristics:

- **Android**: Like an open-source community, phone manufacturers can deeply customize it, offering many choices. You can change themes, install third-party app stores, and tinker with various tricks. But because of its openness, fragmentation is severe, with significant experience differences between brands.

- **iOS**: Like a carefully tended garden, Apple controls both hardware and software, with a unified and smooth experience. App quality is high, privacy protection is good, but freedom is low—many things must follow Apple's rules.

- **HarmonyOS**: Huawei's new species, focusing on distributed capabilities, allowing different devices to collaborate seamlessly. We'll cover it specifically later.

All three have extremely rich ecosystems—just choose the lifestyle you prefer. There's no absolute good or bad, only what suits you.

Phone Ports Explained

Phone ports are bridges connecting to the outside world, and different ports vary greatly:

Micro USB

- **Physical characteristics**: Flat trapezoidal connector, with distinct front and back (you can't insert it reversed). It has been largely phased out on phones, but some low-end devices and old equipment still use it.

- **Transmission limitations**: Only supports USB 2.0 standard, with a maximum transfer rate of 480Mbps; charging power usually doesn't exceed 10W.

- **Characteristics**: Front and back are not interchangeable, short insertion lifespan, slow transfer speed, low charging power.

Lightning (Apple Lightning connector, discontinued)

- **Physical characteristics**: Apple's self-developed 8-pin connector, reversible. Used on iPhone 14 and earlier models, now fully discontinued.

- **Transmission limitations**: The Lightning connector physically only supports USB 2.0 standard, with a maximum transfer rate of 480Mbps. Even using a USB-C to Lightning cable, transfer speed is limited by the Lightning connector itself.

- **Reason for discontinuation**: The EU passed regulations requiring all phones to use USB-C connectors starting in 2024; Apple switched to Type-C from the iPhone 15 series. The Lightning connector has officially exited the stage.

- **Note**: Apple's MFi-certified cables contain a certification chip; non-certified cables may not charge or transfer data properly. If you're still using an older iPhone, it's recommended to keep Lightning cables on hand.

USB Type-C

- **Physical characteristics**: 24-pin oval connector, reversible. It has become the standard connector for phones and most electronic devices. The EU has legislated that all phones, tablets, earphones, and other portable devices must use USB-C connectors starting in 2024, and other regions worldwide are following suit.

- **Transmission protocols**: Type-C is just the connector form; actual transmission capability depends on the internal protocol. Supports USB 2.0 (480Mbps), USB 3.0/3.1/3.2 (5-20Gbps), and even USB4 (40Gbps).

- **Important distinction**: Not all Type-C ports support USB 3.0! Many phones use a Type-C connector but only connect USB 2.0 pins internally, making transfer speeds just as slow as older ports. The iPhone 16/17 standard models only support USB 2.0 speeds, while the Pro models support USB 3.0 (requires a USB-C to USB-C cable).

- **Charging capability**: Supports USB PD and other fast charging protocols, up to 240W charging.

- **Video output**: Supports DisplayPort Alt Mode, can output video to a monitor via an adapter cable.

5mm Audio Connector

- **Physical characteristics**: Round 3-segment or 4-segment jack; 3-segment transmits left and right channel audio, 4-segment adds a microphone channel.

- **Protocol**: Analog signal transmission, plug-and-play, no drivers or conversion needed. Stable sound quality, zero latency.

- **Current status**: Most flagship phones have removed the 3.5mm connector, requiring a Type-C adapter or Bluetooth earphones. Some mid-range phones and gaming phones still retain it.

Phone Product Lines and Naming Conventions

Faced with the dazzling product lines of various brands, understanding naming conventions can help you quickly locate the right model for you.

Mainstream Brand Product Lines:

- **Apple**: iPhone (standard) / iPhone Plus (large screen) / iPhone Pro (professional) / iPhone Pro Max (large screen professional) / iPhone SE (budget) / iPhone 16e (budget AI version, equipped with A18 chip and Apple's self-developed C1 baseband). The 2026 iPhone 17 lineup is fully equipped with the A19 Pro chip, with further enhanced AI capabilities.
- **Huawei**: Mate series (Business flagship) / P/Pura series (Imaging flagship) / Mate X series (Foldable flagship) / nova series (Youth mid-range) / Enjoy series (Entry-level)
- **Xiaomi**: Xiaomi Digital series (Flagship) / Xiaomi MIX (Exploration flagship/Foldable) / Redmi K series (Performance flagship) / Redmi Note series (Mid-range) / Redmi Digital series (Entry-level)
- **OPPO**: Find X series (Flagship) / Reno series (Mid-range) / A series (Entry-level) / OnePlus Digital series (Performance flagship)
- **vivo**: X series (Imaging flagship) / iQOO Digital series (Performance flagship) / S series (Selfie mid-range) / Y series (Entry-level)
- **Samsung**: Galaxy S series (Flagship) / Galaxy Z Fold/Flip (Foldable flagship) / Galaxy A series (Mid-range)

Common Suffix Meanings:

- **Pro**: Abbreviation for Professional, the professional version. Usually upgrades camera, screen, processor, and other core configurations on the basis of the standard version.
- **Max**: The maximum version. Largest screen, largest battery, but configuration may not be higher than Pro.
- **Ultra**: The ultimate version. Usually represents the highest configuration of the series, with the most powerful hardware and the highest price. Such as Samsung S24 Ultra, Xiaomi 14 Ultra.
- **SE**: Special Edition, actually mostly a budget version. Reduced configuration, lower price. Such as iPhone SE.
- **Plus**: The enlarged version. Larger screen than the standard version, larger battery, but basically the same configuration.
- **Lite/Youth Edition**: The lightweight version. Streamlined configuration, lower price.
- **Note**: Originally referred to large-screen phones, now used differently by various brands. Redmi Note is a mid-range series.
- **Neo**: The new sharp version. Usually refers to performance-oriented mid-range models.

Baseband and Antenna: The Unsung Heroes of Phone Communication

Your phone can make calls and access the internet thanks to the partnership of the baseband chip and antenna.

Baseband Chip:

The baseband is the processor in the phone responsible for wireless communication. It modulates digital signals into radio waves for transmission, and demodulates received radio waves back into digital signals. Baseband performance directly affects call quality, internet speed, and signal stability. Qualcomm is the largest mobile baseband supplier; MediaTek and Samsung also have self-developed basebands. Starting from iPhone 16e, Apple has equipped its self-developed C1 baseband chip, marking Apple's official entry into the self-developed baseband era; the C1 baseband uses a 4nm process with excellent power consumption. iPhone 16/16 Plus/16 Pro/16 Pro Max still use Qualcomm basebands (Snapdragon X75 5G), with subsequent models expected to gradually transition to self-developed basebands. Earlier iPhone XS/11 series using Intel basebands had poor signal performance, which was a pain point for many users.

Antenna Design:

Phone antennas are hidden inside the body, usually arranged around the frame. Antenna design directly affects signal reception performance:

- **MIMO technology**: Multiple-input multiple-output, using multiple antennas to simultaneously send and receive data, improving speed and stability. 4×4 MIMO is faster than 2×2 MIMO.
- **Number of antennas**: Flagship phones usually have a dozen antennas (cellular, Wi-Fi, Bluetooth, NFC, GPS each need independent antennas).
- **Impact of metal body**: Metal shields signals, so metal-body phones need antenna bands designed in (plastic strips on the frame). Glass and plastic bodies have less impact on signals.

Network Technology Evolution:

- **: Currently the widest coverage, sufficient speed (theoretically up to 100Mbps+), relatively low power consumption.**: Theoretical peak rate 1Gbps downlink / 500Mbps uplink (Cat.18), actual experience usually 50-150Mbps. Widest coverage, sufficient for use.

- **5G NR**: Theoretical peak rate 10Gbps+ downlink. Sub-6GHz band has better coverage; mmWave millimeter wave is extremely fast but has very limited coverage.
- **5G-A (5G Advanced)**: An enhanced version of 5G, also known as 5.5G. Downlink speeds can reach 10Gbps, uplink 1Gbps, with significantly improved connection density and positioning accuracy. Since 2024, China Mobile, China Telecom, and China Unicom have been deploying 5G-A networks in key cities. Flagship phones like the Huawei Mate 70/80 series and Xiaomi 15/16 series already support 5G-A. 5G-A also brings "integrated communication and sensing" capabilities, where base stations can simultaneously communicate and sense the environment, providing infrastructure for low-altitude economy (drone delivery) and vehicle-to-everything (V2X).

Network Band Classification:

The network bands supported by a phone determine which countries and carriers' networks it can be used on:

- **FDD-LTE Band (B series)**: Such as B1, B3, B5, etc., 4G bands. Different countries use different bands; China mainly uses B1/B3/B5/B8/B34/B38/B39/B40/B41.
- **TDD-LTE Band**: Another 4G standard, heavily used in China with B38/B39/B40/B41.
- **5G NR Band (N series)**: Such as n1, n3, n28, n41, n78, n79, etc. n78 (3.5GHz) is China's main 5G band; n41 (2.5GHz) is used more by China Mobile.
- **Purchase advice**: If you need to use the phone abroad, confirm before purchase whether it supports the target country's bands. China mainland phones usually have the most complete support for Chinese bands, but may lack some overseas bands.

How Many Roles Does Your Phone Actually Play?

It has long been more than just a phone:

1. **Communication tool**: Phone, SMS, WeChat, email...
2. **Smart camera**: Photos, videos, document scanning...
3. **Wallet**: Payments, transit cards, bank cards, membership cards...
4. **Information portal**: News, weather, stocks, knowledge...
5. **Gaming console**: From match-3 to AAA titles...
6. **Mobile office**: Documents, spreadsheets, meetings, collaboration...
7. **Health assistant**: Step counting, heart rate, sleep, exercise...

8. **Smart home remote**: Control lights, AC, TV, robot vacuums...
9. **Digital ID**: Electronic ID cards, electronic driver's license, health codes...
10. **Entertainment center**: Music, videos, e-books, podcasts...

Think about it—how much life is packed into one tiny device! So, learning to use it well is learning to live better.

Hands-on: Understand Hardware Configuration

In [Settings], see if you can find the processor model, RAM, and ROM (storage) sizes. Get a sense of your phone's capabilities. Android users can download "AnTuTu" or "DevCheck" to view detailed hardware information.

Appendix: Android Phone Memory Expansion Explained

When buying a phone, you often see promotions like "12GB+8GB RAM" —as if 8GB of RAM has appeared out of thin air. But can it really make the phone faster? Could it actually damage the phone? In this section, we'll thoroughly explain the technology of "memory expansion."

Essential Definition

Android phone "memory expansion" (also called memory extension, memory fusion, RAM Plus, etc.)**does not actually physically add RAM chips**; it's a system-level software technology: by dynamically allocating part of the phone's internal storage space (ROM/UFS flash) to simulate virtual RAM for use.

To use an analogy: physical RAM is the "high-speed workbench," while expanded memory is the "temporary storage room." When the workbench isn't enough, the system moves temporarily unused data to the storage room, and moves it back when needed. The storage room has large capacity, but back-and-forth moving is definitely slower than keeping it directly on the workbench.

Technical Principle

Mainstream manufacturers implement it in two main ways:

zRAM Compression

A compressed data space is partitioned within RAM, compressing inactive app code for storage, with a compression ratio of about 2:1. The data remains in RAM—fast and doesn't wear the flash—but requires CPU involvement for compression and decompression.

Swap Partition

Directly partitions a fixed space in UFS flash as virtual memory, writing background process data to storage. Depends on storage bandwidth; read/write speed is far lower than physical memory.

The "12GB+8GB RAM" promoted by manufacturers is usually a combination of 12GB physical RAM + 8GB flash virtual memory. It's worth noting that zRAM doesn't occupy phone storage by default; only when zRAM is completely filled with data does the system create swap files on storage, and this situation is extremely rare.

Speed Gap Is the Key Bottleneck

The fundamental limitation of this technology is the speed difference. Currently, mainstream UFS 4.0 flash has sequential read speeds of about 4200MB/s, only 1/10 – 1/15 of LPDDR5X physical memory. Therefore, during light-load zRAM compression, the speed is close to real RAM, but once entering high-load scenarios, the speed drops to a few dozenths.

Applicable Scenarios and Benefit Boundaries

Memory expansion is not a panacea; effects vary by model and usage scenario:

- **When it helps**: Models with 6GB physical RAM + UFS 3.1 or above, with ≥30GB free space, can keep 3-4 more apps alive in the background after enabling 3GB expansion, with a 68% reduction in reload rate—suitable for heavy multitaskers like delivery riders and ride-hailing drivers.

- **Daily scenarios**: Significant improvements for WeChat multi-instance, browser tab retention, instant messaging response, with noticeably shorter cold-start times for background apps.

- **Limited scenarios**: Limited improvement for high-bandwidth scenarios like large game loading and 4K video editing.

- **When it's harmful**: Models with 8GB RAM + UFS 2.2 and insufficient free space, where enabling it actually increases game frame rate fluctuations, with more battery drain when scrolling short videos and lag when switching.

- **When it's useless**: Models with 16GB+ of large RAM see almost zero benefit—at most keeping one more app alive, but increasing CPU load by 12% and heat by 2-3°C. For models with more than 12GB RAM, it's recommended to simply turn it off to avoid meaningless compression computation.

Naming and Enabling Methods by Manufacturer

Different brands call it differently: Xiaomi calls it "Memory Extension," Huawei calls it "Memory Fusion," Samsung calls it "RAM Plus," vivo names it "Extended RAM," OnePlus calls it "Memory Extension." This feature is a system-native capability and is not directly tied to the underlying Android version.

The general enabling path is: Enter [Settings] [More Settings] find the [Memory] or [Performance] sub-item enable "Memory Expansion" and select the capacity tier (usually 2GB–7GB), taking effect after restart. After enabling, the system will display a memory combination such as "8GB+4GB".

Notes

- Memory expansion occupies part of the phone's storage space; please ensure free storage is no less than 20-30GB, otherwise it may affect system fluidity.
- Using flash as virtual memory causes read/write wear; although modern UFS flash has a long lifespan, long-term high-frequency swap may still accelerate storage aging.
- If the phone itself has small physical memory (such as 4GB or below), enabling memory expansion only provides temporary relief and cannot fundamentally solve lag issues.

Summary

Memory expansion is a mature system-level optimization technology that can improve multitasking capability and background retention for mid-to-low-end models without hardware changes, but it's limited by the huge speed gap between flash and physical memory, and is not a "miracle cure." Whether to enable it and how much to set should be judged based on your model's physical memory size, flash specification, free storage space, and usage habits.

Hands-on: Check If Your Phone Has Memory Expansion

1. Open [Settings] and search for "Memory Extension," "Memory Fusion," "RAM Plus," or "Extended RAM."
2. Once found, check whether it's currently enabled and what the expansion capacity is set to.
3. Based on this section, judge: How large is your model's physical memory? What's the flash specification? Is the free storage space sufficient?
4. Try disabling or reducing the expansion capacity, and observe changes in background retention and heat over a week to find the setting that suits you best.

Chapter 1 Appendix: HarmonyOS Operating System Special Topic

HarmonyOS is Huawei's self-developed operating system. Since its release in 2019, it has gone through several major versions. It's not just "another Android replacement"—it's a new species with a unique design philosophy. Let's dive deep into this rising operating system.

What Is HarmonyOS? Why Is It Different?

HarmonyOS's core philosophy is "interconnection of all things." Unlike traditional operating systems that primarily serve a single device, HarmonyOS was designed from the start with multi-device collaboration in mind.

Distributed architectureis the soul of HarmonyOS. Simply put, it can virtualize multiple devices into a "super terminal." Your phone, tablet, computer, watch, and TV are no longer isolated individuals but integrated "organs" that can call upon each other's capabilities.

For example: You're watching a video on your phone and want to see it on a bigger screen. The traditional way is screen casting; on HarmonyOS, you can directly "stream" the video to the TV to continue watching, with the phone becoming a remote control. This isn't simple screen casting—the app itself is running across devices.

HarmonyOS 4.3: A Mature Version Based on AOSP

HarmonyOS 4.3 is the currently widely deployed version on Huawei phones. It should be noted that HarmonyOS 4.3 is compatible with the code framework of the Android Open Source Project (AOSP), so it can fully support Android apps—most apps you download from the Huawei AppGallery are essentially still Android apps, running through HarmonyOS's compatibility layer. But HarmonyOS 4.3 is not simply a reskinned Android; it adds a large number of Huawei's self-developed distributed capabilities, Ark engine, and other technologies on top of AOSP.

Core Features:

- **Android app compatibility**: Since it's developed based on AOSP, HarmonyOS 4.3 can run almost all Android apps normally; users hardly notice any difference.
- **Ark engine**: Huawei's self-developed compiler and runtime, making apps run more smoothly and power-efficiently. Compared to early versions, fluidity has improved significantly.
- **Super Device**: In the Control Center, you can intuitively see other HarmonyOS devices around you—just drag to connect. It's convenient for the phone to use the computer's keyboard and mouse, or the tablet to use the phone's camera.

- **Service cards**: Similar to iOS widgets, but more flexible. You can swipe up on an app icon to expand a card showing key information without opening the app.
- **or**: Only allows installing apps from the official app store, reducing malware risks.
- **Privacy Center**: Centrally manage app permissions and view app behavior records.

User Interface Features:

- **Magazine lock screen**: The lock screen displays beautiful images and articles; swipe to enter the home screen.
- **Universal cards**: Home screen icons can be expanded into cards to display more information.
- **Super Hub**: A temporary storage area where you can stash text, images, and files, then drag them into any app. Cross-app sharing becomes incredibly smooth.
- **Smart Search**: Global search that can find apps, files, settings, and web pages.

HarmonyOS NEXT (HarmonyOS 6): The True Starting Point of Huawei's Autonomous Operating System

HarmonyOS NEXT (HarmonyOS NEXT 5.0) is fundamentally different from HarmonyOS 4.3—it uses Huawei's self-developed HarmonyOS microkernel, no longer relying on the Linux kernel and AOSP code, and**no longer directly supports Android apps**. This is the key step for HarmonyOS to officially transition from "an Android-compatible transitional form" to "a fully independent operating system."

Major Changes:

- **Fully autonomous kernel**: HarmonyOS NEXT uses Huawei's self-developed kernel, containing no AOSP code. The system is more streamlined, more secure, and more efficient.
- **No longer supports Android apps**: This is the biggest change. HarmonyOS NEXT can only run native HarmonyOS apps and cannot install or run APK files. Before upgrading, please confirm that your frequently used apps already have native HarmonyOS versions.
- **Native app ecosystem**: Huawei is fully pushing developers to migrate apps to the native HarmonyOS platform. Currently, mainstream apps such as WeChat, Alipay, and Taobao have launched native HarmonyOS versions, but some niche apps may not yet be adapted.

- **Star Shield security architecture**: A brand-new security system with full-chain protection from hardware to software. Supports finer-grained permission control, such as "fuzzy location"—apps can only know roughly where you are, not your precise location.
- **Deep AI integration**: System-level AI capabilities, not just a single app having AI. For example, AI subtitles can translate videos in real time, and AI cutout can intelligently identify subjects.
- **Cross-device seamless streaming**: Further distributed capabilities. App state can switch seamlessly between devices—a document you're editing on your phone automatically opens on your computer when you walk up to it.
- **Meta-services**: Lightweight services that can be used without installation. For example, scanning a QR code directly opens a function without first downloading an app.

Must-Read Before Upgrading to HarmonyOS NEXT

After upgrading to HarmonyOS NEXT, your phone will not be able to run Android apps. Please be sure to confirm before upgrading: (1) Your frequently used apps already have native HarmonyOS versions; (2) Your work or life doesn't have scenarios that must depend on Android apps. After upgrading, you cannot roll back to HarmonyOS 4.3.

Operation Logic Changes:

- **Gesture navigation**: Swipe from left or right edge to go back, swipe up from the bottom to return to the home screen, swipe up and pause to enter multitasking.
- **Smart multi-window**: Supports split screen, floating windows, parallel view, and other multitasking forms.
- **Celia calls**: Huawei's video call service, supporting screen sharing, beauty effects, and background blur.

HarmonyOS Ecosystem: Devices and Apps

HarmonyOS is not just a phone system—it's an ecosystem covering multiple devices.

Supported Device Types:

- **Phones and tablets**: Core devices with the most complete functionality.
- **Smart watches and bands**: Run independently or in coordination with the phone.
- **Smart screens (TVs)**: Home entertainment centers.
- **Smart speakers**: Voice interaction entry points.
- **Automobiles**: HarmonyOS cockpit, smart driving cabin experience.

- **Smart home**: Refrigerators, air conditioners, washing machines, etc.

App Ecosystem Status:

The HarmonyOS App Market already has millions of apps, and mainstream apps are essentially all available. Huawei is also actively encouraging developers to create native HarmonyOS apps. As of mid-2026, top apps including WeChat, Alipay, Taobao, Douyin, Meituan, and JD.com have all launched native HarmonyOS versions, making daily use largely barrier-free. Some niche and industry-specific apps may still be in adaptation —before upgrading, it's recommended to confirm whether your frequently used apps are already supported.

HarmonyOS Practical Tips

- **Super Device**: Swipe down from the top right of the screen to open the Control Center, tap "Super Device" to see other devices around you—drag to connect.
- **Super Hub**: Long-press text/images/files, select "Copy to Hub," then long-press to paste in other apps, or drag it out.
- **Service cards**: Swipe up on an app icon to expand the card; long-press the card to resize or add to the home screen.
- **Smart multi-window**: Swipe inward from the left or right edge of the screen and pause to open the sidebar; tap an app to open it as a floating window.
- **Privacy protection**: Settings > Privacy, where you can view app permission usage records, set fuzzy location, etc.

Notes

HarmonyOS is continuously updating and iterating; different versions may have some differences. This description is based on the main features of HarmonyOS 4.3 and 6; specific functions are subject to your phone's actual version. It's recommended to keep the system updated to get the latest features and security patches.

Try It Now: Experience HarmonyOS Features

1. Open the Control Center, find "Super Device," and see if there are other HarmonyOS devices around.
2. Try "Super Hub": Copy some text to the Hub, then drag it out in Notes.
3. Swipe up on a few common app icons to see what their service cards display.
4. Go to Settings > Privacy to see which apps have recently used sensitive permissions.

Chapter 2: Organize Your Digital Room—File Management

Phones get messy over time, and good habits start with managing files. Just like your room needs regular tidying, your digital space needs upkeep too. An organized phone isn't just easier to find things on—it runs more smoothly too.

I. Three-Tier Storage System: Your Digital Warehouse

1. **Device storage**: The core zone—fast, and where the system and apps live. This is your phone's most precious space, so spend it wisely.
2. **SD card (if supported)**: An extra closet—slower, but good for large files like movies and photos. Note: many flagship phones no longer support SD cards.
3. **Cloud storage**: A safe + sync drive. Prevents loss and syncs across multiple devices. iCloud, Baidu Netdisk, and OneDrive are common choices.

II. Understanding the "File Manager"

Android usually has a [Files] or [File Manager] app, while iOS has the built-in [Files] app. These are the "Explorer" you use to view all files on your phone.

Common folder meanings:

- **DCIM**: Photos and videos taken by the camera.
- **Pictures**: Other images, such as screenshots and saved pictures.
- **Download**: Files downloaded by the browser.
- **Music**: Music files.
- **Movies**: Video files.
- **Documents**: Document files.

- **Android**: Android system folder containing data for each app—generally do not delete manually.

III. File System Structure: A Deeper Look

Android is based on Linux, so its file system has a hierarchical structure. While ordinary users don't need to dive deep, knowing some basics helps.

Storage partitions:

- **/system**: System partition, storing operating system files. Ordinary users have no access—root permission is required.
- **/data**: Data partition, storing user data and app data. This is the space you use day to day.
- **/sdcard or /storage/emulated/0**: User-accessible storage, where your photos and downloads live.

App data storage:

- **Internal storage**: The app's private space, inaccessible to other apps. Deleted when the app is uninstalled.
- **External storage**: Files the app stores in the user-accessible area, such as photos and downloads. Not necessarily deleted when the app is uninstalled.

IV. Data Backup and Recovery: Insure Your Data

Data is priceless—backup is a habit you must build.

Backup strategies:

1. **Cloud backup**: Automatic and convenient, but requires network and cloud space. iCloud, Huawei Cloud, Xiaomi Cloud, etc., all support this.
2. **Local backup**: Connect to a computer with a cable and back up using official tools. Fast and spacious, but requires manual operation.
3. **Dual backup**: Cloud + local—the safest option.

What to back up?

- **Photos and videos**: Take up the most space and are the most precious.

- **Contacts**: Most convenient when stored in a cloud account.
- **WeChat chat history**: Use WeChat's built-in backup feature.
- **Important documents**: Keep them on cloud drives or a computer.

V. File Security: Protect Your Privacy

- **Encrypt sensitive files**: Many phones have a "Secure Folder" or "Private Space" feature where you can place sensitive files that require a password or fingerprint to access.
- **Deleting $\neq$ destroying**: Ordinary deletion only marks the space as available to overwrite—the data is still there. To truly delete, use dedicated secure erasure tools or overwrite multiple times.
- **Cloud security**: Choose reputable cloud providers, enable two-factor authentication, and regularly review logged-in devices.

VI. Out of Space? Time for a Deep Clean

- **Clear cache**: Apps' temporary memory—clearing it frees up space. You can do this in Settings > Storage.
- **Find large files**: Use the file manager to sort by size and catch the "culprits" hogging space.
- **Delete leftovers**: After uninstalling apps, junk files are sometimes left behind—clean them up manually.
- **Clean up albums**: Delete duplicate photos, blurry photos, and unneeded screenshots.
- **Clean up WeChat**: WeChat is a space hog—regularly cleaning chat history and cache is essential.

VII. Complete Guide to Common File Formats

All those file extensions on your phone can be dizzying—understanding them helps you manage files better:

Format	**Type**	**Characteristics**	**Recommended way to open**
JPG/JPEG	Image	Lossy compression, small size, best compatibility. Default format for phone photos.	Album/any image viewer
PNG	Image	Lossless compression, supports transparent backgrounds. Common for screenshots and icons. Larger than JPG.	Album/any image viewer
MP3	Audio	Lossy compression, small size, excellent compatibility. The most universal audio format.	Any music player
FLAC	Audio	Lossless compression, CD-quality sound, 3-5x larger than MP3. Audiophiles' top choice.	Music player/professional player app
WAV	Audio	Uncompressed raw audio, best sound quality but huge size. Common for professional recordings.	Music player
M4A	Audio	AAC encoding, better sound quality than MP3 at the same bitrate. Common in the Apple ecosystem.	Apple Music/music player
MP4	Video	The most universal video container format, with excellent compatibility. Default format for phone-recorded videos.	Album/any video player
MKV	Video	Open-source container format,	VLC/nPlayer

Try It Now: Organize the "Downloads" Folder

Open the file manager > go to the "Downloads" folder. See what's inside? Delete the useless stuff, and categorize the useful stuff (for example, create new "Work Materials" and "Memes" folders to put them in). Also check your storage space to see which file types take up the most.

VIII. Image and File System Management: Keep Your Assets in Order

In the previous section we learned to "recognize folders," but recognition alone isn't enough. This section is about**how to proactively establish a set of long-term sustainable management practices**so your files are always findable, not lost, and not snooped on.

File Classification and Naming Conventions

The root of chaos is the lack of rules. Establishing rules is actually simple—the key is**persistence**.

- **"Time + Event" naming method**: Include a timestamp and event description in folder names for easy recall and retrieval later. For example,`202606_Travel_Receipts`and`2025_Year_End_Summary`and`2024_Kids_Birthday`. This way you can see the timeline and know the content at a glance.
- **Three-tier classification system**:
  - **Tier 1**: By file type (Document / Image / Video / Audio)
  - **Tier 2**: By use case (Work / Life / Study)
  - **Tier 3**: By specific event or project (e.g., "2026Q2_ProjectA")
- **Monthly cleanup plan**: Pick a fixed date each month (payday or the 1st is recommended) to do a focused cleanup of the "Downloads" folder:**archive the useful, delete the useless, back up the precious**. Twelve small cleanups a year are far easier than one massive annual overhaul.

This Month's Task List

Pick a free weekend and follow these steps in order:

1. Create a work folder for this month in the file manager in the form`2026Q2_ProjectA`;
2. Go through all the files from this week in the "Downloads" folder—keep the useful ones, archive them, and delete the useless ones;
3. Create annual folders like`2025_Family`and`2026_Family`for this year's family photos.

Redundant File Handling Strategies

Phones “gain weight” over time—mostly because of redundant files.

- **72-hour auto-cleanup**: Temporary files received by social apps like WeChat and QQ (stickers, chat images, short videos) are usually useless after one view. In [WeChat] > [Settings] > [General] > [Storage], turn on ”Auto Cleanup” and set the cache expiration to 72 hours. Your phone will feel instantly lighter, and you won’t lose important records.
- **Targeted storage cleanup**: Android [Settings] > [Storage] > [Cleanup], iOS [Settings] > [General] > [iPhone Storage]—the system will list items by category: cache / temporary files / uninstallable apps / large files. Focus on: (1) cache files; (2) expired installation packages (old versions left after app upgrades); (3) duplicate files (photos with the same name and size).
- **Auto-classification rules**: Most brand album apps support smart classification: people, scenery, documents, screenshots. Once enabled, a newly taken group photo will be automatically filed under ”People > Zhang San,” and a scanned contract under ”Documents.”**Combined with monthly organizing, this greatly reduces the chance of ”not being able to find things.”**.

Images are the most “sensitive” data type on your phone—a casual photo can reveal your location, device, and time. We recommend

establishing a “dual protection” workflow**:**:

Mask visible information

1. **: Use the built-in or third-party tools (such as Meitu XiuXiu, Snapseed) to**redact the middle digits of ID numbers, license plates, house numbers, names, amounts**, and other key fields;**Strip metadata
2. **: Before sending, tap ”Share” or ”Export” in the album—the system will ask whether to keep location info—choose**”Don’t Keep.”**This way the shooting device model, GPS coordinates, and shooting time won’t be sent along with the image;**Temporary watermark
3. **(optional): Add a text watermark like ”For XX processing only” on sensitive screenshots to prevent misuse.**Common methods for metadata (EXIF) cleanup:

Common methods for metadata (EXIF) removal (basic operations; for advanced protection see Chapter 6 “Photo Anonymization”):

- **: iOS [Settings] > [Privacy & Security] > [Camera], you can turn off ”Precise Location”; when sharing, selecting ”Current photo only” will**automatically strip most EXIF**.**.

- **Android**Professional tools
- **: ExifTool (desktop), Metapho (iOS), Photo Metadata Remover (Android) can thoroughly remove all metadata.**For sensitive images that need

long-term storage**(ID cards, bank cards, household register scans, passports), we recommend storing them in an**encrypted album**:**:

- Huawei/Xiaomi: [Album] > long-press photo > "Move to Safe";
- Third-party encrypted apps:
- 360 Safe Album,and,and**, etc., with separate password protection and better concealment.**

Chapter 3: Plugging Your Phone into Its "Lifeline"

Without a network, a smartphone loses half its "smart." Networks are the lifeline of modern phones—understanding them, using them well, and protecting them is required study for every phone user.

Don't Get Lost Online—Telling Wi-Fi and Mobile Data Apart via "Roads" and "Cars"

Many people have both the Wi-Fi and signal icons lit up at the top of their phone screen, but are completely confused about the essential difference between them. The most classic misconception is:**connected to Wi-Fi but pages won't load**—because the router lost its internet connection, and the phone didn't automatically switch back to mobile data. To fully understand this issue, we need to build a "road and car" mental model.

Core Metaphor: Roads and Cars

Imagine the internet as a city. **"Roads"** are the channels between your phone and the internet, and **"cars"** are the data traveling on the roads. Wi-Fi and mobile data are two different roads, but the data (cars) itself is the same.

Essential Difference: Local Channel vs. Remote Highway

Wi-Fi is essentially a**local area network technology**—it's like the road wirelessly laid downstairs in your building, responsible for transmitting data from the router to your phone. But if your broadband bill is unpaid or the fiber cable is broken, the road is still there, but the "highway" beyond is cut off, so naturally you can't go anywhere. Mobile data, on the other hand, is transmitted through carrier 4G/5G base stations—equivalent to driving directly onto the carrier's high-speed dedicated line. As long as there's signal, you can connect directly to the internet.

Cost and Scenarios: Fixed Monthly Fee vs. Pay-per-Use

Wi-Fi is usually a one-time investment (monthly broadband fee)—as long as the power is on, the whole family can use it unlimited, making it ideal for high-traffic scenarios like downloading movies and gaming. Mobile data is billed by usage—exceeding your plan may result in throttling or extra charges, making it more suitable for lightweight tasks in mobile scenarios like subways and business trips.

Try It: Check Your Data Plan

Open your phone’s [Settings] > [Mobile Network] > [Data Management] to see how much data you have left this month and which apps use the most. Knowing your numbers means you won’t panic when you go out.

Security Considerations: Public Wi-Fi Is a Hidden “Phishing Ground”

In terms of security, mobile data is usually safer than public Wi-Fi. Mobile data is encrypted by the carrier, while public Wi-Fi can indeed be set up by hackers as “fake hotspots” to intercept unencrypted traffic. But let’s be clear: in today’s world of ubiquitous HTTPS, what attackers can intercept is**which websites you visit**(domains) and**unencrypted HTTP content**; while HTTPS-encrypted data like WeChat messages and banking app login passwords**cannot be directly read**—unless a more complex SSL stripping attack is performed and the user ignores certificate warnings. Even so, when making payments on the go, using mobile data directly is still the safest choice.

Security Reminder

At airports, malls, cafés, and other public places, do not connect to free Wi-Fi with suspicious names (such as “Free_WiFi_2024”). Even if you do connect, never sign in to banking apps or enter passwords on it.

Checking on Your Phone: Decoding the Status Bar’s “Hidden Language”

How do you know what your phone is using to connect right now? Taking a Huawei phone as an example, when a**two-way arrow icon**appears next to the carrier signal or WLAN signal in the status bar, it means you’re using that method to access the internet; if both light up at the same time, it means network acceleration is in effect and the phone is using both channels simultaneously.

iPhone users have it more intuitive: if the Wi-Fi icon is lit, you’re on Wi-Fi; if not, look at the signal bars in the upper right corner to know you’re on mobile data. Some Android phones also display “5G” or “4G” in the status bar, directly telling you the current mobile network type.

I. Mobile Networks (4G/5G) and Data

Mobile networks are the foundation of staying connected anywhere. From 2G to 5G, speeds have increased a thousandfold, but the principles are much the same.

Network standards:

- **4G LTE**: Currently the widest coverage, sufficient speed (theoretically up to 100Mbps+), relatively low power consumption.
- **5G**: Fast (theoretically up to 10Gbps), but coverage is still expanding and power consumption is high. Best for downloading large files and watching HD video.
- **5G SA vs NSA**: SA is standalone networking; NSA relies on 4G base stations. SA offers a better experience, but requires both carrier and phone support.

Data management:

- **What data is**: The amount of data generated when your phone goes online. Video and images use the most data; text and voice use relatively little.
- **Data-saving tips**: Turn on "Data Saver" in settings, and update apps or watch HD video over Wi-Fi.
- **Signal bars $\neq$ speed**: Strong signal only means a good connection—speed also depends on how busy the base station is and how far you are from it.

Important Warning: Turn off [Data Roaming] Before Going Abroad!

Using mobile data abroad can result in astronomical bills! Before going abroad, be sure to turn off data roaming in settings, or purchase a local data card/international roaming plan in advance.

II. Wi-Fi: Connection, Sharing, and Security

Wi-Fi is the main connection for homes and offices—fast and doesn't use mobile data. But Wi-Fi is constantly evolving, and the differences between standards are significant.

Connecting to Wi-Fi:

Go to [Settings] > [WLAN], select a network, and enter the password. Many phones now support QR code connection, which is even more convenient.

Sharing passwords:

Both Android and iOS let connected phones quickly share passwords with friends. Android generates a QR code; iOS can share automatically by bringing devices close together.

Wi-Fi standard evolution:

Wi-Fi standards are set by IEEE, and each generation brings significant improvements:

Standard	Protocol	Band	Max Speed	Features
Wi-Fi 4	802.11n	2.4/5GHz	600Mbps	Legacy standard, decent coverage
Wi-Fi 5	802.11ac	5GHz	3.5Gbps	Currently the most widespread, sufficient speed
Wi-Fi 6	802.11ax	2.4/5GHz	9.6Gbps	Strong multi-device performance, low latency
Wi-Fi 6E	802.11ax	2.4/5/6GHz	9.6Gbps	Adds 6GHz band, less congestion
Wi-Fi 7	802.11be	2.4/5/6GHz	46Gbps	Ultra-fast transmission, multi-link aggregation

- **Wi-Fi 6 key technologies**: OFDMA (Orthogonal Frequency-Division Multiple Access) allows multiple devices to transmit simultaneously; MU-MIMO (Multi-User Multiple Input Multiple Output) improves multi-device concurrency efficiency; TWT (Target Wake Time) makes devices more power-efficient.

- **Wi-Fi 6E**: Built on Wi-Fi 6, it adds the 6GHz band, providing more channels and reducing interference. Requires support from both the phone and the router.

- **Wi-Fi 7**802.11be

- **2.4GHz vs 5GHz**: 2.4GHz has better wall penetration but slower speed and more interference; 5GHz is faster but weaker at penetrating walls. The 6GHz band of Wi-Fi 6E/7 is the fastest but weakest at wall penetration, suitable for use within the same room.

Public Wi-Fi risks:

Public Wi-Fi may have hackers eavesdropping on data or setting up phishing hotspots. Guidelines:

- Avoid logging into bank accounts or entering passwords.
- Best to use a VPN for encrypted connections.
- Don't auto-connect to unfamiliar Wi-Fi.
- Use HTTPS websites (look for the lock icon in the address bar).

III. Bluetooth: The All-Rounder of Short-Range Wireless Communication

Bluetooth is probably one of the most frequently used wireless technologies on your phone —wireless earbuds, smart watches, car systems, keyboards and mice all rely on it to connect.

Technical principles:

Bluetooth operates in the 2.4GHz ISM band and uses Frequency-Hopping Spread Spectrum (FHSS) to spread data across 79 channels and switch between them rapidly, effectively avoiding interference. Bluetooth range comes in three classes: Class 1 about 100 meters, Class 2 about 10 meters (commonly used by phones), and Class 3 about 1 meter.

Version evolution:

- **Bluetooth 4.0**: Introduced Bluetooth Low Energy (BLE), allowing small-battery devices like wristbands and heart rate monitors to connect wirelessly, extending battery life from days to months.
- **Bluetooth 4.2**: Improved transmission speed and privacy protection, supports IPv6 direct connection.
- **Bluetooth 5.0**: Doubled transmission speed (2Mbps), 4x the range (theoretically 300 meters), 8x the broadcast capacity. Supports dual audio (connecting two pairs of earbuds simultaneously).
- **Bluetooth 5.1**: Introduced direction finding (AoA/AoD), with centimeter-level positioning accuracy.
- **Bluetooth 5.2**: New LE Audio standard, supports LC3 codec (better sound quality, lower power consumption), multi-stream audio, and broadcast audio.

- **Bluetooth 5.3/5.4**: Further optimized power consumption and connection stability, enhanced periodic broadcasting.

LE Audio adoption: In 2025-2026, LE Audio has entered a rapid adoption phase. New true wireless earbuds from major brands (such as Huawei FreeBuds Pro 5, Xiaomi Buds 6 Pro, Samsung Galaxy Buds3 Pro, etc.) already support LE Audio and LC3 codec, offering lower latency and better sound quality. When buying new earbuds, it's recommended to prioritize products that support LE Audio.

Application scenarios:

- **Audio transmission**: Wireless earbuds, car audio. Bluetooth 5.2's LE Audio brings a qualitative leap in both sound quality and latency.
- **Data transmission**: Close-range file and photo transfer. While slower than Wi-Fi Direct, it's simpler.
- **Device pairing**: Smart watches, fitness bands, keyboards and mice, and game controllers.
- **Location services**: Bluetooth beacons for indoor navigation, item tracking (like Apple AirTag).
- **Smart home**: Bluetooth Mesh networks control light bulbs, smart locks, and more.

IV. Infrared Remote Control: Phone Becomes a Universal Remote

Infrared remote control is one of the oldest wireless control technologies, but it's still practical on phones. Some Android phones (such as certain Xiaomi and Huawei models) have built-in infrared emitters that can remote-control TVs, air conditioners, set-top boxes, and other appliances.

Working principle:

An infrared remote works by emitting infrared light pulses at a specific frequency (usually 38kHz) through an infrared LED, and the infrared receiver in the appliance decodes these pulses to carry out the corresponding actions. Different brands use different encoding protocols —common ones include NEC, RC5, RC6, and SONY. The infrared app on your phone ships with a code library for each brand, so picking the brand and model matches the right code.

Limitations:

- **Directionality**: Infrared travels in straight lines and must be aimed at the device— cannot pass through walls.

- **Short range**: Effective distance is generally 5-8 meters.
- **No Apple support**: iPhone has no infrared emitter and cannot use this feature.

V. NFC: The Magic of a Tap

NFC (Near Field Communication) is a very short-range (within 10 cm) high-frequency wireless communication technology, operating at 13.56MHz with a maximum transmission rate of 424Kbps.

Communication principles and modes:

NFC evolved from RFID technology and supports three working modes:

- **Read/Write mode**: The phone reads information from NFC tags, such as triggering actions by reading NFC stickers.
- **Peer-to-Peer mode**: Two NFC devices transmit data to each other, such as Android Beam (discontinued).
- **Card Emulation mode**: The phone emulates a smart card—this is the most commonly used mode. Transit cards, access cards, and bank cards all use this mode.

Security mechanisms:

- **Ultra-short distance**: The communication distance within 10 cm is itself a physical security guarantee—remote eavesdropping is virtually impossible.
- **Secure chip**: The phone has a built-in independent Secure Element (SE) or uses HCE (Host Card Emulation). Payment keys are stored in the secure area and cannot be extracted even if the phone is rooted.
- **Fingerprint/Face verification**: Payments require biometric confirmation—even if the phone is lost, it cannot be used for fraudulent transactions.
- **Tokenization**: Apple Pay, Huawei Pay, etc., do not transmit real card numbers but use one-time tokens—even if intercepted, they cannot be reused.

Typical applications:

- **Transit cards/Access cards**: Write physical card information into the phone—just tap your phone.
- **Mobile payments**: Apple Pay, Huawei Pay, etc.—bind your bank card to your phone and pay with a tap.

- **Device pairing**: Tap to quickly pair Bluetooth earbuds and speakers.
- **Identity verification**: NFC ID card reading, electronic driver's license verification.

VI. Personal Hotspot: Turn Your Phone into a Wi-Fi Router

When you have plenty of data, you can turn on the hotspot for tablets and computers.

- **How to enable**: Settings > Personal Hotspot/Tethering, turn it on.
- **Things to note**: It drains battery and data—remember to set a strong password and limit the number of connected devices.
- **USB tethering**: Connect to a computer with a cable—more stable and faster than wireless hotspot.

Hands-On: Set Up and Manage a Hotspot

1. Open [Settings], find "Personal Hotspot" or "Tethering," turn it on, and set the name and password.
2. Use another device (like a computer) to search for and connect to this Wi-Fi.
3. On your phone, see if you can manage connected devices or kick off unrecognized ones.

Chapter 4: Installing and Managing Apps

Apps make your phone capable of anything, but they need to be managed. Just like you'd assign a new employee a desk and permissions, installing apps requires a standardized process.

I. Official App Stores: The Safest "App Supermarket"

- **Android**: Google Play (outside China) or each brand's built-in store (such as Huawei AppGallery or Xiaomi GetApps).
- **iOS**: Only the App Store.
- **HarmonyOS**: Huawei AppGallery.

Core security value: reviewed, guaranteed updates, payment security. Action guideline:**prioritize downloading only from official stores!**

II. Pure/Protection Mode: Turn On Your Phone's "Antivirus Wall"

Mainstream domestic brands (Huawei, Xiaomi, OPPO, vivo, Honor) all have a built-in**Pure Mode**or**Protection Mode**—a system-level security switch.**Strongly recommended for all users to enable**.

What exactly is it?

The core function of Pure Mode (also called "Protection Mode") is to**block third-party app installations**: when enabled, the system will**only allow app installations from the official app store**. Installation packages downloaded from browsers, QR codes, chat apps, and other channels will be directly intercepted, eliminating malware at the source.

Who it's for

- **Elderly users**: Can't tell "official" from "fake" ads—Pure Mode is the most reliable safeguard.
- **Users insensitive to app permissions**: Don't need to install niche apps, beta versions, or foreign-region games.
- **All users who want to improve device security**: Even for power users, having it on is better than off—if you have a temporary need, just turn it off briefly.

How to enable (taking Huawei/Xiaomi as examples)

- **Huawei/HarmonyOS**: [Settings] > [System & Updates] >**[Pure Mode]**, select "Enable."
- **Xiaomi**: [Settings] > [Privacy Protection] >**[Protect Privacy]**, or [Settings] > [App Settings] > [App Installation], turn off "Install unknown apps."
- **OPPO/vivo**: [Settings] > [Security] >**[Install Apps from External Sources]**, turn off each authorization one by one.

App Store Security Detection Mechanisms

App stores aren't just for show—there's a complete security pipeline behind them:

- **Virus scanning**: Each app is jointly scanned by multiple antivirus engines before listing, to identify known trojan and virus signatures.
- **Permission review**: The store reviews whether the permissions an app requests are "reasonable"—a flashlight app asking for contacts? Directly rejected or delisted.
- **Developer certification**: Listing requires enterprise qualification, personal ID, or credit card verification. If problems arise, someone can be held accountable—not a "black app."
- **Continuous monitoring**: Listed apps are also continuously rescanned, and newly discovered malicious behavior results in forced delisting.

III. Third-Party Sources and "Electronic Parasites": Extremely High Risk!

Downloading apps from webpages or unknown links may bundle viruses, steal privacy, or commit payment fraud. Here we specifically mention a new concept: **"electronic parasites."**

.

What are "electronic parasites" ?

This refers to**.**—you think you “saved money,” but actually “got infected.” Common forms:

- ”Cracked,” ”free,” ”ad-free” versions of paid software;
- ”Like-farming,” ”red packet grabbing,” ”game cheats” and other gray-area tools;
- So-called ”internal versions” of apps recommended by ”small ads” in group chats.

These programs appear usable, but behind the scenes they may be**stealing contacts, reading SMS, eavesdropping on the microphone, mining in the background**. Pure Mode is the**first line of defense**against them—simply not letting you install them.

Second line of defense recommendations:

- **Regular full virus scans**: Use the phone’s built-in security center (like Huawei Phone Manager, Xiaomi Security Center) to scan once a month.
- **Cautiously grant sensitive permissions**: Contacts, SMS, location, camera, microphone—these five ”high-risk permissions” must be strictly reviewed.
- **Watch for battery and data anomalies**: Phone suddenly heating up, battery draining fast, data running at night—most likely a ”parasite” hiding in the background.

If you absolutely must install a third-party app:

1. Ensure the source is absolutely trustworthy.
2. Carefully review permission requests before installing.
3. Scan with a security app afterward.

Ultimate advice: In system settings, turn off the “Install unknown apps” permission.

IV. Install, Update, Uninstall

- **Install**: Tap ”Get” or ”Install” in the store. After installation, the first launch will ask for permissions.
- **Update**:**Must update!**Mainly to fix security vulnerabilities. Recommend enabling auto-update.
- **Uninstall**: Long-press the icon and drag to the uninstall area. For a more thorough clean, go to [Settings] > [App Management] to find and uninstall.

V. Permission Management: You Are the App's "Boss"

Apps requesting permissions (like contacts, location) is sometimes reasonable (maps need location), sometimes not (flashlight needs contacts).

- **Permission options**: "Always allow," "Only while using the app," "Deny," "Ask every time."
- **Management principles**:
  1. Minimization: Don't grant unless necessary.
  2. Post-audit: Regularly check permission settings to see who's "overstepping."
  3. Context-based: Like ride-hailing apps, only grant location permission while in use.

Key Permissions to Watch

- Contacts/SMS: May be used to send spam messages.
- Real-time location: Your movements may be tracked.
- Camera/Microphone: May be used for secret photography/recording.
- File access: May read your private files.

VI. Components of an App Installation Package

Have you ever wondered what's inside that installation package you download from the app store? Understanding the components of an installation package helps you better understand how apps work, and when problems arise, you'll know where to look.

Android APK File Structure

An APK (Android Package Kit) is essentially a ZIP archive. After extraction, you can see the following core directories and files:

- **AndroidManifest.xml**: The app's "ID card" and "instruction manual." Records core information like app package name, version number, required permissions, supported system versions, entry Activity, etc. The system reads this file first when installing the app.
- **classes.dex**: The app's core code. Java/Kotlin source code is compiled into Dalvik bytecode and stored in this file. An app may have multiple dex files (like classes2.dex, classes3.dex).
- **resources.arsc**: The compiled resource index table. Records the IDs and corresponding relationships of all resources.

- **res/ directory**: Resource folder. Contains images, layout files, strings, color definitions, etc. Images for different resolutions are placed in different drawable folders (like drawable-hdpi, drawable-xhdpi).
- **assets/ directory**: Raw resource folder. Stores resources that don't need compilation, such as web files, fonts, audio, etc. Apps can read them via AssetManager at runtime.
- **lib/ directory**: Native library folder. Stores compiled .so files (C/C++ compilation output), categorized by CPU architecture (like arm64-v8a, armeabi-v7a). Games and apps requiring high-performance computation usually have this directory.
- **META-INF/ directory**: Signature information folder. Contains MANIFEST.MF, CERT.SF, CERT.RSA files, used to verify the APK's integrity and signature.

iOS IPA File Structure

IPA (iOS App Store Package) is also a compressed archive, with a slightly different structure:

- **Payload/ directory**: Where the core content lives. Inside is a .app folder containing all the app's content.
- **Info.plist**: The app's configuration file, similar to Android's AndroidManifest.xml. Records app ID, version, permission requirements, supported devices, etc.
- **Executable file**: The main program with the same name as the .app folder, a compiled Mach-O format binary.
- **Frameworks/ directory**: Dynamic libraries and frameworks the app depends on.
- **Resource files**: Images, audio, video, and other resources, usually stored inside the .app folder.
- **_CodeSignature/ directory**: Code signature information, used to verify the app's integrity and origin.

Signature Verification Mechanism

Why are apps downloaded from official stores safer? Signature verification is key.

- **Android signing**: Developers sign APKs with private keys, and the system verifies with public keys during installation. If an APK is tampered with (e.g., implanted with a virus), the signature becomes invalid and the system refuses to install. Google Play also requires App Signing, where Google holds the signing key, further protecting app security.

- **iOS signing**: Apple's review is stricter. Every IPA must be signed with a certificate issued by Apple, otherwise it cannot be installed. This is why iOS can only install apps from the App Store or enterprise certificates. Apps signed with enterprise certificates have limited validity and Apple can revoke them at any time.
- **Role of signatures**:
  1. Verify origin: Confirm the app indeed comes from the claimed developer.
  2. Verify integrity: Ensure the app wasn't tampered with during transmission.
  3. Prevent impersonation: A malicious app cannot disguise itself as another app.

Installation Package Size and Optimization

You may notice that the app size shown in the app store differs from the space it occupies after installation. This is because:

- **Download size**: The compressed APK/IPA size.
- **Installation size**: The extracted size, usually much larger than the download size.
- **Runtime footprint**: Apps also generate cache and data files at runtime, occupying more space.

Modern app optimization methods:

- **App Bundle (Android)**: Google Play's dynamic delivery technology. Users only download resources suited to their device when downloading (e.g., only images for the corresponding screen resolution), greatly reducing download size.
- **App Thinning (iOS)**: Similar technology—Apple automatically trims unneeded resources based on the device.
- **On-demand download**: Large apps like games can download the core package first, then download resource packs later.

Trivia: Why are some app installation packages so large?

Game apps are usually the largest because they contain lots of high-definition textures, 3D models, and audio files. Social apps are also quite large because they need to support various functional modules. Tool apps are usually the smallest. If you find a tool app that's unusually large, be alert to whether it bundles unnecessary features.

Hands-On: Audit 3 Apps' Permissions

Pick 3 commonly used apps (e.g., social, shopping, tool), go to the app permission management page in [Settings], review each item, and turn off or change unnecessary sensitive permissions to "Only while using the app."

Chapter 5: Make Your Phone Understand You Better

A few minutes of setup brings a huge boost to your experience. These settings are like "tuning" your phone to better match your usage habits.

I. Sound and Vibration

Set a ringtone and notification sound you like. Use silent mode (for meetings) and Do Not Disturb mode (for sleeping) flexibly—the world instantly quiets down.

- **Ringtone**: Pick one you can hear and isn't embarrassing.
- **Vibration mode**: Vibration alerts when on silent.
- **Do Not Disturb mode**: Can set exceptions, like allowing calls from contacts.
- **Volume separation**: Media volume, ringtone volume, and alarm volume can be adjusted separately.

II. Display and Brightness

- **Auto brightness**: Lets the phone adjust based on ambient light—worry-free.
- **Eye comfort/Night Shift mode**: Filters blue light, making phone use more comfortable at night.
- **Dark mode**: Saves power (especially effective on OLED screens) and isn't harsh on the eyes at night. Many apps also support dark mode.
- **Screen refresh rate**: Adjustable on some Android phones—higher (like 120Hz) means smoother scrolling.
- **Auto-lock**: Set to 15 or 30 seconds—both secure and power-saving.
- **Font size**: Think the text is too small? Make it bigger.

III. Wallpaper, Themes, and Home Screen

Change to a nice wallpaper (static or dynamic)—even your mood changes. Android brand phones often have theme stores that can change the entire set of icons and style.

Home screen widgets: Put weather, schedule, and music controls on the home screen for one-step access. Both iOS and Android support this—long-press an empty area on the home screen to add.

IV. Efficiency Settings: Make Your Phone Your "External Brain"

These are "once set, never go back" features—each one can significantly boost your daily efficiency.

Focus / Do Not Disturb Mode: Reclaim Attention Stolen by Notifications

- **Core idea**: Not "look at your phone less," but "focus when you should focus, connect when you should connect."
- **Usage suggestions**:
  • Work mode: Only allow "starred" contacts and essential apps through;
  • Sleep mode: Auto-enable, only allow alarms;
  • Fitness / Reading / Meetings: customize based on the current activity.
- **Advanced usage**: Use automation + location / time triggers to switch modes automatically. For example: iOS Shortcuts can set "arrive at company auto-enter work mode."

Notification Management: Stop Red Badges from "Relentlessly Pestering"

- **Turn off marketing notifications**: 90% of push notifications you don't actually need.
- **Sort by importance**: Pin interpersonal communications (phone, SMS, instant messaging) to the top; news, shopping, and social platform notifications can be set to "silent delivery."
- **Group notifications**: Fold similar notifications together to avoid being overwhelmed by dozens of messages with one unlock.

Quick Gestures: Stay One Step Ahead

- **Screenshot**: Power + Volume Down (iOS full-screen) / Power + Volume Down (Android universal).

- **Long screenshot / Scroll screenshot**: After a regular screenshot, tap the "Long Screenshot" option.
- **One-tap scan**: Add a "Scan" icon to Control Center / Quick Settings.
- **Back Tap (iPhone)**: In [Settings] > [Accessibility] > [Touch] > [Back Tap], bind actions like "Screenshot" or "Lock Screen."
- **Voice wake**: Siri / Xiaoai / Xiaoyi / Xiaobu—just call out to check weather or set an alarm.

Input Method: The Most Underestimated Efficiency Tool

- **Choose the right input method**:
  • Sogou / Baidu: Large lexicon, fast trending word updates;
  • iFlytek: Highest voice input recognition rate, a note-taking lifesaver in meetings;
  • Gboard (Google): Clean and simple, strong cross-platform sync, supports gesture typing;
  • iOS built-in: Best privacy, deeply integrated with the system.
- **Essential toggles**: Smart correction, word prediction, cloud lexicon sync, swipe up for numbers and symbols.
- **Privacy tip**: Use "full-sentence prediction" or "personalized recommendations" with caution—they may upload your input content.

Home Screen and Multitasking: Smooth Out Your Workflow

- **App split screen / Picture-in-Picture**: Reply to messages while watching video, take notes while browsing.
- **Small window / Floating window**: WeChat / TikTok support "small window mode"—reply and dismiss.
- **Pin frequently used apps**: Only put the 4 most frequent apps in the Dock (bottom of screen) to reduce search time.

"First Day of Work" Checklist

After getting a new phone or a major version update, spend 10 minutes going through these settings:
(1) Lock screen password + biometrics (2) Notification grouping + turn off marketing pushes (3) Font size / refresh rate / dark mode
(4) Default input method / browser / email (5) Emergency contacts / medical ID (6) Backup settings

V. Accessibility: The Power of Tech for Good

Don't be intimidated by the name "accessibility"—these features are for everyone. They were originally designed for people with disabilities, but they're also useful for ordinary people—temporarily injured hands, holding a baby, broken earbuds on the subway, or needing reading glasses for small text... they can all save the day. Ultimately, accessible design embodies the philosophy of**tech for good**: letting everyone equally enjoy digital life.

"Eyes" for Visually Impaired Users—Complete Screen Reader Guide

Screen readers are the core tool for visually impaired users to use phones. It reads**all visible content on the screen aloud**, and users operate the phone through gestures and voice. Below we introduce the usage of iOS "VoiceOver"and Android "TalkBack"respectively.

iOS VoiceOver

How to enable: [Settings] > [Accessibility] > [VoiceOver], turn on the switch. You can also triple-click the side button (or Home button) to quickly toggle.
Basic gestures:

1. One-finger swipe: Browse screen content element by element;
2. One-finger double-tap: Activate element;
3. :
4. Three-finger swipe: Switch pages;
5. One-finger triple-tap: Zoom or select (different from regular triple-tap).

Personalization: [VoiceOver] > [Speech], you can adjust**speaking rate**,**pitch**, and switch between different voice packs. iOS has built-in "Enhanced Speech" and "Speech Controller"—the former reduces breathing sounds and pauses for a more natural listening experience.

Android TalkBack

How to enable: [Settings] > [Accessibility] > [TalkBack], turn on the switch. Or long-press both volume keys for 3 seconds to quickly enable (supported by some brands).
Basic gestures:

1. One-finger left/right swipe: Switch focus element;
2. One-finger double-tap: Activate element;
3. Two-finger up/down/left/right swipe: Scroll;
4. Three-finger single-tap: Enter reading mode (continuous reading);

5. Four-finger single-tap: Pause/resume reading.

Personalization: [TalkBack] > [Text-to-Speech Settings], you can adjust**speech rate**,**pitch**, and switch**the speech engine**(we recommend installing the Google Text-to-Speech engine for more natural reading).

Make Small Text Big, Make Blurry Clear—Visual Adjustments

Zoom Gestures

- **iOS**: [Settings] > [Accessibility] > [Zoom], after enabling,**three-finger tap the screen**to zoom in. Full-screen zoom is the default, with optional window zoom (a partial magnification pane).**Pinch with two fingers to adjust magnification (1.5x to 15x)**, three-finger double-tap to lock the current zoom ratio.
- **Android**: [Settings] > [Accessibility] > [Magnification], three-finger tap to zoom, three-finger tap again to exit; or directly**three-finger triple-tap the screen**to enable temporary magnification (up to 8x).

High Contrast Text / Bold Text

- **iOS**: [Settings] > [Accessibility] > [Display & Text Size], turn on "Bold Text," "Increase Contrast," "Reduce Transparency," etc. "Smart Invert" can keep text black on light backgrounds, avoiding white-on-white.
- **Android**: [Settings] > [Display] > [Font Size], with four levels: "Small," "Default," "Large," and "Extra Large." Huawei/Xiaomi additionally provide "Bold" and "High Contrast Text" switches.

Color Filters (Color Blind-Friendly)

- **iOS**: [Settings] > [Accessibility] > [Display & Text Size] > [Color Filters], provides**four color blind modes: red/green/blue/yellow**, plus "Grayscale" and "Invert Colors" options. Take the built-in color vision test once before choosing a filter.
- **Android**: [Settings] > [Accessibility] > [Color Correction], includes**three color blind modes**plus an "Invert Colors" switch.

AI Image Description: Let AI Be Your "Eyes"

When a screen reader encounters an image, the traditional approach is to skip it or just read "image." But now AI can**describe image content in a sentence**, which is very helpful for recognizing CAPTCHAs, reading images, and identifying menus.

AI Image Description Support by System

- **iOS 15+**: Enable "Image Descriptions" in [Settings] > [Accessibility] > [VoiceOver] > [Speech Descriptions]. The system calls a**local AI model**to automatically generate text descriptions of images (requires iPhone 12 or later).
- **Newer TalkBack app**: Google TalkBack integrates**image Q& A**in newer versions—you can ask "What's in this image?" and AI answers in real time (not strictly dependent on Android system version—just update the TalkBack app to experience it).
- **Third-party apps**: "Seeing AI" (iOS), "Lookout" (Android), "Xiaoyi Bangbang" (Huawei) are all excellent AI visual assistants that can recognize objects, read documents, and identify facial expressions.

Use Cases

Try asking about a street photo, **"How many people are in this image? What are they doing?"**, or open a food delivery app and let AI read out the dish names in the menu image—**this is true "tech making life more equal"**.

"Ears" for Hearing-Impaired Users—Complete Live Captions Guide

Live Captions is a**system-level speech-to-text**feature—especially useful for hearing-impaired users in noisy environments, libraries, or when communicating in a foreign language.

Enabling and Optimizing Live Captions

iOS: [Settings] > [Accessibility] > [Live Captions], turn on the switch. The system detects all device audio (calls, videos, podcasts, FaceTime, voice memos) and automatically displays captions at the top of the screen.
Android 12+: Press the**volume keys**tap the "Live Caption" icon in the popup; or enable in [Settings] > [Accessibility] > [Live Caption].
Accuracy optimization tips:

1. Use in a**quiet environment**as much as possible, away from air conditioners and machinery noise;
2. The speaker**articulates clearly and speaks at a moderate pace**for the highest recognition accuracy;
3. Multi-person conversations reduce accuracy—use "show only one person" mode;
4. Supports mainstream languages like Mandarin Chinese, Cantonese, and English;**minority languages have lower recognition rates**;
5. Background music or voice interference during speech will cause intermittent recognition.

Dual-Channel Call Scenarios: Speech-to-Text + Text-to-Speech

When users**can neither hear nor speak**(such as sudden voice loss, laryngeal surgery recovery, severe stuttering), the phone can still act as a "spokesperson."

Real-Time Call Transcription (TTY / RTT)

iOS: [Settings] > [Accessibility] > [RTT/TTY], after enabling, a keyboard appears on screen during calls—the other party's speech is transcribed to text in real time, and**your typed text is also read aloud**to them.
Android: [Settings] > [Accessibility] > [Live Transcribe] + [Sound Notifications]. "Live Transcribe" displays the other party's speech as text in real time; "Sound Notifications" converts your input to speech when voice isn't convenient.
Compatibility:

- **RTT (Real-Time Text)**: Both parties need support; text appears as you type with no delay; mainstream carriers now support it;
- **TTY (Teletypewriter)**: Requires a dedicated TTY device; it is gradually being replaced by RTT;
- WeChat/QQ**voice-to-text feature**is also an alternative: long-press a voice message "Convert to Text."

"Small but Practical" Features for Everyone

- **Back Tap**(iOS 14+): [Settings] > [Accessibility] > [Touch] > [Back Tap], can bind**double-tap/triple-tap**to trigger screenshot, lock screen, control center, etc. Especially handy when carrying packages with both hands.
- **Assistive Touch / Floating Ball**: iOS [Accessibility] > [Touch] > [Assistive Touch], Android [Accessibility] > [Floating Navigation]. Virtual buttons replace gestures or physical buttons—lifesaver when buttons are unresponsive or for one-handed use.
- **Switch Control**: Control the phone through external switches, single clicks, or head movements—**designed for users with physical disabilities**. iOS [Accessibility] > [Switch Control], Android [Accessibility] > [Switch Access].
- **Sound Recognition**(iOS): [Accessibility] > [Sound Recognition], after enabling, the phone "listens" for doorbells, baby cries, fire alarms, sirens, etc., and**upon recognition, vibrates + sends a screen notification**—very valuable for hearing-impaired users and during sleep.
- **LED Flash Alerts**: iOS [Accessibility] > [Audio/Visual] > [LED Flash for Alerts], Android [Settings] > [Notifications] > [Flash Alerts]. The flash blinks for incoming calls and notifications—**sensible even without sound in noisy environments**.

- **Mono Audio**: [Settings] > [Accessibility] > [Audio] > [Mono Audio], merges left and right channels. Users with single-ear hearing loss won't lose half the audio information.

Apple's Accessibility Design Philosophy

Apple invests heavily in accessibility, updating and adding assistive features every year. Apple' s website has a dedicated accessibility page detailing how to use each feature. iOS accessibility is typically more complete and more polished than Android' s, which is a major reason many users with disabilities choose Apple.

Spend 30 Seconds to Experience

Find some free time and do the following experiences:

1. Enable**VoiceOver / TalkBack**and try browsing WeChat with three-finger swipes without using your eyes;
2. Turn on**Live Captions**and speak to your phone to check recognition accuracy;
3. Use**three-finger tap**to zoom in on text to 200%—experience the daily life of visually impaired users;
4. Turn off all the above features and reflect on "if I permanently needed these features, how would I want them designed."

Chapter 6: Guarding Your Digital Life

Phone security = privacy security + financial security. In this era where data is an asset, protecting your phone means protecting yourself.

I. Lock Screen: The First Physical Line of Defense

The lock screen is your phone's first line of defense, and also the most easily overlooked.

Security level ranking:

1. **Password/PIN**: Highest security, the ultimate safeguard (relied upon when fingerprint/face fails).
2. **Fingerprint recognition**: The perfect balance of security and convenience.
3. **Face recognition**: Most convenient, but security depends on technology (3D structured light is secure; 2D photos can be fooled).
4. **Pattern unlock**: Convenient but leaves traces that can be guessed.

Setting a strong lock screen:

- Be sure to set a complex password (don't use birthdays, 123456).
- Enable biometrics.
- Set lock screen time to "Immediately" or the shortest option.

II. "Find My Phone" : The Ultimate Anti-Theft Tool

Must enable!It can:

- **Locate**: Find where you left your phone on a map.

- **Play sound**: Make your phone ring—good for finding it at home.
- **Lost Mode**: Remotely lock the phone and display a contact number.
- **Remote erase**: The last resort—wipes all data (but after erasing, it can no longer be located).

Go to [Settings]—Apple users look for "Find My iPhone," Android users look for "Find My Device" or similar brand services, and make sure it's enabled (green).

III. System Updates: Patch Holes, Plug Vulnerabilities

The primary purpose of system updates is to fix security vulnerabilities, just like patching holes in a house. So when you see a system update prompt, upgrade as soon as possible.

Of course, major version updates (like iOS 17 to iOS 18) may have compatibility issues—you can wait a few days before upgrading.

IV. Recognizing and Preventing Telecom Fraud

In today's era of rapid AI development, the traditional "seeing is believing" cognitive model has completely failed. Fraudsters are using AI technologies like Deepfake to achieve a comprehensive upgrade from "scatter-shot" fraud to "precision hunting" crime.

The Most Common Fraud Patterns in the AI Era

(1) AI Face-Swap and Voice Cloning Fraud

Criminals illegally collect high-definition photos published by targets on social media or just a few seconds of voice samples, and use AI deepfake technology to clone "friend/family videos" or "voices" realistic enough to deceive. Common scripts include:**sudden car accidents, emergency loans, kidnapping**, etc. This type of fraud exploits emotional bonds like family and friendship, making it**extremely deceptive**.

(2) Impersonating Authority / Customer Service Fraud

AI technology can precisely simulate the voices and tones of official customer service and law enforcement personnel, and even synthesize highly realistic fake transfer screenshots, electronic documents, arrest warrants, and other files. By**creating panic or authority pressure**.

(3) AI Investment and Wealth Management Fraud

Through AI algorithms, fraudsters forge "guaranteed profit" fake return screenshots, virtual quantitative stock-picking software interfaces, or "master-level" investment analysis reports. Using high returns as bait, they sell expensive investment services or induce victims to transfer funds into trading accounts they control,**ultimately absconding with the money**.

How to Break Through— "Magic Mirror" Defense Thinking

AI can clone voices and faces, but it's not invincible—there are several vulnerabilities to exploit:

- **Action verification method.easily shows flaws like screen stuttering, facial distortion, or edge bleeding**when handling complex dynamic occlusion, rapid motion scenes, and lighting changes.

- **Secret signal verification method**: Establish and use preset private verification questions, such as "What did you order when we last had dinner together?" or "What was the first gift I gave you?"—details only both parties know. If the other party hesitates noticeably or answers irrelevantly, you should**immediately terminate the call and activate safety protocols**.

- **Secondary verification method**: Strictly follow the "**Four Don'ts and One Verify.call back using the known original number or confirm in person**. When making transfers, it's recommended to choose the "24-hour arrival" mode to allow ample time for potential fraud identification and fund recovery.

Traditional Common Patterns (Still Need Vigilance):

- **Lottery/brushing rebates**: Lure with small gains first, then scam big money.

- **Romance scams**: "Pig-butchering" scams—build a relationship then steal money.

- **Impersonating acquaintances**: Borrow money after account theft (even harder to distinguish in the AI era).

Built-in Phone Protections:

- Harassment call blocking.

- Verification code theft prevention.

- SMS link security detection.

Anti-fraud mantra:

Don't readily believe, don't disclose, don't click, don't transfer, verify first when in doubt!When encountering an "emergency," stay calm for 5 minutes—almost all scams reveal their flaws in that moment.

V. Resist "Doxing" and Social Engineering Attacks: Cut the Privacy Leak Chain

The core principle of "doxing" (social engineering attacks) is to**systematically collect, analyze, and piece together personal digital fragments to build a complete personal profile**. The key to defense is drawing clear lines for personal privacy and establishing a multi-layered protection system.

Photo Desensitization

Basic EXIF removal methods are covered in Chapter 2 "Image and File System Management." Before sharing photos on social media, further steps are needed:**completely delete the photo's metadata (EXIF information)**, including shooting time, device model, GPS location coordinates, and other sensitive data.**house numbers, license plates, ticket barcodes, ID numbers**and other key information—strictly redact or blur them to avoid unintentionally revealing your whereabouts.

- iOS: [Settings] > [Privacy & Security] > [Camera] can turn off "Precise Location"; selecting "Current photo only" when sharing strips metadata.
- Android: Some album apps (like Google Photos) prompt whether to keep location info when sharing.
- General tools: ExifTool, Metapho, etc., can view/edit EXIF.

App Permission Convergence

Basic permission management methods are detailed in Chapter 4 "Permission Management." For "anti-doxing" purposes, you should also periodically review and disable unnecessary permissions for e-commerce and utility apps:**microphone, contacts, location**, to prevent apps from "eavesdropping" on daily conversations in the background or illegally collecting personal relationship networks. Uninstall apps that are idle for a long time but still hold high permissions, reducing privacy leak risk points at the source.

- When you see a "Contacts permission" request, ask yourself: Does this app really need to know all my contacts?
- Haven't used an app in three months? Uninstall it without hesitation. Permissions should get tighter over time, not looser.

Smart Device Security Hardening

IoT devices like smart cameras, smart door locks, and smart speakers must have their**default login passwords changed immediately**upon first use. Set high-strength independent passwords containing uppercase and lowercase letters, numbers, and special symbols.

Keep rarely used remote access features off, regularly check for device firmware updates, and promptly patch security vulnerabilities to**prevent hackers from intruding and snooping on your home's private space**.

Real Case Warning

In 2022, police in a certain area reported that hackers cracked the default passwords of a brand of home cameras (many users never changed them), stealing bedroom footage from thousands of households and selling it online.**Not changing the default password = leaving your front door wide open**.

VI. Privacy Protection: Your Data Is Your Property

1. The "Minimization Principle" for App Permissions:

Location

- **Location**: Use "Allow all the time" for ride-hailing/map apps; "Only while using the app" for everything else.
- **: Only grant when necessary (like adding friends on WeChat); otherwise deny.**Album / All Photos
- **: Since iOS 14+, you can authorize "Selected Photos"; Android 13+ also supports granular authorization.**Microphone / Camera
- **Microphone / Camera**: Keep them off by default and enable when needed. If you suspect covert recording, swipe open the Control Center and check for the "green dot / orange dot".
- **Background tracking**: Since iOS 14.5+, apps must ask before using IDFA; Android can turn off "Ad ID."

Privacy Management in Practice:

- **Privacy Report (iOS)**: [Settings] > [Privacy & Security] > [App Privacy Report], see which apps are secretly accessing your location, camera, microphone.
- **Permission Usage History (Android)**: [Settings] > [Privacy] > [Permission Usage History], see who accessed sensitive permissions and when.
- **Limit ad tracking**: Turn off "Ad ID" or "Limit Ad Tracking" in system settings to reduce "targeted ad harassment."

VII. "First Aid" Checklist After Loss / Theft

Don't panic if your phone is lost—follow these steps in order:

1. **Immediately freeze the SIM card**: Contact the carrier to report the loss, preventing it from being used to receive verification codes.
2. **Remote lock**: Enable "Lost Mode" through cloud services (iCloud / brand cloud).
3. **Change important passwords**: Immediately change passwords for all important accounts that were logged in on the phone (payments, banks, email).
4. **Report to police**: File a report at the police station and record the IMEI number. This also helps prove identity if the phone is recovered.
5. **Notify family and friends**: Prevent scammers from borrowing money in your name.
6. **(Last resort) Remote erase**: Only do this when recovery is hopeless—after erasing, the location feature stops working.

VIII. SIM Card and eSIM Security

- **Set a SIM card PIN**: Once enabled, you need to enter the PIN every time you restart the phone or swap cards, preventing "pull the card, swap the phone" bypass of the lock screen.
- **eSIM advantages**: No physical card to be pulled out, higher security; also supports multi-number switching, very convenient for travel.
- **Secondary card swap risk**: Beware of the "carrier secondary card swap" scam—scammers use your identity to reissue a SIM card, thereby taking over your phone number. Setting "card replacement requires the account holder to appear in person at a store with ID" is the most effective defense.

Things You Must Never Do

- Use weak passwords like birthdays / 123456 / qwerty;
- Send verification codes to anyone (even "customer service");
- Click unfamiliar links in text messages, especially those from overseas numbers starting with 00;
- Log into online banking or make payments on public Wi-Fi;
- Enable "screen sharing" or "remote control" for strangers.

Try Enabling and Simulating ”Remote Lock”

1. Make sure your ”Find My Phone” feature is enabled.
2. On a computer browser or another phone, log into the corresponding cloud service account (iCloud.com or brand cloud service website).
3. Find the ”Find Device” feature, try initiating a ”Play Sound” or simulating a ”Lock” command on your own phone to familiarize yourself with the entire process.

Chapter 7: Comprehensive Analysis of Cutting-edge Features

Supported models:

Apple

: iPhone 14 series and later support satellite emergency SOS. iPhone 16 series further supports satellite iMessage and SMS.

Supported models:

: Mate 50 series started supporting BeiDou satellite messages (text only); Mate 60 Pro series further supports Tiantong satellite calls (can make phone calls); Mate 70 series has further enhanced satellite communication capabilities.

Things to note:

- **Apple**: Starting from the iPhone 14 series, satellite emergency SOS is supported. The iPhone 17 series further supports sending iMessages and text messages via satellite, with expanded coverage in more regions.
- **Huawei**Use cases:
- **Outdoor adventure, mountaineering emergencies.**Maritime emergencies.
- **Natural disasters causing ground communication outages.**How to use:

Make sure you're in an open area with a view of the sky.

- Turn on your phone's satellite communication feature (usually in the emergency call interface).
- Follow the on-screen instructions to aim at the satellite.
- Send preset distress messages and your location.

Trivia

1. Satellite communication requires special hardware antennas in the phone—not all phones support it. Additionally, satellite communication usually requires a paid subscription service, but the emergency SOS feature is generally free.

2. II. Off-Grid Communication: A New Way of Close-Range Communication

3. "Off-grid communication" refers to technology that allows two phones to communicate directly with each other without mobile networks or Wi-Fi.

4. Technical principle:

Uses Bluetooth or dedicated radio frequencies to establish point-to-point connections over short distances (usually a few hundred meters to a few kilometers).

Application scenarios:

Outdoor activities

: Communication between teammates during hiking and camping.

Disaster relief

: When communication is disrupted after earthquakes and floods.

Smart home

- **: When crowds at concerts and sporting events cause poor signal.**Supported platforms:

- **Apple**: iOS 17's "NameDrop" and "Find Friends Offline" features.

- **Huawei**: The Celia Chat app supports off-grid calls.

Xiaomi

- **Apple**III. Crash Detection: A Lifesaver at Critical Moments

- **Huawei**Working principle:

- **Xiaomi**Pop up a countdown interface, asking if you're okay.

If you don't respond before the countdown ends, automatically dial emergency numbers.

Send your location to emergency contacts.

Supported models:

Apple

1. : iPhone 14 series and later.
2. Android
3. : Some high-end models have successively added support.

Things to note:

- **Apple**May trigger false alarms (like roller coasters, intense exercise)—just cancel in time.
- **Android**IV. SparkLink Technology: Next-Generation Wireless Connectivity

SparkLink is a new-generation wireless communication technology led by Huawei, aiming to replace Bluetooth and Wi-Fi in certain scenarios. It matches or even exceeds existing solutions on key metrics: air interface rate up to 900Mbps (far exceeding Bluetooth's 50Mbps), latency as low as 20 μ s (Bluetooth about 15ms), and a single network supports 4096 devices connected simultaneously (Bluetooth usually under 8).

- Technical features:
- Ultra-low latency
- : End-to-end latency of only 20 microseconds (0.02ms), two orders of magnitude lower than Bluetooth—gaming and music sync are virtually imperceptible.

High speed

: Single-link peak rate of 900Mbps, dozens of times that of Bluetooth audio, capable of transmitting lossless audio and even HD video.

Low power consumption

- **: Standby power consumption comparable to Bluetooth, but with higher transmission efficiency and lower energy consumption per unit of data.**Wide coverage + high concurrency

- **: Connection distance over 50% farther than Bluetooth, with a single network supporting thousands of devices online simultaneously.**Application scenarios:
- **Wireless earbuds**: Lower latency, perfect audio-video sync for gaming.
- **Smart watches**: More stable connections.

Smart home

- **: More devices connected simultaneously.**Automotive connectivity
- **: Smoother phone-to-car connections.**Supported devices:
- **Smart home**Technology Outlook
- **SparkLink technology represents the future direction of wireless communication. As more manufacturers join, we may see more devices supporting SparkLink, enjoying a better wireless experience.**Go Check: Does Your Phone Support These Features

Search for “satellite” in settings to see if there are satellite communication options.

Search for “crash detection” or “emergency SOS” to check if it’s enabled.

If your phone is Huawei, see if it supports SparkLink device connections.

Learn how to use these features—they may come in handy at critical moments.

II. Software (Chapters 8-14)

Chapter 8: Communication —Texts, Calls, and Social

Communication is the phone's original and core mission. Today it has grown far beyond simple calls and texts into a whole ecosystem of instant messaging, social networks, and email.

Advanced features of the Phone app

The Phone app on modern phones is way more than a dialer. It packs in call recording, spam blocking, smart caller ID, and other powerful tools.

Call recording (mind the legal fine print):

- What it's for: Save important calls for work notes, interviews, customer commitments, or evidence in a dispute.
- Legal note: In mainland China, a call recording is generally admissible as evidence only with the other party's consent, or when it protects your own lawful rights.
- One-sided vs. two-sided consent: Legally, a recording made "without the other party's consent" is not automatically invalid. Under relevant Supreme People's Court interpretations, it can still be submitted as evidence as long as the method of recording did not seriously infringe others' lawful rights, violate prohibitive legal rules, or go against public order and good morals. That said, some provincial courts apply stricter standards for "secretly recorded" audio in civil cases; criminal cases usually face even stricter legality review.
- Regional differences: Courts in places like Beijing, Shanghai, and Guangdong are relatively lenient about one-party recordings as civil evidence. Recordings involving trade secrets or personal privacy need extra care anywhere —they may be excluded for infringing privacy rights.
- How to do it: Android usually shows a clear "Record" button on the call screen. iOS has no built-in call recording, so you'll need a third-party app or a Bluetooth headset with a recording feature.

Hard rules for using recordings

- Never use recordings for extortion, spreading them around, or publicly insulting others.
- Do not casually share content that involves third-party privacy (for example, accidentally catching someone else's conversation).
- In important situations, say out loud at the start of the call that "this call will be recorded." That keeps things compliant and makes the recording stronger as evidence.

A closer look at spam-call blocking:

- How it works: The Phone app checks a huge online number database and shows "spam call," "scam call," or a company name when the phone rings.
- How to turn it on: On Android, open the Phone app > Settings > Spam blocking. On iOS, go to Settings > Phone > Call Blocking & Identification. For brand-by-brand switch paths and recommended options, see the comparison table in Chapter 13, "Useful System Tools."
- Main labeling platforms: In China, number labeling is mainly handled by three data providers —Tencent Mobile Manager, 360 Mobile Security, and Dianhuabang (Tapir). Carrier features like "caller name" display and "spam blocking" often pull from the same data. Phone makers (Huawei, Xiaomi, OPPO, vivo) mostly partner with these providers for their label databases.
- What to do if you're mislabeled: If your legitimate work number is wrongly tagged as "food delivery," "courier," or "spam," you can appeal through these channels:
- Dianhuabang (电话邦): visit the number-appeal platform haoma.tk.cn and submit identity verification.
- 360: Submit via the "Number appeal" feature inside the 360 Mobile Security app.
- Tencent: Submit on the "Number appeal" page in the "Tencent Mobile Manager" mini program.
- The big three carriers: China Mobile users can text "KTFSR" to 10086 to enable spam protection. China Telecom users can follow the "Tianyi Anti-Harassment" official account. China Unicom users can set it up in "Unicom Mobile Manager."

Next-level spam blocking: Turn on your carrier's "high-frequency call blocking" (such as China Mobile's high-frequency call limit) to automatically block AI robocalls.

Common built-in phone blocking features:

Blacklist / whitelist: Manually block known spam numbers. Add contacts and courier/delivery numbers to the whitelist so they don't get caught by mistake.

SMS keyword filtering: Messages containing words like ”prize,” ”loan,” or ”click this link” are automatically sent to the blocked box.

iOS Silence Unknown Callers: Calls from numbers not in your contacts go quietly to voicemail without interrupting you. Handy if you get lots of spam —just watch out for important calls from new numbers.

Do Not Disturb integration: At night or during meetings, allow only contacts through, on a schedule. Your sleep quality will thank you.

How apps work together: Prefer the phone's built-in blocking first, then layer on the National Anti-Fraud Center app and carrier spam protection. For cleanup tools, lightweight options like Qingzhuo are fine, but one security/cleanup suite is enough —see Chapter 13.

What VoLTE/VoNR are and how to turn them on:

- VoLTE (Voice over LTE): Voice calls over the 4G network. No more falling back to 2G/3G mid-call, so you can talk and browse at the same time, with clearer audio and faster call setup.

- VoNR (Voice over NR): Voice over 5G —the next step after VoLTE. Lower latency, better sound, and support for HD video calls. You need a 5G phone, a 5G plan, and local 5G coverage.

- How to enable: On iOS, go to Settings > Cellular > Cellular Data Options > Voice & Data and turn on ”VoLTE.” On Android, find the ”VoLTE calls” switch under Settings > Mobile network > Mobile data > Advanced. Carriers usually turn it on by default. If your data drops during calls, VoLTE is probably off.

Quick tip

If your data suddenly slows or drops during a call, VoLTE/VoNR is probably off and you've fallen back to 2G/3G. Turn it on and you can “talk and scroll” at the same time.

SMS and iMessage/RCS

Traditional SMS isn't dead. It's still the reliable channel for verification codes, bank alerts, and official notices.

- iMessage: Messages sent over the internet between Apple devices, shown as blue bubbles. Supports high-quality photos and videos, read receipts, and more —free and end-to-end encrypted.

- RCS: A next-generation messaging protocol pushed by carriers worldwide and by Google. Features similar to iMessage or WeChat, but based on phone numbers.

- iMessage and RCS interoperability: Starting with iOS 18, Apple supports the RCS standard. That means iPhones and Android phones can exchange HD photos, read receipts, and typing indicators —goodbye ”green bubble” gap. Both carriers need to support RCS, and both phones need a new enough OS.

RCS rollout in China:

- China Mobile: An early RCS adopter. Some provinces already support ”5G Messages” with rich media and interactive cards. Needs a supported handset and activation at a store.
- China Telecom: Offers ”Telecom 5G Messages.” Some plans include it by default; older accounts may need to send a specific SMS command to activate.
- China Unicom: Rolling out 5G Messages in major cities, but device support is uneven. Many Android models still need ”Enhanced messages” or ”5G Messages” switched on manually in the Messages app.
- Real-world experience: RCS has rolled out more slowly than expected in China, mainly because WeChat dominates and device support is inconsistent. Day-to-day chat still runs on WeChat; RCS mostly handles ”official push” messages such as bank alerts and government notices.

Verification-code safety rules

- Never forward codes: Anyone who asks you to ”read out a code” or ”forward a text” may be a scammer —including people claiming to be ”support,” ”couriers,” or ”police, prosecutors, or courts.” Your verification code = your money.
- Phishing SMS: Be wary of links in texts about ”ETC expired,” ”health card frozen,” or ”lost package compensation.” Legitimate organizations never ask for card numbers or passwords via SMS links.
- Spotting the sender: Official bank texts usually come from short codes starting with 95 or from 1069 + a company signature. A ”bank notice” from a normal 11-digit mobile number is almost always a scam.
- Two-factor backups: Don’t casually change the phone number tied to important accounts. If you must, unlink all key accounts first —otherwise the new owner of your old number could reset your passwords via SMS codes.

WeChat/QQ: Managing China’s national super-apps

WeChat and QQ have outgrown simple messaging and become “super-apps” for social, payments, and information.

Backing up and migrating chat history:

1. Log into WeChat on your PC or Mac.
2. Click the menu at the bottom left > [Migrate & Backup] > [Backup & Restore].
3. Choose "Back up chat history to computer," then pick the chats on your phone.
4. Migrating to a new phone: On the old phone, open WeChat [Me] > [Settings] > [General] > [Chat Log Migration & Backup] > [Migrate] > [Migrate to Phone/Tablet], then scan the QR code.

Deep cleaning storage space:

- Standard path: WeChat [Me] > [Settings] > [General] > [Storage] —clear cache and clean chat history by time or file type.
- Clean inside a chat: Open a chat > "..." at the top right > [Chat history] > [Manage storage] —delete photos, videos, and files in that chat while keeping the text.
- Original photos/videos: HD originals from group chats take the most space. In [Manage storage], select "Originals" to delete them in bulk.
- Turn off auto-download: [Settings] > [General] > [Photos, Videos, Files & Calls] > turn off "Auto-download" so group files don't silently pile up on Wi-Fi.
- Mini program cache: In [Discover] > [Mini Programs], long-press unused ones to delete and free their local cache.
- Compressed chat backups: When backing up on desktop WeChat, check "Compress" to shrink multi-GB chat histories to 30–50% of their size.

Group management tips:

- Group announcements: Owners/admins post them via "..." at the top right > [Group Announcement]. Everyone gets a strong notification —great for important notices. New members see it automatically.
- Group to-dos: The owner can turn a pinned message into a "Group To-do." A red badge appears above members' heads and clears when they tap done. Perfect for signups and task collection.
- Group chains (sign-up lists): Type "#chain" in the input box to start one —great for ordering food or tallying signups. Results auto-summarize at the top without flooding the chat.
- Group live: "+" at the top right > [Group Live] for a live stream only this group can watch. Good for company training, family gatherings, and product beta tests.

- Mute a group: Swipe a group name left in the list > [Mute] to tuck unimportant groups into "Muted groups," notifying you only when @mentioned.
- Only owners/admins can @everyone: In groups of 500+, regular members can't @everyone —less noise for everyone.

Fine-grained Moments privacy:

- Group visibility: When posting, tap "Who can see" and choose "Partially visible" or "Hide from," using tag groups you set up earlier (e.g., "Colleagues," "Family," "Clients").
- How far back friends can look: [Me] > [Settings] > [Friends' Permissions] > [Moments] > [Allow friends to view Moments] —choose "Last 3 days / 1 month / 6 months / All." "6 months" balances privacy and social life nicely.
- Hide someone's Moments: Long-press a post > [Hide their Moments] or [Unfollow] — they won't know.
- Moments ads: [Me] > [Settings] > [About WeChat] > [Personal Info & Permissions] > [Personalized Ads Management] to turn off behavior-based recommendations.
- Strangers see ten posts: [Friends' Permissions] > [Moments] > [Allow strangers to view 10 Moments] —turn this off so strangers can't scroll your history.

Video call comparison

Each video call tool has its strengths. Picking the right one for the situation makes conversations smoother.

- WeChat video: Ubiquitous in China, easy for all ages. Great for everyday chats with family and friends. Supports up to 20 people (via the "Group Video" mini program). Downside: quality drops noticeably on poor networks.
- FaceTime: Apple-only, excellent quality, end-to-end encrypted. Since iOS 15, you can invite Android/Web users via a link, but the host must be on an Apple device. Great for cross-device family calls.
- Zoom: The go-to for international meetings. Free 40-minute meetings for up to 100 people. Screen sharing, virtual backgrounds, breakout rooms —solid for remote and cross-border work.
- Tencent Meeting: The office standard in China, deeply integrated with WeChat (scan to join). Up to 300 participants (paid). Cloud recording, live captions, meeting minutes — great for company training and large meetings.
- Which to pick: Everyday 1-on-1 WeChat. Apple-to-Apple FaceTime. Formal business Zoom or Tencent Meeting (Tencent in China, Zoom across borders).

Bandwidth and data use reference:

- WeChat video calls: About 500 Kbps–1 Mbps; roughly 3–5 MB per minute at standard quality, 8–12 MB at HD.
- FaceTime: About 1 Mbps at SD, 2 Mbps for 720p, and 4+ Mbps for 1080p. Roughly 5–15 MB per minute.
- Zoom/Tencent Meeting: About 1.5–3 Mbps for 720p, 3–5 Mbps for 1080p. Roughly 300–800 MB per hour; hosts of multi-person meetings use more.
- Tip: Use Wi-Fi for long video calls. On mobile data, lower the quality yourself (turn off "HD video" in app settings) to save data.

Video call quality tips:

- Lighting: Face a light source with a window behind it (natural light is best). Avoid backlighting that turns your face into a silhouette.
- Angle: Keep the camera slightly above eye level to avoid a double chin. In landscape mode, hold the phone with both hands for stability.
- Audio: A headset prevents echo and howling. In a quiet room, don't use speakerphone.
- Network: Prefer Wi-Fi. On 4G/5G, stay near a window for a stronger signal. In group meetings, turn off other videos and keep only the speaker.
- Background blur: Tencent Meeting and Zoom both support AI blur to hide messy backgrounds. Wear solid colors so you don't "green-screen" into the background.

Quick tip

When the connection lags, turn off video and keep audio before hanging up —more polite than dropping the call. Before important meetings, run Tencent Meeting's built-in "Network Test."

Email: Set up and use it on your phone

- Add an email account: On iOS, Settings > Mail > Accounts > Add Account. On Android, there's usually a built-in "Email" app.
- Tips: Set different notifications for different accounts. Use swipe gestures to quickly archive, delete, or flag mail.

IMAP vs POP3 vs Exchange:

- POP3: Downloads mail and deletes the server copy by default. Good for a single device that needs offline reading. Downside: no history on new devices. No longer recommended.
- IMAP: Mail stays on the server; local is just a cache. Phones/PCs/tablets stay in sync —read on one, read everywhere. Today's mainstream recommendation.
- Exchange (including ActiveSync): Microsoft's protocol. Syncs calendar, contacts, and tasks as well as mail. First choice for work email. Supported by Outlook, NetEase Enterprise Mail, and Tencent Enterprise Mail.
- What to choose: Personal email IMAP. Work email Exchange first, IMAP second. Avoid POP3 unless you have a special archiving need.

Work email setup:

1. iOS: [Settings] > [Mail] > [Accounts] > [Add Account] > choose "Microsoft Exchange" or "Other."
2. Android: In the built-in "Email" app or Outlook, choose "Add account" and enter your full address and password.
3. Server settings (Exchange example): Server is often "mail.yourcompany.com" or "outlook.office365.com"; domain is your company short name.
4. If you see "Cannot verify server," enter settings manually: Incoming server (IMAP) port 993 with SSL; outgoing server (SMTP) port 465 or 587 with TLS.
5. After signing in, turn on "Push" in [Account Settings] for real-time mail, and turn off message preview to protect confidential content.

Heads-up

Work email usually needs an admin-issued "client authorization password" or "IMAP/SMTP auth code" —not your web login password. If login fails, generate a code in webmail settings first, then enter it on your phone.

Hands-on: Back up your WeChat chats to a computer

1. Start the backup on your computer: Open WeChat for desktop > menu at the bottom left > [Migrate & Backup] > [Backup & Restore].
2. Choose back up to computer: Click "Back up chat history to computer."
3. Select what to back up on your phone, then wait for the backup to finish.

Chapter 9: Mobile Office and Learning

Microsoft 365

: If your company uses genuine Office, this is the best choice. Format compatibility with the desktop version is nearly perfect, avoiding the slight layout shifts that WPS may have with some complex layouts. Deeply integrated with OneDrive, the document you were halfway through editing on your computer opens as the latest version on your phone.

Shimo Docs / Tencent Docs

- **: These two represent "online collaboration". They are suitable for multiple people editing the same document simultaneously, with each person's changes shown in real time. I used Shimo Docs to plan a trip with a friend —both of us filling in information into a spreadsheet at the same time was far more efficient than doing it alone.**Practical Workflow
- **: My current habit is —important files go to OneDrive, daily editing uses WPS, and team collaboration opens Shimo Docs. Since my files went to the cloud, I've never again had the embarrassment of "forgetting my USB drive".**2. Note-Taking Apps: From Quick Notes to a Knowledge System
- **When it comes to taking notes, many people want to build a complex system right from the start, only to give up after three days. My advice is to start light and then go heavy —first build the habit of "recording", then consider the problem of "managing".**Quick Notes —The built-in system is enough

: iOS Notes launches extremely fast and supports document scanning, handwriting, and audio recording. Android's built-in memo apps by various brands are

also decent, and can be launched quickly even from the lock screen. These tools are suitable for recording temporary ideas, shopping lists, and meeting key points. The key is fast launch —you can record as soon as you think of something, without waiting for an app to load. Read It Later —Collecting Information

: WeChat's "Float" feature is actually a hidden gem. When you see a good article but don't have time to read it carefully, tap "Float" in the upper-right corner, and it shrinks into a small dot attached to the edge of the screen, ready to be tapped open to continue reading anytime. It's much more convenient than forwarding to "File Transfer Assistant". In addition, read-later tools like Cubox or Pocket can collect good content you come across anywhere, across platforms.

Building a System —Notion and Feishu

- **: When you feel that scattered notes are becoming hard to find, it's time to move to Notion. This tool is truly flexible —you can build a "reading notes database" that automatically organizes by book title, author, and rating; or set up a "project board" to visualize the progress of work tasks. Feishu's "Bitable" is similar, and has faster access speeds in China. I use Notion to manage all my writing materials, study notes, and project progress —I've been using it for over three years.**A Practical Note-Taking Strategy

- **: My own habit is a "three-piece set" —temporary ideas go to the system memo (instant launch), articles to read go to WeChat's float (handy), and knowledge that needs long-term accumulation goes to Notion (systematized). Don't try to stuff everything into one tool from the start —that makes it easier to give up.**3. Scanning and OCR: Bringing the Paper World into Your Phone

- **Phone scanning, when used well, can really save a lot of trips to the print shop. Last time I applied for a visa and needed electronic copies of a bunch of materials, I got them all done with my phone.**iPhone users

: The built-in scanning feature of iOS Notes is already quite good. Open Notes tap the camera icon select "Scan Documents", aim at the document and it will automatically recognize the border and correct the perspective. After shooting, you can adjust the color —the "Black & White" mode produces results closest to a scanner, and "Grayscale" is suitable for materials with images. Android users

: Many brands' cameras have a built-in "Document Scan" mode. If you can't find it, download Microsoft Lens from the app store (free, and integrated with OneDrive).

OCR Text Recognition

- **: If you need to extract text from scanned documents for editing, you need OCR. WeChat's built-in "Extract Text" feature is the simplest —long-press the image to use it, and the recognition accuracy is decent. For bulk processing, you can use the Baimiao app or CamScanner. For Chinese recognition, Baidu's OCR engine works quite well; WPS's image-to-text feature is also very accurate.**Scanning Tips
- **Look Up Tips**4. How to Choose a Cloud Drive: There's No Best, Only What's Best for You
- **I thought about choosing a cloud drive for a long time, and finally realized it really depends on what your "primary ecosystem" is.**Apple Ecosystem iCloud
- **: Seamless syncing that makes you barely notice its existence. Photos you take on your phone automatically appear on your computer a few minutes later. And Apple's privacy protection is the strictest. But the drawbacks are also obvious: cross-platform sharing with Android or Windows users is cumbersome, the free tier is only 5GB, and expansion costs money.**Windows Primary OneDrive

: If you use a Windows PC + Office 365, OneDrive is a natural match. Save a document in Word, and it's automatically stored in the cloud. The 1TB space bundled with the Office subscription is a great deal. The mobile app's auto photo backup feature is also very handy.

Domestic Sharing Baidu Netdisk

- **: To be honest, Baidu Netdisk's download speed limit has always been criticized, but its sharing ecosystem is truly unmatched. A large amount of study materials and film/TV resources on the Chinese internet are shared via Baidu Netdisk. If you only occasionally download files shared by others, you can make do without a membership. If you're a heavy resource collector, just pay for the membership —the time saved is worth much more.**Aliyun Drive / Quark Drive

- **: These are the rising stars in recent years. Aliyun Drive has no upload/download speed limits (currently), and Quark Drive is deeply integrated with the Quark browser, so you can directly save to the drive when searching for materials, which is very convenient.**My Combined Solution
- **: iCloud manages personal photos and daily files (privacy first), Baidu Netdisk stores resources and receives shares (ecosystem first), and OneDrive manages work documents (Office integration first). Three cloud drives, each with its own job, not interfering with each other.**5. A Real Mobile Office Scenario
- **Let me share a real experience of my own. Once on a business trip out of town, the client suddenly needed to revise contract terms. My workflow was as follows:**At the hotel, I received the client's revision comments via WeChat (text messages + a photo of handwritten annotations).

Opened OneDrive to find the original contract, opened and edited it with WPS, revising the terms one by one according to the client's WeChat messages. After revisions, used WPS's "Export as PDF" feature, and sent it to the client's WeChat group for confirmation.

The client raised a second round of comments, so I used WPS's brush tool to directly circle the revision positions on the PDF, with voice annotations explaining the revision approach.

After confirming everything was correct, used WPS's "Electronic Signature" feature to sign online, and sent it to the other party to sign as well.

1. Finally, archived the signed contract PDF to both Baidu Netdisk and iCloud, and made a note in my work log in Notion.
2. The whole process took about forty minutes from start to finish, without turning on a computer at all. To be honest, three years ago I would never have believed a phone could do these things, but now it really can. The key is to familiarize yourself with these tools in advance, so you don't scramble to download and register when you actually need them.
3. Try it out: Build your mobile office workflow
4. Choose your primary document app (WPS or Microsoft 365), log in and link it to a cloud drive.
5. Open Notes or Memo, use the scan feature to scan a document, and export it as a PDF to send to yourself.
6. Choose a cloud drive app and set it to auto-backup your phone photos (select "Backup only on Wi-Fi" to save data).

Create a new page in Notion or Feishu, and try building a simple "work log" template.

Chapter 10: Mobile Payment and Life Services

1. To be honest, my parents' generation was initially resistant to the idea of "paying with a phone". "If the money is in the phone, what if it gets stolen?" This concern is very normal, but the reality is: the cash in your pocket has a much higher chance of being lost than mobile payment being fraudulently charged. Mobile payment is not about storing money in your phone —it's just a key, a key that opens your bank account.

2. 1. How Exactly Do You Bind a Bank Card?

3. Whether you use Alipay or WeChat, the first time you bind a card, you need to go through these steps:

4. Open the app, find "Me" or "Wallet". Tap "Add Bank Card".

Chapter 10: Mobile Payment and Life Services

The bank will send a verification code SMS to your reserved mobile number. Enter it and set a 6-digit payment password.

This payment password, your bank card password, and your lock screen password should ideally be three different passwords — don't set them the same for convenience.

A little tip: if you're concerned about security, you can open a Tier II or Tier III electronic account in your bank app specifically for binding to mobile payment. Only keep your everyday spending money in this card, so even if something happens, the loss is limited.

1. 2. The Payment Code Is "Whoever Sees It, Takes the Money"
2. Many people are confused about the difference between a payment code and a collection code, and this distinction is very important.
3. Payment Code
4. : The barcode + QR code you show to the cashier to scan. This code is equivalent to your "electronic wallet" —whoever scans it can deduct money from your account. So

absolutely never screenshot it and send it to anyone

. This code automatically refreshes once per minute precisely to prevent fraudulent use.

Collection Code

- **: Others scan this code to transfer money to you. This code can be screenshotted, saved, and sent to others —it's safe.**Remember this distinction, and you

can avoid many fraud traps. Anyone who asks you to "screenshot your payment code and send it over" is 100% a scammer.**3. These Security Settings —5 Minutes to Set Up Can Save You a Lot of Worry**Fingerprint/Face Payment

- **: In Alipay and WeChat's payment settings, turn on biometric payment. This way, every payment requires your fingerprint or face scan, so even if your phone is lost, the person who finds it can't spend your money.**Security Lock

: Both Alipay and WeChat have a "Security Lock" feature. Once enabled, entering the app's "Wallet" page requires an additional identity verification. Path: Alipay Me Settings Account & Security Security Center Security Lock.

Password-Free Payment Check

- **: Many people have signed up for various password-free payments and forgotten about them. Go to "Payment Settings" in Alipay and WeChat, find "Password-Free Payment / Auto Deduction", and see which services you've signed up for —turn off the ones you don't need.**Login Device Management

- **: Regularly check which devices have logged into your account. If you find an unfamiliar device, kick it out immediately. Path: Alipay Settings Account & Security Login Device Management.**4. Bus and Subway: Your Phone Is Your Transit Card

- **Nowadays, very few people use physical transit cards when taking the bus or subway. There are two paths for swiping your phone at the turnstile:**Virtual Transit Card in Phone Wallet

- **: Open a virtual transit card directly in Apple Wallet, Huawei Wallet, or Xiaomi Wallet. The advantage of this solution is: you don't need to unlock your phone when swiping —just hold it close, the response is fastest, and it works even without a network. Some cities' virtual transit cards also offer the same transfer discounts as physical cards.**Alipay/WeChat Ride Code

: Search for “Ride Code” in the app, and activate the bus or subway ride code for your city. The advantage of this solution is: no need to top up —it deducts directly from your balance or linked bank card. But it requires a network connection, and you need to open the app and find the code before swiping.

I personally recommend using the virtual transit card in your phone wallet. Although you need to top up 10 or 20 yuan the first time, it’s really much more convenient to use, especially when rushing for the subway —you don’t have to stop to unlock your phone and find the app.

- **5. Utility Bills, Food Delivery, Ride-Hailing**Water/Electricity/Gas Bills
- **: In Alipay or WeChat’s ”Utility Bills”, bind your household water, electricity, and gas account numbers. Then each month when you receive a bill, just tap in to pay. The account number can usually be found on your paper bill, or you can call customer service to ask.**Food Delivery Price Comparison

: The same restaurant often has different prices and promotions on Meituan and Ele.me. Check both before placing an order —sometimes you can save several yuan. Also, Meituan’s “Pin Hao Fan” feature lets you order together with friends for a noticeably cheaper price.

Ride-Hailing Safety

- **: In Didi or Amap Ride-Hailing, remember to set up ”Emergency Contact” and ”Trip Sharing”. When you take a ride, the app automatically shares your trip with your designated contacts. Also, for late-night rides, prioritize ”Taxi” over ”Express” —taxis have industry regulation and a higher safety level.**6. Handle Medical Insurance, Social Security, and Government Affairs on Your Phone
- **Things have changed a lot in recent years —many things that used to require queuing at a service hall can now be done in a few minutes on your phone.**Electronic Medical Insurance Certificate
- **: Search for ”Electronic Medical Insurance Certificate” in Alipay or WeChat. After activation, you can just swipe your phone when seeing a doctor or buying medicine at a pharmacy —no need to carry your physical medical insurance card. The first time requires facial recognition verification, and afterwards a dynamic code is generated each time you use it.**Social Security Inquiry

: Download the “Palm 12333” app or use “Citizen Center” in Alipay to check social security contribution records, pension account balances, unemployment benefit claims, and more.

Individual Income Tax

- **: The annual tax reconciliation from March to June each year can be handled on the ”Individual Income Tax” app. Remember to check if there are any special additional deductions you can claim (children’s education, mortgage interest, supporting the elderly, etc.) —these can all help you pay less tax or get a refund.**Remember These to Avoid Scams and Stay Safe
- **Police, prosecutors, banks, and customer service will never ask you to transfer money to a ”safe account”.**Anyone who asks you to screenshot your payment code and send it over is a scammer.
- **Don’t click unfamiliar links, especially SMS messages like ”ETC expired” or ”Social security card suspended”.**Online shopping refunds are returned via the original payment method —no need to provide your bank card number.

Do It Now: Give Your Payment Apps a Security Checkup

- Open Alipay and WeChat, confirm that fingerprint/face payment is enabled.
- Find the ”Security Lock” feature and turn it on.
- Check the ”Password-Free Payment / Auto Deduction” list, and turn off services you no longer need.
- View the ”Login Devices” list, and remove any unfamiliar devices.

Activate your electronic medical insurance certificate, so next time you visit the hospital you won’t need to bring the physical card.

1. Chapter 11: Photography and Photo Management
2. To be honest, most people’s phone photos fall far short of the phone’s actual potential. It’s not that your skills are bad —it’s that you haven’t noticed many small settings and habits. This chapter skips the fluff and only covers things you can use right away.
3. 1. Have You Made These Mistakes Too?
4. Let’s first talk about the most common problems —see how many you’ve hit:
5. The lens is dirty

Chapter 11: Photography and Image Management

You never adjust exposure

Have You Made These Mistakes Too?

The flash is always on

- **: The effect of a phone's LED flash is really ugly —the light is harsh and cold, making people look like ghosts. Unless it's truly pitch black, turn it off. In low light, Night Mode works far better than the flash.**Zooming by pinching with two fingers
- **: Pinch-to-zoom on the screen is digital zoom, which essentially crops and enlarges the image, noticeably reducing quality. If you want to shoot something far away, use the zoom buttons on the viewfinder (0.5x/1x/2x/3x) — these are optical zoom and don't lose quality.**2. A Few Small Tips to Immediately Boost Your Hit Rate
- **These don't require any theoretical knowledge —use them once and you'll see the effect:**Rule of Thirds + Grid Lines
- **: Turn on "Grid Lines" (also called reference lines) in camera settings. When shooting, place what you want to highlight at the intersections of the four lines, and the composition will immediately "click". This is the simplest and most effective rule in photography —no mysticism, it just matches the visual habits of the human eye.**: Pinch-zooming on the screen is digital zoom —essentially cropping and enlarging the image, which noticeably reduces quality. To zoom in on distant subjects, use the zoom buttons in the viewfinder (0.5x/1x/2x/3x). The 0.5x (ultrawide) and 1x (main camera) are native optical lenses with no quality loss; 2x/3x are typically "lossless quality zoom" —achieved by cropping from high-resolution sensors, with near-optical quality but not a separate lens. Beyond 3x is usually hybrid or digital zoom, with noticeable quality degradation.

A Few Small Tips to Immediately Boost Your Hit Rate

Don't always shoot standing up

- **: Most people always shoot at their own eye level, so their photos all have the same perspective. Try crouching down for a low angle, or finding a high spot to shoot downward —the same place can produce a completely different feel. When shooting kids and pets especially, you need to crouch down —only eye-level shots produce a natural feel.**Light is the top priority
- **: Shooting portraits under the bright midday sun produces hard, ugly shadows on the face. The best light is during the "golden hours" of early morning and late evening —the sunlight is angled and warm-toned, making everything look good. Overcast-day light isn't flashy, but it's even and soft, making it actually easier to shoot portraits than under bright sun.**Night Mode essentials
- **: After turning on Night Mode, the phone automatically takes multiple shots and composites them. The most critical thing at this point is —your hand absolutely cannot shake. Lean against a wall if you can, rest on a table if you can, or hold your breath if you must. Night Mode requires you to hold steady for 2 to 4 seconds —any shake and the image blurs.**3. Portrait Mode and Pro Mode
- **Portrait Mode**: It produces the effect of a blurred background with a sharp subject (shallow depth of field). Because phones have small lens sizes, their real bokeh capability is limited, so this relies on AI algorithms —simply put, the AI cuts you out and blurs the background. The best results come at a distance of 1-2 meters; too far or too close and the algorithm easily makes mistakes. Also, when shooting portraits, make sure the background doesn't have overly complex lines, or the AI cutout may "fail" around the hair edges.
- **Pro Mode**: If you want full control over the image effect, try Pro Mode. A few key parameters to know: the lower the ISO, the cleaner the image quality, but the darker the image; the slower the shutter, the more light comes in, but hand shake causes blur; white balance controls color temperature —set poorly, photos will look blue or yellow. Most people don't need to use Pro Mode year-round, but understanding these parameters at least lets you know what the camera is doing.

Portrait Mode and Pro Mode

- **Many people misunderstand photo editing, thinking that editing makes it "unreal". In fact, photos taken by a camera are already different from what the naked eye sees —appropriate adjustment is to restore the scene you saw at the time, not to fake it.**Snapseed

- **: A free editing tool from Google, professional features with no ads. My daily editing flow is: first use "Tune Image" to adjust brightness and contrast use "Details" to sharpen use "White Balance" to correct color casts finally "Crop" to the desired composition. Snapseed's "Selective" feature is very useful —you can adjust only a specific area of the image without affecting the whole thing.**Xingtu / Meitu

Photo Editing: Don't Overdo It —Just Right Is Enough

Lightroom Mobile

- **: If you're particular about color, this is the most professional. It has advanced features like curve adjustment and HSL per-color adjustment. But for most people, it's overkill.**Editing Workflow Suggestion
- **: My own habit is to shoot the original photo first, then edit slowly on a computer screen or iPad at home —the phone screen is too small and the color isn't quite accurate. If you must edit on the phone, remember to set the screen brightness to medium-high, and don't edit in power-saving mode.**5. Too Many Photos? Organization Strategies
- **Thousands of photos piled up in the phone album, taking forever to find a specific one —everyone has had this experience. Here are a few practical organization methods:**Make good use of search
- **: Now both iPhone and most Android phone albums support smart search. You can search for keywords like "cat", "seaside", "ID card", "flowers", and the system will use AI to identify image content and find them for you. You can also search by date and location, like "last May" or "Beijing". Not many people know about this feature, but it's really useful.**Regularly clean up junk

Too Many Photos? Organization Strategies

Create albums —don't rely solely on default sorting

- **: The system sorting by time is fine, but for cross-time content like "child's growth photos", "travel photos", and "important documents", you should manually create albums to categorize them. Apple's "Shared Albums" can even let family members add photos together —one person creates it, and the whole family can view it.**Backup is the bottom line
- **: Photos are the most irreplaceable data —files can be recreated, chat histories can be lost and that's that, but a child's photos or an elderly person's videos,**

once lost, are truly gone. You must enable cloud backup. iCloud Photos is the most worry-free; on Android, each brand has its own cloud service, or you can use Baidu Netdisk / Aliyun Drive for auto backup. The principle is: have at least one cloud backup —don't put all your eggs in one basket.6. Screenshots and Screen Recording

- **Screenshots**: Press the power button + volume up button simultaneously —a universal method for all phones. Many Android phones also support three-finger swipe down screenshots and knuckle double-tap screenshots. Long screenshots (scrolling screenshots) are well-established on Android —after taking one shot, tap the "Scroll" button, and the phone automatically scrolls down and stitches together a long image, suitable for capturing chat histories and web pages.
- **Screen Recording**: Add "Screen Recording" from the Control Center, tap once to start recording, tap again to stop. Before recording, long-press the record button to choose whether to record microphone sound —turn on the mic if you're making a tutorial with narration, turn it off if you're just recording operations. The recorded video is automatically saved to the album.

Screenshots and Screen Recording

- **Turn on "Grid Lines" in camera settings, find a spot with natural light, and take three photos (remember to wipe the lens and adjust exposure).**Select the best one, and use Snapseed for basic adjustments (brightness, contrast, cropping).
- **Create a dedicated album for this photo, named "My Works".**Check whether cloud backup is enabled, to ensure the photo has a copy in the cloud.

Chapter 11 Appendix: Professional Imaging Techniques

To truly understand the mysteries of phone photography and video, you need to know some professional imaging technology. This knowledge not only helps you better use your phone's camera, but also makes you more discerning when choosing devices.

Resolution, Frame Rate, and Bitrate: The Three Elements of Video Quality

Resolution:

Resolution determines image clarity, expressed in pixel count.

- **720P (HD)**: 1280×720 pixels, entry-level clarity.
- **1080P (Full HD)**: 1920×1080 pixels, currently the most mainstream resolution.

- **2K (QHD)**: 2560×1440 pixels, clearer than 1080P.
- **4K (UHD)**: 3840×2160 pixels, ultra HD with rich detail.
- **8K**: 7680×4320 pixels, the future standard, currently with limited device support.

The higher the resolution, the clearer the image, but the larger the file and the higher the demands on storage and processing power.

Frame Rate:

Frame rate determines image smoothness, expressed in frames per second (fps).

- **24fps**: The standard cinema frame rate, with a unique "cinematic feel".
- **30fps**: Commonly used for TV and web video, smooth and natural.
- **60fps**: High frame rate, suitable for sports scenes and game recording, very smooth.
- **120fps/240fps**: Ultra-high frame rate, can be used for slow-motion video.

The higher the frame rate, the smoother the image and the better the slow-motion effect, but the larger the file.

Bitrate:

Bitrate determines the data density of video, expressed in data per second (Mbps).

- The higher the bitrate, the better the image quality and the more detail preserved, but the larger the file.
- Too low a bitrate causes compression artifacts like mosaic blocks and color banding in the image.
- Professional video shooting usually requires high bitrate to preserve more room for post-processing.

Video Coding Formats: H.264, H.265, and ProRes

H.264 (AVC):

- **Characteristics**: The most widely used coding format, with excellent compatibility.
- **Pros**: Almost all devices can play it, with moderate compression efficiency.
- **Cons**: Compared with newer formats, lower compression efficiency and larger files.
- **Applicable scenarios**: Web video sharing and everyday shooting for ordinary users.

H.265 (HEVC):

- **Characteristics**: An upgrade to H.264, with about 50% better compression efficiency.
- **Pros**: Smaller files at the same quality, supports higher resolutions and frame rates.
- **Cons**: Encoding and decoding require a more powerful processor; some older devices are incompatible.
- **Applicable scenarios**: 4K video shooting and scenarios with limited storage space.

Apple ProRes:

- **Characteristics**: A professional video codec developed by Apple, with nearly lossless compression.
- **Pros**: Extremely high image quality, large room for post-processing, accurate color.
- **Cons**: Extremely large files, requiring substantial storage and professional hardware to process.
- **Applicable scenarios**: Professional video production and cinema-grade shooting.

ProRes RAW:

- **Characteristics**: Combines the advantages of ProRes and RAW formats.
- **What is RAW**: Unprocessed raw sensor data, preserving maximum information.
- **Pros**: Huge room for post adjustment—exposure, white balance, and other parameters can be changed without any quality loss.
- **Cons**: Extremely huge files, requiring professional post-production software to process.
- **Applicable scenarios**: High-end professional production. iPhone 15 Pro series and later support Apple ProRes video recording (up to 4K 60fps), and iPhone 17 Pro further supports ProRes RAW direct in-device recording.

HDR: High Dynamic Range Technology

HDR (High Dynamic Range) technology preserves detail in both the brightest and darkest parts of the image at the same time.

- **HDR10**: The most basic HDR standard, with good compatibility.
- **HDR10+**: A standard promoted by Samsung, supporting dynamic metadata.
- **Dolby Vision**: The top-tier HDR standard developed by Dolby, with the best image quality, but requires licensing.

- **HLG**: Broadcast-grade HDR standard, compatible with ordinary displays.

When shooting HDR video, you need an HDR-capable display to see the full effect.

Color Space and Color Depth

Color Space:

- **sRGB**: The standard color space, with the best compatibility.
- **Display P3**: A wide-gamut standard promoted by Apple, with richer colors.
- **Rec.2020**: The standard color space for HDR video, with the widest gamut.

Color Depth:

- **8-bit**: 16.7 million colors, the standard for ordinary video.
- **10-bit**: 1.07 billion colors, the standard for professional video, with smoother color transitions.
- **12-bit**: 68 billion colors, a high-end professional standard.

Practical Advice

- Everyday shooting: 1080P/4K + 30fps + H.265 encoding, balancing quality and file size.
- Sports shooting: 1080P/4K + 60fps, to ensure smoothness.
- Professional creation: 4K + ProRes (if supported), to preserve maximum post-production room.
- Storage advice: High-bitrate video takes up a lot of space —back up to your computer or cloud drive regularly.

Explore: Your Phone's Video Settings

1. Open the Camera app and switch to video mode.
2. View the resolution and frame rate options, and try different combinations.
3. If supported, check whether there are advanced options like HDR and ProRes.
4. Shoot a few video clips with different settings, and compare the image quality and file size.

V. RAW vs HEIF vs JPEG Comparison

What Is RAW Format:

RAW format is the “raw data” recorded by the camera sensor —pure, unprocessed original information without any compression. It's like raw ingredients that need post-processing “cooking” to become a finished product. Phones, due to storage and processing limitations, default to “pre-cooked” JPEG or HEIF. RAW must be explicitly enabled.

RAW vs HEIF vs JPEG Comparison:

- **JPEG**: The oldest format, best compatibility (any device can open it). 8-bit color depth, lossy compression, smallest files (2-5MB per photo). Limited post-processing room — forcing detail out of shadows produces banding artifacts.
- **HEIF/HEIC**: Apple's newer format, files are half the size of JPEG at the same quality (1-3MB per photo). Supports 10-bit color depth and transparency channel. Drawback: some older devices and Windows require plugins to open.
- **RAW**: Uncompressed raw data, largest files (20-100MB per photo). Color depth typically 12-16 bit, maximum post-processing room. Drawback: requires professional software (Lightroom, Snapseed, etc.) to open and process —cannot be shared directly.

Why Professional Photographers Shoot RAW:

- **Lossless White Balance Adjustment in Post**: If white balance is wrong in JPEG (e.g., yellow under tungsten lighting), correcting it in post degrades quality; RAW lets you reset color temperature with zero loss.
- **Greater Exposure Tolerance**: In JPEG, shadows underexposed by 2 stops are basically dead black —brightening produces noise. RAW can recover usable detail from shadows underexposed by 3-4 stops. Same applies to overexposure.
- **Richer Shadow and Highlight Detail**: RAW's dynamic range far exceeds JPEG — much more latitude for preserving highlights without blowing out and shadows without crushing.
- **Sharpening, Noise Reduction, and Lens Correction All Controllable in Post**: In-camera processing is a ”one-shot deal.” RAW can be adjusted repeatedly, always starting from the original data.

How to Enable RAW on Phones:

- **iPhone ProRAW**: Only iPhone 12 Pro and later Pro models. Go to ”Settings Camera Formats” and enable ”Apple ProRAW.” A ”RAW” button appears at the top of the camera interface —tap to enable. Files are about 25MB each —enable only when needed.

- **Android Motion Raw / Ultra RAW**: Naming varies by brand. Xiaomi/OPPO/vivo flagships have RAW toggle in camera "Pro Mode" or "More"; Huawei enables RAW in "Pro Mode"; Samsung Galaxy S series has RAW option in "Pro" mode. Files are typically 20-50MB.
- **General Advice**: No need to enable RAW for casual daily shooting —files are too large and waste space. Only enable for important scenes (landscape, portrait, post-processing needed).

VI. Computational Photography Principles

Modern phone photography is inseparable from "computational photography" —using algorithms, not just lenses and sensors, to generate photos. Understanding these core principles helps you understand what your phone does the moment you press the shutter.

Multi-Frame Fusion:

When you press the shutter once, the phone actually takes multiple photos. The algorithm aligns and fuses the frames to produce one high-quality image.

- **Night Mode**: Takes multiple underexposed photos within 1-3 seconds, aligns and stacks them —noise cancels out, brightness accumulates. The result: clean images even in low light.
- **HDR (High Dynamic Range)**: Shoots underexposed, normal, and overexposed frames (or more), then composites them into one photo with detail in both highlights and shadows. Especially effective for backlit scenes.
- **Deep Fusion (Apple) / Photonic Engine**: For medium-to-low light scenes, analyzes and fuses multiple frames pixel by pixel, focusing on improving texture and detail — sweaters, hair, and similar details become noticeably clearer.

Semantic Segmentation:

AI doesn't process the entire image uniformly —it first "understands" what's in the scene —sky, faces, grass, buildings, clothing —then applies different processing strategies to each region.

- **Sky**: Selectively enhance blue, reduce brightness to prevent overexposure.
- **Faces**: Identifies skin tone areas, selectively brightens and smooths skin, leaving facial features and hair untouched.
- **Grass/Leaves**: Enhance green saturation, boost detail sharpening.
- **Background Blur**: In Portrait mode, AI identifies the subject's outline, blurring the background while keeping the subject sharp.

Deep Learning Applications:

- **AI Scene Recognition**: The camera automatically recognizes scenes like "food," "pets," "night," "document" and optimizes parameters accordingly. For example, food mode enhances warm tones and saturation to make food look more appetizing.
- **AI Portrait Bokeh**: Phone lenses physically can't achieve DSLR-level shallow depth of field. Deep learning models predict subject edges, then blur the background. Latest algorithms correctly handle details like hair strands and glass edges.
- **AI Eraser/AI Outpainting**: Select passersby or objects in the frame, and AI "fills in" the occluded areas based on context. This is generative AI applied to imaging.

Computational vs Optical Photography:

- **Computational Photography Advantages**: Good image quality in a small form factor, usable shots in extreme scenes like low light and backlight, and ordinary users can take "good-looking" photos without photography knowledge.
- **Computational Photography Disadvantages**: Images can be "over-processed" — excessive sharpening, color distortion, unnatural sky colors; multi-frame fusion creates ghosting when subjects move; detail is "computed" rather than truly recorded.
- **Optical Photography Advantages**: Large sensor + wide-aperture lens truly captures light, natural image quality, large post-processing room; no artifacts from algorithmic "hallucination."
- **Trend**: Modern flagship phones combine "optical + computational" —large sensors capture real data, algorithms optimize the result. The two aren't opposed.

VII. Lens Types and Focal Lengths

Modern flagship phones typically have three to four lenses, each corresponding to a different focal length. Understanding focal lengths helps you choose the right lens for each scenario.

Common Lenses and Equivalent Focal Lengths:

- **Ultrawide (13-24mm equivalent)**: Widest field of view, suitable for landscapes, architecture, and interior spaces. Captures expansive scenes, but edges have distortion (straight lines curve), and faces stretch in portraits. Good for "big scenes," not for "close-ups."
- **Main Camera (24-28mm equivalent)**: The most commonly used lens, best image quality, widest aperture. Balances field of view and quality —works for both people and scenery. The "1x" on most phones is the main camera.

- **Telephoto (70-200mm equivalent)**: For distant subjects and portrait close-ups. Telephoto compresses spatial perception (distant mountains and nearby people appear closer together), producing more natural facial proportions and background blur in portraits.
- **Periscope Telephoto (200mm+ equivalent)**: Uses a folded reflective optical path to achieve longer focal lengths within the phone's thickness. Captures clear detail at great distances —suitable for concerts, sports, wildlife. Only on flagship models.

Sensor Size vs Pixel Size Trade-off:

The sensor is the "foundation," determining how much light can be captured. The industry says "a bigger sensor crushes the competition"—at the same pixel count, a larger sensor means larger individual pixels, more light intake, and better image quality.

- **Large Sensor**: More light intake, better night image quality, wider dynamic range. Disadvantages: phone must be thicker (camera bump), higher cost.
- **High Pixel Count, Small Sensor**: In sufficient daylight, high pixel count produces sharper photos, convenient for post-cropping. But in low light, each pixel receives insufficient light, producing more noise.
- **Mainstream Flagship Choices**: Main cameras are typically 1/1.3-inch to 1-inch (some models already use 1-inch sensors), around 50MP —the balance point between image quality and size.

Pixel Binning Technology:

To balance high pixel count in daylight and large pixels at night, manufacturers use "pixel binning" —merging multiple small pixels into one large pixel for output.

- **Quad Bayer (4-in-1)**: 50MP sensor outputs 12.5MP photos. Each output pixel is synthesized from 4 adjacent pixels, increasing light intake 4x for cleaner images.
- **Nonacell (9-in-1)**: Used for 100MP+ sensors, outputs approximately 11-12MP. Same principle, even greater light intake.
- **Practical effect**: In default shooting mode, you get the binned low-pixel photo —good quality, small file. When you need maximum detail (e.g., documents, distant subjects), manually switch to "high-resolution mode" for full-resolution output.

VIII. Special Shooting Modes

Beyond everyday photography, your phone has several special modes that, when used correctly, can capture shots you normally can't get.

Panorama Mode:

- Move the phone slowly and steadily in the direction of the arrow —don't go fast, or stitching misalignment will occur.
- Keep the phone vertically stable and only pan horizontally. If it bobs up and down, the horizon will become wavy.
- Use with caution when there are moving people or cars —stitching may produce "half bodies" or "ghosts."
- Panorama can also be shot vertically —vertical shooting with horizontal stitching, suitable for tall buildings or waterfalls.

Slow Motion and Time-Lapse:

- **Slow Motion**: Shot at 120fps or 240fps high frame rates, played back as slow motion. Great for capturing water droplets, sports moments, and pets jumping. Requires ample light —slow motion mode has shorter exposure time, resulting in poor low-light quality.
- **Time-Lapse**: Takes photos at intervals and combines them into a fast-forward video. Great for traffic, cloud movement, sunsets, and flower blooming. Shooting can take minutes to hours —always use a tripod to secure the phone; handheld shots will be blurry.
- **Tip**: For time-lapse cloud photography, set up the phone and shoot for 15-30 minutes. The result is about 10 seconds of stunning time-lapse footage.

Starry Sky / Astronomy Mode:

- You need to get away from urban light pollution —find a rural area or mountain with no streetlight interference at all. You can't capture the Milky Way in the city.
- A tripod is mandatory —handheld shots will be blurry. The phone must be held steady for several seconds to tens of seconds.
- Weather should be clear and cloudless, with the moon not too large (a full moon will overwhelm starlight). Dark nights around the beginning or end of the lunar month are best.
- iPhone's "Night Mode" automatically detects starry skies and extends exposure time (up to 30 seconds); Android flagships' "Star Mode" or "Astronomy Mode" works similarly. Be patient —don't move the phone after pressing the shutter.

Macro Mode:

- Used for photographing flowers, insects, jewelry, text close-ups, and other small objects. Can focus as close as 1-2 centimeters.
- Some phones have a dedicated macro lens (usually a 2MP or 5MP small-sensor lens) with average quality; flagship phones use the ultra-wide lens to double as macro, producing better quality.
- When shooting up close, the lens is very near the subject and may block light —use a desk lamp or flashlight to supplement lighting if needed.
- The focus distance is extremely close, creating a very shallow depth of field —moving the subject a few millimeters forward or back can cause focus loss. Keep your hands steady and ideally have the subject stay still.

Practical Tips for Still Photography

- Daily shooting: HEIF/JPEG default is fine —small file size and adequate quality.
- Important scenes (landscape, portrait, post-processing needed): Enable RAW to preserve maximum post-processing flexibility.
- Choose the right lens: ultra-wide for landscapes, telephoto for portraits, main camera for everyday use.
- Night scenes and starry skies: Always secure the phone and enable Night Mode or Astronomy Mode.
- Phones with large sensors (e.g., 1-inch sensor models) have higher quality ceilings, but shooting technique remains the key.

Try It Yourself: Understand Your Camera's Capabilities

1. Check if your phone supports RAW shooting (Settings Camera Format). Try enabling it and take a shot, then compare the JPEG and RAW file sizes of the same scene.
2. Switch between the ultrawide, main camera, and telephoto focal lengths and shoot the same distant scene with each, to feel the differences in perspective and quality.
3. Find a scene with Night Mode or Starry Sky Mode, secure the phone, and take a night photo.
4. Try shooting a wide scene in Panorama Mode, keeping the phone moving smoothly and steadily.

Chapter 12: Audio-Visual Entertainment and Reading

QQ Music

: Has the most complete licensing —popular artists like Jay Chou and Mayday are basically only on QQ Music. The sound quality options are also very rich, and members can enable lossless quality. If you care about "being able to find every song you want to listen to", QQ Music is the most reliable choice.

Apple Music

- **: The first choice for Apple users. Clean interface, no ads, good sound quality, and supports lossless and Dolby Atmos. But its recommendation algorithm doesn't understand Chinese users' preferences as well as domestic apps, and the playlist culture isn't as developed as NetEase Cloud's. Suitable for people who like a clean experience and don't care much about social features.**Spotify
- **: Has the most users globally, and is widely recognized as having the strongest recommendation algorithm. "Discover Weekly" and "Daily Mix" have scarily high accuracy —the longer you use it, the better it knows you. The downside is that the free version has many ads, and some Chinese songs have incomplete licensing. Requires a special network to use.**A few music-listening tips
- **: If you use Bluetooth headphones, check the music app settings for "Sound Effects" or "Equalizer" —slightly adjust the bass and treble for a noticeable improvement. Also, streaming music uses a lot of data —remember to turn on "Play lossless only on Wi-Fi" in settings, or download your favorite playlists in advance for offline listening.**2. Podcasts: The "Ear Companion" on Your Commute
- **Podcasts —people who haven't listened think they have nothing to do with them, but once they start, there's no going back. Essentially, it's "the Netflix**

version of radio shows" —some talk tech, some discuss history, some read stories —whatever type you want, it's there. Xiaoyuzhou App

: The best podcast app in China, with a clean interface and no clutter. Its "Daily Recommendations" and "Editor's Picks" help you discover new shows, and the comment section is very active. I personally listen most to "Hu Zuo Hu You" (culture and history) and "Bu He Shi Yi" (social issues) —each episode feels like eavesdropping on a conversation among smart people. Apple Podcasts

: Built into iPhone, no need to download an additional app. Best integration with CarPlay, very convenient when driving.

Listening tips

- **: Most podcast apps support 1.2x or 1.5x playback speed —once you're used to it, you can save a lot of time. I usually listen to Chinese at 1.2x and English at 1.5x. Also, definitely turn on "Auto-download new episodes" and "Auto-delete after playing", so your phone won't pile up with audio files you've already heard.**3. Watching Video: The Right Way to Open Players and Streaming
- **Many people don't know that the formats a phone can play are far more than what streaming apps offer. A good local player can solve many problems.**VLC Player
- **: Open source and free, can play almost any format —MKV, AVI, FLAC, even ISO disc images. If you downloaded a video that plays on your computer but not on your phone, it's most likely a format issue —install VLC and it's solved. It also supports external subtitles —put the .srt subtitle file and the video in the same folder, and VLC will auto-load it.**nPlayer

: If you want a more refined experience, this paid app is worth buying. Nice interface, smooth gesture controls —swipe up and down on the left to adjust brightness, on the right to adjust volume, swipe left and right to fast-forward and rewind. Supports DTS and Dolby audio, giving a much stronger immersive feel when watching blockbusters.

Bilibili (B Site)

- **: No longer just an ACG site —the learning resources on it are now absurdly abundant. Postgraduate exam prep, programming, photography, cooking —**

tutorials on everything. The bullet-screen (danmaku) culture is the soul of Bilibili —watching videos with danmaku on feels like having a group of people watching with you. However, when watching educational videos, it's recommended to turn off danmaku, otherwise it's easy to get distracted. Casting to TV

- **: No matter how big your phone screen is, it's not as comfortable as a TV. On the same Wi-Fi, open the "TV" cast button in the upper-right corner of the video app to cast the image to your TV. Most smart TVs and projectors support it —the experience of watching shows and movies is so much better. iPhone users can also use AirPlay to directly mirror the entire screen.**4. E-Book Reading: You Can Read Well on a Phone Too

- **Some say reading on a phone hurts the eyes and is distracting —these points are indeed valid. But the advantages of phone reading are also obvious — anytime, anywhere, maximizing the use of fragmented time. I've read quite a few books on my phone —on the subway, while queuing, before bed.**WeChat Reading

- **: Currently the best reading app experience in China. The library is very comprehensive, with both published books and web novels. It uses a "reading time for book coins" model —the more you read, the more free books you can exchange for, which is very friendly to students. Its "Underline" feature lets you see which passages others have marked —sometimes seeing good sentences highlighted by others really resonates. However, the "Reading Leaderboard" thing is a bit competitive, easily turning into "reading just to rack up hours" —I suggest turning it off.**Kindle App

: If you have a Kindle device, the phone Kindle app can sync your reading progress. Genuine Kindle e-books have much better layout quality than pirated PDFs, but the Kindle China store has closed, so you need to switch to an overseas region.

Apple Books

- **: Built into iPhone, suitable for reading EPUB and PDF. Minimalist interface, very comfortable page-turning animation. If you download an e-book file from the internet, just "Share to Books" to import it.**Reading Settings Advice

- **: No matter which app you use, be sure to set the background to a warm yellow tone (eye-care mode), increase the font size, and widen the line spacing. When reading at night, turn on "Dark Mode" —white text on black is much**

easier on the eyes. Never read with white background, black text, and small fonts in low light for too long —eye fatigue accumulates.5. Mobile Gaming: Play Well, Without Harming Your Phone

- **My attitude toward mobile gaming is: playing is fine, but don't let it control you in return. Use gaming tools wisely to play more comfortably without delaying your real business.**Game Mode / Game Space
- **: Mainstream phones now all have this feature. When turned on, the phone automatically clears background apps, blocks notifications, and prevents accidental touches —if you're in the middle of a team battle and a phone call suddenly pops up, Game Mode can block it for you. Some phones also support Performance Mode / E-sports Mode, forcing the processor to run at full speed, but at the cost of faster heating and battery drain.**How to adjust image quality and frame rate

: Don't just max everything out. First ensure frame rate (smoothness), then pursue image quality. 60 fps is much smoother than 30 fps, but 120 fps puts a lot of pressure on the phone —mid-range phones shouldn't force it. If you play competitive games (like Honor of Kings or PUBG Mobile), frame rate priority is far more important than image quality —smooth control feel is a hundred times more important than pretty grass textures.

What to do when your phone heats up while gaming

- **: This is normal, but prolonged high temperatures will damage battery life. A few cooling methods: remove the phone case for better heat dissipation; lower screen brightness to around 70% (brightness is a major heat source); don't charge and play games at the same time (compounded heating kills the battery fast). If the phone gets too hot to hold, exit the game immediately and let it rest for a while.**Anti-addiction tools
- **: iOS "Screen Time" and Android "Digital Wellbeing" both let you set daily time limits for game apps. I set myself a maximum of one hour on weekdays and two hours on weekends. When the time is up, the app icon turns gray and can't be opened. This limit is genuinely useful, not just for show.**Cloud Gaming
- **: If your phone isn't powerful enough but you want to play big titles, you can try cloud gaming platforms (like Tencent START Cloud Gaming or NetEase**

: Don't just max everything out. First ensure frame rate (smoothness), then pursue image quality. 60 fps is much smoother than 30 fps, but 120 fps puts a lot of pressure on the phone —mid-range phones shouldn't force it. If you play competitive games (like Honor of Kings or PUBG Mobile), frame rate priority is far more important than image quality —smooth control feel is a hundred times more important than pretty grass textures.

Cloud Gaming): The game runs on a remote server, and the image is streamed to your phone as video. It demands a lot from your network but almost nothing from your phone's performance. The downside is that latency is unavoidable —suitable for strategy and RPG games, not for FPS shooters. Get Hands-On: Build Your Mobile Entertainment System

- **In your music app, download your 3 favorite albums or playlists to ensure offline listening.**Subscribe to 3 channels of interest on Xiaoyuzhou or Apple Podcasts, and download an episode to try.
- **Install VLC Player, and try using it to play a video file copied from your computer.**Find a book you want to read in WeChat Reading, set a warm background and eye-care mode, and adjust the font to a comfortable size.

If you play games, open Game Mode and see what settings you can optimize (do-not-disturb, frame rate priority, etc.).

1. Chapter 13: Recommended Tool Apps
2. This chapter selects tool apps that can truly boost efficiency, promote health, and facilitate travel.
3. 1. Productivity Tools: Manage Your Time and Attention
4. To-Do Lists
5. : System memo, Microsoft To Do (lightweight); TickTick, Todoist (full-featured).

Chapter 13: Recommended Utility Apps

This chapter picks productivity, health, and travel apps, plus practical system and cross-device tools: cleaners like Qingzhuo and spam blocking, built-in phone blocking switches, weather and earthquake alerts; remote desktop with Sunlogin/ToDesk, AI office tools like Trae, live wallpapers, LocalSend for LAN transfers, cross-device clipboard, Aegis/Microsoft Authenticator for one-time codes, TickTick schedules, and paper scanning.

Productivity: Manage your time and attention

Productivity apps are the “backbone” of digital life. They help you organize scattered ideas, tasks, and files. Picks by scenario below.

To-do lists (task management)

- TickTick. Best for: People who want work, life, and study tasks plus a calendar in one place. A top pick in this section. Highlights: Sync across phone/PC/tablet/web; to-dos + calendar + Pomodoro in one; natural language like ”meeting tomorrow at 3pm” or ”weekly report every Monday” auto-parses times and repeats; supports subtasks, tags, priorities, attachments, and shared lists. Great for ”schedule + daily check-ins + review” in one flow.
- Getting more out of TickTick:
- Calendar view: Drag meetings and deadlines onto the calendar to spot conflicts at a glance —more intuitive than a plain list.
- Recurring tasks: Set meds, bills, and workouts as repeating rules instead of retyping every time.
- Pomodoro: Start focus sessions on a task and see how much you actually finished this week.
- Phone integration: Say ”remind me⋯” to your voice assistant to create system reminders. Double up important deadlines in the system calendar too.

Todoist. Best for: Minimalists who work across platforms. Highlights: Clean, restrained interface; clear hierarchy (task subtask sub-subtask); karma points that nudge good habits.

Microsoft To Do. Best for: Office workers and Outlook users. Highlights: Completely free, no ads; "My Day" clears each morning and eases mental load; deep Microsoft ecosystem integration.

System Notes/Reminders. Best for: Light users who'd rather not install third-party apps. Highlights: Native experience, zero lag, add via Siri/Xiao AI, works with system widgets.

Pomodoro (focus training)

- Forest. Best for: People who can't keep their hands off their phone. Highlights: Grow a tree while you focus; leave the app early and the tree dies. Virtual life as positive feedback; plant together with friends.
- Tide. Best for: People who need white noise to focus or fall asleep. Highlights: Nature sounds (rain, waves, forest) + Pomodoro + meditation in one. Calm, healing, minimal interface.
- The rule of thumb: Classic 25 minutes focus + 5 minutes break. Adjust to 50/10 or 90/20 to match your energy.

Calendars (visualize your time)

- System calendar. Best for: Everyone. Highlights: Most reliable native alerts, very low battery use, add via Siri/Xiao AI.
- Google Calendar. Best for: Multi-device users and shared schedules. Highlights: "Goals" auto-schedules fitness/study time; Gmail smartly pulls events from email.

Notes (capture knowledge)

- Notion. Best for: People who want notes, tasks, databases, and wikis in one place — students, freelancers, teams. Highlights: All-in-one knowledge base with block editing; build homepages, reading lists, project boards; rich template ecosystem.
- Obsidian. Best for: Serious knowledge workers, researchers, writers. Highlights: Local Markdown storage, backlinks + graph view —let your notes form a "second brain." Your data is fully yours.
- flomo. Best for: Capturing quick ideas without writing long essays. Highlights: Card-style notes, as easy as posting a social update; # tags for grouping; browse by timeline.
- Apple Notes. Best for: iPhone/Mac users with light note-taking needs. Highlights: Fast launch; scans documents, sketches, checklists; iCloud sync. Free and deeply integrated.

- Evernote. Best for: Power users who clip lots of web pages and archive materials. Highlights: Powerful web clipper, OCR for text in images; long-standing and stable —but Premium is paid.

PDF tools

- PDF Expert. Best for: iPad/iPhone users, professionals annotating contracts, students. Highlights: Widely considered the best PDF editor on iOS —smooth markup, signing, merge/split, form filling.

- WPS. Best for: Office users juggling Word/Excel/PPT/PDF. Highlights: One-stop office suite, plays well with Chinese cloud drives and WeChat files; free tier is plenty for daily use.

- Adobe Acrobat. Best for: People who need strict PDF compliance (legal, design, publishing). Highlights: Official standard, reliable PDF/A conversion, professional cross-device sync and e-signatures.

Scanning (digitize paper)

Paper contracts, receipts, homework, whiteboard notes, ID copies⋯your phone camera can replace most scanners. Core flow: Shoot clearly auto crop & perspective fix export PDF/image optional OCR.

- CamScanner. Best for: Business people and students who often scan contracts, receipts, and IDs. Highlights: Auto edge detection, perspective correction, OCR; multi-page PDFs with cloud sync.

- Microsoft Lens. Best for: Students who need whiteboards/pages as Word docs. Highlights: Free, no ads, one-tap export to OneNote/Word, accurate text recognition.

- Apple Notes scanning. Best for: iPhone users with light scanning needs. Highlights: No extra app —tap the camera icon in Notes; auto shadow removal, sign and markup. Free and solid.

- Baimiao / CamScanner web / brand camera ”Document Scan”: Huawei, Xiaomi, OPPO, and vivo cameras mostly include a Documents/Scan mode. Android can also use Google Camera or the system scanner. Handy for snapping a one-page contract in a hurry.

- Shooting tips: Even lighting, keep the phone parallel to the page, aim at all four corners. For multi-page docs, shoot in order then merge to PDF. Watermark sensitive IDs and watch where you store them; clear them from ”Recently Deleted” when done.

Health and fitness

Health apps are the phone's most underrated "personal assistant." They track sleep, heart rate, and activity continuously —reflecting long-term trends better than the occasional hospital visit.

Workouts

- Keep. Best for: Home workouts, total beginners, fat loss and toning. Highlights: Huge free course library from HIIT to yoga; AI builds your training plan; lively community.
- Nike Training Club (NTC). Best for: People who want professional coaching and structured training. Highlights: Official Nike app, all courses free, demos by real coaches, organized by muscle group/equipment.
- Apple Fitness. Best for: Apple Watch users who want ring-based motivation. Highlights: Three daily activity rings (Move/Exercise/Stand), visual goals, seamless heart-rate data from the watch.

Sleep tracking

- AutoSleep. Best for: Apple Watch users who want deep/light sleep breakdowns. Highlights: Tracks automatically while you sleep with the watch on —no start/stop; quality score and bedtime tips.
- Sleep Cycle. Best for: People who sleep without a watch and want phone-based tracking. Highlights: Uses the mic to analyze movement and snoring; smart alarm in light sleep so waking up hurts less.
- Mi Fitness. Best for: Xiaomi Band/watch users. Highlights: Free and tightly integrated with Xiaomi bands; heart rate, SpO2, and sleep in one; great value.

Meditation and relaxation

- Calm. Best for: Stressed professionals and people who struggle to fall asleep. Highlights: "Daily Calm" guided meditations + Sleep Stories (celebrity narrators); beautiful interface; rich English content.
- Tide. Best for: Chinese-speaking users who need white noise to focus or sleep. Highlights: Polished domestic app —nature sounds + meditation courses + Pomodoro; friendly Chinese guidance.
- Little Sleep. Best for: People who need sleep sounds and want to record sleep talk/snoring. Highlights: Stackable white noise (rain + fan + campfire), sleep timer; sleep-talk recordings that are fun and useful.

Women's health

- Flo. Best for: Cycle tracking, trying to conceive, and women's health. Highlights: Accurate AI predictions for periods/ovulation; anonymous community Q&A; polished design.
- Meiyou. Best for: Users in China who want Chinese community content and education. Highlights: Rich local content tailored by stage —period/TTC/pregnancy/parenting.

Medication reminders

- Medication Assistant. Best for: People on long-term meds and adult children managing parents' medicine. Highlights: Scan to ID drugs, auto-build a med list, pop-up reminders, share adherence logs with family.

Data hubs

- Apple Health / Google Fit. Best for: All smartphone users. Highlights: Aggregates data from your phone, band, and third-party health apps (steps, heart rate, sleep, cycles) into one "health record center."

Travel and getting around

On the road, a few good apps save you from countless headaches. Picks by stage of the trip below.

Flights and trains

- Umetrip. Best for: Frequent flyers. Highlights: Official CAAC data —most authoritative flight status, gate changes, delay alerts; aircraft type and seat maps.
- 12306. Best for: Anyone taking trains in China. Highlights: The only official ticket channel; waitlist purchases (auto-buys when someone refunds); easy student/group tickets.
- VariFlight. Best for: People sensitive to delays and those doing airport pickups. Highlights: Accurate delay predictions; share your trip so family can watch your location and ETA.

Maps and navigation

- Amap (Gaode). Best for: City users in China, drivers and ride-hailing. Highlights: Accurate live traffic, great at avoiding jams; ride-hailing one-stop; rich transit transfer options.

- Baidu Maps. Best for: People who need detailed POIs and indoor navigation. Highlights: Street View/AR navigation; detailed indoor maps for malls and attractions; fun voice packs.
- Google Maps. Best for: International travelers. Highlights: Best global coverage, honest local reviews; Street View and offline maps are lifesavers abroad.

Translation

- Google Translate. Best for: Travelers and cross-language learners. Highlights: "Conversation" real-time two-way translation and camera instant lookup —travel essentials; offline language packs supported.

Currency conversion

- Minimalist Currency. Best for: Travel, overseas shopping, cross-border buys. Highlights: Ultra-clean interface, 160+ currencies, live rates, multi-currency compare; long-press a number to convert quickly.

Weather

- System Weather / China Weather. Best for: Everyone —the first stop for forecasts and official alerts. Highlights: OEM weather apps and official channels like "China Weather" push rainstorm, typhoon, and heat alerts as banners/notifications. Authoritative.
- Caiyun Weather. Best for: People who need minute-level rain forecasts. Highlights: Industry-leading "will it rain in the next two hours" accuracy; animated rain map; proactive alerts before rain starts.
- Carrot Weather. Best for: Weather fans who want fun and rich widgets. Highlights: Snarky AI voice reports, tons of custom widgets, pro radar. Paid version unlocks more.
- Alert essentials: Weather apps need notification permission to deliver alerts. Alerts come in blue, yellow, orange, and red; red means the highest risk of severe weather —adjust your plans. See Section 7 of this chapter for more setup tips ("Weather alerts / disaster alerts / earthquake alerts").

Travel guides

- Mafengwo. Best for: Detailed planners, road trips, deep travel. Highlights: Rich user trip reports; thorough destination tips/routes/what to avoid; downloadable offline guides.
- Xiaohongshu (RED). Best for: Young travelers hunting real experiences, hidden spots, and food. Highlights: Visual posts and short videos; fast-updating "3 days in XX city" guides; great search.

Automation for beginners

Automation is leverage: configure once, save effort for a long time —compressing dozens of daily taps to zero. This section is just a concept intro; see Chapter 23, “Computational Thinking and Automation,” for details.

- iOS Shortcuts: Apple's official automation tool. No coding needed —chain actions like "when I get home: turn on Wi-Fi + dim the screen + read the weather" with one tap.
- Android automation: Built-in scene tools (like Xiaomi "Automations" or Huawei "Smart Scene") are the easiest to start with. Tasker is powerful but has a steep learning curve —better for tinkerers.
- Core idea: First spot the things you repeat every day, then ask "can one trigger + several actions replace this?" For detailed examples and advanced tricks, see Chapter 23.

Browsers and search engines: Your gateway to information

A browser is your window to the internet; a search engine is your compass in the sea of information. Use them well and you'll avoid a lot of detours.

How a browser works:

When you type a web address, the browser: (1) asks a DNS server for the IP behind the domain; (2) sends an HTTP/HTTPS request to that server; (3) the server returns page code (HTML, CSS, JavaScript); (4) the browser parses and renders it into the page you see.

Recommended browsers:

- Microsoft Edge: Chromium-based, fast and compatible. Built-in Copilot AI can summarize pages, translate, and chat. Cross-device sync. Strong personal recommendation as a main browser.
- Safari: Apple's default browser, deeply integrated and power-efficient. Good privacy, but fewer extensions.
- Chrome: Most widely used worldwide, with a huge extension library. But it's memory-hungry and drains phone batteries faster.

Choosing a search engine:

- Bing: Microsoft's search engine, tightly integrated with Edge. AI summaries give answers, not just links —very efficient. Image search is solid too.

- Yandex: Russia's largest search engine, strong at image search and reverse image search. Great for finding similar images or identifying objects.
- Baidu: The main option for searching in China, but it shows a lot of ads —you need a critical eye.
- Google: The most precise search engine globally, but it needs special network access inside China.

Guide to efficient research:

- Use AI search: Bing's AI summaries and Edge's Copilot can answer questions directly and pull key facts out of huge result lists —much faster than clicking link after link.
- Spot ads and false info: Results marked "Ad" or "Sponsored" are paid placements, not organic. Check the source's authority and prefer official sites and known media.
- Download apps safely: Search "app name + official site," like "WeChat official site," and click results with the official domain. Avoid third-party download sites —they often bundle junkware.
- Spotting the real official site: For Windows 11 go to microsoft.com; for Chrome go to google.com. Matching the domain to the brand is the most basic check.

Website domain basics:

A domain is a site's "street address." Reading it helps you judge what a site is and where it comes from:

- Common suffixes:
- .cn —China's country domain
- .org —Nonprofit organizations
- .net —Network service providers
- .edu —Educational institutions
- .gov —Government agencies
- .xyz/.tech/.info —Newer generic domains, cheap to get, use caution

Country/region domains:

.kr —Korea

.uk —United Kingdom

.de —Germany

.tw —Taiwan

.hk —Hong Kong

Safety tip: .gov and .edu sites are usually more trustworthy because registration is strict. Cheap domains like .xyz and .top are easy for phishing sites to grab —stay alert.

Ad blocking:

- AdGuard. Best for: Anyone annoyed by pop-ups, banner ads, and in-app ads. Highlights: System-wide ad blocking (not just the browser), filters in-app banners and splash tracking, saves data, speeds up pages. Paid version covers all devices.

Privacy browsers:

- Brave. Best for: Everyday browsers who care about privacy and hate ad tracking. Highlights: Chromium-based, blocks ads and trackers out of the box, built-in Tor private windows, faster and lighter than Chrome.

- DuckDuckGo. Best for: People who don't want one search to follow them everywhere with ads. Highlights: Built-in search engine (no search history), one-tap "Fire Button" clears all traces, consistent cross-platform experience.

Browser comparison:

Browser	**Highlights**	**Best for**
Microsoft Edge	Chromium + built-in Copilot AI: summarize pages, translate, chat	Productivity-focused office users, heavy AI users (recommended main browser)
Safari	Deep Apple integration, power-efficient, strong privacy	iPhone/Mac users who care about battery and system integration
Chrome	Most widely used globally, richest extension ecosystem	Power users who need many extensions, cross-device sync fans
Brave	Blocks ads and trackers out of the box, faster	Privacy-minded users who want ad blocking without installing add-ons
DuckDuckGo	Search engine keeps no history; one tap clears traces	Privacy-sensitive users who don't want personalized tracking

Try building a ”wind down before bed” automation

1. Create a trigger: a set time (say 22:30).
2. Add actions: lower volume > turn on Do Not Disturb > play a chill playlist > dim the screen.
3. Save and test.

Useful system tools: Cleanup, spam blocking, and safety

Beyond productivity and health apps, daily life often comes down to three things: fewer interruptions, enough storage, and keeping scammers out. This section first covers handy utilities, then zooms in on built-in phone spam blocking —many people own a phone for years and never flip these switches on.

System cleanup and storage

- Qingzhuo. Best for: Android users who want a lightweight way to clear cache, leftover files, and empty folders without a bloated ”phone manager” suite. Highlights: Tiny, no ads; deep scan for leftovers, large-file finder, storage breakdown. More restrained than commercial ”one-tap boosters” —good for hands-on users doing regular maintenance. Note: Check paths before cleaning. Don’t blindly select WeChat, gallery, or document folders. Prefer the brand’s built-in storage cleaner for system caches; treat third-party tools as supplements.
- Built-in storage cleaner. Best for: Everyone, especially older users and people who want zero hassle. Highlights: Android [Settings] > [Storage] > [Clean up]; iOS [Settings] > [General] > [iPhone Storage]. The system knows its own app cache structure best and is least likely to delete something important. Make this your first choice.
- WeChat/QQ built-in storage tools. Best for: Heavy chatters. Highlights: WeChat [Me] > [Settings] > [General] > [Storage] —clean by chat or file type. Safer than outside ”WeChat cleaner” apps.
- Rule of thumb: If the system or app has a built-in option, use that instead of a third party. If you can select by path or type, avoid ”clean everything.” The goal is freeing space —not wiping important data along with it.

Anti-spam and safety utilities

- National Anti-Fraud Center. Best for: Everyone —especially parents, finance-related jobs, and people who shop online or take unknown calls often. Highlights: Official app; call warnings, SMS risk tips, suspicious URL checks. Linked to police anti-fraud databases —the most official layer of the ”three lines of defense.”

- Brand security centers / phone managers. Best for: Users of that brand. Highlights: Huawei Phone Manager, Xiaomi Security Center, OPPO Phone Manager, vivo Phone Manager, and so on. Bundle spam blocking, virus scans, permission management, and payment protection. Usually free and better integrated with the system —don't install a second commercial security suite on top and cause permission clashes.

- Carrier spam blocking. Best for: People bombed by robocalls and AI voice calls. Highlights: China Mobile: text KTFSR to 10086. China Telecom: follow "Tianyi Anti-Harassment." China Unicom: set up in "Unicom Mobile Manager." Blocking happens on the carrier side, so calls never ring —a batch of spam numbers gets stopped before they reach you.

- AdGuard. Best for: Anyone annoyed by pop-ups, banner ads, and in-app ads. Highlights: System-wide ad blocking (not just the browser), filters in-app banners and splash tracking, saves data, speeds up pages. Paid version covers all devices.

- Be careful with "cleaner / spam blocker" all-in-one suites: Some third-party managers sit in the background and keep asking for contacts and SMS permissions. Their blocking may be no better than the system's, and they burn battery and raise privacy risks. Prefer built-in features first, then the official anti-fraud app.

How to pick utility apps

- System first: For blocking, cleanup, and permissions, turn on the phone's built-in switches first.

- Official extras: The National Anti-Fraud Center and carrier spam blocking are official channels you can layer on.

- Go easy on third parties: One app per job is enough. Avoid permission fights and background resource grabs.

- Less is more: The more tools you install, the bigger the notification and permission surface —and the more "the tools themselves" interrupt you.

Built-in spam blocking: How to turn it on by brand

Entries by brand below. Names may vary slightly by system version —search Settings for "spam," "block," or "unknown callers." This pairs with Chapter 8 on spam calls: that chapter covers number labeling and mislabel appeals; this one focuses on where the switches live.

Platform	Where to find it	Recommended switches
iPhone / iOS	[Settings] > [Phone] / [Messages]	Silence Unknown Callers; Blocked Contacts; "Filter Unknown Senders" in Messages
Xiaomi / Redmi	Phone app > Settings > Spam blocking; or Security app	Smart blocking, block one-ring hangs, SMS keyword filter
Huawei / Honor	Phone > Spam blocking; Phone Manager > Spam blocking	Caller ID, blacklist, smart SMS blocking
OPPO / realme / OnePlus	Phone app > Spam blocking; or This phone > Spam blocking	Call blocking, scam call detection, SMS filter rules
vivo / iQOO	Phone > Spam call blocking; i Manager	Spam blocking, scam call warnings, message filtering
Samsung	Phone > Settings > Block numbers / Caller ID and spam protection	Block unknown numbers, spam call ID (varies by region)

Layering three lines of defense

Everyday setup: "system blocking + carrier spam blocking + National Anti-Fraud Center." The system handles labels and blacklists; the carrier stops robocalls on the network; the anti-fraud app adds official warnings. You don't need max settings on all three —or you'll block couriers, food delivery, and your doctor. Whitelist contacts and recent calls.

Hands-on: Turn on spam blocking today

1. Open Phone app settings, find "Spam blocking / Silence Unknown Callers," and turn on smart blocking.
2. Do the same on your parents' phones: same path + enable carrier spam blocking (China Mobile: text KTFSR to 10086).
3. Install "National Anti-Fraud Center" from the app store, sign up, and turn on call warnings.
4. Call a known spam test number (or check the block log) to confirm it works.

Weather alerts, disaster warnings, and earthquake early warnings

Spam blocking stops phone scams; early warnings stop natural disasters and extreme weather. Many people install a weather app just for the temperature, but never turn on notifications or earthquake warnings. In the seconds to tens of seconds before a red rainstorm alert, typhoon, or quake, your phone could buy you precious time to get to safety.

Weather alerts: Rainstorms, typhoons, heat

- Official channels first: Alerts from the China Meteorological Administration, National Meteorological Center, and provincial bureaus are the most authoritative. Follow the "China Weather" WeChat account and Weibo, or install official apps like "China Weather."
- System weather apps: Huawei Weather, Mi Weather, OPPO Weather, vivo Weather, Apple Weather, and others usually push local observatory alerts. Open the weather app settings and make sure "Severe weather alerts" notifications are on.
- Minute-level rain apps like Caiyun Weather: Handy before commuting or school pickup to decide "umbrella or not in the next 30 minutes." For official warnings, still rely on the meteorological authority.
- Alert colors (general):
- Yellow: Be extra careful; cut unnecessary outdoor activity.
- Orange: Serious. Stay indoors if you can and keep away from danger zones.
- Red: Extremely serious. Follow closures and evacuation guidance. Don't push through flooded roads, flash-flood areas, or high-wind zones.

Common alert types: rainstorm, blizzard, typhoon, heat wave, cold snap, gale, lightning, hail, icy roads, fog, drought, geological hazard risk.

Natural disaster warnings: Not just "rain"

- Who issues them: Weather bureaus handle meteorological alerts. Geological hazards and flash floods are often issued jointly by natural resources, water resources, and emergency management departments. Local governments also push via emergency radio, SMS, and 12379.
- Get warnings on multiple channels:
- Follow local "Emergency Management" and "Weather Bureau" accounts on WeChat.

- 12379 national alert channels (SMS/services, depending on your region).
- TV, emergency radio, village/community loudspeakers —in mountains or flood season, don't rely on your phone alone.

Outdoors and travel tips: In a typhoon, stay away from billboards and trees. In heavy rain, avoid underground garages, tunnels, and underpasses. In mountain areas, if a "geological hazard weather risk" alert is issued, leave steep slopes and valleys immediately.

Pair with the emergency handbook: For post-disaster contact loss, dead phones, or location-based rescue calls, see Chapter 32 (earthquake/disaster scenarios) and Chapter 7 (satellite messaging without a network).

Earthquake early warning: Not the same as "earthquake prediction"

Earthquake Early Warning (EEW) pushes an alert seconds to tens of seconds before damaging waves (especially S-waves) reach you. It uses the fact that radio waves beat seismic waves and stations detect P-waves first. It cannot predict whether a quake will happen, but it can buy time to "take cover nearby, cut fire sources, get out of elevators." Closer to the epicenter, the warning window shrinks —sometimes to nothing.

- Official and industry options: In mainland China, common sources include capabilities tied to the China Earthquake Networks Center and services promoted by groups like the Institute of Care-Life (Chengdu). Some phones ship with or can receive these warnings online.
- Third-party apps: Search the app store for "earthquake warning" apps from official or research bodies. Check the developer before installing —avoid fake "quake predictor" apps that demand a pile of unrelated permissions.
- What to do when an alert sounds: Indoors, take cover nearby (under a table, inside corners of load-bearing walls, next to sturdy furniture) and protect your head and neck. Stay away from windows, exterior walls, and tall furniture. Don't blindly rush the stairs or jump. If you're in a lift, get off at the nearest floor. Outdoors, move to open ground away from power lines, billboards, and unstable buildings. After the shaking, check gas and wiring and watch for aftershocks.
- Don't silence it: Earthquake alerts should be allowed to break through silent/Do Not Disturb. Keep your phone within reach at night, and don't set the ringer to zero.

Built-in earthquake warnings: How to enable by brand

Entries below may vary by system version. Search Settings for "earthquake," "early warning," or "safety alert." iOS has limited built-in support for warnings in China —you'll usually need a reliable third-party app with location and notification permissions.

Platform	Common path	Notes
Xiaomi / Redmi	Phone Manager / Security > Earthquake warning; or search "earthquake" in Settings	Some versions include a built-in warning service; needs internet and location
Huawei / Honor	Settings > Safety > Emergency alerts / Earthquake warning	May be named "Emergency alerts" and cover earthquakes among others
OPPO / realme / OnePlus	Settings > Safety (SOS emergency) > Earthquake warning	Depends on model and regional service coverage
vivo / iQOO	Weather app or Settings > Safety/More > Earthquake warning	Follow the menus on your actual system version
iPhone / iOS	Often no full built-in switch; install a reliable earthquake warning app and allow notifications	App needs background run and location; don't block its alerts in Do Not Disturb
Samsung, etc.	Settings > Safety and emergency / Wireless emergency alerts	"Emergency alerts" coverage varies by region and carrier

Common myths about earthquake early warning

- Warning ≠ prediction: It can't tell you "will there be a quake tomorrow." It only alerts after a quake starts and before the waves arrive.
- You don't always get many seconds: Near the epicenter you may get only a few seconds —or none. Warnings mainly help surrounding areas.
- Mute it and it's useless: Even the best warning fails if it's silenced, killed in the background, or denied location.
- Bogus apps: Be wary of "earthquake predictor" apps that want contacts and SMS. Prefer built-in system options or reputable institutional apps.

Putting warnings together

Daily setup: "system weather alerts + official weather/emergency channels + earthquake warning (system or trusted app)." Like spam blocking, you don't need a pile of similar apps. What matters: notifications on, location allowed, and alerts that can break through Do Not Disturb. After you finish spam blocking on your parents' phones, check their weather and earthquake alerts too.

Hands-on: 10-minute warning settings check

1. Open the system weather app, go to settings, and turn on "Severe weather alerts" notifications.
2. Search Settings for "earthquake" or "warning" and enable earthquake/emergency alerts (Android). On iPhone, install a trusted earthquake warning app and allow notifications.
3. Follow your local weather bureau or emergency management account on WeChat and allow its pushes.
4. Confirm alerts can still sound with Do Not Disturb on. Repeat the check on your parents' phones.

Cross-device teamwork: Remote control, fast transfer, account safety

A phone feels smooth when it teams up with your computer and tablet. This section adds high-frequency tools: remote-control your PC from a phone, AI office work and remote collaboration, live wallpapers, LAN file transfer, cross-device clipboard, authenticator 2FA, and linking paper scanning with schedules. For systematic multi-device setups, see Chapter 17.

Control your computer from a phone: Sunlogin / ToDesk

- Sunlogin Remote Control. Best for: People who need to operate a home/office Windows or Mac while out —also popular for helping parents fix phones remotely (see Chapter 27). Highlights: Mature domestic ecosystem; phone app controls the desktop, transfers files, and can power on/off (some features need hardware). Free tier is enough for light personal use.
- ToDesk. Best for: People who want stable, low-latency remote work/ops. Highlights: Device ID + password connection; phone controls PC (and sometimes PC controls phone, depending on version); file transfer and multi-monitor switching feel smooth. Domestic nodes usually perform well.
- Basic setup: Install the client on the computer and sign in / stay online install the app on the phone connect with device ID + access password (or the same account) use the phone as a trackpad/gestures for the desktop.
- Safety rules (memorize these):
- For personal use: Turn on two-step verification, change default passwords, sign out or disable unattended access when done. On work computers, follow company security

policy.

- In public, remote work carefully —watch for people peeking at your screen and passwords.

AI office apps: Trae / WorkBuddy / Dumate

- Trae. Best for: Coding and document collaboration, friendly in Chinese environments. Highlights: ByteDance's AI IDE/assistant; SOLO mode can spin up a project from one sentence. Efficient for daily coding and document edits. If it runs on a computer, you can pair it with remote control or official companion features on mobile to check/follow tasks (check the latest product form). See Chapter 41 for the AI coding tools landscape.
- WorkBuddy. Best for: Office users who want AI help with meeting notes, email, reports, and work docs. Highlights: An AI assistant aimed at office scenarios —great for draft polish summarize flows. Start some workflows on the phone; leave heavy lifting to the computer.
- Dumate. Best for: Teams/individuals who want multi-person collaboration with docs and knowledge management in one place. Highlights: AI collaboration office tool focused on gathering materials and co-editing. View and lightly edit on the phone; do complex layout on a bigger computer screen.
- What "phone remote-controls computer" means here: Many AI office features run on desktop/web. From your phone, connect back via Sunlogin/ToDesk or use the product's own mobile app/webpage to "remote" AI tasks on the computer during your commute. Rule: heavy compute and long docs on the computer; check progress and tweak a few lines on the phone.
- Privacy note: Before uploading contracts, ID cards, or customer data, read the product's privacy terms and company compliance rules. For work materials, prefer tools your organization specifies.

Live wallpapers: miHoYo Desktop Mate (phone + PC)

- Desktop Mate. Best for: Fans of miHoYo characters who want free live wallpapers. Highlights: Free live wallpaper app from miHoYo on phone and PC. Character Live2D/animated performances can sit on your desktop; some devices/systems allow lock screen or home wallpaper use.
- Battery and performance: Live wallpapers use more power and GPU than static images. On low battery or older phones, use them only while charging, or lower the frame rate if the app offers it.
- Install source: Prefer the official site or a trusted app store. Avoid third-party "cracked" builds with bundled malware. On PC, watch the install path and whether auto-start is needed.

Fast LAN file transfer: LocalSend / Feige / AirDrop

- LocalSend. Best for: Mixed Android/Windows/iOS/Mac/Linux setups that want cross-platform transfers without public cloud. Highlights: Open source, direct LAN connection. Send photos, videos, and docs on the same Wi-Fi —no cable, no vendor servers. Speed depends on your router.

- Feige (IPMsg). Best for: Office Windows PCs and older LAN office setups. Highlights: A classic LAN transfer/chat tool, good for big files on an intranet. Download from a trusted source; on company networks follow IT policy.

- AirDrop. Best for: iPhone, iPad, and Mac users. Highlights: One-tap sharing in the Apple ecosystem. Bluetooth discovery + Wi-Fi Direct, no mobile data. Set "Contacts Only" to reduce strangers pinging you.

- Also worth knowing: Mi Share, Huawei Share, vivo Share, and similar brand tools are very fast within the same brand. WeChat "File Transfer Helper" is convenient but limited by file size and signed-in devices. For big files, prefer LocalSend/AirDrop.

- Tips: Both devices on the same Wi-Fi. Public Wi-Fi is risky for sensitive files —prefer a personal hotspot. After transfer, confirm the file opens completely on the receiving side.

Cross-device clipboard: KDE Connect / brand phone-PC link

- Apple Universal Clipboard. Best for: iPhone + Mac users. Highlights: Sign in with the same Apple ID and enable Handoff —copy on the phone, paste on the Mac (and vice versa). Works for text and some files.

- KDE Connect. Best for: Android + Windows/Linux/Mac users who want open-source clipboard sync, notification mirroring, and remote input. Highlights: Pair on the LAN to sync clipboards, push phone notifications to the PC, use the phone as a slide clicker/trackpad. Data doesn't go through third-party accounts.

- Brand phone-PC links: Huawei Multi-Screen Collaboration / Super Device, Mi Continuity, vivo Share, OPPO Connect, Honor Connect, and others often include cross-screen clipboard and drag-and-drop. Best experience within the same brand.

- Windows + Android official path: "Phone Link / Link to Windows" (evolved from Your Phone) lets you see phone notifications and photos on the PC and copy/paste in supported cases. Deeper on Samsung and some other models.

- Safety note: Clipboards can retain passwords and codes. On shared computers, be careful with global sync. After pasting into a password field, avoid a second paste that leaks it. Some keyboards also read the clipboard —check permissions.

Authenticator 2FA: Aegis / Microsoft Authenticator

- Why an authenticator instead of SMS: SMS codes can be hijacked via SIM swap. TOTP codes are generated locally on a time algorithm, don't need phone signal, and resist phishing/interception better (still protect against malware and screenshots).
- Aegis Authenticator. Best for: Android users who prefer open source and local control. Highlights: Free and open source, encrypted backup and import/export, clean UI, unlock token list with fingerprint/face.
- Microsoft Authenticator. Best for: Microsoft account, Office 365, and work account users; works on iOS/Android. Highlights: Push approval for sign-ins, password monitoring, and account backup (depending on version). Tightly integrated with Windows and enterprise identity.
- Google Authenticator / 2FAS, etc.: Common in their ecosystems too. What matters is encrypted backup so you don't lose tokens when you switch phones.
- Must-do when switching phones: Transfer from the old device, or use the service's "backup codes / recovery key." Print backup codes and store them in two places — not just on the same phone.
- Password managers: Some password vaults can store TOTP too. Convenient, but risk is concentrated. For high-value accounts, split the job: "password manager + separate authenticator." See Chapter 14.

Suggested combo: How a day fits together

Example workflow

- Morning: Check today's schedule in TickTick glance at weather/alerts open Trae/docs on the computer, phone aside.
- Daytime: Scan paper materials to PDF with a scanning app, send to the computer via LocalSend/AirDrop; draft long docs on the computer with AI office tools.
- Out and about: Connect back to the computer with ToDesk/Sunlogin to continue tasks; sign in to important accounts with Aegis/Microsoft Authenticator codes.
- Evening: Check off done items in TickTick; if you like, change the wallpaper with Desktop Mate to unwind.

Remote control and account safety rules

- If a stranger asks you to install remote-control software hang up and find official support yourself.
- Never tell anyone your verification codes or authenticator codes —not even "couriers"

or ”support.”

- Before remote-controlling a work computer or installing AI software on it, ask IT first.
- Be extra careful transferring files or logging in remotely on public networks.

Hands-on: Run the cross-device chain once

1. Put phone and computer on the same Wi-Fi and send a photo with LocalSend (or AirDrop).
2. Create a TickTick task ”Tonight 8pm: organize scans” and confirm it shows on both phone and computer.
3. Turn on 2FA for your email or WeChat (if supported), bind it to Aegis or Microsoft Authenticator, and save backup codes.
4. Install Sunlogin or ToDesk on your own computer, connect once from your own phone, then sign out or strengthen the password —learn the flow without leaving a back door.

Chapter 14: Password Managers

In the digital age, your passwords are the keys to your front door. But with too many keys, all too complex—what then? A password manager is your "universal key butler".

I. Why Do You Need a Password Manager?

Modern people have dozens or even hundreds of online accounts on average. If every account uses the same password, one hacked website puts all accounts at risk. If every account uses a different complex password, they are impossible to remember.

A password manager solves this dilemma: you only need to remember one master password, and it remembers all the others for you.

How Password Managers Work

1. **Encrypted Storage**: All passwords are stored with high-strength encryption, and only you can decrypt them.

2. **Master Password**: You set a master password to unlock the password vault. The master password itself is not stored on the server.

3. **Auto-Fill**: On website or app login pages, the password manager automatically fills in your account and password.

4. **Password Generation**: Can generate random, complex passwords to improve security.

III. Introduction to Encryption Principles

Understanding the encryption principles behind password managers helps you judge whether a service provider is trustworthy. Below are several core concepts explained in plain language.

AES-256 Encryption

AES (Advanced Encryption Standard) is a symmetric encryption algorithm certified by the U.S. National Institute of Standards and Technology (NIST), widely used by banks, military, and government. "256" refers to the 256-bit key length —it would theoretically take longer than the age of the universe to brute-force. Mainstream password managers (1Password, Bitwarden, KeePass, etc.) all use AES-256 to encrypt your password vault.

Zero-Knowledge Architecture

"Zero-knowledge" doesn't mean the service provider stores nothing —it means that**encryption and decryption are both done locally on your device, and the server only stores encrypted ciphertext**. The specific process:

1. You enter your master password in the client.
2. The client derives an encryption key from the master password (this happens on your phone/computer).
3. Only the encrypted ciphertext is uploaded to the server.
4. On next login, the server sends back the ciphertext, and the client decrypts it again using your master password.

Key Point:**The master password itself never leaves your device**, and the server doesn't have it either. So even if the service provider is hacked, the attacker only gets a pile of unbreakable ciphertext —unless they can guess your master password.

Why does "zero-knowledge" mean the service provider can't see your passwords either?

Because the decryption key is derived from your master password, which exists only in your mind and is used only on your local device. The service provider's server has neither the master password nor the decryption key —only ciphertext. That's why legitimate providers like Bitwarden and 1Password explicitly state "we cannot see your passwords either" — it's not that they don't want to, it's that they technically can't.

Key Derivation Functions: PBKDF2 and Argon2

The master password can't be used directly as an encryption key —it needs to be processed by a "key derivation function." The purpose of this function is to**deliberately slow down computation**, making the cost of brute-force attacks prohibitively high.

- **PBKDF2**: A well-established standard, widely used by iCloud Keychain, Bitwarden, LastPass, and others. Slows down cracking through repeated iteration (typically 100,000+ times) of hashing.

- **Argon2**: The winner of the 2015 Password Hashing Competition, more modern than PBKDF2. In addition to "computational cost," it introduces "memory cost" —making it difficult for specialized hardware like GPUs/ASICs to accelerate cracking. Already supported by Bitwarden, KeePassXC, and others.

Higher iteration counts are more secure but also slower to unlock. Legitimate providers publicly disclose their derivation algorithm and iteration counts for security community auditing.

Case Study: The LastPass 2022 Data Breach

In August 2022, the veteran password manager LastPass announced that its development environment was breached; in November of the same year, it further disclosed that hackers had taken backups of users' password vaults (encrypted ciphertext). Key lessons from the incident:

- **Zero-Knowledge Architecture Held the Line**: Since the password vault was encrypted, hackers got the ciphertext but couldn't decrypt it —provided the user's master password was strong enough.
- **Users with Weak Master Passwords Are Most at Risk**: Hackers can brute-force weak master passwords offline, so users with weak or reused master passwords face real risk.
- **Metadata Was Also Leaked**: Although password contents were encrypted, metadata like site URLs and notes may not have been, revealing which services the user used.

Lessons Learned from LastPass

First, your master password must be long and unique (16+ characters recommended, with upper/lower case, digits, and symbols); second, prioritize providers that support Argon2id with high iteration counts; third, always enable independent two-factor authentication for high-value accounts (bank, email) —don't rely solely on a password manager.

IV. Mainstream Password Manager Recommendations

System Built-in:

- **iCloud Keychain**: The first choice for Apple ecosystem users. Seamlessly integrated with Safari and apps, free, secure. The downside is that cross-platform use is inconvenient.
- **Google Password Manager**: Convenient for Android users. Integrates with Chrome and the Android system, free.
- **Huawei Password Vault**: Built into HarmonyOS, suitable for Huawei users.

Third-Party Professional Tools:

- **Bitwarden**: Open source and free, powerful, supports all platforms. Highly recommended for privacy-conscious users.
- **1Password**: Beautiful interface, rich features, suitable for families and teams. Paid.
- **LastPass**: A long-established password manager, with limited features in the free version.
- **KeePass**: Completely local storage, open source and free. Suitable for geek users.

V. Real Password Strength Assessment

Password managers generally display a "password strength" indicator bar (red/yellow/green), but this assessment often only considers length and character variety, which isn't comprehensive. To truly judge whether a password is secure, you need to consider "entropy" and "whether it has been leaked."

Entropy: The Scientific Measure of Password Strength

"Entropy" measures the uncertainty of a password, measured in bits. The higher the entropy, the more attempts are needed for brute-force cracking.

- **"123456"**: Extremely low entropy, cracked instantly.
- **"P@ssw0rd"**: Appears complex, but appears in all dictionary attack lists, so entropy is still low.
- **"correct horse battery staple"**: Four random words, approximately 44 bits of entropy, far harder to crack than "P@ssw0rd."
- **16 random characters (including upper/lower case + digits + symbols)**: Approximately 100 bits of entropy, uncrackable by brute force with current computing power.

Conclusion: **"Length + randomness" is more important than "complex symbols"** . This is why the random long passwords generated by password managers are truly secure.

HaveIBeenPwned: Check If Your Password Has Been Leaked

Maintained by security researcher Troy Hunt,haveibeenpwned.comit collects accounts and passwords leaked in major data breaches. You can:

1. Enter your email to check if your account was exposed in a breach.

2. Use the Pwned Passwords feature to check if a password is in the known leaked database (password managers and browsers have this built in).

If a password appears in the leaked database,**no matter how complex it looks, you must change it immediately**—because dictionary attacks prioritize these known passwords.

k-Anonymity Query Principle

Worried that entering your password on HaveIBeenPwned will cause another leak? No need to worry. Modern queries use the "k-anonymity" protocol: the browser only sends the first 5 characters of the password hash to the server, the server returns all hashes starting with those 5 characters, and the comparison is done locally. In other words, the server never sees your complete password hash. 1Password and Bitwarden's "leaked password detection" both work this way.

Built-in Monitoring Features in Password Managers

- **1Password Watchtower**: Real-time monitoring of weak passwords, reused passwords, leaked passwords, expired passwords, and sites not using 2FA, displayed with red/orange/yellow severity levels.
- **Bitwarden Vault Health**: Free version shows weak and reused passwords; paid version can detect leaked passwords (based on HaveIBeenPwned).
- **iCloud Keychain "Security Recommendations"**: iOS Settings Passwords Security Recommendations, alerts you which passwords appear in known leaks and which are reused.
- **Google Password Manager "Password Checkup"**: Built into Chrome, can one-tap check for leaked, weak, and reused passwords.

Check Now: Are Your Passwords Safe?

1. Go to haveibeenpwned.com, enter your commonly used email, and see which breach events it appeared in.
2. iPhone users: Settings Passwords Security Recommendations, address each red warning one by one.
3. Chrome users: Settings Passwords Check Passwords, follow prompts to change.
4. 1Password/Bitwarden users: Open Watchtower/Health Reports, prioritize "leaked" and "reused" items.

VI. Passkey: The Future of Passwordless Login

In 2024-2026, password technology is undergoing a revolution —**Passkey**is poised to completely replace traditional passwords. Jointly promoted by Apple, Google, and Microsoft, en-

dorsed by the FIDO Alliance and the World Wide Web Consortium (W3C), this is the biggest change in login methods in decades.

What Is Passkey?

Passkey is based on the FIDO2 / WebAuthn standard, using**on-device key pairs**to replace passwords. The principle can be simplified as:

- Your phone generates a key pair:**Public key**(registered with the website) +**Private key**(permanently stored in your device's secure chip).
- When logging in, the website sends a "challenge," your phone signs it with the private key, and the website verifies it with the public key.
- Before signing, you need to confirm your identity via biometrics (Face ID / fingerprint) or device PIN.
- Throughout this process,**no password is transmitted over the network**.

Passkey vs. Traditional Passwords

Dimension	Traditional Password	Passkey
Phishing Resistance	Poor, phishing sites can steal passwords	Naturally phishing-resistant, private key only signs for real domains
Server Breach Impact	Password hashes may be cracked	Server only stores public key, no impact from breach
Convenience	Requires memorization/entry/management	Face ID/fingerprint login in one second
Replay Attack	May be intercepted and replayed by man-in-the-middle	Each signature is one-time, cannot be replayed

Platform Support Status

- **Apple iCloud Keychain Passkey**: Native support on iOS 16+ / macOS Ventura+, Passkeys sync across all devices signed in with the same Apple ID, no need to recreate on new devices.
- **Google Password Manager Passkey**: Native support on Android + Chrome, Passkeys stored in Google Account, sync across Android devices; since 2023, Google personal accounts support Passkey login.
- **Microsoft**: Windows 11 / Edge already supports Passkey; Microsoft Account, Outlook, Office and other services are gradually opening up Passkey login, manageable in Microsoft Authenticator.

- **1Password / Bitwarden**: Already supports Passkey creation, storage, and cross-platform sync, freeing you from being locked into a single ecosystem.

Cross-Device Sync Is the Key Breakthrough

The biggest pain point of early Passkey was being "bound to a single device" —you had to recreate it when switching phones. Starting in 2023, Apple, Google, and Microsoft implemented cloud sync for Passkeys: iCloud Keychain syncs across Apple devices, Google Password Manager syncs across Android devices. Third parties like 1Password have enabled cross-ecosystem sync. This means Passkey is now practical enough to replace passwords.

How to Create and Use Passkey on Your Phone

iOS (iPhone) Path:

1. On the account security page of a Passkey-supporting website (such as Google, Microsoft, Amazon, eBay, PayPal), select "Add passkey."
2. The system shows a Face ID / Touch ID verification window. Once verified, the Passkey is automatically generated and saved to iCloud Keychain.
3. Next time you log in to this website, enter your username and select "Sign in with iPhone" —just a Face ID scan and you're in.
4. Management: Settings Passwords find the corresponding website to see the Passkey entry.

Android Path:

1. On the account security page of a Passkey-supporting website, select "Add passkey."
2. Google Password Manager popup select your Google account screen lock verification (fingerprint/pattern/PIN).
3. Passkey is automatically saved to Google Password Manager and synced across Android devices with the same account.
4. Next time you log in, select "Use Passkey" —just unlock your screen and you're in.
5. Management: Android Settings Passwords, passkeys & autofill Password Manager google.com.

Cross-Device Passkey Login

If you're on a Windows PC logging into a website that only has a Passkey on your iPhone, you can authorize via QR code scan:

1. On the website login page, select "Use Passkey."

2. Select ”Use another device.”
3. The PC displays a QR code —scan it with your iPhone.
4. iPhone shows a Face ID verification, and upon confirmation, the website login is completed on the PC.

This process uses dual encrypted communication via Bluetooth + internet —the Passkey private key never leaves your iPhone.

Try It Now: Experience Passwordless Login

1. Sign in to your Google Account Security Passkeys Add a passkey, experience Face ID/fingerprint login.
2. Sign in to your Microsoft account Security Advanced Security Options Add a passkey.
3. Try it on services that already support Passkey, such as Taobao, JD.com, eBay, and PayPal.
4. Keep traditional passwords as backup, gradually migrate high-value accounts to Passkey.

Transition Period Recommendations

Passkey hasn't completely replaced passwords —many websites are still in a “Passkey + password” coexistence phase. Recommendation: prioritize enabling Passkey for high-value accounts, but**don't immediately delete traditional passwords**, keep them as backup; continue using your password manager, since not all websites support Passkey. Treat Passkey as a “powerful supplement to password managers” rather than an “immediate replacement.”

VII. Hardware Security Keys

If your security requirements exceed those of ordinary users —for example, if you hold significant cryptocurrency, are an executive/journalist/activist, or protect enterprise core accounts —hardware security keys are a more reliable choice than SMS verification codes.

What Is a Hardware Security Key?

A hardware key is a physical device (shaped like a USB drive) with an internal security chip that stores your private key, requiring physical contact or NFC to complete authentication. Representative products:

- **YubiKey**(Yubico): The most mainstream, supports FIDO2/WebAuthn, U2F, OTP, PIV, OpenPGP, with USB-C / USB-A / NFC and other interfaces. Series 5 costs $50-70.

- **Google Titan**: Made by Google, USB-C + NFC / Bluetooth, about $30-40.
- **OnlyKey**: Open-source alternative, supports multiple protocols.
- **Domestic Alternatives**: Such as Feitian ePasskey, more affordable.

Why Are Hardware Keys Safer Than SMS Verification Codes?

- **Phishing Resistance**: Like Passkey, hardware keys only respond to the real domain, so phishing sites can't trick them into providing verification codes.
- **SIM-Swap Resistance**: SMS verification codes can be intercepted via SIM-swap attacks, while hardware keys don't need a phone number.
- **Man-in-the-Middle Resistance**: The FIDO2 protocol includes a challenge value in each signature, preventing replay.
- **Physical Isolation**: The private key remains inside the hardware chip forever —even if malware compromises your computer, it can't read it.

Applicable Scenarios

- Cryptocurrency exchange accounts, wallet recovery.
- Enterprise SSO, remote work VPN, SSH server login.
- Email, GitHub, social media and other "master accounts" —once compromised, all other accounts can be reset.
- Journalists, lawyers, executives, and other high-sensitivity individuals.

How to Configure a Hardware Key as Two-Factor Authentication

Using a Google Account + YubiKey as an example:

1. Purchase a YubiKey (recommended to buy two at once —one for daily use and one as backup, stored in different locations).
2. Sign in to myaccount.google.com Security 2-Step Verification.
3. Select "Add security key" insert the YubiKey into the USB-C port or tap it against your phone's NFC sensor.
4. Follow the prompt to touch the YubiKey's metal contact to complete registration.
5. It's recommended to register a second backup key.
6. In the "2-Step Verification" settings, set the hardware key as the preferred verification method, with SMS as backup.

Websites with mainstream support: Google, GitHub, Microsoft, Apple ID, Facebook, Twitter, Dropbox, GitLab, AWS, Coinbase, Binance, and more.

Dual-Key Strategy

The biggest risk of hardware keys is "loss." Recommendations: (1) Register two different keys simultaneously; (2) Carry one with you (on a keychain) and store one in a home safe or bank deposit box; (3) Some websites allow printing "backup recovery codes" for offline storage. This way, even if the primary key is lost, you won't be permanently locked out of your accounts.

Be Careful with Purchase Channels

Always purchase from official stores or authorized dealers —don't buy used or keys from unknown sources, as pre-configured hardware could theoretically have been implanted with a backdoor. YubiKey firmware cannot be updated after manufacturing, but Yubico continuously publishes security advisories —follow their official website.

VIII. Family/Team Password Sharing

In family scenarios, Netflix, home Wi-Fi, router admin, and smart lock passwords need to be shared among the whole family; in team scenarios, servers, databases, and social media accounts require multi-person collaboration.**Sending passwords via WeChat or writing them in shared documents are high-risk practices**—once chat history or document permissions are leaked, the passwords are leaked too.

1Password's "Shared Vaults"

1Password is the benchmark product for family and team sharing. Its core mechanism is**multiple vaults**:

- **Personal Vault**: Only you can see it, for private accounts like banks and email.
- **Shared Vault**: Family or team members can all see it, for shared items like Netflix, home Wi-Fi, and shared credit cards.
- **Permission Tiers**: The team version can set certain members to "view only" or "view but not edit."

Family or team members don't need to share a master password —each logs in with their own account and can see the shared vault content they're authorized for. Password changes sync in real-time to all members.

Bitwarden's "Organizations" Feature

Bitwarden's free version supports individual password sharing (via the "Send" feature for time-limited sharing of a single password); the paid version can create "Organizations" to share entries with family or team colleagues with fine-grained permissions. Bitwarden also offers a self-hosted option, suitable for enterprises with high data sovereignty requirements.

Family Scenarios: Secure Ways to Share Netflix, Wi-Fi, and Other Passwords

- **Netflix / Streaming Sites**: Store login passwords in 1Password Families' "Shared Vault" —family members log in individually without sharing in a group chat.
- **Home Wi-Fi Password**: iOS devices can share Wi-Fi passwords directly via AirDrop to visiting guests without reading it aloud; Android 12+ also has a "Nearby Share Wi-Fi" feature.
- **Smart Locks, Smart Appliances**: Use the manufacturer app's "Family Members" feature to create individual accounts for each person, rather than sharing one admin account.
- **Home Router Admin**: Store the admin password in a 1Password Shared Vault, and set up a separate SSID for guest Wi-Fi.

1Password Families Value for Money

1Password Families costs about $4.99/month, supporting up to 5 family members, each with an independent personal vault + 4 shared vaults. That's less than $1/person/month — currently the best family sharing experience. Bitwarden's family plan at $10/year supports 6 people, offering better value but with a slightly less polished interface.

Team Scenarios: Server/Database Password Sharing

- **1Password Teams / Business**: Can create multiple vaults by department and project team, with fine-grained permission control and auditable access logs.
- **Bitwarden Teams / Enterprise**: Open-source solution, self-hostable, enterprise version supports SSO integration and directory sync.
- **Separation of Duties Principle**: Admin passwords should not be held by only one person —designate 2-3 "emergency contacts" to ensure business continuity when key personnel leave or take vacation.
- **Offboarding Process**: Immediately revoke departing employees' password manager access, and rotate all high-sensitivity passwords they had access to if necessary.

Emergency Access: In Case Something Happens to You

Imagine a scenario: you alone manage all the family's important accounts, but you're unexpectedly hospitalized or pass away, and your family can't log into the bank, email, or insurance accounts.**The "Emergency Access"feature**is designed exactly for this —authorize a trusted family member/friend to access your password vault when you can't respond.

Bitwarden Emergency Access Setup:

1. Log in to Bitwarden Web Vault Settings Emergency Access.
2. Add the trusted contact's email and invite them to accept.
3. Set a "wait period" (7-14 days recommended): the time window from when they make a request to when you confirm.
4. If you don't respond during the wait period (decline or take no action), the other person automatically gains access after it expires.
5. You can manually reject requests or revoke emergency access permissions at any time.

1Password Emergency Access:

1. 1Password doesn't call it "Emergency Access" directly, but it can be achieved through the "Family/Team Organization" feature —after adding family members to the family plan, place some entries in a "Shared Vault."
2. For individual plan users, it's recommended to tell one highly trusted family member your master password, or write it in a sealed envelope stored in a safe.
3. 1Password also provides a "Recovery Code" —be sure to print and save it when enabling two-factor authentication.

Set Up Now: Leave a "Spare Key" for Your Family

1. Choose a trusted family member as your "emergency contact."
2. Set up emergency access in your password manager (Bitwarden) or add them to a family shared vault (1Password).
3. Write the master password + recovery code on paper, seal in an envelope, store in a home safe or inform a trusted family member.
4. Review every six months to confirm the emergency contact is still appropriate.

IX. Password Manager Best Practices

- **Set a Strong Master Password**: The master password is the last line of defense — make sure to set a long, complex password and memorize it firmly.

- **Enable Two-Factor Authentication**: Turn on 2FA for your password manager account to add a layer of protection.
- **Regularly Check for Weak Passwords**: Most password managers have a "security audit" feature that helps you find weak and duplicate passwords.
- **Don't Forget the Master Password**: The master password cannot be recovered — forgetting it means losing all passwords. It's recommended to write it down on paper and keep it in a safe place.
- **Emergency Access**: Set up an emergency contact, so if you can't access your account, a trusted person can obtain your passwords.

Important Reminder

Password managers are convenient but come with risks: if your master password leaks, all your passwords are exposed. So your master password must never reuse any other password, nor be written somewhere easily found. Also, choose a provider with a good security track record.

Try It Now: Get Started with a Password Manager

1. Pick a password manager (Bitwarden or your system's built-in one is recommended).
2. Register an account and set a strong master password.
3. Import existing passwords (if any) or start adding new passwords.
4. Install the browser extension or mobile app, and experience the auto-fill feature.
5. Check the security audit report to find passwords that need improvement.

III. Advanced (Chapters 15-20)

Chapter 15: Battery Science and Battery Life Optimization

Afraid of heat

What's the Deal with Batteries? Cycles and Health

Aging

- **: Over time, the maximum capacity will decrease—this is normal.**Core Metrics: Cycle Count and Battery Health
- **One cycle is not one charge. For example, if you use 60% today and recharge to full, then use 40% tomorrow and recharge to full, the total 100% consumed counts as one cycle.**Battery health is the current maximum capacity as a percentage of brand-new. Below 80%, you'll noticeably feel the battery life shorten.
- **How to check?**iPhone: Settings > Battery > Battery Health & Charging; Android: usually search "battery health" in Settings.

Charging Mysticism? Fast Charging, Wireless Charging, and the Golden Rule

- Does fast charging damage the battery?
- Using original or certified fast-charging accessories causes minimal harm. The phone and charger have temperature control and will automatically reduce power if overheated.
- **Wireless charging**: Convenient but less efficient, with more noticeable heat. Suitable for overnight charging.

Charging Mysticism? Fast Charging, Wireless Charging, and the Golden Rule

- **: Try to keep the battery level between 20%-80%. Don't keep it starving (0%) or stuffed (100%).**3. Complete Guide to Fast Charging Protocols: Why Won't Your Charger Charge Fast?
- **Wireless charging**1. Basic Principles of Fast Charging
- **Charging power = voltage × current (P = U × I). To increase power, you either raise the voltage, raise the current, or both. Different protocols choose different technical approaches.**Fast charging workflow:

iPhone "80% Charge Limit" Feature —Turning the Golden Rule into a System Setting

The iPhone 15/16 series (iOS 17+) has turned the "20%-80% rule" into a system feature. When enabled, the phone automatically stops charging at 80%, significantly slowing battery aging. Settings path:

`Settings > Battery > Battery Health & Charging > Charging Optimization > 80% Limit`

The three options mean: (1)**No Limit**—charges to 100%; (2)**Optimized Battery Charging**—learns your routine, only delays to 100% during the last portion of overnight charging; (3)**80% Limit**—stops at 80% directly, best for battery longevity. For people who are always at home/office with charging available, the 80% Limit is strongly recommended; for frequent travelers who need a full charge on the go, Optimized Battery Charging is a good choice.

GaN Chargers —A Traveler's Best Companion

You may have noticed that fast chargers are getting smaller and smaller —the hero behind this is**GaN (Gallium Nitride)**this new semiconductor material. Compared to traditional silicon-based chargers, GaN chargers have several clear advantages:

- **Compact Size**: At the same power output, GaN chargers can be 1/2 or even 1/3 the size of traditional silicon ones. A 65W GaN charger is smaller than Apple's old 5W charger.
- **High Efficiency, Low Heat**: Gallium nitride's switching loss is far lower than silicon, with energy conversion efficiency reaching 95%+, so it doesn't get hot during charging.
- **Travel-Friendly**: A single 65W or 100W multi-port GaN charger can simultaneously charge a phone, tablet, and laptop —much lighter than carrying three "bricks."
- **Buying Advice**: Look for GaN chargers that support the PD protocol (65W+ can

charge laptops). Well-known brands like Baseus, Anker, Nubia, and Xiaomi are all good choices. Make sure to check that the cable is compatible (high-power charging requires a Type-C cable with an E-Marker chip).

Handshake phase

: The phone and charger communicate through the data cable, exchanging their supported protocols and power information.

Negotiation phase

: Both parties determine a commonly supported maximum power level.

Charging phase

1. **: The charger outputs voltage and current according to the negotiated values, and the phone manages the power.**Adjustment phase
2. **: As the battery level increases, the phone gradually reduces the charging power to protect the battery.**2. Mainstream Fast Charging Protocols in Detail
3. **USB PD (Power Delivery)**—King of Universal Protocols
4. **Features**: Official standard developed by USB-IF organization, with the widest coverage. PD 3.0 supports up to 100W (20V/5A), PD 3.1 extends to 240W (48V/5A).

Voltage levels

: 5V, 9V, 15V, 20V; PD 3.1 also supports 28V, 36V, 48V. Compatibility

- **Protocol**Advantages
- **: Strong universality—one charger can charge multiple devices; supports PPS (Programmable Power Supply) for fine-tuning voltage and current.**Disadvantages
- **QC 5.0**Qualcomm QC (Quick Charge)
- **—Veteran of the Android Camp**QC 3.0
- **: Supports adjustable voltage from 3.6V-20V in 0.2V steps, up to 18W.**QC 4.0/4+

: Compatible with USB PD protocol, supports up to 27W, adds dual-path charging technology.QC 5.0

- **: Supports 100W+, 0-50% in just 5 minutes, significantly improved heat.**Compatibility
- **: Most phones with Qualcomm Snapdragon processors support it, but brands usually prioritize their own proprietary protocols.**OPPO VOOC / OnePlus WARP / Realme Dart
- **240W**Technical approach
- **QC 5.0**VOOC 1.0

: 20W (5V/4A), released in 2014, pioneered the fast charging era. SuperVOOC

- **Widest**SuperVOOC 2.0
- **: 65W, fully charges a 4000mAh battery in 30 minutes.**150W/240W Super Flash Charge
- **: Currently the fastest mass-produced fast charging technology.**Features
- **: Requires dedicated charger and cable (with extra contacts), not compatible with other protocols.**Huawei SuperCharge
- **—Evolution of High-Voltage Fast Charging**FCP (Fast Charge Protocol)
- **Protocol**SCP (Super Charge Protocol)

: Supports higher power, from 22.5W to 66W to 100W. Technical features

- **: Uses charge pump technology to convert high voltage into low voltage, high current input to the battery, with efficiency up to 98%.**Compatibility
- **: Exclusive to Huawei/Honor phones; some chargers support PD protocol.**Xiaomi Surge Fast Charging
- **: Only supports USB PD protocol.**Surge P1 chip
- **QC 5.0**Surge G1 chip

: Battery management chip, works with P1 to achieve more efficient fast charging. Technical features

- **: Uses dual-cell series + charge pump solution, supports multi-protocol compatibility.**Apple Fast Charging
- **—Conservative but Stable**Protocol
- **: Only supports USB PD protocol.**Power

Wireless charging also has its own protocol standards: Features

- **240W**Accessory requirements
- **: Requires USB-C to Lightning cable (or USB-C to USB-C cable) and PD charger.**3. Fast Charging Protocol Comparison Table
- **Protocol**Max Power
- **Technical Approach**Compatibility

USB PD 3.1

240W	**High voltage/Multi-level**	**Widest**	**QC 5.0**
100W+ 240W 100W ~27W Wireless charging also has its own protocol standards:	Charge pump Low voltage, high current Charge pump PD protocol Qi standard	Qualcomm models OPPO family exclusive Huawei/Honor iPhone/iPad : Developed by WPC (Wireless Power Consortium), the most universal standard. Basic Qi supports 5W-15W.	VOOC/Dart SuperCharge Apple Fast Charging 4. Wireless Charging Protocols Qi2 standard

: Released in 2023, introduces magnetic alignment (similar to Apple MagSafe), supports 15W+.

Proprietary protocols

- **: Each brand has its own high-speed wireless charging, such as Xiaomi 50W wireless charging, Huawei 50W wireless charging, etc., requiring dedicated chargers.**5. Fast Charging Purchasing Guide
- **Check protocol matching**: When purchasing a charger, confirm it supports your phone's fast charging protocol. Third-party chargers usually list supported protocols.
- **Check the cable**: High-power fast charging requires a dedicated cable. Ordinary cables may only support below 60W.

Check certification

- **: Apple users should choose MFi-certified cables; Android users should choose original brand or well-known third-party.**Check heat
- **: It's normal for the phone to heat up during fast charging, but if it's burning hot, that's not normal—pause charging or change the charger.**Fast Charging Safety Tips
- **Avoid playing heavy games while charging, which will increase heat.**Do not use low-quality chargers and cables, which may cause safety hazards.
- **Remove thick phone cases while charging to help with heat dissipation.**If the charger or phone heats abnormally or has an unusual smell, stop using it immediately.

4. These Are the Real Battery-Saving Tips

- Screen (the power hog)
- : Lower brightness, shorten auto-lock time. OLED screen phones can save a lot of power with dark mode.
- Poor signal
- : In elevators and basements, the phone will desperately search for signal, draining power quickly.

IV. New Battery Technology: Silicon-Carbon Anode Batteries

If you've been paying attention to 2024-2026 flagship phone launches, "silicon-carbon anode" has practically become synonymous with premium batteries. This is an important upgrade to lithium battery anode materials, and the principle isn't complicated:

- **Traditional Graphite Anode**: Phone batteries have always used graphite as the anode—the technology is mature and stable, but energy density has approached the theoretical limit (approximately 372mAh/g).
- **Adding Silicon Material**: Silicon's theoretical specific capacity is about 10 times that of graphite (approximately 4200mAh/g). By adding a small amount of silicon to graphite to create a "silicon-carbon composite anode," the overall energy density of the battery can be significantly improved.
- **Energy Density Improvement**: At the same volume, silicon-carbon anode batteries can increase capacity by approximately**15%-20%**. This is the key reason 2024-2026 flagship phones can fit 5500-6000mAh batteries in 7-9mm bodies.

Which Models Use It?

- **Honor**: The Magic series pioneered the commercial use of silicon-carbon anodes. The Magic6 Pro, Magic7 series, and Magic V3 foldable all use it.
- **Xiaomi**: The Xiaomi 14 Ultra and Xiaomi 15 series feature a new generation of silicon-carbon anode batteries.
- **vivo**: The X100 Ultra and X200 Pro series use a high-density silicon-carbon anode solution.
- **Huawei/OPPO, etc.**: 2025-2026 flagships are widely following suit, and silicon-carbon anodes are transitioning from "black tech" to standard equipment for premium phones.

Practical Impact on Users: The biggest benefit of silicon-carbon anodes is "larger battery capacity at the same size"—phones can be thinner without sacrificing battery life, or the body stays the same thickness but the battery goes from 4500mAh to 5500mAh. In other words, you no longer need to pay the price of "thick and heavy" for battery life. Note that silicon-carbon anode batteries still need optimization with fast-charging chips for low-temperature and full-charge management, which is one reason manufacturers recommend keeping charging at 80% long-term.

V. The Real Battery-Saving Tips

- **: If the signal is poor, manually switch back to 4G, and battery life will be much better.**Location services
- **: For non-navigation apps, set location permission to "Only while using".**Limit background activity
- **: iPhone in Settings > General > Background App Refresh; Android in app battery settings.**Let's Check: Give Your Battery a Health Exam
- **Check health: Find and note your battery's maximum capacity.**Analyze power consumption details: Look at the charts and app list in battery settings to find the power-hungry culprits.
- **Formulate a strategy: Restrict apps with abnormal background power consumption.**Chapter 16: Account Systems and Cloud Services

I have a friend whose phone was stolen last year. His first thought wasn't "how much was the phone worth", but "can I get my photos back?" Unfortunately, he hadn't enabled cloud backup. A phone is just a physical object, but the data in your account—once truly lost, can never be recovered. In this chapter, we'll talk about how to put real insurance on your digital life.

1. 1. How Much Does Your Phone Account Actually Manage?
2. Many people think an account is just for downloading apps. In reality, a phone account is tied to much more than you imagine:
3. Contacts

Chapter 16: Account Systems and Cloud Services

Photos and videos

: From your child's birth to graduation ceremony, from first date to wedding trip.

Notes

- **Contacts**App purchase history
- **Photos and videos**Find My Phone
- **: After your phone is lost, this is the only tool that can help you locate, lock, and remotely wipe data.**If you use an Apple phone, all of the above is tied to your Apple ID. If you use Android, it's tied to your Huawei/Xiaomi/OPPO/vivo account, plus your Google account. These accounts are the "household register" of your digital life—lose the register, and many things become unprovable.
- **2. Three Must-Do Things, Right Now**Enable two-factor authentication
- **: Whether you use Apple or Android, the first thing to do is go to account settings and turn on "Two-Factor Authentication" or "Two-Step Verification". This is like adding a deadbolt to your door—password alone isn't enough, your phone also needs to receive a verification code. Even if someone guesses your password online, without your phone, they can't get in.**Bind a reliable recovery method

: Two-factor authentication is secure, but there's a risk—what if you lose your own phone? So the system will prompt you to set up a "recovery key" or "backup email/phone number". This is your only way to get your account back in an emergency. Best to write it on paper, don't store it on your phone.

Confirm “Find My Phone” is enabled

1. **: This feature can help you locate, lock, play sound, and even remotely wipe all data after your phone is lost. It’s the last line of defense. Many people skip this step when buying a phone, and when they want to use it later, they find it wasn’t enabled—and it’s too late.**3. iCloud: The ”Air” for Apple Users
2. **If you use an iPhone, sooner or later you’ll face a soul-searching question: what to do when iCloud storage is full?**Apple gives you 5GB for free, and honestly, this 5GB isn’t even enough for one backup. So most people eventually receive that annoying popup: ”iCloud Storage Full”. At this point, you have three options:
3. **Clean up**: Go to Settings Your Name iCloud Manage Storage to see who’s taking up your space. Generally, photos and backups are the biggest. You can delete unnecessary old backups first.

Upgrade

: 50GB is 6 yuan per month, 200GB is 21 yuan per month. Personally, I think 50GB is enough for most people, mainly for backups and photo thumbnails.

Workaround

- **: If you don’t want to pay, you can periodically export photos to your computer, then turn off iCloud photo sync. But then you lose the convenience of automatic photo backup.**An important reminder: iCloud Photo Library is not a ”cloud drive”—it’s ”sync”. If you delete a photo on your phone, it will also be deleted from iCloud. If what you want is ”backup to cloud then clear phone”, then iCloud Photos is not what you need—you need iCloud Backup, or a third-party cloud drive.
- **4. Don’t Confuse Sync with Backup**These two words sound similar, but the consequences are completely different:
- **Sync**is bidirectional: Delete on phone, delete on cloud; delete on cloud, delete on phone. iCloud Photos, Contacts, Notes are all sync. The benefit is consistency across multiple devices; the downside is ”once deleted, it’s really gone”.

Backup

iCloud+: The Privacy Trio for Paid Users

As long as you upgrade to any paid iCloud tier (50GB/200GB/2TB, etc.), you automatically become an**iCloud+**user, unlocking three privacy protection features:

- **Private Relay**: Apple’s version of a ”VPN.” Safari browsing traffic is relayed through Apple servers before reaching the website, so no one (including Apple) can simultane-

ously see who you are and what you're accessing. Ideal for protecting privacy on public Wi-Fi. Settings path:`Settings > [Your Name] > iCloud > Private Relay`.

- **Hide My Email**: Generates one-time random email addresses when registering on websites/apps, with emails automatically forwarded to your real email. If you no longer want emails from a site, simply turn off that random address —your real email is never exposed. Settings path:`Settings > [Your Name] > iCloud > Hide My Email`.
- **Custom Email Domain**: You can use your own domain (e.g., @yourname.com) to send and receive iCloud email, suitable for personal branding or family email management.

Note: Private Relay is not available in some countries/regions, and domestic users may need alternative solutions.

iCloud Advanced Data Protection (ADP) —True End-to-End Encryption

In late 2022, Apple launched**iCloud Advanced Data Protection (ADP)**, extending end-to-end encryption (E2EE) from the original limited data like Health and Passwords to**the vast majority of iCloud data**—including Photos, Notes, iCloud Backup, device information, and 23 categories in total.

- **What Is E2EE**: Encryption keys exist only on your devices —even Apple's servers don't have them. Even if Apple is hacked or law enforcement requests data, Apple can't produce decryptable plaintext data.
- `::Settings > [Your Name] > iCloud > Advanced Data Protection`, and follow the prompts to set up Recovery Contacts/Recovery Key (**Must Do**—if you lose the key, your data can't be recovered either).
- **Notes**: (1) After enabling, you must properly safeguard the Recovery Key, otherwise data cannot be recovered if you lose account access; (2) Shared Albums, iCloud Shared Photo Library, and a few other features don't participate in E2EE; (3) Apple IDs in mainland China do not currently support ADP —you need to switch to an account in another region.

One-sentence recommendation: If you have important private data in your account and your account region supports it, strongly consider enabling ADP —this is the strongest cloud data protection available to ordinary users.

is one-way: The phone takes a “snapshot” of its current state and stores it in the cloud. After that, if you delete something on the phone, the copy in the cloud backup is still there. iCloud Backup, local computer backup—these are backups. The benefit is safer data; the downside is they take up space and can’t update in real time.

is one-way: The phone takes a “snapshot” of its current state and stores it in the cloud. After that, if you delete something on the phone, the copy in the cloud backup is still there. iCloud Backup, local computer backup—these are backups. The benefit is safer data; the downside is they take up space and can’t update in real time.

5. The Account Matrix for Android Users

- **The situation on the Android side is more complicated than Apple, because your data may be scattered across several accounts:**Brand account
- **(Huawei/Xiaomi/OPPO/vivo): Manages your cloud space, Find My Phone, app store, wallet. Each brand has its own cloud service, usually a basic-free + paid-expansion model.**Google account

: If you use Gmail, Google Photos, Google Contacts, these are all in your Google account. Google gives 15GB for free, more generous than Apple, but it’s also shared with Gmail and Drive.

WeChat/QQ

- **: They are independent of the phone system. Your chat history and WeChat favorites are neither in the brand account nor in the Google account—they only exist on WeChat’s own servers.**So when Android users do backups, they need to check separately: brand cloud backup, Google backup, WeChat chat history backup—all three are essential.
- **6. Family Sharing: Let Your Family Benefit Too**Apple’s ”Family Sharing” lets you share app purchases, music, movies, iCloud space with your family, and even share location. You can add up to 6 people. Setup method: Settings Your Name Family Sharing. Huawei and Xiaomi also have similar features, findable in account settings as ”Family Sharing” or ”Family Cloud”.
- **A very practical scenario: Set up Family Sharing on your parents’ phones, so you can always see their location (provided they agree). If an elderly person wanders off or can’t be reached by phone, you have a location.**The Consequences of Account Theft Are More Serious Than You Think

If a hacker gets your account password, they can: remotely lock your phone and extort ransom, view your photos and contacts, make purchases with your payment methods. More

troublingly, if your account is tied to too many things, recovering the account itself is a nightmare. So, two-factor authentication—go enable it now.

Hands-on: Give Your Account a Comprehensive Health Check

Open phone settings, go to the account page, confirm two-factor authentication is enabled.

Check whether “trusted phone numbers” and “recovery key” are set up; write the key on paper.

Go to iCloud or cloud space management, see what data is syncing, turn off what you don’t need.

Confirm “Find My Phone” is enabled.

If you use Android, check whether brand cloud backup, Google backup, and WeChat backup are all covered.

1. Chapter 17: Linking Your Phone with Other Devices
2. The phone in your hand is actually a super remote control and data center. It can not only make calls, but also connect your computer, TV, watch, speaker, and car together, so these devices are no longer islands. In this chapter, we’ll talk about how to make them ”team up”.
3. 1. Collaboration with Computer: Phone and Computer Are No Longer Two Worlds
4. Honestly, what I used to hate most was taking a photo on my phone and wanting to edit it on the computer—I had to find a data cable and wait for transfer. Later I discovered that you don’t actually need a cable at all; good integration is ten times faster than a cable.
5. Apple ecosystem (AirDrop + Universal Clipboard)

Chapter 17: Interconnection Between Phone and Other Devices

Android + Windows (Link to Windows)

: Microsoft has an app called "Link to Windows", built into Samsung phones, and other Android phones can download it from the app store. After pairing, you can directly view phone notifications, reply to texts, browse photos, and even make and receive calls on your computer. I've used it for two years—during meetings I don't have to keep looking down at my phone; a notification popping up on the computer screen is enough.

Brand multi-screen Collaboration

- **: Huawei's "Multi-Screen Collaboration" and Xiaomi's "Xiaomi Smart Connect" go even further—directly project your phone screen onto your computer. You can use the mouse to drag files from phone to computer, and also browse TikTok and reply to WeChat on the computer. Cross-device copy-paste and file dragging all work, with a very smooth experience.**2. Screen Casting to TV: Watch Together with the Whole Family

- **When I went home for Chinese New Year, my cousin wanted to play her graduation trip video from her phone on the TV. After fumbling for a long time, we found it was actually very simple:**iPhone TV

- **: Swipe down from the top right to open Control Center, tap the icon with two overlapping squares (Screen Mirroring), find your TV's name, and tap to connect. The prerequisite is that the TV and phone are on the same Wi-Fi, and the TV supports AirPlay (most smart TVs do).**Android TV

: Each brand calls it differently—Huawei calls it "Wireless Projection", Xiaomi calls it "Cast", OPPO/vivo calls it "Multi-Screen Interaction". Usually findable in the quick settings panel; if not, search "screen cast" in Settings.

Direct cast from video apps

- **: If you just want to watch iQiyi, Bilibili, or YouTube on TV, it's even simpler—find a small rectangle icon with a Wi-Fi symbol on the playback interface, tap it, select the TV, and the video goes over. At this point, the phone can do other things while the video continues playing on the TV, without interfering with each other.**What if screen casting fails
- **: 90% of the time, it's because they're not on the same Wi-Fi. The other 10% is that the TV's screen casting function isn't enabled—look for the "Multi-Screen Interaction" or "AirPlay" switch in TV settings.**3. With Smart Watches/Bands: A Health Manager on Your Wrist
- **I bought my mom a fitness band. At first she said "I don't exercise, why buy this?" After two weeks of use, she discovered she was only sleeping five hours a day, and started proactively adjusting her schedule. This is the most practical value of watches/bands—turning invisible health data into visible numbers.**Notification filter
- **: When your phone keeps vibrating in your bag or pocket, just raise your wrist to take a look—is it a WeChat from the boss or a Pinduoduo promotion? Judge in a second, and let messages not worth pulling out the phone for go.**Health data sync

: The watch monitors heart rate, blood oxygen, sleep, and steps 24 hours a day, and this data is automatically aggregated into the health app on your phone. Over time, you can see your trends—have you been walking too little this month, has your sleep quality improved this week.

Transit card/access card sync

- **: Transit cards and access cards bound on your phone will mostly auto-sync to your watch. When taking the subway, you don't need to pull out your phone—just tap your wrist against the turnstile, and you really get a "the future is here" feeling.**Find phone lifesaver

- **: Watches generally have a ”find phone” feature—tap it and the phone will ring. I use this feature far more than expected—I just put the phone down somewhere and forget where.**4. With Smart Home: The Phone Is the Home's ”Master Switch”

- **Smart home sounds mysterious, but it boils down to three things: remote control via phone, set automation rules, voice control.**Let me talk about a few of my own actual use cases, so you can see how practical this thing is:

- **Air conditioner**: On the way home in summer, use your phone to turn on the AC in advance—when you walk in, it's already cool. Buy an AC companion socket for a hundred-something yuan to achieve this, no need to replace the AC.

Lights

: Lying in bed before sleep, too lazy to get up to turn off the lights—just tap on your phone or say “Hey Siri/Xiaoai, turn off the lights” .

Robot vacuum

- **: Before going out, tap ”Start Cleaning” on your phone—when you come back, the floor is clean. You can also set a daily scheduled cleaning.**Camera

- **: If you have elderly, children, or pets at home, you can check on the home situation anytime from your phone.**Which ecosystem to choose? One-sentence summary:

- **Using iPhone prioritize devices that support Apple HomeKit (a bit more expensive but better privacy protection).**Many devices, want cost-effectiveness Xiaomi Mi Jia ecosystem is the most complete—lights, sockets, robot vacuums, everything.

- **Using Huawei phone prioritize HarmonyOS Connect, best experience with Huawei devices.**5. Bluetooth Headphones and Speakers: Wireless Freedom

Switching from wired to Bluetooth headphones was, for me, one of the most noticeable upgrades in phone experience. No more pulling out a tangled mess from the bag, no more worrying about the cable catching on door handles.

- Pairing is simple

- : Headphones generally enter pairing mode automatically when first turned on (flashing light). Turn on Bluetooth on your phone to find them, tap to connect. After that, they auto-connect every time you take them out—nothing to do.

- Same brand has bonuses

: AirPods with iPhone will auto-pop up to show battery, and can seamlessly switch between multiple Apple devices. Huawei FreeBuds with Huawei phones, Xiaomi earbuds with Xiaomi phones, also have similar pop-ups and optimizations.

Bluetooth speakers

- **: For family gatherings or outdoor camping, a Bluetooth speaker can double the atmosphere. After connecting, your phone becomes your DJ booth.**Watch out for latency
- **: When playing games with Bluetooth headphones, there may be slight latency (you hear the gunshot half a second after it fires). If you're sensitive to latency, either use wired headphones or buy Bluetooth headphones that support "low latency mode".**6. With Car Systems: A Safe Partner While Driving
- **Looking at your phone while driving is illegal and life-threatening. But sometimes you really need navigation, phone calls, and music. This is where CarPlay (Apple) and Android Auto (Android) come in handy.**How to connect
- **: If your car supports it (most new cars in recent years do), use a data cable to plug your phone into the car's USB port, and the car screen will automatically switch to the CarPlay or Android Auto interface. Some new cars support wireless connection, even more convenient.**What it can do

UWB (Ultra-Wideband) Technology —More Precise "Spatial Awareness" Than Bluetooth

In addition to Bluetooth, new phones also contain a UWB (Ultra-Wideband) chip. Its capability is**precise distance measurement and spatial positioning**—it can calculate how far another device is and in which direction, with centimeter-level precision. Bluetooth can only tell you "it's nearby," while UWB can tell you "it's 1.2 meters to your front-left." Common applications:

- **AirTag / Precise Finding**: When an iPhone finds an AirTag, it shows directional arrow navigation like "7 meters away, go left" —this relies on UWB. Regular Bluetooth can only tell you whether it's within range.
- **Digital Car Keys**: iPhones and Android flagships with UWB can serve as car keys —automatically unlock when approaching the vehicle, lock when walking away, and distinguish between driver and passenger positions to prevent relay attacks.
- **Indoor Navigation**: "Aisle-level" navigation in malls and airports via UWB beacons, much more accurate than GPS.

- **Supported Models**: iPhone 11 and later all have UWB chips (U1/U2); on the Android side, some flagships from Xiaomi, Samsung, vivo, and OPPO are also equipped.

LE Audio and Auracast —The New Generation Bluetooth Audio Standard

The classic Bluetooth audio (A2DP) used for twenty years is being replaced by**LE Audio**(Low Energy Audio), the new generation audio standard from Bluetooth 5.2 onwards:

- **Lower Power Consumption, Better Sound Quality**: Uses the all-new LC3 codec, with better sound quality than the old SBC standard and lower power consumption —earbud battery life is noticeably extended.
- **Dual-Path Audio**: Left and right earbuds transmit independently —switching to single-ear mode won't cause disconnection.
- **Auracast Broadcast Audio**: The signature feature of LE Audio. A single phone/TV can simultaneously broadcast audio to**countless pairs of**earbuds —airport waiting areas, museums, gyms, and classrooms can all use a single transmitter for multiple people to listen with their own earbuds.
- **Current status**: Flagship earbuds and phones released in 2024-2026 generally support LE Audio, but older devices require hardware upgrades and can't be software-unlocked. When buying new earbuds, check for the LE Audio/Auracast label.

: Navigation (much better than built-in car navigation), phone calls (answer from steering wheel), music (music apps on your phone directly cast to the car), voice WeChat (Siri or Xiaoai helps read messages and reply).

Old cars can use it too

- **: If your car is older and doesn't support it, buy an external CarPlay screen for a few hundred yuan, stick it on the dashboard—the experience is about the same.**7. Security and Privacy in Integration
- **The more devices connected, the larger the surface area for privacy leaks. A few points to note:**Before screen casting in public, first check whether your phone notification bar has any private messages—don't let the whole family see what they shouldn't.
- **Before giving or selling a device to someone else, unbind it first—go to phone settings, find that device, and select "Remove" or "Unpair".**For smart home cameras, buy branded, encrypted ones—don't buy no-name ones for a few dozen yuan. There's been too much news about cameras being hacked for live streaming.

Try It: Cast Your Phone Screen to TV and Set Up a Smart Scene

Make sure your phone and TV are on the same Wi-Fi.

- Find ”Screen Mirroring” or ”AirPlay” on the TV and turn it on.
- On your phone: iPhone taps ”Screen Mirroring” from Control Center; Android swipes down to find ”Wireless Projection”.
- Cast your favorite photo to the TV for your family to see.

If you have smart home devices, try creating a ”Leave Home Mode” in the phone app: one-tap to turn off lights, turn off AC, and start the robot vacuum.

1. Chapter 18: Data Backup and Migration
2. A neighbor of mine dropped her phone into a pool, and by the time she fished it out, it wouldn’t turn on. What hurt her most wasn’t the phone itself, but the three years of unbacked-up photos—every record of her child from birth to learning to walk. After hearing her story, I felt terrible, because this was completely avoidable. In this chapter, we’ll seriously put insurance on your data.
3. 1. First, Figure Out: What in Your Phone Is Gone Forever If Lost
4. Different data has different ”survival states”—you need to distinguish them:
5. Stored locally on the phone, truly gone if lost

Case 1: Battery Life Optimization in Practice

- **Scenario**: Xiaomi 14 user with poor battery life —out of power by midday.
- **Steps**: (1) Settings Battery Check power consumption ranking (2) Identify abnormal background apps (3) Switch from 5G to 4G (4) Restrict background activity (5) Enable power saving mode (6) Remove case while charging for heat dissipation.
- **Expected Result**: Battery life extended by 30-50%, battery health data readable.

Chapter 18: Data Backup and Migration

Stored on app servers, available by re-logging in on a new phone

: WeChat friend list, Taobao orders, transaction records in banking apps. These are stored on the corresponding company's servers, not on your phone.

In a gray area, requiring special handling

- **: WeChat chat history (although WeChat servers store it, you can't download it from the server yourself), device credentials bound to mobile banking, keys in two-step verification apps (like Google Authenticator).**So what you need to focus on protecting is the first and third categories.
- **2. Full Phone Backup: Get Everything in One Go**A full phone backup is like taking a complete "snapshot" of your phone—current system settings, home screen layout, app data, photos, contacts, all packaged up.
- **Option 1: Local Computer Backup (Most Reliable, Recommended for People with Computers)**iPhone

: Connect your phone to the computer with a data cable, open iTunes (older Windows version) or Finder (Mac), and select "Back Up Now" on the device page. Note: check "Encrypt backup", otherwise passwords and health data won't be included.

Android

: Each brand has its own computer software, such as Huawei Phone Assistant, Xiaomi Assistant, Samsung Smart Switch. Connect to the computer and follow the instructions.

The benefit of computer backup is that you don't need to pay for cloud space, and the restore speed is much faster than cloud restore. The downside is that you need to remember to plug in and back up periodically—most people can't manage this.

- **Android**: Each brand's cloud has backup functionality. Huawei phones search "cloud backup" in Settings, Xiaomi goes to "Xiaomi Cloud Service", OPPO/vivo similarly. There's also a Google-built backup (Settings System Backup)—turn this on too.

The biggest benefit of cloud backup is "you don't have to worry about it"—it's automatic. The biggest downside is that restoring takes a long time to download, and the free space is usually not enough.

- **iPhone**Full phone backup is the safety net, but some things are too important and deserve a separate extra copy:
- **Android**Dual insurance plan
- : Enable iCloud Photos/Google Photos auto-sync, and every few months manually copy photos to your computer or external hard drive. This way, one copy in the cloud, one copy locally—mutual backup.

Free plan

: Google Photos gives 15GB for free, can be set to "compressed quality" to save space. Or every month, use a data cable to copy all photos to your computer.

Contacts (Easily Overlooked but Extremely Important)

- **iPhone contacts sync with iCloud by default—as long as sync is on, no extra backup needed. But if you want to be safer, you can export a vCard file from iCloud.com and save it on your computer.**Android contacts usually sync with Google or brand cloud. Go to the Contacts app settings to export a .vcf file—that's a complete backup.
- **WeChat Chat History (Handle Separately, Top Priority)**This must be discussed separately, because WeChat chat history is often incomplete in full phone backups, and phone migration tools simply can't handle it.

Method 1: Migrate to new phone

- . WeChat Me Settings General Chat History Migration & Backup Migrate Migrate to Phone/Tablet. Connect both old and new phones to the same Wi-Fi, scan the QR code to transfer. Note: Don't log out of WeChat on the old phone until migration is complete.

- Method 2: Back up to computer

. Install WeChat PC version on your computer, three bars in the bottom left Migration & Backup Backup & Restore Back up chat history to computer. Afterward, when you change phones, you can restore it to the new phone.

Method 3: Back up to cloud

- **. WeChat also supports backing up to iCloud or phone cloud space, but note: this backup can only be restored on your current phone, not cross-device. To change phones, you still need to use the first two methods.**4. Phone Replacement Guide: From Old Phone to New Phone

- **Getting a new phone is a happy event, but if the data doesn't transfer over, it becomes a headache. Three scenarios, handled separately:**Same Brand Upgrade (Simplest)

- **iPhone New iPhone**: When turning on and activating, place the old phone nearby, and "Quick Start" will automatically pop up. Use the old phone to scan the pattern on the new phone. All data, settings, and app layout will be transferred intact, including Wi-Fi passwords. Takes about 20-30 minutes.

Android same brand

: Use the brand's built-in phone replacement tool. Huawei uses "Phone Clone", Xiaomi uses "Xiaomi Phone Transfer", OPPO uses "Phone Move", vivo uses "EasyShare". The steps are similar: install the same app on both old and new phones, scan to connect, check what to transfer, start.

Cross-Brand Phone Replacement (Android to Android, Different Brands)

- **Use third-party phone replacement tools or the old brand's built-in replacement app (most brands' replacement apps also support transferring to other Android phones). What can be transferred: contacts, photos, text messages, call logs, some apps. What can't be transferred: in-app login status (you need to re-login), WeChat chat history (use WeChat's own migration feature).**Cross-Platform Phone Replacement (Android to iPhone, or Vice Versa)

- **Android iPhone**: Apple has a "Move to iOS" app. Download it on your Android phone, and when activating the new iPhone, select "Move Data from Android". Can transfer contacts, photos, text messages and other basic data, but app data can't be transferred. WeChat chat history still needs WeChat's own migration feature.

iPhone Android

: Each Android brand has corresponding migration tools, but honestly, cross-platform migration experience isn't as good as same-platform. Do the most important things first: export contacts, back up photos, migrate WeChat chat history. Take your time with the rest.

The Golden Rule of Backup: The 3-2-1 Principle

- **This is a classic principle in the data security field, simple and easy to remember:**3
- **: Keep at least three copies of important data (one original, two backups).**2

: Use two different storage media for backups (e.g., one in the cloud, one on an external hard drive).

1

- **: At least one backup should be in a different physical location (to prevent fire, theft, etc. from destroying all data at once).**For ordinary people, achieving "one in the cloud, one on the computer" is already quite good. Don't make it too complicated—the key is that you actually do it.
- **Special Attention for WeChat Chat History**WeChat chat history cannot be migrated by any phone replacement tool! It's not covered by full phone backup. You must use WeChat's built-in "Chat History Migration" feature, or back up to the computer version of WeChat. Many people discover after getting a new phone that all chat history is gone—and by then it's too late. The old phone may have already been factory reset.
- **Practice: Do a Complete Backup Drill**Check whether iCloud Backup or brand cloud backup is enabled, confirm the last backup time.

Manually back up photos: Use a data cable to connect to the computer, copy the DCIM and Pictures folders to the computer.

Back up contacts: iPhone users export vCard from iCloud.com; Android users export .vcf file in Contacts app settings.

Back up WeChat chat history: Open WeChat Me Settings General Chat History Migration & Backup Back up to computer.

Copy the exported contacts vcf and photo folders to an external hard drive or USB drive, completing the "one offline copy elsewhere" of the 3-2-1 rule.

Chapter 19: Troubleshooting and Self-Rescue

Phones, like people, occasionally "get sick". Lagging, crashing, black screen, won't charge... most of these issues aren't that scary, and you can try fixing them yourself before sending it for repair. This chapter is your "phone emergency manual"—try low-cost methods first, then seek a professional doctor if needed.

Phone Lagging and Overheating—The Most Frequently Asked Question

Honestly, it's totally normal for a phone to start lagging after about a year of use. The reason isn't necessarily that the phone is broken—more often it's "dust accumulation"—junk files, background processes, storage space slowly filling up.

Diagnose First, Don't Rush to Fix:

1. **Check storage space**: Go to Settings Storage to see remaining space. If it's under 10%, the system will become very laggy because it's struggling to free up space. My experience: keep at least 15%-20% free space for the phone to run smoothly.

2. **Check power consumption ranking**: Go to battery settings to see the power consumption ranking over the past 24 hours. If an app takes a large share but you barely used it, it's running in the background. Uninstall it directly or restrict background activity.

3. **Think about what you did recently**: Did it start lagging after a system upgrade? (It's normal to lag for the first few days after a major version update, as the system rebuilds indexes in the background.) Or did it start lagging after installing a certain new app? (Try uninstalling it.)

4. **Look at the usage environment**: If the phone is left in the blazing sun, or in a thick case while charging and playing games, overheating is inevitable. High temperature also causes the CPU to automatically throttle, making the phone feel suddenly laggy. Let it cool down and try again.

Treatment methods (from simple to complex): Restart the phone (solves 60% of problems) Clean up storage space (delete useless apps and videos) If a specific app is causing trouble, clear its cache or uninstall and reinstall Last resort: Back up data and factory reset.

App Crashes and Unresponsiveness—When Apps Misbehave

An app crashes when you open it, or suddenly freezes while in use. The handling process is like a doctor's consultation—step by step:

1. **Close and reopen first**: Swipe away the app from the background, then reopen it. Most temporary bugs are fixed with this one move.
2. **Update the app**: Check the app store for updates. Sometimes it's because the app version is too old and incompatible with the system.
3. **Check permissions**: Some apps need a certain permission to work properly, but you didn't grant it. For example, a camera app crashes when opened because it doesn't have camera permission. Go to Settings, find that app, and grant the necessary permissions.
4. **Clear cache**: Go to Settings App Management Find that app Storage Clear Cache. Note: "Clear Cache", not "Clear Data"! Clearing data will delete all your login status and settings in the app, equivalent to factory resetting the app.
5. **Uninstall and reinstall**: The last resort. Before uninstalling, confirm whether the app's data will be lost—for WeChat, back up chat history first; for games, confirm whether your account is bound to a phone number.

Won't Charge or Charges Slowly—It Might Not Be the Battery's Fault

Not being able to charge is a particularly anxiety-inducing problem, but often the cause is with the accessories, not the phone being broken.

- **Check the charging port first**: Use a toothpick or non-conductive small tool to gently clean out dust and lint from the charging port. I've encountered this myself—a thread from my pocket happened to block the charging port. When I plugged it in, I thought it was poor contact, but after cleaning it out, it was fine.
- **Try a different cable and charger**: Cables have a much higher failure rate than chargers, especially cables that are often rolled up. Swap in a known-good cable and charger for cross-testing to quickly determine which accessory is broken.
- **Wireless charging issues**: If you use wireless charging, check whether the phone case is too thick, or whether there's a metal plate on the back of the case (like a magnetic

mount). Also, place the phone precisely in the center of the charging pad—if it's off, it may not charge.

- **Slow charging**: Confirm the charger's power is sufficient (iPhone needs at least a 20W PD charger for fast charging), the cable supports fast charging (not all cables can handle 3A+ current), and the phone isn't too hot (too hot will automatically reduce power to protect the battery).

Can't Connect to Network, Wi-Fi Keeps Dropping

Mobile data works fine outside, but can't connect to Wi-Fi at home; or Wi-Fi is connected but there's no internet. Troubleshoot in this order:

1. **Toggle Airplane Mode**: Swipe down to Control Center, turn on Airplane Mode, wait 5 seconds, then turn it off. This is equivalent to making the phone search for the network again, very effective.
2. **"Forget" Wi-Fi and reconnect**: Go to Wi-Fi settings, tap the exclamation mark next to the network name (gear icon on Android), select "Ignore This Network" or "Forget", then re-enter the password to connect.
3. **Restart the router**: Many people's home routers haven't been turned off for months, and over time they become unstable. Unplug the power, wait 30 seconds, then plug it back in—most Wi-Fi problems are solved.
4. **Check if it's the phone's problem**: Do other devices connect to the same Wi-Fi normally? If they all work fine but your phone doesn't, the phone may need a restart or network settings reset (Settings General Transfer or Reset Reset Network Settings—won't delete your data, but will clear all Wi-Fi passwords).

Screen Issues: Unresponsive Touch, Flickering, Image Retention

- **Unresponsive touch**: First peel off the tempered glass film to see if it's the film's fault—cheap tempered films can indeed affect touch over time. Water on the screen or oil on your hands can also cause touch drift. If neither, restart and try again.
- **Marks/ghost image on screen (burn-in)**: Displaying the same image for a long time (like keeping the screen on while browsing TikTok) will leave "ghost image" on OLED screens. This isn't screen damage, just temporary. Turning off the screen and resting for half an hour usually recovers it.
- **Screen flickering**: If it flickers at low brightness, it may be a PWM dimming issue, more common on OLED screens. Look for "DC dimming" or "anti-flicker" option in Settings—turning it on helps a lot. Not all phones have this feature.

- **A completely dark/bright area on the screen**: This is more serious—likely physical screen damage, requiring screen replacement. Non-official repair screens usually have inferior color and brightness compared to originals, and also affect water resistance.

Won't Turn On, Stuck at Logo—Don't Panic First

This situation is most likely to make your heart race, but stay calm and try these first:

- **Charge for half an hour first**: Often the battery is just completely drained. Plug in the original charger, wait at least 30 minutes, then try to turn on. Some phones need to charge to a certain level before responding after being fully drained.

- **Force restart**: For iPhone full-screen models, press volume up once, volume down once, then press and hold the side power button until the Apple Logo appears (may need to hold for 10-15 seconds). General Android method: press and hold the power button for 10+ seconds until vibration or screen lights up.

- **Stuck at boot screen, rebooting repeatedly**: This is called "white Apple" (iPhone) or "stuck at Logo" (Android). If force restart doesn't work, you may need to enter recovery mode and use a computer to repair. Connect iPhone to computer, open iTunes/Finder, enter recovery mode, then select "Update" (not "Restore")—this will reinstall the system but preserve data. For Android, enter Fastboot or Recovery mode; key combinations vary by brand, recommend searching your phone model + "enter recovery mode".

- **If none of the above work**: Go to Wi-Fi settings, "Forget" your current Wi-Fi, re-enter password to connect.

Sound Issues: Speaker Not Working, Low Call Volume

- **No speaker sound**: First confirm it's not silent mode (iPhone side silent switch), Do Not Disturb mode, or volume turned all the way down. Sounds silly, but this accounts for 40% of cases. Then check whether Bluetooth is connected to a headphone or speaker.

- **Caller can't hear you**: Check whether the microphone hole is blocked—the small hole at the bottom of the phone is easily blocked by dust. Clean gently with a soft brush. Don't poke hard with a needle or toothpick, you'll damage the dust mesh.

- **Distorted/cracking sound**: If caused by water in the speaker—don't use a hair dryer on hot! Use a paper towel to absorb surface moisture, then let it air dry naturally. Online suggestions of water-ejection apps (playing low-frequency sound to vibrate water out) have some effect, you can try.

Finding Official Repair, Avoiding Pitfalls

If you've tried all the above methods and it still doesn't work, you need professional repair. But the repair market is full of pitfalls—let me share my own experiences and lessons learned:

- **Find real official, not "looks official"**: Go to the brand's official website (Apple.com.cn, Huawei.com.cn, etc.) to find the "authorized service provider" list. Non-authorized shops on the street with an Apple logo may not use original parts—after repair, your water resistance is gone in a flash.
- **Must-do before sending for repair**: Back up all data (don't assume data won't be lost during repair), log out of all important accounts (Apple ID, WeChat, Alipay), turn off "Find My Phone". Some repair shops will ask for your lock screen password, which has privacy risks—you'd better log out of accounts yourself before sending it over.
- **Screen replacement pitfalls**: Non-official screens come in "original-pressed" (original glass replaced), "aftermarket-pressed" (non-original glass), and "assembled" (all non-original). Prices differ by several times, but display quality and durability also differ a lot. If your phone is still under warranty, stick with official repair.
- **Battery replacement pitfalls**: Non-original batteries may have falsely labeled capacity, and after half a year of use, degrade more than original batteries do in two years. Apple's official battery replacement price isn't much more expensive than non-official—recommend going official.
- **Data recovery pitfalls**: If your phone gets water-damaged or broken, and the repair shop says "data recovery costs extra", be careful—some unscrupulous merchants will deliberately exaggerate the damage to charge more. Legitimate data recovery quotes a price first, then operates, and doesn't charge if recovery fails.

The Universal First Move: Restart Your Phone

No matter what problem you encounter—lagging, crashing, can't connect to network, sound issues—first restart your phone. This sounds like a brush-off, but in my own experience, at least 60% of minor issues are resolved after a restart. Phones, like computers, accumulate all kinds of junk in memory over time—restart clears it and starts fresh. So next time your phone has issues, don't rush to search "what to do"—restart first, then see.

Memorize Now: These Lifesaving Operations Should Be Second Nature

1. **Force restart**: Full-screen iPhone: press volume up volume down long press power button until Logo appears. Android: long press power button for 10-15 seconds until vibration.
2. **Clean charging port**: Find a small tool (toothpick or SIM ejector pin), gently clean dust from the charging port.

3. **Wi-Fi reconnect**: Go to Wi-Fi settings, "Forget" your current Wi-Fi, re-enter password to connect.
4. **Toggle Airplane Mode**: Swipe down to Control Center, turn on Airplane Mode, count to 5 in your head, turn it off, see if network recovers.
5. Screenshot the steps for these operations, or write them in Notes. When problems arise, you don't need to frantically search—just flip through Notes.

Chapter 20: Developer Mode and Advanced Permissions

This chapter is for users who want to deeply explore the "hidden features" of their phone. Developer Mode, Root permissions, BL lock... these geeky-sounding concepts are actually not far from ordinary users.

Developer Mode: Unlock Hidden Settings

Developer Mode is a hidden menu built into the phone system, containing many features that ordinary users don't need but developers do.

How to enable:

- **Android**: Go to Settings > About Phone > tap "Build Number" 7 times continuously, until it prompts "You are now in developer mode".
- **HarmonyOS**: Settings > About Phone > tap "HarmonyOS Version" 7 times continuously.
- **iOS**: Ordinary users cannot enable full Developer Mode—it requires a paid Apple Developer account.

Commonly Used Features Overview:

- **USB Debugging**: A must-enable switch for connecting to a computer for advanced operations (such as flashing, data recovery, ADB commands). See the "ADB Practical Commands" section in this chapter.
- **Show Taps**: Shows touch points on screen (a small dot following your finger), suitable for recording tutorials or demos.

Performance & Smoothness Related:

- **Animation Speed Adjustment (Window Animation Scale / Transition Animation Scale / Animator Duration Scale)**: These three options control the animation

duration when opening apps and switching pages. The default is 1x; setting to 0.5x makes the phone ”feel” faster (the actual program speed hasn't changed, just the animation is quicker); setting to off disables animations entirely. Most geeks set it to 0.5x —it preserves transition feel while seeming fast. Completely disabling animations can make operations feel abrupt and may mask real program stuttering, misleading you into thinking it's ”very smooth.”

- **Force GPU Rendering**: Forces all 2D drawing to be processed by the GPU instead of the CPU. Can improve scrolling smoothness on older models or with poorly optimized apps, but also increases GPU load and power consumption. Modern phones already use GPU rendering extensively by default, so this toggle has limited impact on mid-to-high-end models —it's more of a legacy option. Ordinary users should keep the default.
- **Profile GPU Rendering**: Displays a real-time bar chart on screen showing the rendering time of each frame. The green horizontal line represents 16.6ms (the frame time threshold for 60fps). Bars above the green line indicate dropped frames. This is a developer tool for troubleshooting app stuttering —ordinary users can also use it to compare which apps are poorly optimized. For example, if an app's bars frequently spike above the red line during scrolling, it's basically that app's fault.

Background & Battery Related:

- **Settings in Developer Mode may affect system stability—don't change options you don't understand. After enabling USB Debugging, be careful when connecting to computers, and don't connect to untrusted computers.**: You can set ”Standard limit,” ”At most 1 process,” ”At most 2 processes,” etc. On the surface, limiting background processes seems like it would save battery, but Android's built-in background management is already quite good. Forcibly limiting it causes apps to reload frequently (cold starts actually consume more power and worsen the experience). This option is more suited for debugging —ordinary users shouldn't keep it enabled long-term; ”Standard limit” is fine.
- **Don't Keep Activities**: Destroys every activity as soon as you leave it. Highly discouraged —causes frequent white screens and reloads, only for testing app recovery logic.

Notes

: Prevents malicious software from being flashed into the system.

II. OEM Unlocking: The “Flashing Switch” in Developer Options

In Developer Options, there's an inconspicuous but important toggle —“OEM Unlocking” (some brands call it “Allow OEM Unlocking” or “Device Unlocking”). This toggle doesn't unlock the Bootloader itself —it's a “prerequisite permission” : only when enabled will the manufacturer allow you to unlock the BL via fastboot commands or brand tools.

- **Purpose of the Toggle**: Equivalent to declaring to the manufacturer "I agree to waive my warranty and accept the risks —please allow me to unlock the BL." If this toggle is off, any BL unlock command will be rejected outright.
- **Relationship with the BL Lock**: OEM Unlocking is the "qualification application," while BL Unlock is the "actual operation." The former is just a tap in phone settings; the latter requires applying for an unlock code from the manufacturer's website and running fastboot commands on a computer (such as`fastboot flashing unlock`), and both are indispensable.
- **Does Factory Reset Reset It?**: Yes. The "OEM Unlocking" toggle returns to off after a factory reset, but once the BL is unlocked, it's a physical-level state —a factory reset won't relock it. You need to manually execute`fastboot flashing lock`to relock it.
- **When Should You Enable It**: Unless you're certain you want to flash or Root, keep it off. It and "USB Debugging" are independent toggles —both usually need to be enabled simultaneously when flashing.

Trivia: Why Do Manufacturers Include This Toggle

Early Android didn't have this toggle, so malware could silently flash the system as long as it got permissions. Google later required all Android manufacturers to include the "OEM Unlocking" toggle in Developer Options as a "user informed consent" confirmation. This is why even if you've unlocked the BL, reflashing will still fail if the OEM Unlocking toggle is turned off.

III. Bootloader Lock: The First Gate to Flashing

: Even if the phone is flashed, it can't be used without the original account password.

Warranty restriction

- **: Unlocking BL usually means losing official warranty.**BL lock policies by brand:
- **Xiaomi**: Can be unlocked, requires account binding waiting period (7-30 days).
- **Huawei**: Stopped providing unlock codes after 2018, basically cannot be officially unlocked.

OnePlus

- **Xiaomi**OPPO/vivo
- **Huawei**Samsung
- **: Can be unlocked, but triggers Knox fuse, losing Samsung Pay and other features.**Apple

- **[Install Apps from External Sources]**3. Root Permission: Android's "Super Administrator"
- **Samsung**Benefits of Root:
- **Apple**Use advanced feature apps (like Titanium Backup, Greenify).

IV. ADB Practical Commands: Controlling Your Phone via Computer

ADB stands for Android Debug Bridge, an official Google command-line tool that lets a computer communicate with an Android phone via USB or Wi-Fi. It doesn't require Root — as long as the phone has "USB Debugging" enabled, you can use it. Think of it as a "remote control that sends commands from your computer to your phone" —flashing, installing apps, transferring files, checking logs, all rely on it.

How to Connect via USB:

1. On the phone: Enter Developer Options enable "USB Debugging."
2. On the computer: Download Platform Tools from Google's official site (includes`adb`verb`fastboot`two executable files), extract to any directory.
3. Connect the phone and computer with a data cable. The phone will show a popup "Allow USB debugging?" —check "Always allow" and confirm.
4. In the computer's command line (Windows PowerShell or cmd), execute`adb devices`. If the output is similar to`List of devices attached /xxxxxxxx device`, the connection is successful.

Don't Ignore Driver and Authorization Popups

If Windows users see`adb devices`showing "unauthorized" or not listing the device at all, it's usually because the driver isn't installed properly or you forgot to tap "Allow" on the phone. Samsung/Xiaomi and other brands can install their respective "USB Drivers" or "Phone Assistant" to fix the driver; Mac and Linux generally don't need drivers. If you accidentally dismissed the phone popup, just unplug and replug the data cable and it will pop up again.

Commonly Used ADB Commands:

- `adb install app.apk`: Installs an APK from the computer to the phone (`-r`means reinstall while keeping data;`-s`installs to SD card).
- `adb uninstall com.xxx.xxx`: Uninstalls an app by package name.
- `adb push [computer path] /sdcard/[target path]`: Pushes a file from the computer to the phone.

- `adb pull /sdcard/[file] [computer path]`: Pulls a file from the phone to the computer (much faster than sending it to yourself on WeChat and downloading).
- `adb shell`: Enters the phone's Linux shell, can directly execute`ls`and`pm`and`am`and other commands.
- `adb reboot bootloader`: Reboots the phone into Fastboot mode (the entry point for flashing).
- `adb reboot recovery`: Reboots into Recovery mode.
- `adb shell pm list packages`: Lists all installed app package names on the phone (essential for finding package names). Add`-s`to see only system apps, add`-3`to see only third-party apps.
- `adb shell pm uninstall --user 0 [package name]`: Uninstalls pre-installed apps for the current user —no Root required. See the practical guide below.

Practical Guide: Uninstalling Pre-installed Apps Without Root:

This is one of ADB's most practical features: using the`pm uninstall --user 0`command, you can uninstall pre-installed apps for the "current user" without Root and without affecting the system partition. Uninstalled apps "disappear" from the system but are actually still in the system partition —they'll come back after a factory reset.

1. First use`adb shell pm list packages -s`to list all system apps and find the package name you want to uninstall. For example, Xiaomi Browser is`com.android.browser`, and Huawei AppGallery is`com.vmall.client`.
2. Not sure which app a package name corresponds to? You can use`adb shell pm list packages | findstr [keyword]`to search on the computer, or install a third-party app "Package Viewer" on the phone to directly view each app's package name.
3. Execute the uninstall:`adb shell pm uninstall --user 0 [package name]`. For example,`adb shell pm uninstall --user 0 com.android.browser`. If it returns`Success`, it's successful.
4. Want to restore? Two methods: (1) Factory reset (most thorough but cumbersome); (2) Reinstall:`adb shell cmd package install-existing [package name]`.

Never Delete Critical System Packages

Although "pm uninstall –user 0" is a safe method that doesn't require Root, deleting critical system apps (such as`com.android.systemui`status bar,`com.android.phone`phone,`com.android.settings`settings) can cause serious problems like system crashes, black screens, and inability to boot. Before deleting, Google "can this package name be deleted" —better to

delete less than to delete wrong. Recommend deleting only one or two at a time and confirming everything is fine before continuing.

Trivia: What Does –user 0 Mean?

Android 4.2+ supports multi-user, where each user has an independent ID (the owner is 0, additional users start from 10).`--user 0`means “uninstall for the current owner only”—the APK file in the system partition still exists, it’s just not shown to the owner. That’s why it doesn’t need Root —it doesn’t touch system files, it just changes the “user configuration.” This is also the biggest difference from Root uninstalling: Root uninstalling actually deletes from the system, so no user has it anymore; ADB uninstalling just “hides it for the current user.”

V. Root Access: Android’s “Super Administrator”

Root access is equivalent to the superuser in Linux (full control over the / directory, can read/write any file), the highest privilege level in Android. It fundamentally differs from Windows Administrator: Windows imposes many restrictions on administrators for security (the true highest privilege is actually TrustedInstaller), and even Administrator can’t directly execute like Linux root can.`sudo rm -rf /*`. After obtaining Root access, you can modify system files, uninstall pre-installed apps, and use advanced apps that require Root.

Risks of Root:

- Loss of warranty
- : Most manufacturers don’t warranty Rooted phones.
- Security risks
- : Malicious apps may obtain Root permission, causing serious consequences.

System instability

- **XDA Developers**Banking apps won’t work
- **: Many banking and payment apps refuse to run when Root is detected.**OTA update failure
- **: May not be able to receive official system updates normally.**Severe Warning
- **Root operations carry certain risks and may brick your phone (won’t turn on). If you’re a beginner, recommend fully understanding the relevant knowledge first and ensuring you have a complete backup. Read tutorials carefully before operating, and confirm your phone model and system version.**4. Advice for Ordinary Users

- **Developer Mode**: Can be enabled to experience animation acceleration and other features, but don't randomly change other settings.

BL lock

: If you don't plan to flash, keeping it locked is safer.

VI. SafetyNet and Play Integrity Certification: Why Bank Apps Don't Work After Root

Many Root users have experienced this devastating moment: after Rooting and excitedly installing a bunch of modules, you open your bank app — "Your device is insecure and cannot be used," Google Pay doesn't work, and even Netflix refuses HD playback. Behind this is Google's two device integrity certification mechanisms: the older SafetyNet and the newer Play Integrity API.

What Are SafetyNet / Play Integrity:

- **SafetyNet Attestation (Legacy, since 2017)**: Google's early device integrity check API, where apps query Google servers to check "whether this device passed CTS compatibility testing and is in an unmodified official state." Results are divided into Basic Attestation and Strong Attestation (relies on hardware keys). Google has announced that SafetyNet will be gradually deprecated after 2024.

- **Play Integrity API (New, since 2022)**: Google's replacement, stricter and more comprehensive than SafetyNet. In addition to device integrity, it includes "app integrity" (whether the app has been tampered with) and "account/licensing integrity" (whether the user account is trusted by Google). New apps should migrate to Play Integrity.

- **Who Uses It**: Banking, financial, and payment apps (such as Google Pay, various banks' mobile banking, some scenarios of Alipay/WeChat Pay), streaming media (Netflix HD copyright protection), some game anti-cheat (such as some Tencent/NetEase games), enterprise office apps (MDM device compliance checks).

Play Integrity's Three Certification Levels:

- **MEETS_DEVICE_INTEGRITY**: The device runs genuine Android, Bootloader is locked, no Root, CTS compatibility test passed. Banking and payment apps work normally.

- **MEETS_BASIC_INTEGRITY**: The device is basically trustworthy, but may have minor anomalies. Some apps will degrade (e.g., Netflix still plays but limits resolution), some strict bank apps will refuse outright.

- **MEETS_STRONG_INTEGRITY**: The highest level, requires hardware key endorsement (such as the Titan M chip in Google Pixel series). Most third-party brand devices can't reach this level even without Root.
- **No Label (NO_INTEGRITY)**: The device is Rooted, BL is unlocked, a third-party ROM is flashed, or it's identified as an emulator. The vast majority of payment/banking apps will refuse to run.

Why Do These Apps Fail After Root:

Apps initiate certification requests to Google servers through the Play Integrity API. Google checks multiple device indicators —Bootloader lock status, whether the system partition has been modified, whether su binary files exist, whether SELinux is in permissive mode, etc. Any anomaly causes the returned certification level to be lowered or empty, and bank apps see "device insecure" and naturally refuse service.

Modern Hiding Solutions: Magisk Hide / Shamiko / Zygisk:

To enjoy Root convenience while still using banking apps, the community has developed a series of solutions to "hide the Root state." The core idea is the same: make the detection program see a "normal-looking" device instead of a modified system.

- **: Old-school Root solution, currently no longer updated.**: The current mainstream Systemless Root solution —doesn't modify the system partition, but achieves Root through mounting. The benefit of this design is a lower chance of OTA update failure and cleaner uninstall. It's the foundation for all subsequent hiding solutions. However, using Magisk still requires unlocking the BL.
- **Magisk Hide (Discontinued)**: Magisk's early built-in hiding module, could hide Root from specified apps. But since 2021 it was removed by Magisk officially, because Google continuously upgraded detection methods, and the two sides fell into a "cat-and-mouse game."
- **Zygisk**: A new mechanism introduced in Magisk 24.0+, similar to Riru, injects into the Zygote process to load modules, more stealthy than directly modifying the system. Enabling Zygisk is a prerequisite for using modern hiding solutions.
- **Shamiko**: The current mainstream hiding module, works with Zygisk, can hide Root for specified apps ("reverse whitelist" mode), hide Magisk itself, and fake the Bootloader lock status. Shamiko is distributed through the LSPosed repository or GitHub.
- **Play Integrity Fix(osm0sis)**: A Magisk module specifically targeting Google Play Integrity detection, modifies the device fingerprint to make Google think your device is an unrooted official model.

The Cat-and-Mouse Game Never Ends

Google updates detection rules periodically, and app developers follow up with Play Integrity upgrades. A hiding solution that works today may be detected tomorrow. If you fully rely on banking/payment apps (such as daily transfers, QR code payments), the safest choice remains: don't Root, don't unlock the BL. Hiding solutions are more suitable for a "main phone without Root + backup phone for Root experimentation" enthusiast setup.

VII. Advice for Ordinary Users

- **: Not recommended for ordinary users—risks outweigh benefits. If you really need certain features, first look into whether there are alternatives that don't require Root.**: You can enable it to experience animation acceleration, GPU rendering analysis, and other features, but don't randomly change other options you don't understand.
- **OEM Unlocking and BL Lock**: If you don't plan to flash or Root, keep both toggles in their default (off) state for maximum safety.
- **ADB**: A powerful no-Root tool that ordinary people can use too —for example, using`pm uninstall --user 0`to uninstall pre-installed apps. But before deleting, always check whether the package name can be safely deleted.
- **Root**: Root is not recommended for ordinary users —the risks outweigh the benefits. If you truly need certain features, first check whether there are no-Root alternatives (such as using ADB to uninstall pre-installed apps, or using Shizuku to replace some Root functionality).

Hands-On: Enable Developer Mode and Adjust Animation Speed

1. Go to Settings > About Phone.
2. Tap "Build Number" 7 times in a row until it says Developer Mode is enabled.
3. Go back to Settings and find "Developer Options" (usually at the bottom of Settings or under "System").
4. Find "Window animation scale", "Transition animation scale", and "Animator duration scale", and set them all to 0.5x or off.
5. Feel whether the phone has become faster.

Chapter 20 Appendix: Termux—A Portable Linux Terminal Environment on Android Devices

If you feel Developer Mode isn't "hardcore" enough and want to experience a complete Linux command line on your phone, then Termux is for you. It requires no root permission,

yet can turn an ordinary Android phone into a portable Linux workstation—a powerful tool for advanced users to explore mobile development and operations.

What Is Termux

Termux is an open-source terminal emulator and Linux environment application designed specifically for Android devices. It provides a complete Linux shell experience on your phone without requiring root permission. As a lightweight application (installation package only about 30MB), Termux supports a rich software ecosystem through the APT package management system, allowing users to run tools like Python, Node.js, Git, SSH on mobile devices, and even set up servers or perform penetration testing.

Termux's core advantages lie in its lightweight nature and high extensibility, suitable for development, learning, and system administration scenarios. It combines standard software packages with precise terminal emulation in an open-source solution.

Core Features and Characteristics

Complete Linux Environment

Termux**Only Bash is Built-in**is the only built-in shell. If you want to use Zsh, fish, or other shells, you need to install them manually via the package manager, for example:`pkg install zsh`or`pkg install fish`. Termux also supports editors like Vim, Nano, and Emacs, providing a near-native command-line experience. Users can install 2000+ packages via "pkg install," including GCC, OpenSSH, SQLMap, and other development and security tools.

Powerful Package Management

Uses the APT package manager (similar to Debian and Ubuntu), can install thousands of Linux software packages, covering development tools, network tools, and more. Supports multiple programming language environments including Python, Ruby, Node.js, Go, Rust, and Swift.

Security and Network Features

- **SSH remote connection**: Supports SSH client and server features—connect to other devices remotely or turn your phone into an SSH host.
- **Encryption tools**: Provides encryption tools like OpenSSL, supports data encryption and secure communication.
- **Server setup**: Can configure Nginx or Apache to set up a static web server.

System Integration and Extension

- Access phone storage permissions for local file management.
- Call Android hardware features like sensors and cameras through the Termux:API plugin.
- Supports Bluetooth keyboard and external display connection, with full mouse support.

Plugin Ecosystem

Termux's core application can be extended through several optional plugin apps, providing additional functionality and integration:

Plugin Name	Feature Description
Termux:API	Exposes Android API functionality to command-line programs
Termux: Boot	Allows running scripts when the device boots
Termux: Float	Provides a floating terminal window
Termux: Styling	Enables color schemes and font customization for the terminal
Termux: Tasker	Integrates Termux scripts with the Tasker automation app
Termux: Widget	Allows running scripts from home screen widgets

Important Note

Termux and its plugins must be installed from the same source, because they share the same sharedUserId. Mixing installations from different sources (such as F-Droid and GitHub) will result in installation errors.

Installation and Configuration Guide

Installation Steps

1. **Download the app**: Download and install Termux from official channels (F-Droid or GitHub), no root permission required.
2. **Initialize the system**: Wait for "Installing bootstrap packages" to complete after first launch.
3. **Get storage permission**: Execute`termux-setup-storage`to get phone file system permission.
4. **Update package sources**: Execute`: Use the`to update the package list.

Basic Optimization

- **Change mirror source**: Use the`Enter`command to change the mirror source through a graphical interface (Tsinghua University mirror recommended).
- **Shortcut settings**: Volume down key simulates Ctrl, volume up key simulates Esc, improving input efficiency.
- **Interface beautification**: Install Zsh + Oh My Zsh, configure themes and plugins.

Typical Use Cases

Mobile Development Testing

Set up a complete development environment on your phone, write and run Python scripts, compile C programs with GCC, manage code repositories with Git, and sync with remote servers via SSH.

Server Operations

Remotely manage servers via SSH, perform system maintenance, log analysis, and other tasks, handling server issues anytime, anywhere.

Learning Linux Operations

Provides a real Linux environment, an ideal platform for learning the command line and system administration—you can practice Shell scripting and Linux file permission management (such as`Network Security Testing`Install security tools like Nmap, SQLMap, Hydra, and Metasploit for network scanning and penetration testing.`6.Version and Compatibility`The latest stable version of Termux is v0.118.2, suitable for devices with Android $\geq$ 7. Note that Termux may have instability on Android 12 and higher versions; recommend following the official update log for the latest compatibility information.`Provides a real Linux environment, making it an ideal platform for learning command line and system administration.You can practice Shell scripting, Linux file permission management (such as`chmod 755

Network Security Testing

Install Nmap, SQLMap, Hydra, Metasploit, and other security tools for network scanning and penetration testing.

Versions and Compatibility

The latest stable version of Termux is v0.118.2, for devices running Android 7.0 and above. Note that Termux may be unstable on Android 12 and higher; follow the official changelog for

the latest compatibility information.

Summary

Termux brings the powerful Linux terminal experience to Android devices, making the phone a portable development and learning tool. Whether developers, system administrators, or tech enthusiasts, all can use Termux to enjoy a complete command-line experience on mobile devices, realizing the concept of “Linux in your pocket”.

Hands-on: Run Your First Linux Command on Your Phone

1. Download and install Termux from F-Droid, wait for initialization to complete on first launch.
2. Execute`pkg update && pkg upgrade`to update packages.
3. Install Python:`pkg install python`.
4. Enter`python3 -c "print('Hello from Termux!')"`, and experience the joy of running code on your phone.

IV. Beyond (Chapters 21-26)

Chapter 21: In-depth Exploration of Mobile Networks and Information Security

Using a VPN

: Your phone VPN encrypted tunnel VPN server target website. The data is encrypted and encapsulated, so "peeping toms" can only see a bunch of garbled text. Additionally, the target website sees the IP address of the VPN server, not your real IP.

So what can a VPN actually do? Simply put: first,

- **protect your privacy on public Wi-Fi**; second,
- **bypass geographical restrictions**(such as watching content only available in certain countries); third,

prevent your Internet Service Provider (ISP) from logging your browsing habits**. But I must remind you:**don't use free VPNs!**Free often means your data is their "commodity." I have a friend who used a free VPN, and his phone started popping up ads for no reason. Later we found out that the VPN had packaged and sold his browsing data to advertisers. So when choosing a VPN, either choose a paid big brand (like NordVPN, ExpressVPN), or simply don't use one.**Also, VPN is not omnipotent. It can only protect data "in transit." If the target website itself is insecure, or your phone already has malware, a VPN can't help you.**2. The "Sweet Trap" of Public Wi-Fi—More Dangerous Than You Think**You might think, it's just connecting to free Wi-Fi, how much risk can there be? Let me tell you a true story: security researchers once did an experiment where they set up a Wi-Fi hotspot called "Starbucks_Free" at an airport, and within an hour, over a hundred people connected. In reality, this hotspot was set up by the researchers themselves, and all**unencrypted traffic**passing through this hotspot—such as the

content of HTTP web pages you visit and App communications without TLS enabled—they could indeed intercept and view.

But note a key premise: today, the WeChat, online banking, and Alipay you use daily have mostly already adopted HTTPS encryption. In other words, even if attackers intercept the data packets, what they see is just a bunch of garbled text, and they

cannot directly read your chat content or payment password

. However, they can still know which websites you visited (domains). If certain old Apps or web pages are still using HTTP plaintext transmission, the content will be completely exposed. This type of attack is called a “Man-in-the-Middle”attack, and the principle is that the attacker “inserts themselves” between you and the real Wi-Fi router. What’s more frightening is that many people’s phones are set to “automatically connect to known Wi-Fi.” Attackers only need to set their hotspot name to one you’ve connected to before (such as “ChinaNet” or “CMCC”), and your phone will automatically connect to it.**So how to defend against this? Here are a few practical suggestions:**Look for HTTPS

: The address bar starts with https://, and the extra “s”stands for Secure. When accessing any website that requires a password, if you see http:// (without the s), stop immediately and don’t enter anything! Now mainstream browsers will display an “insecure” warning before non-HTTPS websites, so be alert when you see this mark.**Turn off “auto-connect”**: Whether iPhone or Android, turn off “auto-join” or “auto-connect” in Wi-Fi settings. Rather than letting your phone judge for itself, manually tap each time.

Use mobile data for sensitive operations

- **: Transfers, logging into online banking, entering ID numbers... honestly use your own 4G/5G data for these operations, don’t save that little bit of data.**Forget infrequently used Wi-Fi
- **: After returning from a business trip or travel, ”forget” all the hotel Wi-Fi and airport Wi-Fi you temporarily connected to before. This way, next time your phone won’t automatically connect to a fake hotspot with the same name.**3. How Does HTTPS Protect You?—SSL/TLS Handshake
- **I just mentioned HTTPS, so how does it actually work? Every time you visit an HTTPS website, an ”SSL/TLS handshake” occurs between the browser and the server. The whole process goes something like this:**Your browser says: ”Hey, server, I want to communicate securely with you, here’s my list of encryption capabilities (supported encryption algorithms, etc.).”
- **The server replies: ”Okay, I’ve selected this set of encryption schemes from your list, here’s my digital certificate, which contains my public key, signed and certified by a certain CA organization.”**The browser verifies the certificate’s

authenticity: Was it issued by a trusted CA? Has it expired? Does the domain name match?

After verification passes, both parties use public key encryption to negotiate a "session key," and all subsequent communication is encrypted with this key.

This is like going to a bank to do business: you first hand over your ID card (certificate), the teller verifies your identity, then the two of you agree on a secret code, and subsequent conversations can only be understood by the two of you. Even if someone eavesdrops nearby, they can only hear encrypted garbled text.

1. But note: HTTPS only guarantees that the "transmission process" is encrypted, not that the target website is a good actor. Phishing websites can also use HTTPS—that little lock icon only indicates that you're connected to "some website," not that the website is the one you want to visit. So you still need to verify the URL yourself.

2. 4. DNS Security: Is Your "Network Phonebook" Safe?

3. DNS (Domain Name System) is like the internet's phonebook: you type "baidu.com," and DNS helps you look up which IP address it corresponds to (such as 110.242.68.66), and then the browser can connect. The problem is that traditional DNS queries are

4. plaintext

It's just like going to a bank: you hand over your ID (the certificate), the teller verifies who you are, and then the two of you agree on a secret code. From that point on, only you two can understand the conversation —even if someone eavesdrops nearby, all they hear is encrypted gibberish.

not encrypted

! This means your ISP, Wi-Fi provider, or any intermediate node can see which websites you've visited.

What's even more disgusting is "DNS hijacking": attackers tamper with the DNS process to point "baidu.com" to a fake phishing website. You clearly typed the correct URL, but you're taken to an identical but actually fake login page, and once you enter your account password, you're done for.**Fortunately, there are now encrypted DNS solutions:**DNS over HTTPS (DoH)**: Hides DNS queries in HTTPS traffic, mixed in with normal web browsing traffic, making it impossible to distinguish which ones are DNS queries.**DNS over TLS (DoT)

: Uses the TLS encryption protocol to specifically protect DNS queries, with an independent port. Network administrators can know you're using DoT but can't see the content.

I personally recommend using Cloudflare's 1.1.1.1 or Google's 8.8.8.8 as DNS servers; both support encrypted DNS. In Android's "Private DNS" setting, just enter "one.one.one.one" or "dns.google," super simple. iPhone is a bit more troublesome, requiring you to install a profile or set it up through an App, but it's worth doing.

- **5. Does "Incognito Browsing" Really Make You Invisible?**First, the conclusion:
- **Incognito mode is your window curtain, not an invisibility cloak.**What it does: It doesn't leave traces locally—after closing the incognito window, the browser doesn't save your browsing history, cookies, form fill-in records, or search history. It's appropriate to use incognito mode when looking something up on someone else's computer or buying a gift without family members finding out from browsing records.

What it can't do:

The website you visit

: When closed, it's the size of a regular phone; when opened, it's nearly square like a small tablet. Representatives: Huawei Mate X series, Samsung Galaxy Z Fold series, Xiaomi MIX Fold series, vivo X Fold series. Main selling point: "one phone = phone + tablet" .**; your IP address, device information, and browsing behavior are clear on the website's end.**

Your network operator

can still see which websites you visited

- , just not the specific content (if HTTPS is used).**Company/school network administrators**can also monitor
- your internet behavior.**Files you download and bookmarks you add will**be retained
- and won't disappear when you close the window.**So remember: Incognito mode is a "local eraser," not a "network invisibility cloak." If you really need to hide your internet traces, you need a combination punch like VPN + Tor browser, but for daily use, knowing this much is enough.**6. Browser Privacy: Don't Let the Browser Sell You Out
- You may not realize that the browser itself is a huge information collector. Cookies, browsing history, cache, autofill, location permissions... each one could leak your privacy. A few suggestions:**Clean browsing data regularly**: Every once in a while, go to settings and clean out cookies, cache, and history. Not to prevent "being tracked"—that's no longer very effective by clearing cookies—but to prevent your phone from getting slower and slower. Browser cache can take up several GB of space.

Give up the “Do Not Track” fantasy

: The “Send ’Do Not Track’ request” option in browsers is essentially “asking” websites not to track you. But whether websites listen is another matter entirely; most websites simply ignore it. Don’t count on it.

Make good use of privacy-protecting browsers

- **: Firefox Focus, DuckDuckGo browser, Brave—these browsers block trackers and ads by default, much cleaner than the default settings of Chrome and Safari.**Search engines matter too
- **: Use DuckDuckGo instead of Baidu/Google search; it doesn’t record your search history or build user profiles. Honestly, the search results are indeed not as precise as Google, but it’s enough for daily research.**Check App permissions
- **: Go to phone settings and see what permissions your browser has been granted. Does it really need to access your location, microphone, contacts? Probably not.**7. Practical Security Checklist: Protect Yourself Starting Now
- **Having said all this, here’s a simple and straightforward security checklist. Spend half an hour to get it done, and your phone’s security will have a qualitative leap:**Turn off Wi-Fi auto-connect
- **: iPhone in Settings-Wireless LAN, Android in Wi-Fi settings.**Switch to an encrypted DNS

: Android directly set Private DNS to “dns.google” ; iPhone use an App or profile to set up 1.1.1.1’s DoH.

Enable Two-Factor Authentication (2FA)

1. **: WeChat, Alipay, Apple ID, Google account—enable 2FA wherever possible. SMS verification codes are already the lowest level of 2FA; using TOTP (such as Google Authenticator, Microsoft Authenticator) is even better.**Review all App permissions
2. **: Go through each App’s permissions, especially these five: location, contacts, SMS, microphone, camera. You’ll find some flashlight Apps surprisingly want your contacts—uninstall them immediately.**Clean browser data regularly

3. **: Set a calendar reminder, once a month, to clean cache and cookies.**Don’t use data cables from public charging stations

4. **: USB ports can both charge and transmit data; malicious charging stations (called ”Juice Jacking”) can steal data while you charge. Bring your own charger head and cable when going out.**Do It Now: Give Your Phone a ”Security Checkup”

5. **DNS upgrade**: Android: Settings > Connections & Sharing > Private DNS > enter ”one.one.one.one” or ”dns.google”. iPhone: Use Safari to search ”1.1.1.1 DNS profile” to install the profile, or download Cloudflare’s 1.1.1.1 App.

6. **Wi-Fi audit**: Open the Wi-Fi list and ”forget” all unfamiliar, infrequently used, and strangely named hotspots.

Permission cleanup

1. **: Go to Settings > Privacy/Permission management, review permissions for each App. Mark Apps with unreasonable permission requirements (such as a calculator wanting contacts), and consider uninstalling or using alternatives.**Check account security

2. **: Open the security page of your Apple ID or Google account, see if there are any unknown device logins, confirm 2FA is enabled, and check whether recovery contacts are trustworthy.**Chapter 22: How AI Works in Mobile Phones

3. **Welcome to the coolest ”magic show” on mobile phones, starring: Artificial Intelligence. The ”snap-and-get” beautiful photos you use every day, the voice assistant that responds as soon as you open your mouth, the input method that understands you better the more you use it—all have this invisible brain standing behind them.**1. AI Camera: Your ”Post-Production Darkroom” Moved into the Phone

4. **Today’s AI computational photography is more like stuffing a professional retoucher + photographer team directly into the shutter button. Before you press the shutter, AI has already started working: scanning the scene, identifying ”this is blue sky,” ”this is a face,” ”this is a night scene.”**2. Voice Assistant: How Does Your ”Voice Secretary” Understand You?

Chapter 22: How AI Works in Phones

: Converts sound waves into text.

Understanding (natural language understanding)

: Understands intent, extracts key entities.

Execution and response

1. **: Calls the corresponding App or service, then "speaks" to you through speech synthesis.**3. On-Device AI vs. Cloud AI
2. **On-device AI keeps computation in the phone, with fast speed, high privacy, and offline capability. Cloud AI requires internet connection but can handle more complex tasks.**4. AI LLMs and Mobile Phones: Your Portable Smart Assistant
3. **In 2024-2025, the explosion of AI LLMs (such as DeepSeek, ChatGPT, Wenxin Yiyan, Tongyi Qianwen, etc.) profoundly changed the way mobile phones are used. Phones are no longer just tools that execute instructions, but intelligent partners that can understand, reason, and create.**What is an AI LLM?

AI LLMs (Large Language Models) are AI systems trained on massive amounts of text data that can understand and generate natural language. You can imagine it as a "super scholar" who has read almost all the text on the internet, capable of answering questions, writing articles, translating, programming, and analyzing data.

AI LLM Applications on Mobile Phones:

Conversational assistants

In 2025-2026, the explosion of AI large models (such as DeepSeek R1, ChatGPT-5, ERNIE Bot 5.0, Tongyi Qianwen 3, etc.) profoundly changed how phones are used. Phones are no longer just tools that execute commands —they have become intelligent partners that can understand, reason, and create. On-device AI chips (NPU computing power exceeding 80 TOPS) made large model inference run locally on phones for the first time, enabling AI without an internet connection.

Writing and creation

: Let AI help you write emails, summarize meeting minutes, polish articles, and generate creative copy.

Image generation and editing

- **: AI can one-click remove passersby from photos, expand image boundaries, convert styles, and even generate images based on text descriptions.**Real-time translation
- **: Real-time call translation, face-to-face dialogue translation, photo translation—no more language barriers when traveling abroad.**Learning tutoring
- **: AI can explain concepts, create practice questions, and grade homework; it's a portable tutor.**On-Device LLMs: Phones Run AI Themselves
- **Real-time translation**No network needed
- **: AI functions work even offline—on airplanes, in subways, anywhere.**More secure privacy

: Your conversations and data are not uploaded to the cloud; everything is processed locally on the phone.

Faster response

- **: No network latency; AI replies are almost instantaneous.**Current limitations
- **: On-device models still can't match cloud-based LLMs in capability; complex tasks still require internet. But as chip AI computing power improves, on-device models will become increasingly powerful.**System-Level AI Integration:
- **Apple Apple Intelligence**: The AI system introduced in iOS 18, integrating writing tools, image magic wand, enhanced Siri, notification summaries, and more. On-device processing first; only complex tasks go to the cloud.

- **Huawei Pangu LLM**: Integrated into the HarmonyOS system, the Xiaoyi assistant gains LLM capabilities, supporting document summaries, smart screen reading, AI photo editing, and more.

Xiaomi Pengpai AI

- **: System-level AI capabilities, supporting AI image search, AI summaries, AI real-time captions, on-device LLM conversations, and more.**Samsung Galaxy AI
- **: Real-time call translation, note AI summaries, image generation and editing, search circle selection, and other functions.**5. Several Typical Application Scenarios and Practical Operations
- **To give you a more intuitive feel for the "superpowers" of mobile AI, below we use five specific scenarios to show how AI "comes to the rescue" in daily life:**Scenario 1: Real-Time Translation—Your Personal Translator for Traveling Abroad
- **Actual performance**: Can't speak French when ordering in Paris? iPhone's "Translate" App supports a face-to-face dual-person dialogue mode, where both parties see subtitles in their own language, and can even do simultaneous interpretation. On the Android side, Samsung Galaxy AI's real-time call translation lets you speak Chinese directly on the phone, and the other party hears English, and vice versa.

Underlying principle

: A three-step pipeline of speech recognition (ASR) machine translation (NMT) speech synthesis (TTS), with end-to-end latency now controllable within 1 second.

Scenario 2: AI Erase—Saving Tourist Photos "Ruined" by Passersby

Actual performance: Took a beautiful selfie at a scenic spot, but the background is crowded with other tourists? Use "Magic Erase" to circle the passersby, and the phone will automatically identify and "erase" them, then intelligently fill in the background. The results are often stunning.

Underlying principle: The image segmentation (Inpainting) model first identifies and deletes the selected area, then "imagines" reasonable fill content based on surrounding pixels and semantic understanding. The AI model behind this is trained on millions of images.

Scenario 3: AI Photo Editing—One-Click to Become a "Photography Master"

Actual performance: Took a gray, hazy landscape photo on a cloudy day? AI "Magic Sky" can one-click replace the sky with clear sky, sunset, or even starry sky. AI image expansion

can turn a vertical composition into a horizontal one, with the phone "imagining"scenes beyond the frame.

Underlying principle: Based on Generative AI's Diffusion Model, it understands the semantic content of the image and then generates new pixels. This is the "cutting edge" of current AI image processing.

Scenario 4: Document Processing—An Efficiency Multiplier for Office Workers

Actual performance: After a meeting, throw the meeting recording to AI, and it can automatically organize structured meeting minutes, extract action items, and generate summaries. A 10-page English PDF report can be distilled by AI in 10 seconds to give you the core conclusions.

Underlying principle: Long Context LLMs can "read through" documents of tens of thousands of words, performing multi-round reasoning and information extraction. Apple Intelligence, Huawei Xiaoyi, WPS AI, Tencent Yuanbao, etc., all have this capability.

Scenario 5: AI Image Search and Object Recognition—Your "Visual Search Engine"

Actual performance: See an unfamiliar flower on the street, open the camera, and AI immediately tells you "This is a hibiscus." Looking through your album for photos taken at the beach last year, search for "ocean," and AI automatically finds all photos containing the sea, without manual scrolling.

Underlying principle: Computer Vision (CV) models vectorize (Embedding) image content, then match it with text queries for similarity. This "multimodal search" is a major leap in phone album management.

Prompt Optimization: The "Communication Formula"That Makes AI Follow Your Commands

Actual performancemost practical, easiest-to-use "Role-Background-Task-Constraint" communication approach

Underlying principle Why do we need a framework?

Many people use AI like using Baidu—typing in a few keywords. But AI is not a search engine; it's a language model that "reads, understands, and then answers."

Give it more context, clearly state what you want, and constrain it from going off track**, and it can give you precise results. A framework translates "what a good answer**

looks like" in your head into instructions AI can understand.1. Role: Tell AI Who It Is

Give AI a specific "identity," and it will draw on the professional knowledge and expression style of the corresponding field.

The more specific the role setting, the more professional the output**.**Practical examples:

"You are a

Xiaomi phone repair engineer with 10 years of experience**. "(Technical question)**.

The tone should be warm and natural, like chatting with a friend

- knowledge scope**"You are a**book editor with good aesthetics, well-read, and your own style
- knowledge scope**"You are a**cooking blogger who makes breakfast for office workers every day
- knowledge scope**"You are a**patient, gentle Traditional Chinese Medicine practitioner who never laughs at beginners
- knowledge scope**Why is it effective?**The role sets AI's
- knowledge scope**,**expression style

, then send to AI:professional depth**. An AI without a role is a "jack of all trades but master of none" ; with a role, it can "enter" that identity and think like that professional.**and**Clearly state "who you are, your current situation, your constraints" in advance, and AI can give answers that are**verb**, rather than generic talk.**Practical examples:

"My phone is a

Xiaomi 14, bought half a year ago, recently overheating severely when playing games**. "(Device situation)"** I am

The tone should be warm and natural, like chatting with a friend

- ." (User profile)**"I am**preparing for the June 2026 CET-6 English exam, with average foundation
- ." (Study background)**"I run**a small coffee shop in a third-tier city, with monthly revenue of 30,000 yuan and two employees

- ." (Business background)**"I**just had a baby and am doing postpartum confinement at home, severely sleep-deprived
- ." (Life stage)**Why is it effective?**Background information lets AI know
- "who to serve"**just had a baby and I'm on postpartum confinement at home, severely short of sleep**"under what conditions"

, then send to AI:3. Task: Clearly Tell AI What to Do**This step should start with a**verb**(write, list, compare, translate, explain, summarize...), and describe the task**specifically and executably

. Vague tasks only get vague answers.

Practical examples: **"Please**list 5 permissions I should immediately turn off**and explain why." (Clear action)** "Please

The tone should be warm and natural, like chatting with a friend

- The clearer the verb and the more specific the requirements, the better AI knows "what to deliver""**"Please**compare these four phones in the 3,000 yuan price range
- The clearer the verb and the more specific the requirements, the better AI knows "what to deliver""**"Please**explain 'what is 5G' in words a primary school student can understand
- The clearer the verb and the more specific the requirements, the better AI knows "what to deliver""**"Please**write three short video scripts for my breakfast shop
- The clearer the verb and the more specific the requirements, the better AI knows "what to deliver"**Why is it effective?**The task part is the "command center" for AI's actions.
- The clearer the verb and the more specific the requirements, the better AI knows "what to deliver"**. Vague words like "do something," "make a," "take a look" are taboos for AI—it will guess your meaning, and the results often go off track.**4. Constraints: Tell AI What Not to Do

, then send to AI:making AI converge on the output that suits you best**. Common dimensions include: word count, format, tone, things to avoid, things to include.**Practical examples:

"

No more than 200 words, list in bullet points**. "(Length + format)"**

The tone should be warm and natural, like chatting with a friend

- , only explain principles." (Boundaries)"Don't use technical terms
- , only explain principles." (Boundaries)"Each paragraph no more than 3 lines
- , only explain principles." (Boundaries)"Must include: usage scenarios, pros and cons, my budget match
- , only explain principles." (Boundaries)"Don't give specific investment advice
- , only explain principles." (Boundaries)**Why is it effective?**Without constraints, AI tends to "freestyle"—either too academic, too long, or off-topic. Clear constraints = clear "delivery standards," aligning expectations on both sides, making results controllable.
- , only explain principles." (Boundaries)**Writing good prompts is not a one-time thing.**After writing, first self-check these five points

, then send to AI: Specific rather than abstract

: Not "write an article," but "write an 800-word, 3-section phone buying guide for new moms."

Include background rather than isolated**: Tell AI "who I am, why I'm asking this," so it can give you a "customized" answer.**Have a clear output format

1. **: Is it a table, list, paragraph, or JSON? State it in advance.**Constraints in place
2. **: Word count, style, things to avoid—any one of these three brings noticeable improvement.**Verifiable by AI
3. **: After asking, have AI repeat your requirements; if it can't articulate them clearly, you haven't been clear.**6. Common Problems and Solutions
4. **Symptom**Solution
5. **Answer too generic, not practical**Add background (who you are/what scenario)

Answer too academic/too official

Add tone constraints + reject "bureaucratic language" list	**Answer off-topic/not answering the question**
Restate the task with verbs and require "only answer this"	Answer too long to finish reading
Hard limit on word count (within 200 words)	AI speaks nonsense with a straight face
Add "if uncertain, mark 'I'm not sure'" + self-check important facts	Not satisfied with the first version
Don't rewrite; append "a bit more..." to the original prompt	7. Three Complete Scenario Cases
Scenario 1: Life Assistant (Breakfast Pairing Suggestions)	Prompt
:	You are a breakfast blogger who excels at nutritional pairing (

Role

). I am an office worker mom with only 15 minutes every morning to make breakfast, with a 6-year-old child and a 60-year-old mother-in-law at home (

discussing the Q2 marketing promotion plan:
list 7 days of non-repeating breakfast pairings(Task)**. Each breakfast requires: (1) Preparation time no more than 15 minutes (2) Nutritionally balanced (protein + carbs + fruits/vegetables) (3) Suitable for all ages (4) Ingredients available at ordinary supermarkets** (Constraints)**.**Why it's effective**: Role sets professionalism, background gives the commonalities of three types of people, task clearly defines deliverables, and 4 constraints ensure executability.**Scenario 2: Study Assistance (CET-6 Exam Preparation)**Prompt**Fill in the repository name, such as
: When unsatisfied with AI's answer, you can say "This is too professional, please explain again in words a primary school student can understand." experienced CET-6 English teacher

(Role). I am an ordinary undergraduate student,

discussing the Q2 marketing promotion plan:
Please organize the following meeting notes into structured minutes**(Task). Requirements: (1) Clear time allocation for listening, speaking, reading, and writing each week (2) Give the key breakthrough direction for each month (3) Recommend 3 suitable review materials (explain reasons in Chinese) (4) Output in table format (Constraints).**Why it's effective**: Role sets professional background, background gives score and time anchors, task clearly defines "phased plan," constraints require "tabular"and "executable."**Scenario 3: Work Efficiency (Meeting

Minutes Organization)**Prompt**:
: When unsatisfied with AI's answer, you can say "This is too professional, please explain again in words a primary school student can understand." senior administrative assistant

(Role). I just finished a 1-hour department meeting,

discussing the Q2 marketing promotion plan:
Please organize the following meeting notes into structured minutes**(Task). Requirements: (1) Divide into three parts: "Resolutions / Action Items / Risks and Dissents" (2) Mark the responsible person and deadline for action items (if not mentioned, write "to be confirmed") (3) Total word count within 600 words (4) Output in markdown format (Constraints).**Why it's effective**: Role sets administrative perspective, background gives meeting topic, task clearly defines structured output, 4 constraints make the result directly reusable.**Advanced Techniques: Make AI Your "Personal Assistant" **Try these advanced uses to experience AI's true power:**Multi-round dialogue optimization
: When unsatisfied with AI's answer, you can say "This is too professional, please explain again in words a primary school student can understand." Chain of Thought (CoT)

: Have AI "think step by step" ; answer quality is usually higher: "Please analyze this math problem step by step."

Multi-AI collaboration

- **: Use A to write the first draft, B to polish, C to translate—combining them works even better.**System prompts
- **: Preset your identity, preferences, taboos, etc. in AI settings to make answers better fit your needs.**7. AI's Limitations and Challenges: Maintain Clear Judgment
- **Although AI is powerful, it's by no means "omnipotent." Clearly understanding its limitations is key to using it better:**"Hallucination" problem
- **Challenge 2: AI Photo Editing**Knowledge cutoff

: Most AI's training data has a cutoff date (such as early 2024); it doesn't know what happened after that.

Limited reasoning ability

- **: Complex multi-step reasoning, mathematical proofs, and logical traps—AI still easily makes mistakes.**Privacy risks

- **: In cloud AI services, your conversations may be used for model training. For sensitive information, prioritize on-device AI.**Bias and values
- **: AI may carry biases from training data; you need to maintain independent thinking.**AI Usage Tips
- **AI-generated content may be inaccurate ("hallucination" problem); important information must be verified.**Don't enter personal privacy information into AI (ID numbers, passwords, bank information, etc.).
- **Make good use of prompts: The more specific and clear the description, the higher the quality of AI's answer.**AI is a tool, not an authority; maintain independent thinking and judgment.

For key decisions (medical, legal, financial), consult professionals rather than relying on AI.

- Advanced Techniques: Make AI Your "Personal Assistant"
- Try these advanced uses to experience AI's true power:
- Multi-round dialogue optimization
- : When unsatisfied with AI's answer, you can say "This is too professional, please explain again in words a primary school student can understand."
- Chain of Thought (CoT)

Challenge 3: Prompt Optimization

1. **: First use a simple prompt "write a travel guide" to ask AI, then use the CRISPE framework to write a detailed prompt. Compare the quality difference of the two answers.**Chapter 23: Programming Thinking and Automation Enlightenment
2. **In the previous two chapters, we built a secure "boundary wall" for our phones and witnessed its increasingly smart "AI brain" within the wall. Now, let's unlock one of the phone's ultimate potentials: letting it "think" like a programmer and automatically complete the continuous tasks you set. You might say: "I don't know how to write code, what does this have to do with me?" Don't worry, this chapter doesn't require you to write a single line of code—modern phones have lowered the threshold for "programming" to the point where you can do it with simple dragging and dropping.**1. What Exactly Is Automation?
3. **In plain language, automation is:**letting your phone automatically do a series of things for you when a certain condition is met.

Case 2: Complete Phone Replacement Data Migration

- **Scenario**: iPhone 13 Xiaomi 15 cross-brand phone replacement.
- **Steps**: (1) Old phone iCloud/WeChat built-in backup (2) Sign out of Apple ID + turn off ”Find” (3) New phone uses ”Xiaomi Migration” app to scan QR code (4) Select migration items (5) WeChat uses built-in ”Chat History Migration” separately (6) Verify key data (contacts/photos/WeChat) (7) Old phone encrypted wipe.
- **Expected Result**: Zero data loss, old phone can be safely resold.

Chapter 23: Programming Thinking and Automation Enlightenment

For example: every morning when you wake up, you manually turn off Do Not Disturb mode, open WeChat to check messages, open the weather App to see what to wear today, and open NetEase Cloud Music to play a song. If you package these steps into an automated flow, set it to "automatically execute at 7 AM every day," then when you wake up, you just need to pick up your phone, and everything is already prepared. Doesn't it feel great just thinking about it?

Automation is essentially the "if... then..." logic chain, the if-else in programming. Once you understand this way of thinking, even if you never write code in your life, your problem-solving organization will be much stronger. Core elements:

Trigger condition**: When to start? It can be "every morning at 7 AM," "when I connect to home Wi-Fi," "when I arrive at a certain location," "when phone battery is below 20%."** Action to execute

: What specifically to do? It can be sending a WeChat message, opening an App, adjusting volume, switching wallpaper, playing music, sending notifications... almost everything you can do manually on your phone, automation can do.

Intermediate logic

1. **(optional but powerful): If situation A, do X; otherwise do Y. For example, "if it's a workday, play news podcast; on weekends, play music"—this is called conditional branching.**2. iOS "Shortcuts"—The Automation Magic Tool for Apple Users

2. **Apple's "Shortcuts" App is a built-in application that doesn't need to be downloaded. Its core concept is to put various operations together like building blocks. First, let's cover a few key concepts:**"Shortcut"

3. **: A complete automation flow that can be manually tapped to run, or triggered by Siri voice. For example, you say "Hey Siri, going home mode," and it automatically executes a series of operations you've set up.**"Automation"

: Instructions that run automatically based on trigger conditions, without needing you to manually trigger them. For example, "when the phone connects to car Bluetooth, automatically open the Maps App." Automations are set up under the "Automation" tab, separate from "Shortcuts."

"Action"

- **: The smallest execution unit. Opening an App is an action, sending a message is an action, getting weather is an action, adjusting brightness is an action. You can chain hundreds or thousands of actions together.**Let me show you a few genuinely useful examples; you'll understand after following along:
- **Example 1: One-Click "Good Morning Routine"**Every morning, tap once (or say "Good morning" to Siri), and the phone will automatically: get today's weather read out the weather and temperature show today's calendar schedule open your frequently used music App to start playing. Steps: Create new shortcut add action "Get Current Weather" add "Speak Text" add "Get Calendar Events" add "Show Reminder" add "Open App" and select NetEase Cloud Music. It's that simple, all drag-and-drop.
- **Example 2: Automatic "Power Saving Mode"**In the "Automation" tab, create new automation select "Battery Level" set "below 30%" add actions "Set Low Power Mode to On," "Reduce Screen Brightness to 30%," "Turn off Bluetooth." From now on, when phone battery drops below 30%, it automatically enters power saving mode without you worrying about it.

Example 3: Scan Code Automatic Bookkeeping

Every time you scan a code to pay with WeChat/Alipay, the phone reads the payment notification and automatically records the amount and merchant to a memo or Numbers spreadsheet. This requires using the "Receive SMS/Notification" trigger in Shortcuts' "Automation" (iOS 17+ supports reading Wallet transactions), combined with the "Append to Numbers" action. For people who want to keep accounts but are too lazy to do it manually, this is a lifesaver.

There's also a clever use many people don't know about: you can add shortcuts to your home screen, turning them into an App icon. For example, make a "Call a Ride Home"

Tasker is universally recognized as the most powerful automation tool on the Android platform, consistently topping the charts. Honestly, its learning curve is a bit steep and the interface isn't pretty, but once you get the hang of it, you'll feel your phone suddenly becomes omnipotent. Tasker's core is also the "trigger task" pattern:

shortcut, put the icon on the desktop, and with one tap, it directly opens Didi and sets the destination to home—much faster than opening the App yourself and slowly typing.

Android Automation—Tasker and Its Friends

On the Android side, things are a bit more complicated, but the ceiling is also higher. Let's start with the simple ones:

Entry Level: Auto Clicker

Many Android phones come with "Auto Clicker" built-in, or you can download it from the app store. Its principle is super simple: you manually operate once, it records the coordinates and time intervals of your taps, then can play them back at high speed. For example, if a certain App gives daily check-in rewards, you record the tapping process once, and then it automatically runs every day. The downside is: it only recognizes coordinates; if the interface layout changes (such as a pop-up ad), it taps the wrong place.

Intermediate Level: Tasker—The King of Android Automation

Tasker is universally recognized as the most powerful automation tool on the Android platform, consistently topping the charts. Honestly, its learning curve is a bit steep and the interface isn't pretty, but once you get the hang of it, you'll feel your phone suddenly becomes omnipotent. Tasker's core is also the "trigger task" pattern:

Profile

: Defines "when to trigger." Trigger conditions can be time, location, Wi-Fi connection status, Bluetooth connection, receiving specific notifications, battery status, screen orientation... Tasker supports an absurd number of trigger conditions, dozens of them.

Task

: Defines "what to do after triggering." A task can contain multiple actions, executed in order. Action types include: adjusting system settings (volume, brightness, Wi-Fi, Bluetooth, airplane mode), sending SMS, HTTP requests, executing Shell commands, controlling media playback, popping up dialogs, and so on.

Here are some classic practical Tasker scenarios:

- **Auto-turn on Wi-Fi when arriving home**: Set the trigger condition to "connect to specific cell tower" (because you'll connect to a fixed tower near your home) task "Turn on Wi-Fi." No GPS needed and no power consumption; the phone automatically turns on Wi-Fi as soon as it enters the community area.
- **Meeting mode**: Trigger condition "calendar event title contains 'meeting'" task "Set silent mode, turn off media volume, auto-reply to WeChat (requires plugin support)." Never worry about your phone suddenly ringing during a meeting again.

Late-night eye protection

- **: Trigger condition "time 22:00-06:00" task "Turn on eye protection mode, reduce brightness to 20%, set screen to black and white (grayscale mode)." Black and white screen has a miraculous effect on the desire to scroll short videos—once the picture goes gray, you don't want to scroll anymore.**Auto-play podcast when headphones inserted
- **: Trigger condition "wired headphones inserted/Bluetooth headphones connected" task "Open Xiaoyuzhou App and start playing the latest episode." Put on headphones and start listening, without unlocking the phone to find the App.**If you find Tasker too hard to learn, try its simplified sibling:
- **MacroDroid**. The interface is much friendlier, with rich templates, suitable for users who don't want to tinker. Also, many domestic Android systems (Xiaomi's MIUI, Huawei's HarmonyOS, OPPO's ColorOS) now come with "Scenes" or "Smart Assistant" features; while not as powerful as Tasker, they're completely sufficient for daily automation.
- **4. Daily Applications of Automation Thinking**The biggest gain from learning automation is actually not saving that little bit of time, but

changing the way you think about problems**. Once you're used to the "if X happens, automatically do Y" thinking pattern, you'll find that a large number of repetitive tasks in life can be eliminated. For example:**After taking photos each time, automatically back up to cloud storage and then compress local photos to small sizes to save space.

Every morning, automatically read today's schedule from the calendar and generate a text message sent to family via WeChat.

After receiving a delivery sign-off SMS, automatically add a "pick up package" to-do in the memo.**When the phone is fully charged, automatically send a voice reminder to unplug the charger (to avoid overcharging damage to the battery).**Every day at quitting time, if still near the company (through cell tower positioning), automatically remind "time to get off work."

- These sound advanced, but actually, as long as you master the two basic concepts of "trigger + action," combined with Shortcuts or Tasker, you can make them all. The key is
- you first need to realize "this thing can be automated"
- —many people don't lack the ability, they just never think in that direction.
- 5. Pitfall Guide: Common Problems with Automation
- Automation isn't all smooth sailing; I've stepped into a few pitfalls for you first:

Location triggers may not work reliably**: iOS's "arrive/leave location" trigger uses geofencing technology, and accuracy depends on GPS signal and system optimization. Sometimes it triggers only 10 minutes after you've arrived home. It's recommended to use Wi-Fi connection or cell tower triggers instead of location triggers—more reliable and more power-efficient.**Shortcuts "Automation" requires authorization

: In iOS, most triggers under the "Automation"tab require manual confirmation to run (iOS's security mechanism). However, Wi-Fi connection, Bluetooth connection, and alarm stopped triggers can run fully automatically without confirmation.

Tasker requires accessibility permission

- **: Many advanced features (such as simulating taps) require accessibility service permission, which makes the phone display a persistent notification in settings. This is Android's security design and can't be removed.**Don't make automation too complicated
- **: When starting automation, begin with small flows of "2-3 actions." Don't jump straight into a 20-step super flow—you'll go crazy debugging it.**Tip: The Right Posture for Getting Started with Automation

- **First spend 5 minutes thinking: What are the top 3 things you repeat most on your phone every day? For example, your morning routine, your habits after connecting Bluetooth headphones, your settings before going out... Start with these high-frequency small things and make a simple automation for each. Even if each automation only saves you 5 seconds, it adds up over time. And the sense of accomplishment after making them is truly addictive.**Practice: Three Beginner Automation Challenges

- **Choose the one most suitable for you to complete, and you'll be on your way after finishing:**Challenge 1: Good Morning Automation

: iOS: In the Shortcuts App, create a new shortcut add "Get Current Weather" "Speak Text" "Open App and select NetEase Cloud Music." After completion, you can say "Good morning" to Siri to trigger it. Android: In Tasker, create a new task add "Get Weather" "Speak" "Open App" set trigger time to 7:00 every day.

Challenge 2: Low Battery Auto Power Saving

: iOS: Automation tab New "Battery Level" below 25% add "Set Low Power Mode On" + "Set Brightness 25%." Android: Tasker new Profile State Power Battery 0-25% Task "Power Saving Mode On" + "Lower Brightness."

Challenge 3: Connect Car Bluetooth Auto Navigation

1. **: iOS: Automation New "Bluetooth" select your car Bluetooth add "Open App and select Amap" + "Media Volume to 80%." Android: Tasker Profile State Bluetooth Connected select car Bluetooth Task "Open Amap" + "Raise Media Volume."**

Case 3: Automated "Good Morning Routine"

- **Scenario**: Automatically announce weather + turn off silent mode + open news at 7:00 every day.
- **Steps**: iOS: Shortcuts Automation Time Trigger Add "Get Weather" + "Set Volume" + "Open App"; Android: MacroDroid Trigger (Time 7:00) Actions (Media Volume / Open URL / TTS Read Weather).
- **Expected Result**: Can be implemented without installation, lowering the barrier to entry.

Chapter 23 Appendix: GitHub Beginner's Guide—From Code Hosting to Collaboration Platform

Chapter 23 Appendix: GitHub Beginner's Guide—From Code Hosting to Collaboration Platform

What Exactly Is GitHub?

1. What Exactly Is GitHub?**Simply put, GitHub is a**based on Git. It launched in 2008, was acquired by Microsoft in 2018, and remains the world's largest open-source code hosting platform, hosting hundreds of millions of repositories covering almost all mainstream programming languages.

If you use an analogy to understand:**Git is the diary on your computer, and GitHub is a public blogging platform**—you can publish your work, follow others' projects, and participate in discussions and collaboration. It's not just a "programmer's back-end repository," but more like a "technology city" built by global developers together—projects are born here, iterated, discussed, and reused.

For ordinary people, GitHub is not out of reach either. Many non-programmers use it to host notes, e-books, personal blogs, and even to back up configuration files. As long as you have a slight need to "organize your things and share them," GitHub is worth knowing about.

The Difference Between Git and GitHub (Key Concepts!)

Many people confuse these two terms when first encountering them; let me clarify for you:

Comparison Item	Git	GitHub
Nature	Version control tool (software)	Code hosting platform (website)
Running location	On local computer	In the cloud (browser access)
Core responsibility	Record every change to files (what changed, who changed it, when)	Put Git repositories in the cloud for easy storage, display, and collaboration
Network required?	Not required, works offline	Requires internet access
Ownership	Open source software, created by Linus Torvalds (father of Linux)	Microsoft product
Similar competitors	SVN, Mercurial	GitLab, Gitee, Bitbucket

Simply put:**Git manages "how to record changes," GitHub manages "where changes are stored and how to collaborate"** . Git is a tool that runs locally; GitHub is a platform that puts Git repositories in the cloud for easy collaboration and display. You can use Git without GitHub (such as doing version management for your own code), but using GitHub requires using Git (because GitHub is based on Git).

GitHub's Core Concepts and Features

Basic Concepts

Before getting started, first get to know a few high-frequency words, otherwise you'll be confused when looking at GitHub's interface:

- **Repository (Repo for short)**: One project corresponds to one repository, which stores all the code files, historical versions, and documentation for that project. You can think of it as a "project folder."
- **Commit**: A record of one modification to the code. Each Commit comes with a description, recording "what was changed this time, why it was changed." This is the foundation of Git/GitHub's ability to "time travel."
- **Branch**: An independent development line split from the main line. For example, if you want to add a new feature, you can open a branch to develop separately, and merge it back to the main line after development, without affecting others.
- **Clone**: Copy a complete version of the cloud repository to your local computer, after which you can modify the code locally.
- **Push/Pull**: Push is uploading local code to GitHub, Pull is downloading the latest code from GitHub to local. These are the two core actions for syncing code during team collaboration.

- **Pull Request (PR for short)**: When you've finished modifying code on a branch and want to merge it into the main branch, you initiate a PR, letting team members review the code before merging. This is GitHub's core collaboration mechanism.
- **Issue**: Used to record bugs, feature requests, and discussion topics. Each Issue is like a discussion thread, where you can assign responsible persons and add labels.
- **Fork**: Copy someone else's repository to your own account, where you can modify your copy freely without affecting the original repository. After modifications, you can send a PR to the original author, requesting to merge your improvements.

Core Features

GitHub provides more than just code storage, including:

- **Version control**: Records the history of every code change; if you break the code, you can always return to the last working version.
- **Team collaboration**: Supports parallel development of the same project by multiple users; multiple people developing simultaneously won't overwrite or conflict.
- **Issue tracking**: Use Issues to record bugs and feature requests, making it easy for teams to follow up.
- **Project management**: Integrates to-do items, project boards (Projects), and other management tools, equivalent to a built-in lightweight Trello.
- **Automation**: Through**GitHub Actions**, achieve automatic compilation, testing, packaging, and deployment after code submission—the core tool of modern "CI/CD (Continuous Integration/Continuous Deployment)."
- **Documentation management**: README introduces the project, Wiki stores detailed architecture and development standards, and Pages can even directly host static websites.
- **Code hosting and display**: Each repository has a beautiful web display with syntax highlighting—it's the "front door" of your technical work.
- **Open source community**: You can Star (favorite) projects you like, Watch (follow) project updates, Follow (follow) developers, and participate in global open source collaboration.

Registration and Getting Started: Create Your First Repository in Five Minutes

Register an Account

1. Visitgithub.comand click "Sign up."

2. Fill in your email, set a password and username (the username will appear in your personal homepage address; English is recommended, such as`github.com/yourname`).
3. Complete email verification and human verification.
4. Select the Free plan, which is completely sufficient for individual users.

Create Your First Repository

1. After logging in, click the "+" in the top right corner and select "New repository."
2. Fill in the repository name, such as`my-first-project`.
3. Write a short description so others know what this repository is for.
4. Choose Public or Private. Public repositories are visible to everyone; Private is only visible to you and collaborators. Free accounts also support unlimited private repositories.
5. Check "Add a README file"—README is the project's "instruction manual," recommended for every repository.
6. Click "Create repository," and your first GitHub repository is born!

README.md is a Markdown format file that is the first thing people see when they open your repository. A good README usually includes: project introduction, installation method, usage examples, license, and contact information. Writing a good README is a basic quality of programmers and the first threshold for whether a project can be used by others.

Basic Workflow: From Local to Cloud

To understand GitHub's workflow, the core is to remember this cycle:**local modification staging commit push cloud update**. Below, each step is explained in plain language:

1. **Modify files**: Write code or modify documents using an editor on your local computer.
2. **git add**: Add the modified files to the "staging area," equivalent to telling Git "I'm ready to record these changes."
3. **git commit**: Formally record the changes in the staging area as a "commit," with a description, such as "fix login bug." This step is only done locally.
4. **git push**: Upload local commits to GitHub cloud, so others can see them.
5. **git pull**: Before starting work, first pull the latest code from the cloud to ensure you're in sync with the rest of the team.

For beginners, remember this mnemonic: **"Pull first then modify, add after modifying, commit with description, finally push up"** . Develop this habit, and team collaboration won't be chaotic.

Team Collaboration Process: How Pull Request Works

GitHub's most powerful aspect is its collaboration mechanism. A typical team collaboration process goes like this:

1. **Open a branch**: Open a new branch from the main branch (usually called main or master), such as`feature/login`, and develop new features on this branch.

2. **Develop and commit**: Write code on the new branch, with multiple commits to record progress.

3. **Push to GitHub**: Push the branch to the cloud repository.

4. **Initiate Pull Request**: Initiate a PR on the GitHub webpage, requesting to merge your branch into the main branch. Clearly state what changes this PR makes and why.

5. **Code Review**: Team members review your code, offering feedback or suggesting changes. You can continue to commit based on feedback, and the PR will automatically update.

6. **Merge**: After review passes, click "Merge" to merge the code into the main branch. Your contribution is officially part of the project!

This process ensures that**all code is reviewed before entering the main branch**, greatly reducing the chance of errors. Open source projects also use this mechanism to accept contributions from strangers worldwide—you can completely submit a PR to a well-known open source project, and once adopted, you become a contributor to that project.

GitHub's Typical Application Scenarios

Students Participating in Competitions and Course Projects

Computer science students participating in ACM, hackathons, and graduation designs almost all can't do without GitHub. It serves both as a team collaboration platform and as a showcase for work—competition judges can directly view your repository to understand code quality and the collaboration process. Many companies also require attaching a GitHub homepage when recruiting, as proof of technical ability.

Personal Knowledge Base and Note Hosting

Non-programmers can also use GitHub. Many people use it to host study notes, technical blogs (combined with GitHub Pages for free website building), and e-books. Markdown writing + Git version management + cloud backup, all in one.

Participating in Open Source Projects

GitHub is the core base of the global open source movement. You can Fork projects you like, modify them and send PRs to give back to the community; or simply Star to favorite good projects and track updates. Participating in open source is an excellent way to improve your skills and build reputation.

Automated Deployment and CI/CD

With GitHub Actions, you can achieve a fully automated process of "code pushed, automatically tested + automatically deployed to server." Many personal blogs and documentation sites are built this way—after writing an article, just push, and the website automatically updates.

Code Backup and Multi-Device Sync

Use code repositories as cloud drives; when changing computers or working on multiple devices, just git pull to get the latest code—no more USB drive copying back and forth.

Advice for Beginners

- **Write clear commit descriptions**: Don't write useless descriptions like "update." A good commit message says "fix the issue of jumping to the wrong page after user login" —half a year later, you'll still know what you did.
- **Commit frequently, take small steps**: Commit once for each small feature completed, rather than writing all day before committing once. This makes it easier to locate problems.
- **Develop on branches, don't directly modify the main branch**Even for personal projects, develop the habit of opening branches, so you won't be flustered when collaborating on teams in the future.
- **Make good use of .gitignore**: This file tells Git which files not to include in version management (such as password configs, build artifacts, dependency folders), avoiding uploading junk files.

- **Look at excellent open source projects more**: Browse GitHub Trending, look at the code structure, README writing, and collaboration process of high-star projects—it's the fastest free way to improve.
- **Apply for GitHub Student Pack**: After student verification, you can get free access to many paid tools and services (Copilot, domains, cloud services, etc.), worth thousands of dollars.

Security Reminder

- Never commit sensitive information like passwords, API keys, database connection strings to GitHub, especially public repositories—once leaked, they'll be scraped by crawlers in seconds, with serious consequences.
- If you accidentally commit sensitive information, just deleting the file isn't enough (it's still in the history); use`git filter-branch`or professional tools to clean history, and immediately change the leaked keys.
- Private repositories aren't absolutely secure either; sensitive information is best managed with environment variables, not in code repositories.

Notes on Accessing GitHub from Within China

GitHub access from within China is sometimes unstable, with situations like not opening or slow cloning. You can try: (1) Use domestic mirror sites (such as GitClone, ghproxy) to accelerate cloning; (2) Configure Hosts or use a proxy; (3) If mainly for domestic collaboration, you can also consider the domestic alternative**Gitee**, which has similar features and fast access. Many teams host on both GitHub and Gitee simultaneously, balancing internationalization and domestic speed.

Advanced Technique: Build a Free Personal Blog with GitHub Pages

GitHub Pages allows you to host a static website for free. Combined with static site generators like Jekyll, Hexo, and Hugo, you can easily build a personal blog. After writing a Markdown article and pushing to the repository, the blog automatically updates, with HTTPS and custom domain names included. Countless tech bloggers' blogs are built this way—zero cost, no ads, version controllable.

Summary: Why Everyone Should Understand GitHub

GitHub has long been more than just "a programmer's tool"; it's more like a**way of working in the digital age**: version management makes everything traceable, collaboration mechanisms make teams run efficiently, and the open source spirit lets knowledge flow freely. Even if you don't write code, understanding concepts like "commit, branch, merge" can help you better manage any content that needs "repeated modifications + multi-person collaboration"—from papers to documents, from notes to proposals.

For computer science students, GitHub is an essential skill: it's your portfolio, collaboration platform, learning resource library, and the most persuasive "resume" for future job hunting. Register an account early, upload your course assignments and small projects, accumulate slowly, and in a few years you'll have a technical resume that makes interviewers' eyes light up.

Get Hands-On: Complete Your GitHub First Experience

1. Register a GitHub account and complete your profile (avatar, bio, technical fields of interest).
2. Create a public repository named`hello-github`, and check "Add a README file."
3. Enter the repository, click the edit button (pencil icon) on README.md, write a self-introduction in Markdown, and commit the changes.
4. In the GitHub search bar, search for a keyword you're interested in (such as`python-tutorial`), and Star two projects you like.
5. (Advanced) Install Git on your computer, Clone the repository to local, modify the README and push back to cloud, completing a full "local cloud" workflow.

Chapter 24: Root Access and the Bootloader Lock Explained

This chapter is an in-depth expansion of Chapter 20, suitable for users who want to truly understand the underlying mechanisms of phones or even try flashing. Please note that these operations carry certain risks, so proceed with caution.

Deep Understanding of Bootloader

Bootloader is the first program to run when the phone starts; its task is to initialize hardware and load the operating system kernel. You can think of Bootloader as the phone's "gatekeeper."

How BL Lock Works:

- **Verify signature**: BL lock verifies the digital signature of the boot image; only manufacturer-signed systems can boot.
- **Prevent downgrade**: Prevents users from flashing older system versions to exploit known vulnerabilities.
- **Protect data**: Even if a new system is flashed, user data cannot be accessed without the original account password.

Detailed Process for Unlocking BL Lock (Using Xiaomi as an Example)

Apply for unlock permission

: Apply for unlocking on the Xiaomi Unlock Tool website.

Bind account

1. : Bind your Xiaomi account in phone settings.
2. Hardware-Level Storage and Verification

3. The BL lock status (Locked/Unlocked) and the manufacturer's public key are typically stored in protected hardware areas (such as Qualcomm's devinfo partition, eFuse, or RPMB partition) to prevent software tampering. On each boot, the Bootloader reads the status from this secure area and decides whether to perform strict signature verification.

Digital Signature Verification

When manufacturers release system updates or images, they sign them with their private keys. At device boot, the Bootloader verifies the system images (such as boot.img, recovery.img) using the built-in public key. The process works as follows:

Recalculate the hash value of the system image.

Decrypt the image's built-in signature with the public key to obtain a hash value.

Compare the two hash values —if they match, verification passes. This ensures the system image hasn't been tampered with by third parties.	**Secure Boot Chain**	**The BL lock is the first link in the entire secure boot chain. Only after it passes verification does it load and execute the next stage bootloader (such as the kernel), ensuring the trustworthiness of the system boot environment layer by layer.**
Locked vs. Unlocked: The Dual Impact of BL Lock	To more intuitively understand the pros and cons of the BL lock, here's a comparison table showing its impact across 5 core dimensions:	Locked State (Default)
Unlocked State	System Security	High security: only loads officially signed images, effectively resists malicious firmware tampering
	Prevents flashing unsigned unofficial systems, protects system integrity and low-level security	
Reduced security: allows flashing third-party ROMs, system integrity verification fails, malware with Root access can deeply tamper with the system	System Stability	Stable: uses official firmware, complete features, reliable OTA updates
	Ensures the phone runs the manufacturer-tested and optimized official system, guaranteeing stability and smoothness	
Increased risk: third-party ROMs may have compatibility issues, potentially causing system crashes, bricking, hardware function anomalies (such as camera, fast charging), OTA update failures	Warranty Rights	Warranty valid: enjoy official free warranty service (non-artificial damage)
	Manufacturers provide warranty service by locking BL to ensure a stable official system	
Warranty void: most manufacturers consider BL unlocking as voluntarily waiving warranty. Even if relocked, historical unlock records may be detected and warranty refused	Data Security	Data security: in daily use, BL lock limits the ability of malware to modify the system partition
	The unlock process usually forces data wipe to prevent old data from being maliciously exploited in the new system environment	

Detailed Process for Unlocking BL Lock (Using Xiaomi as an Example)

1. **Feature Limitations**Ensures security-dependent features (such as payment, banking apps) work normally
2. **Complete features: all official features work normally**Feature limited: some security-sensitive apps (such as Google Pay, banking apps) may refuse to run upon detecting BL unlock status; DRM services may be downgraded, affecting HD video playback
3. **Waiting period**: Depending on account level, you need to wait 7-30 days.
4. **Unlock operation**: Use the Xiaomi Unlock Tool to connect to your computer to unlock.

Multiple Root Solutions

- **Magisk**: Modern Root solution, Systemless design, can hide Root status.
- **Kernelsu**: Kernel-based Root solution, more low-level, better concealment.
- **SuperSU**: Old-school Root solution, currently no longer updated.

Flashing Introduction

Flashing refers to installing a new operating system on the phone. Common flashing types:

- **Flash official ROM**: Install the official system, used for rescuing bricked phones or downgrading.
- **Flash third-party Recovery**: Such as TWRP, used for backup and recovery, flashing third-party ROMs.
- **Flash third-party ROM**: Such as LineageOS, Pixel Experience, etc., to experience different systems.

V. Samsung Knox: The Irreversible Cost of Unlocking the BL

Flashing carries risks; proceed with caution. May lead to:**Phone bricking (cannot boot)**Loss of warranty

- **Data loss**V. Samsung Knox: The Irreversible Cost of BL Unlocking
- **Unlike other Android manufacturers, Samsung's BL unlock mechanism has a unique "fuse" design:**For Samsung phones, unlocking triggers Knox fuse tripping,

permanently losing some security features (such as Samsung Pay); clears all protected data; affects warranty

- **. Specifically:**The Fuse Is One-Time
- **: There's an eFuse inside the phone that permanently burns out once it detects BL has been unlocked —it cannot be restored.**Affected Features

: Samsung Pay (transit cards/bank cards), Knox Secure Folder, enterprise MDM management, and some health insurance apps will all stop working.

Forced Data Wipe**: After the fuse is triggered, the phone forces a factory reset —all local data is erased (cloud data is not affected).**Warranty Impact

Important Warning

Flashing carries risks—proceed with caution. It may cause:

- Bricked phone (won' t boot)
- Loss of warranty
- Data loss
- Payment function abnormalities

Before flashing, be sure to make a complete backup, carefully read the tutorial, and confirm that the model and version match.

Recommended Resources

- **XDA Developers**: The world's largest Android developer community, with rich flashing tutorials and ROM resources.
- **Coolapk**: A well-known domestic digital community, with many Chinese tutorials and discussions.
- **Various model forums**: Such as Xiaomi Community, OnePlus Community, etc., with official and community tutorials.

VI. FAQ (Q& A)

Q1: Can BL Unlock Be Reversed?

A:Yes, it can be relocked. Usually requires flashing the official complete firmware package and using a specific flashing script (such as Xiaomi's`flash_all_lock.bat`) to relock. However, note that after relocking, some security fuses triggered by unlocking (such as Samsung's Knox) may be irreversible.

Q2: How to Root After Unlocking?

A:Unlocking the BL is just the first step. Rooting usually requires:

1. Flashing a custom Recovery (such as TWRP).
2. Flashing Magisk or other Root managers through Recovery to patch the boot image and gain Root access.

Chapter 25: Second-hand Phone Disposal and Digital Legacy

Execute ultimate "formatting"

: Modern phones' "factory reset" usually includes encrypted erasure.

Core principle: Data cleaning isn't "deletion" —it's "overwriting." Many people think dragging photos to the trash and clearing the album is enough, but after the phone changes hands, the new owner can scan with data recovery software and recover old photos, chat history, and ID photos —this kind of "social death" happens every year.

1. **: After factory reset, fill storage with junk files, then perform another factory reset.**: Complete the last full backup, sign out of all accounts (iCloud, Huawei/Xiaomi/Samsung account, Google account), turn off "Find My Phone" —otherwise the next user will be blocked by Activation Lock.

2. **Apple "Legacy Contact"**: Modern phones' "factory reset" usually includes encrypted wiping —it destroys the encryption key itself. Once the key is gone, all data on the disk instantly becomes garbled.

3. **Final "Camouflage" (Only for Old Devices)**: Modern phones (iOS 8+, Android 6+) already have full disk encryption (FBE) by default. Factory reset destroys the encryption key —data is theoretically unrecoverable,**this step is not needed**. Only for very old non-encrypted devices (pre-2015), after factory reset, should you fill the storage with junk files and then perform another factory reset.

Brand-Specific Data Wipe Quick Reference Table: Although the principles are the same, each brand hides "factory reset" in slightly different locations. Key differences are below.

Brand	**Wipe Entry Point**	**Key Differences & Notes**
Huawei	Settings > System & Updates > Reset > Factory Reset	Must sign out of Huawei account and turn off "Find My Phone" first, otherwise Activation Lock will trigger; may require password verification in Pure Mode.
Xiaomi / Redmi	Settings > My Device > Factory Reset	At the bottom, there are "Format Virtual SD Card" and "Clear All Content & Settings" options —both must be checked; turn off "Find Device" before signing out of Xiaomi account.
OPPO / OnePlus	Settings > Additional Settings > Restore Phone > Completely Clear All Data	ColorOS provides a "thorough clear" mode that overwrites storage multiple times; requires lock screen password confirmation.
vivo	Settings > System Management > Backup & Reset > Clear All Data	Must check "Format Phone Storage" to clean the SD card area; must sign out of vivo account and turn off "Find Phone."
Samsung	Settings > General Management > Reset > Factory Reset	Must remove Samsung account and turn off "Find My Mobile" first; some models have an "auto-overwrite" option, recommended to enable.
iPhone	Settings > General > Transfer or Reset iPhone > Erase All Content and Settings	Must sign out of Apple ID first, otherwise the next user will be blocked by iCloud Activation Lock; iOS 15+ supports "Keep iCloud Data When Signing Out" for later reference.

The Three Most Overlooked Things

- **Not Signing Out of Accounts**: Phone gets remotely locked after selling to the next buyer, and you can't get back in either —ultimately you have to take your ID to an official service center to unlock it.
- **SD Card Not Removed**: Factory reset doesn't format photos on the external SD card —remember to manually remove or separately format it.
- **USB Drive / OTG Peripheral Not Disconnected**: If peripherals are connected during the wipe, it may accidentally clear their data or cause the wipe to be interrupted.

Manual backup

The data in your phone is no longer "virtual." It carries your money, social relationships, creative works, and even your "digital identity." **After a person passes away, how should these assets be handled?**This is no longer an abstract question —it's a real dilemma facing more and more families: relatives don't know the passwords, platforms won't allow inheritance, and cryptocurrency is lost forever.

Legal Basis: Article 49 of the Personal Information Protection Law

Article 49 of the Personal Information Protection Law of the People's Republic of China, effective in 2021, clearly states:**When a natural person dies, their close relatives may, for their own legitimate and proper interests, exercise the rights to access, copy, correct, and delete the deceased's personal information as provided in this Chapter, unless the deceased made other arrangements during their lifetime.** This means:

- **Close relatives have a legal basis**to request platforms to process the deceased's accounts and personal information.
- "Legitimate and proper interests" typically include handling inheritance, closing accounts, and deleting sensitive privacy information.
- If the deceased made other arrangements during their lifetime (such as designating a legacy contact), those arrangements take precedence.
- Platforms cannot use "user privacy" as a blanket reason to refuse cooperation, but specific procedures vary by platform.

WeChat and Alipay: Accounts Can't Be Inherited, But Funds Can

The rules for these two most commonly used apps among Chinese people are different:

- **WeChat Account**: According to the Tencent WeChat Software License and Service Agreement, account ownership belongs to Tencent —users only have usage rights,**the account itself cannot be inherited or transferred**. However, the**Wallet balance and Qian funds**are personal property. Close relatives can apply to Tenpay for withdrawal or transfer with a death certificate, proof of kinship, and an inheritance notarization.
- **Alipay**: The account also cannot be directly inherited, but Alipay balance, Yuebao, and wealth management funds**are inheritable property**. Heirs can handle this through Alipay's customer service hotline or by submitting a death certificate and inheritance notarization offline; Ant Wealth financial products require going through the fund company's process.
- **Chat History**: Legally classified as personal privacy —platforms won't proactively provide it to family members. If it involves inheritance leads, it can be obtained through litigation or notarization application.

Enable "screen sharing" or "remote control" for strangers.

- **Don't directly hand the deceased's account passwords to family members to "keep using"**—this violates platform agreements, and it's harder to prove things if funds are frozen or risk control is triggered.
- **Don't try to use the deceased's phone to receive verification codes for "withdrawals"**—this may be classified as theft and could violate the law.
- **Don't Delay**: Some accounts are automatically deactivated after long periods of inactivity, making fund recovery more difficult.

Apple's "Legacy Contact": Designate in Advance for Half the Effort, Double the Result

Starting from iOS 15.2, Apple launched the "Legacy Contact" feature, allowing users to designate trusted contacts in advance. After the user's passing, the contact can use**death certificate + Legacy Contact access key**to request access to the deceased's iCloud data (photos, notes, contacts, etc.). Steps:

1. On iPhone, go to**[Settings] > [Your Name] > [Sign-In & Security] > [Legacy Contact]**.
2. Tap "Add Legacy Contact" and confirm identity with Face ID.
3. Select a contact (from your contacts), and the system will generate an**Access Key**.
4. Send the access key to the other person via SMS, email, or printed copy, and remind them to keep it safe.
5. Multiple contacts can be added as backup to avoid single-point failure.

The designated person doesn't need an Apple device —they can request access on Apple's official website with the key and death certificate. The entire process typically completes within 7 days.

Google's "Inactive Account Manager" : Automatically Handling Digital Aftermath

Google provides the "Inactive Account Manager," which can automatically execute preset actions when the user hasn't logged in for a long time —this is one of the most comprehensive digital legacy mechanisms available.

1.).**myaccount.google.com/inactive**, sign in to your Google account.
2. Set the timeout period: triggers after 3 / 6 / 12 / 18 months of inactivity.
3. Add trusted contacts (up to 10), and set which data each can access (Gmail, Photos, Drive, etc.).
4. Optional: Set up auto-reply email to inform senders you can no longer respond.
5. Final choice: when the timeout expires, either**Share data with contacts**, or**Delete the account**.
6. Set a phone number as fallback verification to prevent accidental triggering.

Domestic Social Media Accounts: Memorialization and Deletion Process

Domestic mainstream social platforms handle deceased users' accounts in two ways: **"Memorialization" (preserving the account for remembrance)**verb **"Deletion" (complete removal)**. Family members can apply to platform customer service with a death certificate, proof of kinship, and the applicant's ID.

- **Weibo**: Supports account "memorialization." Relatives apply with a death certificate —the account is locked and can't be logged into, but content is preserved. The profile page displays "This account has been set as a memorial account." Deletion can also be requested, requiring an inheritance notarization.
- **Douyin**: Through "Me - Settings - Feedback & Help - Account & Security - Account Deletion" or manual customer service, submit a death certificate, proof of kinship, and applicant ID. After review, the account can be deleted or converted to memorial status.
- **Bilibili**: Submit the deceased's certificate and proof of kinship through customer service channels. You can request account deletion or preservation as a memorial page; handling of the uploader's historical content requires consideration of copyright declarations.
- **Xiaohongshu (RED)**: Appeal through official customer service, submit death certificate and proof of kinship, can request account deletion and content removal.

Key Reminders

- Platform rules may be updated —check the latest official customer service guidelines before proceeding.
- All certification documents should be scanned and backed up before submission. Keep originals safe.
- When handling multiple platforms at once, you can apply for a single**inheritance notarization**to save repeated trips.

The Challenge of Digital Asset Inheritance

In addition to social accounts, more and more people hold**genuinely valuable digital assets**, whose inheritance issues are more complex:

- **Cryptocurrency (Bitcoin, Ethereum, etc.)**: Based on the principle of "private key = ownership,"**no private key means the assets are gone**. Heirs can't appeal "on-chain" through legal documents. Recommendation: store mnemonics/private keys in shards across multiple physical security locations (such as a bank safe deposit box + home safe), and specify the access method in your will.
- **Exchange Accounts**: Similar to bank accounts, can apply to exchanges (such as Binance, OKX) for asset disposition with an inheritance notarization, but cross-border exchange processes are lengthy.
- **Domain Names**: Inheritable property. Heirs can transfer ownership at the domain registrar (such as Alibaba Cloud, GoDaddy) after submitting an inheritance notarization.
- **Intellectual Property (original articles, videos, patents, software copyrights)**: Property rights within copyright can be inherited (personal rights are not inherited), and revenue rights can be enjoyed by heirs until 50 years after the author's death.
- **Cloud Storage and Network Drives**: Content data belongs to the individual and can be inherited; however, some platform agreements stipulate that accounts themselves are non-transferable —you need to download data first before deletion.

The Real Cost of Cryptocurrency "Disappearance"

According to Chainalysis reports, globally**over 3 million Bitcoins**have been permanently lost due to lost private keys —equivalent to one-sixth of the existing total. If you hold crypto assets, be sure to let at least one person you absolutely trust know the**recovery path**— otherwise these assets will sleep forever on the blockchain.

A "Digital Will" Template

Traditional property wills can't manage digital assets. I recommend updating a "digital will" addendum annually, stored alongside your official will. The following template is for

reference:

“Personal Digital Asset Disposition Addendum” (Template)

Part One: Account Inventory(Record only account names and platforms —**do NOT write passwords**)

- Social: WeChat (account xxx), Weibo, Douyin, Bilibili, Xiaohongshu, etc.
- Payment: Alipay, WeChat Pay, various bank card apps
- Cloud: iCloud, Huawei Cloud, Mi Cloud, Google account, Microsoft account
- Email: QQ Mail, NetEase Mail, Gmail, etc.
- Other: Cloud drives, subscription services, game accounts, etc.

Part Two: Password Custody Plan

- Master password storage: Use a password manager (like Bitwarden, 1Password). Write the master password on paper and seal it in a home safe.
- Recovery keys/mnemonics: Store in fragments across at least two physical locations (e.g., bank safe deposit box + home safe).
- Authorized contact: Name, ID number, relationship, contact information.

Part Three: Disposition Wishes

- Whether each type of account should be ”memorialized” or ”deactivated.”
- Whether iCloud / Google data has a designated ”Legacy Contact” or ”Inactive Account Manager.”
- Who inherits crypto assets, with instructions for private key access.
- Who enjoys the proceeds from copyrighted works.

Part Four: Contacts and Platforms

- Trusted executor: name, contact, backup contact.
- Platform customer service contacts and required documentation checklist.
- This addendum updated: _____/___/___. Next review date: _____/___/___.

Legal Note on ”Passwords”

Writing passwords directly into a will poses leak risks, and all heirs can see them once the will is probated.**A safer approach: the will only contains “account inventory +**

disposition wishes + password custody location." Actual passwords are transmitted separately to the trusted executor via the password manager's master password. This satisfies legal requirements while reducing password leak risks.

Give it a second life

A single phone contains dozens of metals including gold, silver, copper, and rare earth elements. Discarding it carelessly pollutes soil and wastes resources. There are three proper recycling channels, each with different strengths:

Channel	Recycle Value	Data Security	What is RAW
Official Trade-in (Huawei/Apple/Xiaomi, etc.)	Medium, often with subsidy vouchers	Highest, official standardized process	Most cost-effective when upgrading to same brand
Third-Party Recycling Platforms (Aihuishou, Zhuanzhuan, Huishoubao, etc.)	Slightly higher, can compare prices	Fairly high, with data wiping report	Want direct cash, not brand-locked
Charitable Donation (e.g., rural schools, charity organizations)	No financial return	Must thoroughly wipe data yourself	Device still usable, want to do good
Repurpose (Keep and Modify)	Save money on new devices	Original owner continues using, no risk	Device still performs well, household has repurposing needs

- **: Convert it into a car navigator, smart home hub, or surveillance camera.**: Most recommended —standardized process, data security guaranteed. Apple, Huawei, Xiaomi, and other brands have "trade-in" entries on their official websites. After assessment, the value can be deducted from the new phone price.
- **Last backup and log out of all accounts.**: Choose a reputable large recycling platform. After placing an order, SF Express picks up the device, and payment is made after inspection. Be sure to choose a**platform that provides a "data wiping report"**, and keep the receipt.
- **Charitable Donation**: Contact local rural schools or community organizations —old phones can be converted into educational tablets or phones for the elderly.**Before donating, be sure to perform a complete wipe and remove all SIM/SD cards**.
- **(Optional) Fill storage with junk files, then factory reset again.**: Repurpose as a car navigator, smart home controller, security camera, baby monitor, or digital photo

frame. An old phone can serve for several more years.

The Truth About E-Waste

A phone's circuit board contains**gold, silver, palladium, copper**and other precious metals. Per ton of scrap circuit boards contains tens of times more gold than gold ore. But if discarded carelessly, the lead, mercury, and cadmium will leach into soil and groundwater, polluting for decades.**Proper recycling protects the environment and allows scarce metals to be recycled.**

Chapter 26: Future Phones and Digital Life

1. Final complete backup: Upload photos to cloud, copy important files to computer, export and archive chat history.
2. Sign out of all accounts: iCloud / Huawei / Xiaomi / Google, turn off "Find My Phone."
3. Remove SIM and SD cards, disconnect all OTG peripherals.
4. Perform encrypted factory reset, manually check "Format storage" if necessary.
5. (Old devices only) Fill storage with junk files, then factory reset again.
6. Choose its next destination: official trade-in, certified recycling platform, charitable donation, or pass to family.
7. Sync and update your "digital legacy inventory" —cross out retired devices.

This Month's Task List

1. Designate an Apple "Legacy Contact" or set up Google "Inactive Account Manager" for your current primary phone.
2. Draft a "Personal Digital Asset Disposition Addendum" using this section's template, and discuss storage location with family.
3. Check if you have 2+ idle old phones at home, and schedule an official or certified platform recycling.

Chapter 26: Current Smartphone Technology Frontier

Having covered old phones' "afterlife," let's look at the present. 2024-2026 is a concentrated explosion period for phone technology: AI moved from cloud to pocket, satellite communication trickled down from flagships to mid-range, foldable screens went from "novelty" to "daily use," and battery technology saw material-level breakthroughs not seen in years. This chapter surveys these cutting-edge technologies to help you decide "should I upgrade now" and "what to upgrade to."

I. AI Phones: Marketing Gimmick or Real Revolution?

Since 2024, nearly all Android manufacturers have branded their flagships as "AI phones," and Apple launched Apple Intelligence in iOS 18. But what truly qualifies as an "AI phone"? The industry has roughly converged on three criteria:

- **On-device LLMs**: The phone can run a language model with around 7B parameters locally (like lightweight versions of Qwen, Llama, DeepSeek), completing text summarization, translation, and copywriting without cloud dependency. Test: if AI features still work in Airplane Mode, it's an on-device model.

- **NPU Computing Power (TOPS)**: NPU (Neural Processing Unit) computing power is measured in TOPS (trillions of operations per second). 2024 flagship chips (like Snapdragon 8 Gen3, Dimensity 9300) have 40-70 TOPS; 2025-2026 new-generation chips generally exceed 100 TOPS —the hardware threshold for running LLMs on-device.

- **AI Agent Scenarios**: Not just "Q& A" —can execute cross-app task chains. For example, "organize yesterday's meeting photos into an album and share with Zhang San" —the system automatically calls Photos, Files, WeChat, and other apps to complete the task. This is the core differentiator between an AI phone and a "phone with an AI assistant."

II. Manufacturer AI Strategy Comparison: Who's Serious, Who's Labeling?

Each manufacturer's AI route differs significantly —worth paying attention to when purchasing:

- **Apple Apple Intelligence (iOS 18+)**: Emphasizes system-level app integration — email summaries, notification priority, writing polish, AI photo editing. Primarily on-device models, with complex tasks offloaded to Private Cloud Compute. Advantages: privacy protection and system smoothness. Disadvantages: slower Chinese language rollout, with some features only gradually supported in 2025.
- **Google Gemini Nano**: Built into Pixel 8/9 series, supports "Circle to Search," Magic Compose, recording summaries, etc. Gemini Nano is a true on-device small model, open-source ecosystem friendly, and the reference implementation for Android AI phones.
- **: Real-time call translation, note AI summaries, image generation and editing, search circle selection, and other functions.**: Integrates HarmonyOS Xiaoyi, supporting on-device copywriting, AI photo editing, AI eraser (removing passersby), and call summaries. After HarmonyOS NEXT 5.0, further system-level integration deeply fuses AI capabilities with the OS.
- **Xiaomi MiLM (Mi Large Model)**: Built into HyperOS, supports AI portraits, AI summaries, AI artist, super resolution. Xiaomi follows an "on-device + cloud collaboration" route —lightweight models run on-device, heavy tasks go to cloud.
- **OPPO (Andes Large Model) and vivo (Blue Heart Large Model)**: Deeply integrated into ColorOS 15 and OriginOS 5, focusing on AI eraser, AI document summaries, AI image Q& A, and natural language photo search ("find photos I took at the beach last year").

Buying Advice: If you frequently need AI for work documents and information organization, prioritize models with strong on-device computing and system-level AI integration. If you only occasionally use AI for photo editing, mainstream mid-range phones are sufficient.

III. Satellite Communication: From Flagship "Gimmick"to Mid-Range Standard

Satellite communication is one of the most substantial technology trickle-downs in recent years. When Huawei Mate 60 and iPhone 14 debuted it in 2022, it was flagship-exclusive; by 2025, it has reached the 2,000-3,000 yuan tier.

- **BeiDou Short Messages Reach Mid-Range**: BeiDou satellite short message functionality (sending texts up to 1,000 Chinese characters + location) has reached mid-range

models from Redmi, realme, iQOO, etc. No monthly fee, free to use. Suitable for hiking, road trips, and similar scenarios.

- **Tiantong Satellite Calls**: Huawei Mate 60/70 series, Honor Magic6 series, etc. support Tiantong-1 satellite calling —can make/receive satellite calls and send satellite messages. Requires a carrier satellite plan (about 10-200 yuan/month), suitable for users who frequently enter remote areas.
- **iPhone Satellite Messages (SOS)**: iPhone 14 and later support satellite SOS —can send text messages to emergency services and share location. Free for 2 years, renewable in some regions.
- **Explore: Your Phone's Video Settings**: Urban users basically don't need it, but outdoor enthusiasts and business travelers to remote areas should strongly consider satellite-capable models —it can save lives at critical moments.

IV. Foldable Screen Ecosystem Maturity: Ready to Be a Primary Phone?

Foldable screens have gone from "tech novelty" to "viable primary phone." To judge foldable maturity, look at three things: hardware durability, app adaptation, and interaction ecosystem.

- **Hardware Durability**: Hinge life generally exceeds 500,000 folds (100 folds/day for 13 years), water-drop hinges reduce screen creases, IPX8 water resistance is now standard. UTG glass + screen protector makes the inner screen less "delicate."
- **App Adaptation**: Mainstream apps (WeChat, Taobao, Douyin, Bilibili, Office, WPS) are adapted for large screens, but about 30% of long-tail apps remain unadapted, showing stretched or letterboxed. Try your frequently used apps before purchasing.
- **Flex Mode Interaction**: When folded to 90°-120°, some apps support a flex mode with "top screen for content, bottom screen for controls," suitable for video calls, selfies, and watching videos. Flex-mode selfies are a foldable-exclusive experience.
- **Split Screen and Multitasking**: On the large screen, you can run two apps in split screen, or even three apps in floating windows. HyperOS, ColorOS, and OriginOS split-screen logic is fairly mature, but iOS (iPadOS-style) split screen isn't available on iPhone.
- **Price**: Horizontal folds (book-style) dropped from 10,000+ yuan to 5,000-7,000 yuan tier; vertical folds (flip-style) dropped to 4,000-5,000 yuan. In 2025, some manufacturers launched 3,000-yuan vertical folds —foldables truly entered the mass market.

Buying Advice: If you're a heavy reader, document processor, or video consumer, horizontal foldables are worth considering. If you prioritize portability and social appeal, vertical foldables are more suitable. Spend at least 30 minutes trying one in-store before deciding.

V. 6G and Wi-Fi 7: Speed and Boundaries of Next-Gen Connectivity

While 6G hasn't truly landed, related technologies are already on the way:

- **6G Progress**: 3GPP plans to complete 6G standardization by 2027, with commercial deployment around 2030. 6G theoretical peak rates can reach 1 Tbps (50x of 5G), latency as low as 0.1ms, supporting "integrated sensing and communication" —phones could even serve as low-precision radar. Currently, manufacturers are still in prototype verification, far from consumer products.
- **Wi-Fi 7(802.11be)**: Became flagship standard from 2024, with theoretical peak rates of 46 Gbps (4x Wi-Fi 6), supporting 320MHz bandwidth, 4K-QAM, and MLO (Multi-Link Operation). Real-world testing shows 2-5 Gbps transfer rates on flagships with Wi-Fi 7 routers, 2-3x faster than Wi-Fi 6E. If you get a Wi-Fi 7 phone, remember to pair it with a Wi-Fi 7 router to unlock full performance.
- **Bluetooth 5.4 and LE Audio**: 2024-2026 phones widely support Bluetooth 5.4 and LE Audio (Low Energy Audio) —can connect two headphone pairs simultaneously to listen to the same song, and Auracast broadcast audio lets multiple people sync to the same audio source in public spaces.

VI. Silicon-Carbon Anode Batteries: Battery Life Finally Gets a Material-Level Breakthrough

Phone battery life has long been criticized as "one charge per day." Since 2024, the adoption of silicon-carbon anode battery technology has brought substantial improvement.

- **Technical Principle**: Traditional graphite anodes have a theoretical capacity of about 372 mAh/g. Silicon's theoretical capacity is as high as 4200 mAh/g (~10x), but silicon expands 300% during charge/discharge, causing pulverization. Silicon-carbon anodes use a nano-silicon-carbon composite structure to control silicon expansion within acceptable limits, boosting energy density by 15%-30%.
- **Actual performance**: 2024-2025 flagships with silicon-carbon anodes (like Xiaomi 14 Ultra, Honor Magic 6 Ultimate, vivo X100 Pro, etc.) fit 5000-6000 mAh batteries in 7-8mm bodies —about 800-1000 mAh more than the previous generation. Heavy use can last a day and a half.
- **100W+ Fast Charging Compatibility**: Silicon-carbon anodes have good fast-charging compatibility —100W/120W fully charging in 17 minutes is no longer news. In 2025, some models support 150W-240W, reaching 80% in 10 minutes.

- **Buying Advice**: If you're a heavy user, prioritize post-2024 models with the "silicon-carbon anode + 100W+ fast charging" combo —battery anxiety will be greatly reduced.

VII. Chapter Overview: Remember These Frontiers in One Sentence

- AI phone three criteria: on-device LLM, NPU TOPS, AI Agent cross-app scenarios.
- Satellite communication: BeiDou short messages reach mid-range, Tiantong calls still flagship-only.
- Foldable screens: hardware mature, 70% app adaptation, flex mode/split screen are unique experiences.
- 6G is still on the way, Wi-Fi 7 has landed, Bluetooth LE Audio is spreading.
- Silicon-carbon anode + 100W+ fast charging: dual breakthrough in battery life and charging.

Try It: Make a "Tech Requirements Checklist" for Your Next Phone

Take a piece of paper and list your needs for AI, satellite, foldable screens, Wi-Fi 7, fast charging, and battery capacity in three tiers: "Must-have / Nice-to-have / Don't need." This way, next time you visit a store, you can quickly judge whether the salesperson's "selling points" are real needs or just upselling. For example:

- Must-have: 5000mAh+ battery, 80W+ fast charging, NFC, IP68 water resistance.
- Nice-to-have: on-device AI summaries, Wi-Fi 7, BeiDou short messages.
- Don't need: foldable screen (I prefer bar phones), satellite calls (I don't travel far), 240W extreme fast charging.

Chapter Summary

- AI phone three criteria: on-device LLM, NPU TOPS, AI Agent scenarios
- Manufacturer AI strategies: Apple emphasizes privacy integration, Google on-device open-source, Huawei/Xiaomi/OPPO/vivo each have their strengths
- Satellite communication: BeiDou short messages reach mid-range, Tiantong calls still flagship-exclusive
- Foldable screens: hardware mature, ~70% app adaptation, flex mode and split screen are unique experiences
- 6G standard by 2027, Wi-Fi 7 widely adopted, Bluetooth LE Audio beginning to land
- Silicon-carbon anode batteries make 5000-6000mAh standard for flagships, greatly

reducing battery anxiety

V. Practical (Chapters 27-32)

Chapter 27: Making Phones Easy for Parents —Complete Senior-Friendly Guide

Honestly, when writing this chapter, I thought of my own parents. Every time a "system update" pops up on my mom's phone, she's afraid to tap it, worried she'll break the phone. My dad's WeChat font is smaller than an ant, but he doesn't know how to enlarge it —he's been squinting at it for two years. It's not that they're "dumb" —it's that phones default to assuming everyone is a twenty-something. In this chapter, I use the plainest language to solve the most frustrating problems elderly users face with their phones, one by one.

First, Get the Mindset Right: It's Not "Seniors Can't Use Phones" —It's "Phones Weren't Designed for Them"

Before making any settings changes, one concept needs to be clear:**Current smartphone interfaces were never really designed with elderly users in mind**. Pop-ups are too small to see, gestures require precise touches, icons all look the same —these aren't user problems, they're product problems. So when helping elderly relatives with phones, the first thing isn't to teach —it's to help them**set up the environment**. Once the environment is set up properly, most problems disappear.

Three iron rules for helping elderly relatives with phones: (1)**Don't rush**—you've used smartphones for ten years, they may have used them for three months. (2)**Use analogies**—"Wi-Fi is like the water pipes at home —turn on the faucet and water flows." This kind of metaphor is infinitely easier to understand than technical jargon. (3)**Write it down**—they might forget verbal instructions. Write the key steps on a piece of paper and stick it to the back of their phone case —it works better than anything else.

I. First Step: Enable "Easy Mode"—The Phone Becomes Clean and Simple

Most phones now have a built-in "Easy Mode" (some call it "Care Mode," "Senior Mode," or "Elder Mode"). After enabling it, the entire phone transforms:

- **Fonts and icons become very large**—no more need for reading glasses to see the screen.
- **Desktop icons greatly simplified**—only Phone, Messages, Camera, Photos, WeChat, and a few other commonly used apps remain; everything else is tucked away.
- **Operations become simpler**—many flashy features (leftmost page, smart assistant, floating windows, etc.) are disabled by default —the interface is clean and straightforward.
- **Sound becomes louder**—ringtone, notification, and call volumes are automatically set to maximum.

How to enable Easy Mode by brand (just follow along):

Brand	Mode Name	How to Enable
iPhone	Accessibility (no separate Easy Mode, but items can be enlarged individually)	Settings > Display & Brightness > Text Size, drag to max. Then go to "Accessibility > Display & Text Size" and enable "Bold Text" and "Button Shapes"
4. Wireless Charging Protocols	Easy Mode	Settings > System & Updates > Easy Mode > Enable
Xiaomi/Redmi	Minimalist Mode	Settings > Additional Settings > Minimalist Mode > Enable
OPPO/OnePlus	Easy Mode	Settings > Special Features > Easy Mode
vivo/iQOO	Easy Desktop	Settings > Home Screen, Lock Screen & Wallpaper > Easy Desktop
Samsung	Easy Mode	Settings > Display > Easy Mode

If you use another brand, search for "simple" or "elder" in the search box at the top of Settings —you'll usually find it. If not, follow the "Manual Enlargement" approach below step by step —the result is the same.

II. If You Don't Use Easy Mode, Manual Adjustment Works Too — "Big Text + Big Icons" in Five Steps

Some elderly relatives don't want to use Easy Mode —they think it looks too "old-fashioned." In that case, manually adjust these five items for the same effect:

1. **Set Font to Maximum**: Settings > Display > Font Size, drag to the far right. Also set "Display Size" (sometimes called "Display Zoom") to maximum —this makes all interface elements larger together.
2. **Make Icons Bigger**: Pinch on the home screen (like zooming out on a photo) to enter desktop settings > Desktop Layout (or Icon Size), and choose the layout with the fewest grid cells (like 3×5) —icons naturally become bigger.
3. **Enable "High Contrast Text"**: Makes text edges sharper —find this toggle in "Accessibility."
4. **Disable Unnecessary Notifications**: Settings > Notifications, turn off all app notifications except Phone, Messages, and WeChat. This way the phone won't buzz every few minutes and won't feel annoying.
5. **Put Frequently Contacted People on the Home Screen**: Long-press empty space on home screen > Add Widget > Contacts, and place speed-dial shortcuts for spouse, children, doctor, and emergency contacts on the home screen. One tap to call —no need to scroll through contacts.

Hands-On: Help Elderly Relatives "Renovate" Their Phone

1. Enable Easy Mode (follow the brand table above).
2. Turn on battery percentage display (Settings > Battery > Battery Percentage), so they always know how much charge is left.
3. Open the notification shade and remove or hide quick toggles like Bluetooth, NFC, and Airplane Mode that elderly relatives don't need.
4. Place four speed-dial contacts on the home screen: spouse, children, family doctor (if any), community/property management.

III. Anti-Fraud —Please Read This Section Together with Your Elderly Relatives

Elderly people are prime targets for telecom fraud. It's not that they're less vigilant — scammers' scripts are carefully designed to exploit elderly people's psychological vulnerabilities. Here are the most common scam types and how to respond. I recommend reading them aloud to your elderly relatives, or printing them out and sticking them next to the phone.

Six Common Scams (Hang Up Immediately If You Encounter These):

- **(1) "Your Grandson Is in Trouble, Send Money Now"**—The scammer says "your grandson/granddaughter got into trouble outside, was arrested by police, and needs bail." Remember:**Any call demanding a transfer to "release someone" is a scam**. Police and government agencies never demand money over the phone.
- **(2) "Your Health Insurance Card Has Been Suspended"**—A text message says your health insurance card is abnormal and asks you to click a link to "activate" it. Remember:**Health insurance and social security bureaus never send text messages asking you to click links**. In this situation, call 12333 (the official social security hotline) to verify.
- **(3) "High-Yield Investment, Guaranteed Profit"**—WeChat groups and text messages recommend various "retirement investments" and "government-supported projects," promising high returns. Remember:**Anything promising annualized returns over 5% that's "guaranteed" is 99.9% a scam**.
- **(4) "I'm Your Boss/Old Friend, I Changed My Number"**—Impersonating an acquaintance to borrow money. Remember:**Calling to verify works better than anything**. If they say "I can't take calls," they're almost certainly a scammer.
- **(5) "Free Eggs / Health Product Seminar"**—Free offline events lure elderly people into attending "health seminars" and then sell overpriced health products. Remember:**There's no free lunch —free events are always businesses in disguise**.
- **(6) "Your Phone Is Infected, Needs Remote Repair"**—The scammer asks you to download a remote control app (like Sunflower, TeamViewer), then remotely controls your phone to transfer money. Remember:**Absolutely never let anyone remotely control your phone!**Phone manufacturers, carriers, and banks will never call you asking you to install remote control software.

Three defense lines that, once set up, can block 90% of scams:

1. **First Line: Built-in Phone Blocking**—On Android, enable "Harassment Blocking" (in the Phone app or Phone Manager app). On iPhone, enable "Silence Unknown Callers" (Settings > Phone > Silence Unknown Callers).
2. **Second Line: Carrier Blocking**—Send text "KTFSR" to 10086 to activate China Mobile's free high-frequency spam call blocking. China Telecom and China Unicom have similar services. Call your carrier's customer service to have them enable it for you.
3. **Third Line: National Anti-Fraud Center App**—Download the "National Anti-Fraud Center" app from the app store, register, and enable incoming call alerts. This app is linked to police databases —scam numbers are identified as soon as they call.

Anti-Fraud Mantra for Elderly Relatives (Print and Stick Next to the Phone)

- **Stranger calls asking for money transfer Hang up!**
- **Text message links asking you to click Delete!**
- **Promising high returns with guaranteed principal Block!**
- **Asking you to install remote control software Hang up immediately!**
- **Real police don't handle cases over the phone, and definitely don't ask for money!**
- **When unsure about anything, call your children first.**

IV. Remote Assistance —Fix Your Parents' Phone from Anywhere

Sometimes elderly relatives encounter problems that you can't explain over the phone — they can't find the right button. That's when "remote assistance" comes in —you can see their phone screen on your own phone and help them operate it. Each brand has a different solution:

- **4. Wireless Charging Protocols**: Built-in "Changlian" app has a "Screen Sharing" feature. Both you and your parents install Changlian, start a video call, and tap "Share Screen" —you can draw circles on their screen to point out "tap here."
- **Xiaomi/Redmi**: Xiaomi Video Call app has "Remote Assistance," supporting screen sharing + simultaneous voice call.
- **[Install Apps from External Sources]**: Built-in "Family Space" or "Remote Guardian" —search for "remote" in Settings to find it.
- **Universal Solution (Works for All Brands)**: Download the "Sunflower Remote Control" app (free version is sufficient). Install it on both phones. Register the same account or add as friends. Have your parents read you the verification code displayed on their phone, and you can remotely view and operate their screen.
- **: If you need to extract text from scanned documents for editing, you need OCR. WeChat's built-in "Extract Text" feature is the simplest —long-press the image to use it, and the recognition accuracy is decent. For bulk processing, you can use the Baimiao app or CamScanner. For Chinese recognition, Baidu's OCR engine works quite well; WPS's image-to-text feature is also very accurate.**: FaceTime supports "Screen Sharing" during calls. While on a FaceTime call, tap the screen share button, and the other person can see your screen. But iOS security prevents remote operation —it's "instructive teaching" only.

V. WeChat "Senior-Friendly Makeover"—Just Tell Your Parents These Things

WeChat is the app elderly people use most, but the interface is too dense and cluttered. Here are some quick, effective adjustments:

- **Enlarge WeChat's Own Font**: WeChat > Me > Settings > General > Font Size, drag to maximum. Note: this only affects the font within WeChat, not the system font.
- **Enable "Care Mode"**: WeChat > Me > Settings > Care Mode > Enable. After enabling, fonts and buttons become larger, color contrast is enhanced, and you can one-tap "Listen to Text Messages" (tap a text message and it reads aloud).
- **Help Them Label Important Contacts**: Change children's and relatives' names in contacts to easily recognizable ones —like "Eldest Son - Xiaoming" or "Second Daughter - Xiaohong," so they immediately know who's who when opening WeChat.
- **Teach Them to Use "Voice Input" Instead of Typing**: On the keyboard next to the chat box, hold the spacebar (Android) or tap the microphone icon (iPhone), and speak directly —it converts to text and sends. Recognition accuracy is already very high —for elderly people who can't type in pinyin, it's a "game-changing" convenience.
- **Disable Unused Features in "Discover"**: WeChat > Me > Settings > General > Discover Page Management, turn off "Moments," "Channels," "Live," "Shopping," "Games" —anything not used —to reduce clutter.

VI. What If the Phone Is Lost? Make an "Emergency Plan" for Elderly Relatives

Preparation prevents problems. While the phone isn't lost yet, do these things first:

1. **Confirm "Find My Phone" Is Enabled**: iPhone —Settings > [Your Name] > Find My > Find My iPhone; Android —search for "Find Device" in Settings. Make sure this toggle is on.
2. **Write Important Passwords and Accounts on a Piece of Paper**: Lock one copy of Apple ID / Huawei account / Xiaomi account passwords in a home drawer, and give another copy to a trusted child. If the phone is lost, you need these accounts to remotely lock and locate it —forgetting the password makes it very difficult.
3. **Teach Elderly Relatives One Sentence**:**"If the phone is lost, the first thing to do is call family."**Family can immediately log into iCloud or Android's Find My Device website to remotely mark it as "Lost Mode" —the person who finds it can't unlock or use it.

VII. Wireless Earphones and Hearing Aid Compatibility —An Overlooked Feature

Many elderly relatives have hearing issues but don't know that phones can work with hearing aids. iPhone and some Android phones support MFi (Made for iPhone) hearing aids, which can directly transmit phone call audio to the hearing aid —much clearer than holding the phone to the ear.

- **Connection Method**: iPhone Settings > Accessibility > Hearing Devices, turn on Bluetooth, and follow the hearing aid's pairing instructions.
- **If Not Using a Hearing Aid**: In "Accessibility > Audio/Visual," enable "Phone Noise Cancellation" and "Mono Audio" to improve call clarity.
- **Android**: In "Accessibility > Hearing Enhancement," there are similar settings —locations vary by brand. Search for "hearing aid" directly.

VIII. Health-Related Features —A Phone Is More Than a Communication Tool

- **SOS Emergency**: Press the power button five times consecutively (on most phones), and the phone automatically calls emergency services and sends location to preset emergency contacts. Set this up for elderly relatives in advance —iPhone: "Settings > Emergency SOS"; Android: "Settings > Safety > Emergency Assistance." Practice once before teaching them, so they don't fumble when stressed.
- **Fall Detection**: Apple Watch has this feature —if it detects a fall followed by no movement, it automatically calls emergency services and notifies family. If budget allows, getting one for elderly relatives can save lives at critical moments.
- **Medication Reminders**: iPhone's "Health" app has a built-in medication reminder —set the medication name, time, and dosage, and it pops up a reminder when it's time. Android can use "alarm clocks" or download "medication reminder" apps.

Chapter Summary

- Enable Easy Mode or manually enlarge fonts/icons first —once the environment is set up, most problems disappear
- Three iron rules for helping elderly relatives with phones: don't rush, use analogies, write it down
- Remind about six common scams repeatedly —three defense lines can block 90% of fraud
- Remote assistance lets you fix your parents' phone from anywhere

- WeChat's "Care Mode" + "Voice Input" are the two most practical features for elderly users
- SOS Emergency and Fall Detection —life-saving configurations worth spending five minutes to set up

Chapter 28: Spend Wisely on Your Dream Phone —The Ultimate Smartphone Buying Guide

When buying a phone, many people spend a week reading reviews, a month comparing prices, then end up spending five or six thousand on a flagship they don't actually need. This chapter doesn't list spec sheets —you can find those on official websites. What I want to give you is a "phone selection mindset" —so no matter what new phone hits the market, you can judge whether it suits you.

Three-Step Phone Selection: Needs Budget Model

Many people ask online "what phone should I buy for 3,000 yuan?" —this question can't actually be answered, because we don't know what you'll primarily use the phone for. The correct thinking order when buying a phone should be:

Three-Step Phone Selection

1. **First, figure out what you care about most**: Photography? Gaming? Battery life? Screen? System smoothness? Or just need to make calls and use WeChat? Prioritize your needs.
2. **Then, Set a Budget Ceiling**: Not "roughly how much" but "the maximum I'm willing to spend." Set a hard ceiling so you don't keep looking at pricier options.
3. **Finally, Pick a Model**: Use your prioritized needs and budget to match with models.

I. How to Read Core Specs —You Don't Need to Understand Everything, Just These Five

Spec sheets have dozens of data points, but ordinary users only need to focus on these five dimensions:

Dimension	How to Evaluate	One-Sentence Summary
Processor (SoC)	Determines how many years the phone will stay smooth. 2026 standards: Snapdragon 8 Elite Gen5 / Dimensity 9500 and above = top tier; Snapdragon 8s Gen4 / Dimensity 8400 = mid-to-high; Snapdragon 7+ Gen4 / Dimensity 7350 = good for three years. Below Snapdragon 6-series —be cautious.	Buy new, not old —SoC determines how many years your phone stays lag-free
Screen	OLED has better colors and is more power-efficient than LCD; 120Hz refresh rate is noticeably smoother than 60Hz; resolution should be at least 1080P. High-frequency PWM dimming (1920Hz+) is more eye-friendly.	OLED + 120Hz + high-frequency dimming = the baseline for a good screen
Using Huawei phone prioritize HarmonyOS Connect, best experience with Huawei devices.	**Don't Look at Megapixels —Look at Sensor Size**. Sensor size matters much more than megapixels. Ultrawide is practical; telephoto (3x+ optical zoom) for distant subjects; macro lenses are mostly filler.	Large-sensor main camera + practical telephoto > multiple filler lenses
Battery and Charging	4500mAh is the baseline; 5000mAh+ easily lasts a day. For fast charging: 60W+ is fast, 100W+ is blazing fast. Wireless charging is convenient but slow.	5000mAh + 60W fast charging = anxiety-free combo
System Update Commitment	Determines how long the phone stays secure. Apple typically provides 5-6 years of system updates; Samsung flagships get 4 years system + 5 years security updates; Google Pixel gets 7 years; domestic mid-range phones generally have shorter commitments (2-3 years).	Want to use it longer — choose a brand with long update commitments

II. Budget-Tier Recommendations (Based on Mid-2026 Market)

The following recommendations are based on the mid-2026 Chinese mainland market. Prices are official starting prices. Actual prices fluctuate —refer to the price at time of purchase.

Under 1,000 Yuan: Can Make Calls, Use WeChat —Good Enough

Who It's For: Students, backup phones, users with minimal requirements, primarily for calls and WeChat.

What You Can Get: Redmi Note series, Honor X series, realme number series —low-to-mid configs. Processors are roughly Snapdragon 6-series or Dimensity 6000-series. Screens are mostly LCD. Cameras can take clear photos but autofocus is slow. System smoothness is average.

Buying Advice: At this price, look at two things: Is the battery big enough (choose 5000mAh+)? Is there enough memory (start with 6GB+128GB, don't buy 4GB —it'll lag)? Redmi has the clearest advantage at this price point.

1,000-2,000 Yuan: Best Value —The Sweet Spot That Lasts 3-4 Years Without Lag

Who It's For: Students, budget-conscious users who want longevity, no extreme demands on photography or gaming.

Representative Models: Redmi Turbo series, iQOO Z series, Honor X series high-spec, OnePlus Ace series entry-level.

Buying Advice: Choose models with Snapdragon 7+ Gen3 or Dimensity 7300+ processors, with 8GB+256GB memory. This configuration easily handles daily use + occasional Honor of Kings, and can last 3-4 years without lag.

2,000-3,500 Yuan: Mainstream —Noticeable Improvements in Performance and Photography

Who It's For: Ordinary users who want decent photography, lag-free gaming, and longevity. This is the price range most people buy in.

Representative Models: Xiaomi number series (like Xiaomi 17 standard after price drops), vivo S series high-spec, OPPO Reno series, Honor number series, Redmi K series.

Buying Advice: The biggest characteristic of this price range is "all-around phones" —no obvious weaknesses. Compared to sub-2,000 yuan phones, the main improvements are: noticeably better cameras (with OIS), high-quality OLED screens, faster charging (80W+ common), and more refined build quality. Choose 12GB+256GB or 12GB+512GB —easily lasts 4-5 years.

3,500-5,000 Yuan: Near-Flagship —More Than Enough for Everyone Except Enthusiasts

Who It's For: Users who pursue photography, those who want to experience premium build quality.

Representative Models: Xiaomi number series standard, vivo X series, OPPO Find X series standard, older iPhone flagships (like iPhone 16).

Buying Advice: At this price, you can already buy last year's flagship models. My advice is:**Buy last year's flagship, not this year's mid-range**. Last year's flagship typically has better materials, build, and camera hardware than this year's mid-range. The prerequisite is accepting it's not the "latest model."

5,000+ Yuan: Flagship Zone —Money No Object or Pursuing Ultimate Experience

Who It's For: Photography enthusiasts, heavy gamers, users seeking top-tier screens and craftsmanship, content creators who shoot video on phones.

Representative Models: iPhone 17 Pro/Pro Max, Samsung S26 Ultra, Huawei Pura 80 Ultra, Xiaomi 17 Ultra, vivo X300 Pro.

Buying Advice: At this price, the differences between brands are shrinking. It mainly comes down to which system you prefer. Like photography —choose vivo or Huawei. Like video —choose iPhone. Like screen and S Pen —choose Samsung. Like balance —choose Xiaomi. Honestly, at this price point,**you can go with your gut —just pick the one that looks right to you**.

III. Choose by Scenario —What Are Your Needs?

Here are the most common "user profiles" —just find the one that matches you:

4. "Ears" for Hearing-Impaired Users—Complete Live Captions Guide	**Key Specs to Look For**	**Recommended Direction**
Photography Enthusiast	Large-sensor main camera (1-inch ideal), OIS, telephoto (3x+ optical zoom), imaging chip	vivo X series, Huawei Pura/Mate series, Xiaomi Ultra series
Gamer	Top-tier processor, high touch sampling rate, cooling system, large battery + fast charging	iQOO Digital series, ROG Phone, RedMagic, Redmi K series
Battery Life Priority	5500mAh+ battery, energy-efficient processor, LTPO display	Honor Magic series, OnePlus, Redmi Note large battery edition
For Elderly Users	Easy mode, large fonts, loud volume, big battery, many service centers	Huawei Enjoy, Honor Play, OPPO A series, Redmi Note
For Students	High value for money, durable, large battery, parental control features	Redmi Note/Turbo, realme GT, iQOO Neo
Apple Ecosystem Users	Already own Mac/iPad/AirPods, iPhone is the only solution	iPhone 17 standard (value) or Pro (full experience)
International Travel	eSIM support, complete global band coverage, Google services	iPhone, Samsung S series, Pixel, OnePlus

IV. Five Common Pitfalls —Check Before You Buy

1. **Pitfall 1: Four or Five Cameras Don't Equal Good Photos**—Manufacturers love to stack lens numbers in specs, but two of them are often 2MP "filler lenses" (macro + depth) that are practically useless. When evaluating a phone's photography,**only look at the main + ultrawide + telephoto**—these three are enough. You can pretend the rest don't exist.

2. **Pitfall 2: Don't Be Fooled by 108 Megapixels**—High megapixel count doesn't mean good image quality. Sensor size, aperture, and algorithm tuning matter far more than pixel count. Many flagship phones use 50MP large-sensor cameras that produce photos far superior to some 108MP budget phones.

3. **Pitfall 3: Don't Buy Carrier-Locked Phones**—"Deposit phone credit, get a free

phone" looks like a great deal, but carrier-customized phones have slow system updates, non-removable pre-installed apps, and are usually carrier-locked (only work with that carrier's SIM card). You think you're saving money, but you're actually getting "locked in."

4. **Pitfall 4: "Curved Screens Are Premium" —Marketing, Not Truth**—Curved screens were once a flagship signature, but more and more people are discovering their problems: hard to apply screen protectors, edge accidental touches, and expensive repairs when cracked. The 2026 trend is returning to "slightly curved" or even "completely flat" screens. Don't choose a curved screen just because "curved means premium."

5. **Pitfall 5: Not All "AI Phones" Are Actually Useful**—In 2026, every manufacturer is shouting "AI phone," but many just renamed their existing voice assistant. Genuinely useful AI features include: real-time translation, AI eraser (removing people from photos), AI photo editing, on-device LLM inference, and localized smart summaries. Other flashy features are mostly gimmicks.

V. When Is the Best Time to Buy?

- **618 Shopping Festival (late May to mid-June)**: The period with the biggest phone price drops of the year. Many manufacturers slash flagship prices by 800-1,500 yuan to boost sales. This is the best window for buying phones in the first half of the year.

- **Double 11 (late October to mid-November)**: The second-half mega sale, with discounts comparable to 618. Many brands release new flagships around this time, so the previous generation sees the biggest price drops.

- **One Month After New Release**: At launch it's full price, but after a month, channel prices (third-party stores) typically drop by 200-500 yuan.

- **Two Weeks Before Chinese New Year to Lantern Festival**: Brick-and-mortar stores clear inventory during the "New Year Goods Festival," and you can negotiate prices with staff.

- **Not Recommended Times**: Buying on launch day —unless you really want to be among the first owners, launch prices are the most expensive.

VI. Where to Buy —Which Channels Are Reliable?

- **Brand Official Store / Tmall Flagship Store / JD.com Self-Operated**: Safest option. No price advantage, but no worry about refurbished or counterfeit goods, and best after-sales service.

- **Pinduoduo "10 Billion Subsidy"**: Lowest prices, but only buy from stores labeled with "10 Billion Subsidy." Avoid unlabeled ones. Record an unboxing video when you receive the goods —you can return if there are issues.
- **Physical Retail Stores**: You can touch and feel the phone. Note: Sales staff will push high-margin models (not the most suitable ones for you). Before visiting, lock in 2-3 target models online, and tell them "I only want to see these."
- **Used Platforms (Xianyu / Zhuanzhuan)**: Cheaper but risky. If buying used, remember three rules: require purchase invoice or order screenshot, require IMEI number for verification,**only use platform transactions, never transfer money privately**.

Hands-On: Use This Method to Choose a Phone

1. Take a piece of paper and write down your two core needs (e.g., battery life first, camera second).
2. Set a budget ceiling (e.g., 3,500 yuan).
3. Open JD.com or Tmall, filter by price, and circle 3-5 candidate models.
4. Search Bilibili for "XX model one-month usage experience" (not first-day reviews, but genuine feelings after at least a month of use), and watch two or three videos.
5. Confirm the channel is reliable before ordering, and record an unboxing video upon delivery.

Chapter Summary

- Three-step phone selection: define needs set budget pick model
- Five core specs to check: processor, display, camera, battery/charging, system update commitment
- 1,500-3,500 yuan is the best value-for-money sweet spot
- Last year's flagship is a better deal than this year's mid-range
- Five pitfalls: multi-camera scam, high-megapixel trap, carrier-customized phones, curved screen myth, AI gimmicks
- 618 and Double 11 are the two best time windows for buying phones each year

Chapter 29: Digital Wellbeing — Building a Healthy Relationship with Your Phone

Have you ever experienced this: you only meant to scroll short videos for 5 minutes, then looked up and realized two hours had passed? Or lying in bed at night scrolling your phone, getting more and more awake, and finally falling asleep at 2 AM? Phones have changed our lives, but they're also quietly consuming our time and health. This chapter isn't about "quitting phones" —it's about helping you take back control.

I. Check Your "Screen Time" —Numbers Are a Wake-Up Call

Before making changes, understand the current situation. Your phone has a feature many people have never opened called "Screen Time" (iPhone) or "Digital Wellbeing" (Android). It tells you exactly how much time you spend on your phone each day and which apps that time goes to.

- **iPhone**: Settings > Screen Time > See All Activity. You'll see daily unlock counts, pickup counts, and time spent in each app.
- **Android**: Settings > Digital Wellbeing & Parental Controls. Huawei calls it "Healthy Phone Usage," Xiaomi calls it "Screen Time Management." Similar features, all showing daily usage data.

80% of people who see this data for the first time are shocked. A friend of mine discovered after checking that they spent over 4 hours a day on Douyin —that's 28 hours a week, equivalent to a part-time job. This kind of "digital shock" is the first step toward change.

II. Set App Time Limits —Install a Speed Limiter for Yourself

Once you know where the problem is, you can set targeted limits. No need to delete apps outright —just set a "daily allowance":

1. **iPhone**: Settings > Screen Time > App Limits > Add Limit. Select the app you want to restrict (e.g., Douyin) and set 30 minutes per day. When the time's up, the app icon grays out, and opening it requires an additional password entry.
2. **Android**: Settings > Digital Wellbeing > App Timers. Select an app and set a daily usage limit; when time's up, the app icon grays out.
3. **Tip**: Set the limit password to something you don't use often and is hard to remember. If it's too easy to enter, you'll be tempted to "extend" every time, making the limit useless.

III. The Right Way to Use "Do Not Disturb" —Not Muting Notifications, But Muting Anxiety

Many people's phones buzz with notifications dozens of times a day, and each buzz interrupts their attention. You think you're just "glancing at a notification," but it actually takes 15-20 minutes to refocus after each interruption. Do Not Disturb isn't about "going offline" —it's about helping you regain control of your attention.

- **Schedule It**: Set Do Not Disturb to automatically turn on from 10 PM to 7 AM. This way, your sleep won't be interrupted by a spam text message.
- **Allow Important Contacts Through**: In Do Not Disturb settings, set "Allow Calls From" to "Favorites" or enable "Repeated Calls" (a second call within 3 minutes will ring normally). This way you won't miss emergency calls from family, but won't be bothered by sales calls.
- **Work Focus Mode**: iPhone's "Focus" mode lets you set custom scenarios —for example, a "Work" focus only allows work-related app notifications, while a "Reading" focus silences all notifications.

IV. Eye Protection Relies on Habits, Not Apps —The 20-20-20 Rule

Prolonged screen time inevitably causes dry, sore eyes. Eye protection mode (blue light filter) helps somewhat, but what really works is changing your eye usage habits:

The 20-20-20 Rule

Every**20 minutes**of screen time, look at something**20 feet (about 6 meters)**away for**20 seconds**. This simple action effectively relieves ciliary muscle fatigue.

- **Enable Eye Protection Mode**: Settings > Display > Eye Protection (or Night Shift). When enabled, the screen's color temperature warms up, reducing blue light. Recommended to set it to auto-enable from sunset to sunrise.

- **Adjust Screen Brightness**: When the environment is dark, the screen should be dim too. Many people keep their screen too bright in the dark —this is the most damaging behavior for your eyes. Enable "Auto-Brightness" and make sure it works properly.
- **Maintain Distance**: Keep the phone at least 30 cm from your eyes. Many people hold their phones very close, which keeps the ciliary muscle in a state of tension, accelerating nearsightedness.

V. Neck Rescue Guide for "Phone Benders"

When looking down at your phone, your neck bears 5-6 times the pressure of normal standing. Long-term neck bending can straighten the cervical curve, cause bone spurs, dizziness, and hand numbness. Here are some simple improvement methods:

- **Raise Your Phone**: When looking at your phone, try to hold it at eye level rather than looking down at your lap. Your arms may get tired, but your neck will thank you.
- **Move Your Neck Every 30 Minutes**: Slowly turn your head left to the limit, hold for 3 seconds, return to center; then turn right, hold for 3 seconds. Repeat 5 times. Then do the "rice character exercise" —trace the character 米 ("rice") in the air with your chin.
- **Wall Standing for 3 Minutes**: Press the back of your head, shoulder blades, buttocks, and heels against a wall. Stand for 3 minutes daily to effectively correct the hunched, head-down posture.

VI. Sleep and Phone —The Bedroom Habit Most Worth Changing

Blue light from your phone screen suppresses melatonin production, making you "more awake the more you look." Many people feel "I'll get sleepy as I scroll," but they're actually being tricked by blue light —when they finally put the phone down, they can't fall asleep. Here are improvement suggestions:

- **No Phone for 1 Hour Before Bed**: This is the most effective yet hardest to follow. If you can't manage it, at least turn on "Night Shift/Eye Protection Mode" to minimize blue light.
- **Put the Phone Out of Reach**: Put the charger at the other end of the bedroom, not on the nightstand. This way you won't be tempted to pick it up and scroll, and when the alarm goes off, you have to get up to turn it off —killing two birds with one stone.
- **Enable "Sleep Mode"**: iPhone's "Sleep" Focus can automatically lock the screen and simplify the lock screen during sleep hours; Android brands have similar features.

- **Use an Alarm Clock Instead of Your Phone**: Buy a cheap physical alarm clock for a dozen or so yuan. Stop using your phone as an alarm, and you'll escape the trap of "just checking the time, then can't help scrolling."

VII. Information Anxiety —Learn “Selective Blindness”

The amount of information we receive daily equals what people in ancient times received in a lifetime. But the human brain wasn't designed to process this much information —overload leads to anxiety, scattered attention, and decision fatigue. The solution isn't “getting more information” but “actively filtering” :

- **Unfollow**: Open your list of followed accounts, bloggers, and creators, and ask yourself one question: "Did I get any real value from this in the past month?" If not, unfollow. Quality over quantity for information sources.
- **Turn Off Non-Essential App Notifications**: Turn off notifications for all apps except WeChat, phone, SMS, and calendar. News doesn't need push notifications —you'll see it when you actively look.
- **Set an "Information Fast Day"**: Pick one day a week (e.g., Sunday) to completely avoid phone news and short videos. You'll find the sky doesn't fall, and you'll feel noticeably better on Monday.

Hands-On: One-Week Digital Health Challenge

1. Today: Open Screen Time / Digital Wellbeing and review last week's usage data.
2. Tomorrow: Set daily limits for your most time-consuming entertainment apps (recommend 30-60 minutes).
3. Day after: Schedule Do Not Disturb (22:00-07:00), allowing calls from favorites.
4. Day 4: No phone for 1 hour before bed; move charger to the other end of the bedroom.
5. Day 5: Unfollow 5 accounts or bloggers you no longer watch.
6. Day 6: Try a full day of "information fast" —no news, no short videos.
7. Day 7: Review this week's Screen Time report and compare with last week.

Chapter Summary

- First check your screen time data —understanding the current state is the first step to change
- Set limits for entertainment apps, schedule Do Not Disturb —regain control of your attention
- The 20-20-20 rule is the simplest and most effective eye care habit

- Looking down at your phone puts 5-6x pressure on your neck —raise your phone and stretch regularly
- No phone 1 hour before bed, charger out of reach
- The antidote to information anxiety is ”active filtering,” not ”getting more”

Chapter 30: Screen Mirroring, Presentations, and Car Connectivity

Your phone screen is at most 7 inches, but your TV is 55 inches and your projector is 100 inches. Casting your phone screen to a larger display is a completely different experience. This chapter covers three high-frequency scenarios: screen casting, presentations, and car connectivity —helping you extend your phone's capabilities to bigger screens.

I. Three Wireless Casting Camps —Don't Mix Them Up

Wireless casting all looks like "phone screen to TV," but there are three underlying technologies that are mutually incompatible:

Technology	**Who Uses It**	**Protocol**
AirPlay	iPhone/iPad/Mac	Apple ecosystem, smoothest experience. Requires Apple TV or AirPlay-enabled TV
Miracast	Android phones	Wi-Fi Direct connection, no router needed. Android phones' "wireless casting" uses this by default
DLNA	All platforms	Only pushes media files (video/music/photos), doesn't mirror screen. Good for pushing video to TV for playback

Operation methods for each platform:

- **iPhone**: Control Center > Screen Mirroring > Select your TV/Apple TV. If the TV doesn't support AirPlay, you can buy an Apple TV box or an AirPlay-enabled TV stick.
- **Android**: Swipe down the notification panel > Find "Wireless Cast" / "Smart Cast"

/ "Multi-Screen Interaction" button > Select TV. Phone and TV must be on the same Wi-Fi.

- **Universal Solution**: If the TV is too old to support any wireless casting, plug a Chromecast or Xiaomi Box / Huawei Box into the TV's HDMI port to receive casts.

II. Wired Casting —More Stable, Zero Latency

Wireless casting is convenient but has latency, making gaming and presentations unpleasant. Wired casting solves this:

- **iPhone**: Requires a Lightning/USB-C to HDMI adapter cable (about 100-200 yuan). Plug in, and the phone screen outputs directly to the TV/projector's HDMI port.
- **Android**: Some phones support USB-C to HDMI direct connection (requires DP Alt Mode support; most flagships support it). Just buy a USB-C to HDMI cable.
- **Meeting Room Scenario**: If you frequently use your phone for presentations, keep an HDMI adapter cable in your bag. Meeting room TVs/projectors almost all have HDMI ports —just plug in and go, far more reliable than fiddling with wireless casting.

III. Phone Becomes Computer —Desktop Mode

Some Android phones support "Desktop Mode"—when connected to a monitor, the phone outputs a computer-like desktop interface that can be operated with mouse and keyboard:

- **Samsung DeX**: Samsung flagship's signature feature. Connecting a monitor brings up a desktop interface with multi-window support, the closest experience to a real computer. With Bluetooth keyboard and mouse, you can travel without a laptop.
- **Huawei Desktop Mode**: Huawei Mate series / Pura series also has similar functionality when connected to a monitor. Supports multi-window, file management, and office apps.
- **Motorola Ready For**: Moto flagships also support it, with features similar to DeX.
- **What is RAW**: Light office work (email, document editing), PPT presentations, watching videos on big screen. Not suitable for heavy Excel work or professional design.

IV. Car Connectivity —The Right Way to Use Your Phone While Driving

Operating your phone while driving is illegal, but navigation, music, and calls are essential. Car connectivity systems let you use your phone's core features on the car's infotainment screen, so the phone can be set aside:

- **Apple CarPlay**: Top choice for iPhone users. Connect to the car system via USB or wirelessly, and the car screen shows a streamlined iPhone interface —navigation (Amap/Baidu Maps), music, phone, messages, podcasts. Siri voice control is entirely hands-free.

- **Android Auto**: Android users' counterpart. Features similar to CarPlay, supporting Google Maps, Spotify, etc. Some features are unavailable in China due to Google service restrictions.

- **Huawei HiCar**: Huawei's proprietary solution, widely supported by domestic car models. Beyond navigation and music, it also supports Huawei video calls, parking camera fusion, and other unique features.

- **Baidu CarLife**: Cross-platform solution, works with both iOS and Android. Fewer features but broad compatibility —many older car models only support CarLife.

Security Reminder

Regardless of which car connectivity solution you use, always rely on voice control while driving. Set your navigation destination and music playlist before starting to drive —never operate the car screen while in motion.

V. Meeting Room Casting in Practice —No More Panic

Meetings are the most frequent casting scenario and also the most prone to problems. Here are practical tips:

- **Arrive Early to Test**: Never wait until the meeting starts to connect casting. Arrive 10 minutes early and test your connection method. Wi-Fi casting fails more often than you'd think.

- **Prepare a Wired Plan B**: Keep an HDMI adapter cable in your bag. When wireless casting won't connect, a wired connection is your lifesaver.

- **Disable Notification Preview**: When casting, your WeChat messages appear on the meeting room big screen. Before casting, turn off "Lock Screen Notification Content Preview" in settings, or enable "Presentation Mode" (some phones support this —automatically hides notifications during casting).

- **"Present Documents" Rather Than "Operate Your Phone"**: If you're presenting a PPT, store the file on your phone and open it in WPS for fullscreen playback, rather than casting your entire phone screen. This is more professional and smoother.

Hands-On: Complete a Phone-to-TV Cast

1. Chapter 18: Data Backup and Migration
2. Open a photo or video on your phone.
3. Try wireless casting (iPhone uses Screen Mirroring, Android uses the wireless cast button).
4. If it fails, check whether the TV supports the corresponding protocol, or try DLNA push (tap the "Cast" button in the Gallery/Video app).
5. After successful casting, try doing other operations on your phone to see if the big screen mirrors them.

Chapter Summary

- Three wireless casting camps: AirPlay (Apple), Miracast (Android), DLNA (cross-platform media push)
- Wired casting is more stable with zero latency —an HDMI adapter cable is essential for meeting rooms
- Samsung DeX / Huawei Desktop Mode turns your phone into a light office computer
- Car connectivity: Four major solutions —CarPlay / Android Auto / HiCar / CarLife
- Meeting room casting golden rules: test early, prepare a Plan B, disable notification preview

Chapter 31: Getting Started with Short Video Shooting and Content Creation

In 2026, short videos are no longer "toys for young people"—they've become the fundamental way everyone gets information and expresses themselves. Whether you want to document your life, share knowledge, or start a side business, your phone is your best creative tool—no need to buy a camera or learn Premiere. A single phone is enough to get started. This chapter helps you start from scratch.

I. Think Before You Shoot: Who's Your Audience?

The first mistake many people make is "picking up the phone and shooting immediately" without knowing who the audience is. Before you start, answer three questions:

1. **Who is my audience?**—Content for kids, peers, or elderly viewers requires completely different styles.
2. **What value can I provide?**—Make people laugh (entertainment), learn (knowledge), crave (food), or feel warm (emotion). You need at least one.
3. **Which platform can I consistently post on?**—Douyin/Kuaishou (general entertainment), Xiaohongshu (lifestyle), Bilibili (in-depth content), WeChat Channels (social circle). Choose one main platform to focus on—don't distribute across all platforms from the start.

II. Phone Camera Basic Settings—5 Things to Do Before Shooting

1. **Set Resolution to 1080P, Frame Rate to 30fps**: This is the standard for short video platforms. 4K files are too large and upload slowly. 60fps has no advantage on social platforms and just creates bigger files.

2. **Clean the Lens**: The first and most important tip. Wipe the lens with your clothes before shooting —the image immediately becomes noticeably clearer.

3. **Lock Focus and Exposure**: Long-press the subject in the viewfinder for 2 seconds until "AE/AF Lock" appears. This prevents brightness fluctuations while moving during the shot.

4. **Horizontal or Vertical?**—For Douyin/Kuaishou/WeChat Channels/Xiaohongshu**shoot vertically**(9:16). For Bilibili/YouTube**shoot horizontally**(16:9). Decide the platform first, then the orientation.

5. **Enable Grid Lines**: Settings > Camera > Grid. The foundation of rule-of-thirds composition, helping you quickly find the visual center.

III. Five Camera Movement Techniques —Make Your Footage Come Alive

The biggest problem for beginner videographers is footage that's too "static" —standing still the entire time. Learn these five basic camera movements, and your footage will instantly feel more professional:

- **Push**: The phone slowly moves toward the subject. Creates tension or intimacy of "approaching."
- **Pull**: The phone slowly moves away from the subject. Pulls from detail to wide shot, establishing the environment.
- **Pan**: The phone stays in place, rotating left or right. Good for landscapes and showing space.
- **Track**: The phone moves parallel, following the subject. Requires steady hands, best with a smartphone gimbal.
- **Follow**: The phone follows behind or in front of the subject. The most commonly used movement in vlogs.

Stability Is the Top Priority: Shaky footage looks very unprofessional. Two-handed grip is more stable than one-handed; when walking, bend your knees slightly and use a "ninja walk" to reduce up-and-down bouncing; if you have budget, buy a smartphone gimbal (DJI OM series about 500-800 yuan) —it's a game-changer.

IV. Editing Basics —CapCut Is Enough

JianYing (the domestic version of CapCut) is currently the most beginner-friendly mobile editing tool —free, feature-rich, and template-heavy. Here's the basic workflow:

1. **Import Footage**: Open CapCut > New Project > Select and import your video clips.
2. **Trim**: Drag the timeline to trim the wasted footage at the beginning and end of each clip. Principle: keep only the parts with information —shorter is better than longer.
3. **Add Transitions**: Tap the "Transition" button between two clips and choose a simple transition effect. Beginners should use "Dissolve" or "Fade to Black" —avoid flashy 3D transitions.
4. **Add Subtitles**: CapCut has an "Auto Captions" feature that generates subtitles with one tap. Accuracy is about 90% —just manually proofread once.
5. **Add Music**: CapCut's music library has many royalty-free tracks. Choose a BGM that matches the rhythm and content, and set the volume to about 1/3 of the original video audio.
6. **Export**: Export at 1080P, 30fps, bitrate set to "Recommended."

V. The Importance of Thumbnails —70% of People Decide Whether to Watch Based on the Cover

A short video's thumbnail is its "storefront" —it determines whether people click. Key points for making thumbnails:

- **Thumbnail Should Have One Sentence**: Write the core content of the video in large text, e.g., "Learn Phone Photo Editing in 3 Minutes." Font should be large, colors bright, with contrast against the background.
- **Thumbnail Must Match Content**: Don't be a "clickbaiter" —if the thumbnail says one thing but the video shows another, viewers will swipe away immediately, and the algorithm will penalize you.
- **Facial Expressions Should Be Vivid**: If the thumbnail has a person, pick a moment with a vivid or exaggerated expression. Human faces attract more clicks than text.
- **Making Thumbnails in CapCut**: In CapCut, select the best frame, add text overlay, and export as an image for the thumbnail.

VI. Three Content Creation Principles

- **First 3 Seconds Determine Life or Death**: On short video platforms, viewers decide within 3 seconds whether to swipe or keep watching. The first 3 seconds must have a "hook" —a question, a suspense, or a surprising visual.

- **One Video, One Topic**: The most common mistake beginners make is cramming too much into one video. One video should solve one problem —thoroughness beats comprehensiveness.
- **Persistence Beats Talent**: Algorithmic recommendation requires "warming up the account." The first 20 videos will get almost no views —this is normal. Keep updating 2-3 per week, let the algorithm learn your content direction, and traffic will gradually grow.

Hands-On: Shoot a 30-Second Short Video

1. Pick a small topic you know well (e.g., "How to make phone fonts larger").
2. Shoot 3-5 vertical clips, each 5-10 seconds.
3. Use CapCut to splice, trim, add subtitles, and add BGM.
4. Create a thumbnail with large text.
5. Export and send to a friend first for feedback before publishing.

Chapter Summary

- Think before shooting: who's the audience, what value you provide, which platform to post on
- 5 things before shooting: 1080P/30fps, clean lens, lock focus/exposure, decide orientation, enable grid
- Five camera movements: Push, Pull, Pan, Track, Follow —stability is priority #1
- Edit with CapCut: trim transitions subtitles BGM export —six steps
- First 3 seconds are make-or-break, one video one topic, persistence beats talent

Chapter 32: Phone Safety Emergency Handbook

This chapter is an emergency plan. We hope you never need it —but if your phone is lost, stolen, or used in a scam, it will help you cut the damage. In earthquakes, rainstorms, and other disasters, a phone is also how you get warnings, share your location, and stay in touch. Read it through now, screenshot the key steps or print it and keep it at home —because in an emergency your phone may already be gone, and in a disaster it may be dead or offline.

Scenario 0: Earthquake / extreme weather —how your phone helps you stay alive

This is about personal safety, not accounts and money. For warning setup, see Chapter 13, "Weather alerts, disaster warnings, and earthquake early warnings." Here we only cover what to do after the alarm sounds.

Earthquake alarm: Drop, cover, hold —don't just film it

1. Take cover nearby: Drop, cover, hold on (under a table, inside corners of load-bearing walls, next to sturdy furniture). Protect head and neck. Stay away from glass, exterior walls, and hanging lights.

2. Don't blindly rush downstairs: The main shock often lasts only seconds to tens of seconds; stairwells can be more dangerous. Never use elevators.

3. Evacuate in order after shaking stops: Use the stairs, watch for aftershocks and falling objects. Reach open ground away from poles, billboards, and unstable buildings.

4. Use your phone wisely: Short messages to emergency contacts ("I'm at XX, safe/hurt, need help") save more battery than long calls. Prefer text/SMS/WeChat when you can.

5. No signal? See Chapter 7 on satellite SOS / satellite messaging (on supported models), or try higher/open ground. In some cases, network-free Bluetooth walkie-talkie apps help too.

6. Turn off non-essential features: In an emergency, toggle airplane mode on a timer, turn off Bluetooth scanning, lower brightness —stretch battery life by hours. Save the last bar for "I'm safe" and rescue messages.

Rainstorm / typhoon / flash flood: Act during the warning stage

- On a red alert: Stay indoors as much as possible. Avoid underground spaces, underpasses, riverbanks, and steep slopes.
- Trapped outdoors: Move to higher floors of sturdy buildings. Don't wade through floodwater (missing manhole covers and live wires). When calling 119/110, give landmarks and water level.
- Before traveling: Check weather apps and local alert accounts. On long drives, avoid mountain roads along water and cliffs.
- Family plan: Agree that "if we lose contact, use the last WeChat/SMS location" so nobody burns battery on endless calls.

Four things to do before you need them

1. Turn on system earthquake warnings / a trusted warning app, and let it break through silent mode.
2. Pin 2–3 emergency contacts at the top of Contacts and show them on the lock screen (iOS/Android Medical ID or lock-screen message).
3. Keep power banks at home and in your bag. Charge up and pack a cable before long trips.
4. Memorize at least one number you can dial without Contacts (a family mobile). Write it on paper in your wallet.

Scenario 1: Phone lost or stolen —the golden 30 minutes

The first 30 minutes after losing a phone are "golden time." Handle them right and you protect your money and privacy. Follow this order and skip nothing:

Standard 30-minute playbook

1. Borrow a phone and call your own number (minute 1): It may just be nearby; a kind stranger can return it.
2. Lock the phone remotely (minutes 2–5): On another phone or computer, sign in to Find My:
3. Android brands: Huawei cloud.huawei.com / Xiaomi i.mi.com / OPPO cloud.oppo.com, etc.

4. Suspend the SIM (minutes 5–10): Call your carrier (China Mobile 10086 / Unicom 10010 / Telecom 10000) and report the phone lost to suspend the SIM. This is the most critical step —if scammers get your SIM, they receive your codes and can reset every password.
5. Freeze payment accounts (minutes 10–20):
6. WeChat: Report to police at 110, then contact WeChat support, or WeChat Security Center > Freeze account
7. Banking apps: Call the number on the back of your bank card and ask to freeze online banking
8. Tell family and friends (minutes 20 – 25): Post on Moments or have family spread the word that your phone is lost, so scammers can't use your identity to trick others.
9. Report to police (minutes 25–30): Call 110 or go to the nearest station and get a report receipt. You'll need it for insurance and bank disputes later.

Scenario 2: You got scammed —the 72-hour window

If you realize you've been scammed, staying calm is critical. Panic leads to bad decisions and more losses. Do this:

1. Stop all transfers immediately: No matter what they say, don't send another cent. "Send one more deposit and we'll refund everything" is the classic second-round scam.
2. Keep every piece of evidence: Don't delete chats! Don't delete call logs! Screenshot chats, transfers, and the other party's account details. These are key for reports and getting money back.
3. Call 96110 right away: China's national anti-fraud hotline. Operators guide emergency stop-payment —if the transfer was recent (usually within 24 hours), the bank may freeze the other account and intercept funds.
4. Call 110: Bring your phone, ID, and transfer receipts to the nearest station for a statement.
5. Contact your bank/payment platform: Report the scam and ask to freeze the accounts involved. For credit-card fraud, ask the bank to start a chargeback.
6. Change every password: On another safe device, change passwords for Apple ID/brand account, WeChat, Alipay, and banking apps, and turn on two-factor authentication.

Scenario 3: Phone infected with malware —what now?

If your phone shows strange pop-ups, installs weird apps on its own, or your bill/data spikes —it's likely malware:

- Step 1: Enter safe mode —on Android, power off with the power button, then power on and hold volume-down after the vibration until ”Safe mode” appears. Third-party apps are disabled in safe mode, making cleanup easier.
- Step 2: Uninstall suspicious apps —in safe mode, Settings > Apps, sort by date, remove unknown recent installs.
- Step 3: Scan with a security app —install ”Tencent Mobile Manager” or ”360 Mobile Security” and run a full scan.
- Step 4: Factory reset (last resort) —if nothing works, back up important data and factory reset. Warning: A factory reset erases everything. Back up photos, contacts, and key files first.
- iPhones almost never ”get viruses” —iOS’s sandbox makes malware hard to survive. If your iPhone acts up, it’s usually an app bug or a settings issue. A factory reset fixes 99% of problems.

Scenario 4: Account hijacked —take back control fast

1. Change your password now: If you can still sign in, change it immediately. Make it completely different from the old one.
2. Kick other devices: In account security settings, find ”Login devices” and sign out anything you don’t recognize.
3. Check login history: Review the last 7 days. Strange locations mean the account really was taken.
4. Turn on two-factor authentication: If you haven’t, do it now. It’s the most effective way to prevent another hijack.
5. Check linked info: Verify the phone number and email on the account weren’t changed. If they were, use account recovery to take it back.
6. Check payment settings: If WeChat/Alipay was hit, look for altered password-free payments or auto-debits, and bank cards added to unfamiliar accounts.

Scenario 5: Blackmail message —don’t panic, don’t pay

Two common scripts: (1) “I have your private video/photos; pay or I’ll send them to everyone in your contacts” ; (2) “Your phone is hacked; pay or I’ll lock it.” If this happens:

- 99.9% is a scam: They don’t have anything on you. They send the same message to tens of thousands of people and wait for someone to pay.

- Don't reply, don't pay: Replying tells them "this number is live" and marks you as a target for more harassment.
- Screenshot the evidence, then block them.
- Change passwords: If it eases your mind, change passwords on your main accounts.
- If there's a real privacy leak risk: Report to police. They have experience with these cases.

Scenario 6: Deepfake video / voice cloning scams —seeing is no longer believing

Since 2024, deepface and voice cloning let scammers "become" your family, boss, or friends on video calls. "Mom, I'm in trouble, send money" —the person looks and sounds exactly like your son, but it's fake. This is one of the hardest and most dangerous scams right now.

Step 1: Spot the tells —four common giveaways in AI fakes

- Abnormal blinking: Real people blink 15–20 times a minute; AI faces often blink too little or with stiff rhythm.
- Lips out of sync: Watch the mouth against the audio —there's often a fraction-of-a-second lag.
- Blurry face edges: Hairlines, ears, and hair edges may show "fuzzy" fringes or flicker.
- Inconsistent lighting: Highlights and shadows on the face don't match the background light, or features "slide" wrong when the head turns.

Step 2: Three ways to check if they're real

- Ask for a specific action: Turn the head, cover the face, block the mouth then open and close it. Deepfakes break on occlusion and big head turns —faces warp or lag behind motion.
- Ask only they would know: Invent a fake question like "which clinic did we take the dog to last time?" Scammers can't answer or will stall.
- Call back on a known number: Hang up and dial the number saved in your contacts. Scammers can't control the real phone number —the truth comes out immediately.

Step 3: If you already lost money —report it

1. Stop transfers now: No matter how "urgent," pause. Anyone pushing "transfer fast, don't hang up" is almost certainly a scammer.
2. Keep every piece of evidence: Screen-record and screenshot video call frames, chats, their account, transfer receipts —key material for technical investigation.

3. Call 96110: Say it's a "deepfake voice/video" scam; operators can start emergency stop-payment.
4. Call 110: Bring all evidence to the station and explain AI forgery was used —this helps police classify the case.
5. Warn family and friends: Scammers may use your fake video on others. Post a heads-up in Moments or the family group chat.

Scenario 7: SIM swap —your number gets "stolen"

Many people haven't heard of this attack, but it's devastating. A SIM swap means an attacker fakes your identity and gets a replacement SIM at a store. Once that succeeds, your phone suddenly loses service, while their phone with the new SIM receives every SMS code — Alipay, banks, WeChat all wide open.

How to notice: Three warning signs of a SIM swap

- Phone suddenly shows "No service" / "No SIM" even though you didn't touch the card and restarting doesn't help —often the moment of the swap.
- Carrier texts say "SIM replacement successful" or "service change successful" —but you never requested it.
- You suddenly stop getting verification codes —and other texts too, because SMS has been hijacked to their card.

Protection —put a lock on your SIM too

- Set a SIM PIN: Path Settings > Phone > SIM PIN. After enabling, every restart or SIM swap needs the PIN. Even if they get the card, they can't use it.
- Enable carrier two-step verification: In China Mobile/Unicom/Telecom apps, find "Security Center" or "Service verification," turn on replacement-SIM alerts and a second password for service changes. Then staff will demand extra ID checks if someone tries to replace your SIM.
- Don't rely on SMS for important accounts: For banks, Apple ID, and other high-value accounts, prefer authenticator apps (Google Authenticator, Microsoft Authenticator) or hardware keys over SMS codes.
- Watermark ID copies: When a business needs a copy, write "For XX use only" — attackers often impersonate you with leaked ID info.

If it already happened —speed is everything

1. Call your carrier immediately to suspend: Mobile 10086 / Unicom 10010 / Telecom 10000.

Freeze SIM replacement and stop the attacker from using your number.

2. Sign in to key accounts from another device: On a computer or family phone, change passwords for banks, Alipay, WeChat, Apple ID and kick unknown devices.
3. Freeze payment accounts: Alipay 95188 report loss, WeChat Security Center freeze, bank support freeze online banking.
4. Go to a carrier store with your ID: Get a new SIM in person and restore your number.
5. Report to police: SIM swap counts as ”infringing citizens’ personal information.” Bring evidence (suspension time records, financial loss, replacement SIM records).
6. Full audit: After getting your number back, check recent logins and transactions for misuse.

Prevention beats cure —five daily habits

1. Lock screen passcode + two-factor authentication —both on
2. ”Find My Phone” confirmed on
3. SIM PIN set (Settings > Phone > SIM PIN) so a lost phone can’t use your card
4. Don’t install shady apps; don’t tap unknown links
5. Back up important data regularly (photos, contacts, chat history)

Hands-on: Make your own emergency contact card

1. On a piece of paper, write: carrier support (Mobile 10086 / Unicom 10010 / Telecom 10000), bank support, 96110 anti-fraud hotline, 110 police, family numbers.
2. Write the registration email and backup contact for your Apple ID / brand account.
3. Keep that paper in your wallet or on the fridge —not only on your phone, because in an emergency your phone may be gone.
4. Check now: Is ”Find My Phone” on? Is the SIM PIN set?

Chapter recap

- Lost phone golden 30 minutes: Borrow a phone lock remotely suspend SIM freeze payments warn contacts report to police
- Scammed 72-hour window: Stop transfers keep evidence call 96110 report contact bank change passwords
- Malware: Safe mode uninstall suspicious apps security scan factory reset (last resort)
- Hijacked account: Change password kick devices check logins enable 2FA check linked info check payments

- Blackmail messages are 99.9% scams —don't reply, don't pay, screenshot and block
- Deepfake voice/video scams: Watch blinking, lips, edges, lighting; ask them to turn their head and cover their face; ask private questions; call back on a known number
- SIM swap: Treat sudden "No service" as a red flag. Set a SIM PIN, enable carrier two-step verification, use authenticator apps for important accounts
- Five daily habits: Lock screen + 2FA, Find My Phone, SIM PIN, no shady apps, regular backups

Part VI: Outlook and Car Connectivity (Chapters 33-34)

Chapter 33: Future Outlook —The Next Journey of Smartphones

Phones won't disappear overnight, but their "monopoly" may be eroded by a new generation of devices. This chapter covers three things: what next-gen devices look like, how the phone may "disappear" to become a new identity foundation, and the ethical issues we must consider ahead of time.

I. AR/VR and Spatial Computing: The "Second Desktop" Beyond Phones

The competition for next-gen devices has begun. The most likely candidates to divert time from phones are AR/VR and spatial computing devices:

- **Vision Pro Successor**: The launch of Apple Vision Pro marked the official arrival of the "spatial computing" concept, but the first generation's price (starting at 29,999 yuan) and weight remain barriers. A lighter, cheaper Vision is expected in 2026-2027, bringing spatial computing from "developer toy" to "early majority." At that point, iPhone may become the computing companion and data hub for Vision glasses.
- **Another future evolution direction for phones may be "invisibility." It will transform into an omnipresent "behind-the-scenes promoter," becoming the sole cornerstone of your digital identity.**: Open a "second desktop" in front of your eyes, making digital information as ubiquitous as air. Meta and Ray-Ban's smart glasses have sold millions of pairs, and domestic brands like Rokid, XREAL, and INMO are also releasing consumer AR glasses. By 2025-2026, AR glasses can do real-time translation, navigation HUD, and AI assistant conversations, but they're still limited by waveguide display and battery life —3-5 years from "replacing phones."
- **Smart Rings and Wearables**: Samsung Galaxy Ring, Oura Ring, and other smart rings are gaining popularity, serving as "body-worn sensors" that complement the phone's health monitoring. The phone's role is shifting from "all-in-one terminal" to "hub + multiple lightweight accessories."

II. Brain-Computer Interface: Thought as Interaction Medium

Thought itself will become an interaction medium, but this path is longer than imagined:

- **Neuralink Latest Progress**: Musk's Neuralink completed its first human implant in 2024 —the first subject, Noland Arbaugh, controlled a computer mouse with his thoughts to play chess. Currently, implantable BCIs (brain-computer interfaces) primarily target assistive scenarios for paralyzed patients, aiming to help them "operate" the digital world again.
- **Non-invasive BCI**: Multiple companies are developing non-invasive headbands (such as Kernel, OpenBCI) that can read brain signals without surgery, but with far lower precision than invasive ones. We're still quite far from "typing with thoughts."
- **Consumer Outlook**: BCI is 10-20 years from consumer-level phone interaction, but may first be used in accessibility assistance and focus monitoring. Ethical and privacy issues ("Whose privacy do my brain waves belong to?") will become debate focal points before the technology matures.

III. The Phone's "Disappearance" and New Identity Foundation

Another future evolution direction for the phone may be "disappearing" —becoming an omnipresent "behind-the-scenes enabler" and the sole foundation of your digital identity:

- **: Your cloud personal data vault is the center.**: Instantly complete "prove you are you" authentication —no need to manually enter verification codes when logging into any service.
- **An ultimate question always exists: Are we "gods" empowered by technology, or "brains in vats" fed by algorithms?**: Pre-set digital asset inheritance plans during new device activation, making "digital afterlife" a factory default.
- **When you unconsciously light up the screen every few minutes, is this your habit or a designed "addictive behavior"?**: Your cloud-based personal data vault is the center; hardware is just an access point. Change phones or devices —your identity and data always follow you.

IV. Ethical Reflection: Are We Still the Masters of Our Phones?

Let automation be your faithful "co-pilot."

- **AI Ethics**: When on-device AI understands your preferences better than you do, is your "choice" an expression of free will or a guided outcome? When AI Agents place orders and reply to messages on your behalf, where do responsibility and boundaries lie?

- **Data Sovereignty**: Does your health data, brain wave data, and biometric data belong to you, the device manufacturer, or the cloud service provider? The Personal Information Protection Law and EU GDPR are just starting points —the true implementation of "data portability rights" and "the right to be forgotten" still has a long way to go.
- **Digital Divide**: AI phones, foldable screens, and satellite communications are widening the gap between those who "can afford" and those who "can't." Equitable access to digital infrastructure will become a new social issue —just as critical as "knowing how to use the internet" was 20 years ago.
- Build your stable "digital rear base."
- Regularly perform "digital decluttering" and "social fasting."

V. Stay Connected, Stay Independent —Your Digital Life Philosophy

No matter how technology evolves, these digital life philosophies will still apply 10 years from now:

1. Try It: Draft Your "Personal Digital Device Terms of Service"
2. Write a simple "terms of service" for you and your phone to define your relationship. For example:
3. Article 1: My attention time belongs to me. You should proactively enter "Do Not Disturb" mode when I am working/spending time with family.
4. Article 2: My personal data is my property. Your primary responsibility is secure encryption and convenient syncing.
5. Article 3: Our common goal is to improve life efficiency and experience. If you make me addicted through manipulative design, I have the right to uninstall you.

V. Appendices

Write a simple "terms of service" for yourself and your phone (and future AR glasses and AI assistants) to define your relationship. For example:

- Appendix A: New Phone Unboxing "Step-by-Step" Guide
- 1. Unboxing Inspection "Three-Step Process":
- Check Appearance

Chapter Summary

- Vision Pro and AR glasses are new computing entry points beyond phones, but won't replace phones in the short term
- BCI is 10-20 years from consumer use; ethical issues will mature before the technology does
- The phone is evolving toward "disappearing" —from device-centric to identity-centric
- AI ethics, data sovereignty, and the digital divide are core issues for the next 10 years
- Staying connected while staying independent is the timeless philosophy of digital life

Chapter 34: In-Depth Guide to Car-Phone Connectivity —The Fusion of Phone and Automobile

Cars are evolving from "four wheels plus a sofa" to "four wheels plus a smartphone." Whether you drive a gas car or an EV, the relationship between your phone and car infotainment system determines your driving experience. This chapter clarifies the three routes for car-phone connectivity, the characteristics of each camp's system, and tips that make driving easier.

I. Three Routes for Car-Phone Connectivity

There are currently three approaches to connecting phones with cars. Understanding them means understanding the entire ecosystem:

- **Phone Casting to Car Screen**: The car screen acts as just a "monitor" and "speaker" —computing and map data come from the phone. Representative solutions: Apple CarPlay, Google Android Auto, Huawei HiCar, Baidu CarLife. The advantage is that the phone ecosystem is directly brought to the car without reinstalling apps; the disadvantage is that the car's own hardware capabilities go unused.

- **Deep Integration of Phone and Car System**: The car system is independent but shares accounts, apps, and data with the phone, and can even "hand off" tasks. Representative solutions: HarmonyOS Cockpit (AITO/Luxeed), BYD DiLink, NIO OS, Li Auto system. The advantage is an integrated experience; the disadvantage is being locked into one brand ecosystem.

- **CarPlay 2.0 Deep Integration**: The next-generation CarPlay released by Apple in 2025 no longer just "casts" —it takes over core functions like the instrument cluster, climate control, and seat adjustment. Ferrari, Aston Martin, and Porsche are among the first to adopt it, marking a new era where "phone manufacturers drive the car infotainment system."

II. CarPlay / Android Auto / HiCar Comparison

The three mainstream casting solutions each have their strengths. Use this table when choosing a car or phone:

Solution	Supported Phone OS	Supported Car Models	Feature Richness	China Availability
Apple CarPlay	iPhone only	Very wide, most car models support it	High — maps/music/phone/messages all work	Good, works natively
Android Auto	Android (domestic phones need service framework)	Fairly wide, mainly joint venture/imported cars	High, comparable to CarPlay	Average, requires VPN to activate
Huawei HiCar	Mainly Huawei phones, some support other Android	Domestic cars and some joint venture models	Fairly high, can integrate with HarmonyOS Cockpit	Excellent, local ecosystem
Baidu CarLife	Both iOS and Android	Fairly wide, common retrofit for older cars	Medium, updates are slow	Excellent, local ecosystem

III. HarmonyOS Cockpit: Seamless Flow Between Phone and Car

If CarPlay is "casting phone to car," HarmonyOS Cockpit is "phone and car are already family." The collaboration between Huawei phones and AITO/Luxeed car systems represents the highest level of domestic car-phone connectivity:

- **Super Desktop**: The car system directly accesses apps on the phone —no need to reinstall on the car. Music playing on the phone and navigation routes automatically continue on the car screen when you get in, and return to the phone when you get out —seamless throughout.
- **App Handoff**: Navigation set on the phone is automatically taken over by the car system when you close the door; when you arrive at the parking lot, the phone continues walking navigation to the final destination. The "last mile" is no longer a gap.
- **Shared Account Between Phone and Car**: Log into your Huawei account once, and the car system automatically syncs your music membership, map favorites, schedule, and contacts. The car and phone are different terminals of the same identity, not two independent systems.

IV. Car Systems from New Automakers

Domestic EV infotainment systems have evolved from "navigation + music" to mobile smart terminals, each with distinctive approaches:

- **NIO OS**: The NOMI voice assistant is an industry benchmark —it can recognize multiple commands and contextual continuous dialogue, with emotional expressions. The NIO Phone turns the phone into a "car key + second screen," deeply integrated with the NIO ecosystem.
- **Li Auto System**: Multi-screen interaction is the highlight —the front passenger screen can independently play shows, and the rear screen can play cartoons. "Task Master" lets you customize scenarios, like "auto-pause music when door opens, resume when closed."
- **Xiaomi HyperOS Car System**: Deeply integrated with Xiaomi phones, watches, and home appliances. Alarms set on the phone remind you on the car system; when navigating home, the home AC turns on in advance. HyperOS connects the "person-car-home ecosystem."
- **BYD DiLink**: The signature rotating screen —vertical for short videos, horizontal for maps, adapting to app orientation. Built-in app store leads in ecosystem richness among traditional automakers.

V. Phone as Car System —Can It Work?

If your car system is too old or too hard to use, can you just use your phone instead? The answer: yes, but it depends on the scenario.

- **Motorola Ready For / Desktop Mode**: Some Motorola, Samsung, and Huawei phones support "Desktop Mode" —connect a monitor and it becomes a simple computer. In theory, it could connect to a car screen, but real-world compatibility varies, and it lacks car-specific interactions (like steering wheel buttons).
- **Phone Mount vs. Built-in Car System**: If the built-in car system lags or maps are outdated, a sturdy phone mount + phone navigation is a practical choice. But note: (1) small phone screen is hard to see while driving; (2) summer heat can cause the phone to thermal-throttle or shut down; (3) long navigation drains battery fast —you need a car charger.
- **Recommendation**: For short trips or temporary use, a phone mount is fine; for daily commuting and long trips, upgrading to a CarPlay/HiCar-enabled system or installing an aftermarket unit is better for both experience and safety.

Security Reminder

Regardless of which solution you use, operating your phone while driving violates traffic rules. A phone mount is only for "viewing navigation" —not for "operating the phone." Set destinations, switch songs, etc. only after pulling over, or use voice control.

VI. Practical Tips

- **Unstable Wireless CarPlay Connection**: (1) Turn off car Bluetooth and re-pair; (2) Delete paired device records on the car side and reconnect; (3) Check if your phone is connected to multiple Bluetooth devices causing conflicts; (4) Update car firmware and phone system to the latest version; (5) If all else fails, switch to wired CarPlay —more stable.

- **Car Data Plan Used Up**: (1) Turn on phone hotspot and connect the car system to it; (2) In car settings, switch "high data consumption" apps (like video and streaming music) to Wi-Fi; (3) Regularly clear the car's offline map cache.

- **Driving Mode Auto-Trigger**: (1) iOS "Focus" set to "auto-enable when connected to car Bluetooth," automatically blocking non-urgent notifications; (2) Android "Driving Mode" or third-party apps (like Tasker) can auto-mute and auto-reply to messages when connected to Bluetooth; (3) Huawei phones can set "smart payment" to auto-switch music to the car system when you get in.

Tips When Choosing a Car

If you care about the infotainment experience, be sure to test at the dealership: (1) Bring your phone and test CarPlay/HiCar connection stability; (2) Test voice assistant response speed and command recognition; (3) Check if the app store has your frequently used apps; (4) Pay attention to OTA update frequency —good car systems are built through software iteration.

Hands-On: Optimize Your Car Connectivity Experience

1. Confirm which connectivity solution your phone and car support (CarPlay / HiCar / Android Auto / CarLife).
2. Clear the car's Bluetooth pairing list once, keeping only your commonly used phones to reduce connection conflicts.
3. Set up a "Driving Focus" that auto-activates when connected to car Bluetooth.
4. Download the car brand app (e.g., HarmonyOS Smart Driving, Li Auto, NIO App) to see if you can remotely control AC and check vehicle status.

Chapter Summary

- Three connectivity routes: casting (CarPlay etc.), deep integration (HarmonyOS Cockpit etc.), CarPlay 2.0 deep integration
- CarPlay has the widest ecosystem; HiCar has the best domestic experience; Android Auto requires workarounds in China; CarLife suits older cars
- HarmonyOS Cockpit's Super Desktop, App Handoff, and shared accounts represent the deep integration direction
- NIO NOMI, Li Auto multi-screen, Xiaomi person-car-home, BYD rotating screen — each has its own style
- Using a phone as car system is only suitable for temporary scenarios —upgrade or install aftermarket for long-term use
- Practical tips: CarPlay reconnection, phone hotspot for data, Bluetooth-triggered driving mode

Children and Phones Special Topic (Chapter 35)

Chapter 35: Children and Phones — An Educational Guide for the Digital Age

This chapter doesn't paint phones as "monsters," nor does it advocate "the earlier the smarter." We'll go step by step by age: giving devices, setting rules, teaching literacy, and protecting health.

I. At What Age Should Kids Get a Phone?

There's no standard answer, but there are age baselines. Giving a phone too early —the child's prefrontal cortex isn't fully developed, so self-control can't keep up with algorithms' "temptation design." Too late —the child may become "socially disconnected" from peers. Here are age-based recommendations:

- **Ages 3-6**: Try to avoid phones entirely. If needed, no more than 15 minutes per session, with parent-selected educational cartoons, and must be**co-viewed with a parent**. Screen time has the greatest impact on vision and attention development at this stage.
- **Ages 7-9**: Can "borrow" a parent's device for online classes, audiobooks, and research. Start establishing "request before use, return after" rules. No dedicated phone.
- **Ages 10-12**: Consider a "feature phone" or kids' smartwatch for school commute communication. If a smartphone is truly needed, enable strict parental controls.
- **Ages 13+**: Entering middle school, social needs increase. A smartphone can be given, but with a "usage agreement" and regular reviews.

The "Good Enough" Principle

A child's first phone doesn't need to be a flagship. "Excess performance" processors often end up running games. Choose a mid-range phone with adequate battery, eye-friendly screen, or an official refurbished model —easier to manage. Budget reference: 1,500-2,500 yuan covers most students' needs.

Smartwatch as a Transition

If you just want to solve the "can I reach them after school" problem, a kids' smartwatch is better than a phone. It can only make calls, send voice messages, and track location —no browser or short videos, naturally limiting "addiction gateways":

- **XiaoTianCai (Little Genius) Phone Watch**: Most complete ecosystem, peer social "tap to add friend" is its core selling point, but the proprietary app ecosystem is relatively closed.
- **Huawei Kids' Watch**: Good integration with Huawei phones, accurate positioning, supports video calls, high value for money.
- **Xiaomi Kids' Watch**: Affordable, complete basic features, suitable for budget-conscious families.

When buying a watch, note: choose one that supports**4G full network compatibility**models (2G/3G are being phased out), and check carrier kids' plan rates.

II. Complete Parental Control Guide

Before giving the device to the child, the first thing isn't turning it on —it's setting up parental controls. Both platforms have native solutions:

iOS: Screen Time

- **App Limits**: Settings > Screen Time > App Limits —set daily time limits per app category (e.g., games 30 minutes).
- **Downtime**: Set a daily "lights out" period (e.g., 22:00-7:00) —all apps gray out, only phone calls available.
- **Content Restrictions**: Filter movies, web content, and app installs by age, and disable in-app purchases.
- **Family Sharing**: Use the parent's iPhone to remotely manage the child's device, approve download requests, and view usage reports.

Android: Google Family Link / Brand Parental Controls

- **Google Family Link**: Google's official solution —remotely manage device usage time, lock devices, view activity reports. Availability on domestic Android varies by model.
- **Xiaomi Kids Space**: Settings > Kids Space —set usage time, whitelist apps, requires parent password to exit.

- **Huawei Student Mode**: Settings > Healthy Phone Usage > Student Mode —restrict usage times and apps, supports remote management.
- **OPPO/vivo Kids Mode**: Enable in ”Settings” or the dedicated ”Kids Space” app — similar principle.

Feature	iOS Screen Time	Android Family Link / Brand Solutions
App Limits	Supported, by category	Supported, per app
Downtime	Supported	Supported (brand solutions vary slightly)
Content Filtering	System-level, more thorough	Depends on brand, varies
Remote Management	Family Sharing	Family Link / Brand Cloud
App Install Approval	Requires parent approval	Family Link supports it

Parental Controls Aren’t ”Once and Done”

Kids can always find ways to bypass them (search “how to crack Screen Time” online — tons of tutorials). Technical measures are just the baseline —the real defense is parent-child communication and trust. I recommend reviewing usage reports together with your child monthly, rather than unilaterally monitoring.

III. Youth Mode

Major content platforms have built-in “Youth Mode” —when enabled, it limits usage time, filters inappropriate content, and disables in-app purchases and tipping. This is a legally mandated feature, but**parents must actively enable it and set a password**:

- **Douyin/Kuaishou Youth Mode**: 40 minutes daily (disabled 22:00-6:00), only curated science and educational videos shown —no livestreaming, no tipping, no recharging.
- **WeChat Youth Mode**: Limits WeChat Pay (requires parent-authorized amount), restricts Official Account and Channels content, Shake and Nearby features are disabled by default.
- **Game Anti-Addiction System**: National-level real-name authentication + time limits. Minors can only log into games on Friday, Saturday, Sunday, and public holidays from 20:00-21:00, for 1 hour per day.

Don’t Use Parents’ IDs for Real-Name Authentication to “Borrow Accounts”

Many kids use parents’ IDs to register game accounts and bypass anti-addiction. This violates user agreements and removes protections. Parents should safeguard their payment passwords and identity information —don’t “lend accounts” to children for convenience.

IV. Digital Literacy Education

Technology can block 90% of risks —the remaining 10% depends on the child's judgment. Rather than "no internet," it's more important to teach them "how to use the internet":

- **Privacy protection**: Don't share real name, school, or home address with strangers. Don't send personal photos, ID cards, or family group photos. Keep location services off by default, only enable when needed.
- **Cyberbullying Recognition and Response**: Being insulted, ostracized, or having photos maliciously edited are all forms of bullying. Tell your child: don't retaliate, don't delete evidence, tell a parent immediately. On WeChat, long-press the message to report —the platform will handle the bully's account.
- **Content Criticism**: Teach children to distinguish "facts" from "opinions," recognize clickbait and sensationalism. Understand that algorithms only push what you want to see, not the whole picture. Treat "too good to be true" ads as always fake.
- **Cybersecurity**: Don't click unfamiliar links, don't scan QR codes from unknown sources, don't download apps from non-official stores. Don't share passwords with anyone (including "best friends"). Anyone asking for verification codes is a scammer.

Hands-On: Sign a "Phone Usage Agreement" with Your Child

Writing rules down, both parties signing, and posting them prominently is far more effective than verbal agreements. Recommended contents:

- Usage times: e.g., "Available 19:30-20:30 after homework is done."
- No-phone zones: at the dinner table, in the bedroom before sleep, in class —phone put away.
- Content boundaries: don't install apps without telling parents, don't private chat with strangers.
- Violation consequences: e.g., "One violation halves next day's usage time."
- Parent commitment: don't "scroll" on the phone in front of the child beyond agreed time —lead by example.

V. Healthy Usage Habits

Children's eyes, neck, and sleep are still developing —phone damage is greater than for adults:

- **Screen Time Recommendations**: Ages 3-6: no more than 1 hour daily; ages 6-12: no limit for educational content, no more than 1 hour for entertainment; ages 12+: negotiate with the child, but entertainment no more than 2 hours.

- **Eye Protection Habits**: Follow the ”20-20-20 rule” —every 20 minutes of screen time, look at something 20 feet (about 6 meters) away for 20 seconds. No phone 1 hour before bed —blue light suppresses melatonin and affects sleep.

- **Posture Management**: Keep the phone screen at eye level, don’t hunch over. Get up and move after every 30 minutes of continuous use.

- **Family Shared Rules**: Set ”no-phone zones” at the dining table, bedroom, and study. During family movie time and outdoor activities, everyone puts their phone away —making ”putting down the phone” a family culture, not a punishment for the child.

Parents Are the Best ”Firewall”

Research shows a strong positive correlation between parents’ phone usage time and children’s. Demanding that children stay off their phones while scrolling short videos until late at night yourself makes the rules ineffective. To change your child, start by putting down your own phone.

VI. Digital Legacy and Children

This section is easily overlooked but very important. Children’s digital assets (accounts, photos, stored value, game items) also face inheritance and privacy issues:

- **Inheritance of Children’s Digital Assets**: When a child unfortunately passes away, the inheritance process for social accounts, game accounts, and cloud photos is the same as for adults —requires death certificate and proof of kinship. Parents should set up ”legacy contacts” or ”emergency contacts” for children’s important accounts in advance (e.g., Apple Legacy Contact, WeChat account inheritance).

- **Digital Privacy of Minors**: The Personal Information Protection Law stipulates that processing personal information of minors under 14 requires guardian consent, with dedicated privacy rules. Parents should regularly review the privacy policies of apps their children use frequently, and turn off ”personalized recommendations” and ”data sharing” toggles.

- **Account Registration Recommendations**: Accounts registered for children should use parent-controlled email/phone numbers for unified management during the minor stage; account transfer can be done when the child reaches adulthood.

The Other Side of "Children's Digital Legacy"

Children are being "recorded" from birth —growth photos posted by parents, videos from kindergarten, health data from smartwatches. Once uploaded, this data is hard to completely delete. Before posting, think: Will the child feel embarrassed by this photo in ten years? Respecting your child's digital identity starts with restraining from "sharenting."

Chapter Summary

- There's no standard age to give a phone, but ages 3-6 should avoid it, and before 13 use a kids' watch or feature phone as transition
- "Good enough": choose a budget phone, not a flagship —excess performance equals excess temptation
- iOS Screen Time and Android Family Link/brand parental controls are two technical defense lines —use them together for better protection
- Youth Mode and game anti-addiction are legally required, but parents must actively enable them and guard the passwords
- Digital literacy is more important than technical blocking: teach kids to recognize bullying, protect privacy, and critically evaluate content
- Healthy habits start with parents —when you put down your phone, your child will too
- Children's digital legacy and privacy are emerging issues —set up "legacy contacts" and restrain from sharenting

Chapter 36: Phones and Smart Home —Make Your House Think

Your phone can do one more increasingly important job: be the remote for your whole home. From a light bulb or plug to a door lock, smart home gear is moving from "geek toy" to everyday setup. This chapter sorts out protocols, ecosystems, hubs, and starter kits, then closes with a safety checklist so "smart" doesn't backfire.

Smart home protocols: Pick the right one

Before buying a smart device, check which protocols it supports —that decides whether your phone and ecosystem can control it. Four mainstream options and their personalities:

- Matter: Released in 2022 by the CSA (Connectivity Standards Alliance), backed by Apple, Google, Amazon, Samsung, and others. One goal: make every brand speak the same language. Rapidly becoming the unified standard from 2023–2026. Matter devices can join Apple HomeKit, Google Home, Amazon Alexa, and Mi Home at once —no more single-ecosystem lock-in.
- Thread: Matter's underlying network "plumbing." Low power, self-healing mesh (if a node drops, signals reroute), fast and smooth. Devices with a Thread border router (HomePod mini, Apple TV 4K, Xiaomi central hub) feel noticeably more stable.
- Wi-Fi: Big bandwidth, connects directly to the router, but power-hungry and can clog channels with many devices. Good for cameras and robot vacuums that move lots of data.
- Bluetooth (BLE): Easy setup, power-efficient, but short range and no mesh. Good for locks and temp/humidity sensors with tiny payloads.
- Zigbee: Classic low-power mesh protocol; needs a dedicated hub. Devices are cheap and stable, but cross-brand interop is weak. Gradually replaced by Matter + Thread.

Rule of thumb

Buying smart gear in 2026? Prefer the "Matter over Thread" label —the most future-proof choice today. Older Wi-Fi or Bluetooth devices don't need to be tossed; keep using them and move to Matter on new purchases.

The big three ecosystems

Protocols are the "language"; ecosystems are the "club." Even with Matter, you'll still pick one main ecosystem for automations and voice. Quick comparison:

Ecosystem	Strengths	Weaknesses	Best for
Apple HomeKit	Strongest privacy (mostly local), smooth iPhone/Siri/Shortcuts integration	Fewer, pricier devices; needs HomePod or Apple TV as hub	Apple households that value privacy and polish
Mi Home (Xiaomi)	Widest device range, best value, solid Xiao AI	More closed ecosystem, more cloud data, more ads	Budget buyers who want one-stop whole-home setup
Huawei HiLink (HarmonyOS)	Seamless with HarmonyOS phone/tablet/watch; device range growing fast	Fewer overseas devices; ecosystem still catching up	Huawei phone users who want Super Device multi-device flow

How to choose

Follow your phone: iPhone HomeKit first. Huawei HiLink first. Other Android Mi Home is easiest. With Matter, devices from different ecosystems can still be controlled by the same phone —no need to pick sides all at once.

Your phone as the home hub

In most starter setups, the phone is your "home hub" —control lights, AC, curtains, and security from an app, and build automations so the house "moves on its own":

- Device control: Open HomeKit/Mi Home/Smart Life to toggle lights, change color temperature, set AC temperature, open/close curtains, and view cameras —locally or remotely.
- Automation scenes: Combine "condition action." Three favorites:
- Away mode: Last person leaves (location leaves or door locks) lights off + AC off + plugs off + cameras on + arm security.

- Sleep mode: Every night 23:00 main lights off + curtains close + AC to 26°C + night light on.

NFC tap: Stick an NFC tag by the door; tap the phone to trigger a scene (tap once for entry lights, twice for sleep). On iPhone use Shortcuts automation; on Android use Wallet or the brand app to write the tag.

Try it: Build a "Coming home" scene

In your smart home app's "Automation," create one: trigger "arrive at place" (home), actions "entry light on," "AC to 26°C," "curtains close." Save it, go out for a walk, and see if the lights turn on when you return —that first success feels better than ordering new gear.

Starter kit picks

You don't need everything at once. These four cover about 80% of daily needs:

- Smart speaker: Xiao AI (Mi Home, great value) / HomePod mini (Apple, also a Thread border router) / Alexa (broad overseas ecosystem). Note: The speaker picks your voice ecosystem —decide the ecosystem before buying.
- Smart bulbs: Yeelight (Mi Home/Matter, adjustable color temperature and brightness, cheap) / Philips Hue (great color and stability, but needs a Hue hub and costs more). Note: Check socket type (E27/B22) and whether you need color changing.
- Smart plugs: Xiaomi Smart Plug (cheap, energy stats) / TP-Link Kasa (good Matter support overseas). Note: For high-power appliances (water heater, space heater), always check the rated wattage and don't overload.
- Smart locks: Loock/Xiaomi (value and Mi Home integration) / Kaadas (traditional lock maker, solid mechanics). Note: Check Matter/your ecosystem support, battery life, and where the emergency keyhole is.

Suggested buying order

Smart speaker first (locks your ecosystem) then bulbs and plugs (low barrier, instant feedback) finally locks and cameras (security-related, choose carefully). Add one category at a time so you don't end up with gear you can't use.

Safety basics

More smart devices mean a bigger attack surface. These four lines are non-negotiable — check them one by one:

1. Change default passwords: Factory passwords for cameras, routers, and locks are public online. First thing after unboxing: set a long, unique password.
2. Update firmware promptly: Vendor updates often fix security holes. Turn on auto-update in the app, or check manually at least monthly.
3. Network isolation: Put IoT devices on a separate Wi-Fi (or guest network) away from your phone and computer. If one device is compromised, it's harder to pivot to your important data.
4. Privacy: Enable camera "privacy zones" (skip bedrooms and bathrooms), turn off 24/7 cloud recording, and don't leave home camera feeds open to third-party apps long-term.

Don't step in these traps

- Don't use remote unlock for strangers —prefer temporary passwords that expire.
- Don't point cameras at beds or out the window —that's your own privacy leaking.
- Don't buy no-name "white label" gear for a bargain —it often never gets firmware updates, basically a back door into your home.

Chapter recap

- Matter + Thread is the 2023–2026 direction —look for that label on new gear
- Each ecosystem has a strength: Apple for privacy/experience, Mi Home for value, Huawei for multi-device flow
- Your phone can be the hub —set up Coming home / Away / Sleep and the house runs itself
- Starter four: smart speaker, bulbs, plugs, locks —add them in that order
- Change default passwords, update firmware, isolate the network, privacy-mask cameras —four safety lines you can't skip

Part VII: AI Coding in Practice (Chapters 37-47)

Chapter 37: The Paradigm Shift — From Writing Code to Directing AI

Congratulations —by opening this book you're standing in the middle of a professional paradigm shift. It started with one programmer's social post in 2025. Dramatic? It really happened that way.

The birth of Vibe Coding: A movement sparked by one sentence

In February 2025, Andrej Karpathy —remember this name, you'll see it again —posted a tweet that got quoted over and over: "There's a new kind of coding called Vibe Coding, where you fully give in to the vibes, ignore the code details, and let AI handle everything. You say what you see, copy-paste, and basically it just works."

That casual-sounding remark precisely named a paradigm shift already underway: humans no longer write code line by line. Instead, we describe intent in natural language and AI implements it. Karpathy's wording was playful, but the insight is deep: once AI code generation crossed a threshold, the bottleneck stopped being "can I write this line?" and became "can I think this through clearly?"

Quick tip: What's a "vibe"?

"Vibe" means mood or feel. Vibe Coding is "directing AI coding by feel" —you describe what you want, AI implements it. Remember the term; it's this movement's birth certificate.

From that moment, "coding" split in two: classic coding turns designs into syntax; new coding turns intent into instructions. The first tests your fingers; the second tests your brain.

Vibe Coding quickly moved from joke to serious engineering concept. By mid-2026 it had grown from a social-media meme into a full methodology covering requirements, context management, iterative validation, and quality assurance. People also realized "coding by vibes" is only the beginner stage —truly efficient AI coding needs a systematic way of thinking. That way of thinking is what this book gives you.

2026 status: The Agentic Engineering era

By August 2026, AI coding had gone through several paradigm shifts. Vibe Coding was the first spark, but people soon found that pure vibes + chat produced wildly unstable code — brilliant sometimes, broken others. So the paradigm evolved: from casual chat coding toward structured agent engineering.

Karpathy again —in early 2026 he proposed Agentic Engineering. Unlike Vibe Coding's "say it and it generates," Agentic Engineering means: give AI a goal and let it plan steps, call tools, verify results, and iterate. AI is no longer just a code generator; it's an agent that perceives its environment, makes decisions, and takes actions.

This shift brought three key changes:

1. Interaction changed: From "one question, one answer" to "set the goal, then delegate."

2. Deliverables changed: From "a piece of code" to "a complete project."

3. Your role changed: From "review every line" to "define boundaries and accept results."

The numbers don't lie: In Cursor 3, released April 2026, Agent mode usage overtook Tab completion for the first time —a clear sign developers had shifted at scale. Claude Code, the terminal-native AI coding tool, hit a $2.5 billion annualized revenue run rate by mid-2026. The chart below shows that crossover moment:

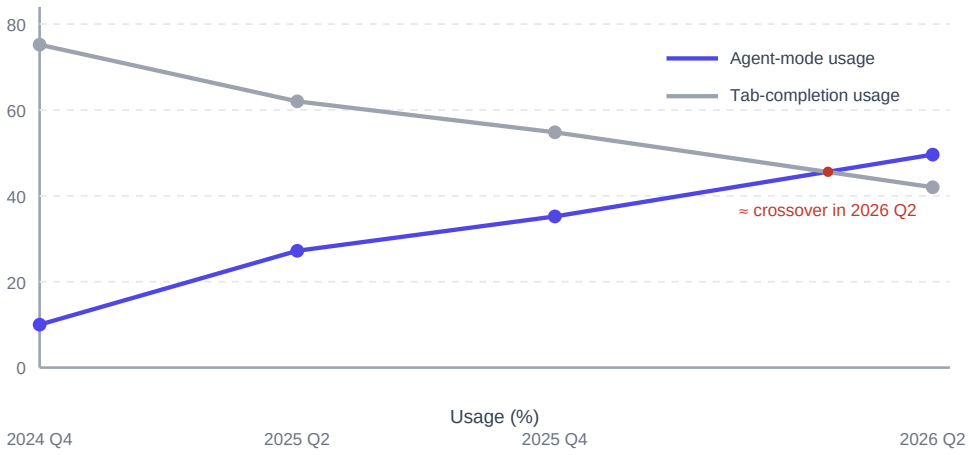


Figure 1: AI coding interaction-mode usage trends (estimated from industry research; Agent mode overtakes Tab completion in 2026 Q2)

Honest truth: tools and models change, but successful AI coding practitioners follow the same underlying methodology whether they use Cursor or Claude Code, GPT-5 or Claude 4.5. That methodology is what this guide teaches.

The core mindset shift: From Coder to Conductor

Traditional coding education teaches you to be a Coder —understand syntax, memorize APIs, debug logic. In the AI coding era, that role is being redefined. You no longer need to type every line yourself. Instead, become a Conductor: define the battlefield, set rules, assign tasks, accept results.

The shift isn't about "can you write code"—it's about "can you think clearly about what to build." A great conductor isn't necessarily the best soldier, but they know the objective, terrain, and troop placement best. Likewise, a great AI coding practitioner doesn't need every syntax detail memorized, but must be crystal clear on goals, architectural constraints, and quality standards.

Worth reading twice —this table captures the whole role change:

Dimension	Traditional Coder	AI Conductor
Core skills	Syntax memory, API calls, debugging	Requirements definition, context management, acceptance judgment
How you work	Write code line by line	Describe intent, review output
Quality control	You know it because you wrote it	Verify AI output with tests and review
Knowledge needed	Deep mastery of a language/framework	Broad tech literacy, deep business logic understanding
Where the bottleneck is	Typing speed and syntax memory	Clarity of thought and communication
Where errors come from	Syntax bugs, logic gaps	Vague requirements, missing context

Traditional Coder vs. AI Conductor —capability comparison

This does NOT mean coding knowledge is obsolete. Far from it: you can only review and correct AI code if you can read it; you can only judge whether AI's design is sound if you understand data structures and algorithms. Knowledge sets your "conductor ceiling" —but its role shifts from "execution tool" to "judgment basis."

Why methodology beats tools

Tools iterate every three months; methodology evolves only every three years. Learn a tool's mechanics and it may be obsolete in three months; master a way of thinking and it still applies three years later.

Many beginners ask first: "Which tool is best?" "Which model is strongest?" Those questions aren't unimportant —they're just the wrong priority. The 2026 tool landscape looks

nothing like 2025: Trae and Qoder rose and fell, Cursor went from plugin to standalone IDE to Cursor 3, Claude Code went from experiment to terminal-coding mainstream. Yet behind all that churn, the thinking principles of successful practitioners barely changed.

That's why this book won't waste pages on one tool's button-by-button walkthrough — those go stale the moment they're written. Instead, it focuses entirely on a proven thinking methodology: how to think about requirements, how to talk to AI, how to verify output, how to keep iterating. Master this and any tool is quick to pick up. Learn only tool steps and every new tool starts you from zero.

Reassurance for absolute beginners

If you're starting from zero, don't let "methodology"scare you. It's not abstract philosophy —it's an operator's manual for "how to think when something breaks." Like cooking: you can follow a recipe (tool steps), but a real chef understands "why season this way" and "how to control heat"(methodology). This book starts from the basics, pairs every principle with concrete examples, and makes sure you understand both "what" and "how."

Hands-on: Take a "role photo" of yourself today

On a piece of paper, answer three questions: (1) How do you currently use AI to write code (if at all)? (2) Are you more like the "Coder" or the "Conductor" in the table above? (3) How would you like to describe your workflow three months from now? Tuck it in the book and revisit after Chapter 47 —you'll be surprised how fast things change.

Chapter recap

- In 2025 Karpathy named the AI coding revolution "Vibe Coding": from line-by-line coding to describing intent in natural language
- By 2026 it evolved into Agentic Engineering: set goals, delegate, AI plans and executes, humans accept results
- Cursor 3's Agent mode overtook Tab completion; Claude Code hit $2.5B annualized revenue
- Your new role is Conductor: define the battlefield, set rules, assign tasks, accept results
- Tools iterate every three months; methodology every three years —learn methodology, not buttons

Chapter 38: Five Thinking Principles of AI Coding

If this book makes you remember only one chapter, make it this one. The five thinking principles are the book's heart. Everything after —context, requirements, prompts, quality — expands these five in different scenarios. Don't worry: none are hard to grasp, but each deserves a few projects to sink into muscle memory.

The story starts with a fun origin: when Andrej Karpathy open-sourced his project, he attached a 70-line CLAUDE.md summarizing four core principles for working with AI. That file earned 70,000+ GitHub Stars and became the most widely cited playbook for AI coding practitioners worldwide. These four principles aren't empty slogans —they're actionable thinking habits distilled from massive practice. This book adds a fifth —Iterate and Verify — completing the five-principle system.

Why is a 70-line file so valuable?

What do 70,000 Stars mean? Most GitHub projects never get close. That a 70-line text file earned them proves one thing: in the AI era the scarcest resource isn't code —it's the ability to "think clearly." Those 70 lines are a concentrated form of clarity.

Principle 1: Think Before Coding

This is the most counterintuitive of the five. If AI can write code for us, why think first? Simple: AI output quality tracks your input quality. If you haven't figured out what you want, AI's answers will trap you in endless revision loops —aka "AI getting lost with you."

"Think first" means answering three questions before prompting AI —an inseparable "thinking triangle" :

1. What problem am I solving? (Goal)
2. What should the result look like? (Acceptance criteria)
3. What constraints must I respect? (Boundaries)

Thinking dimension	Vague input (bad)	Clear input (good)
Problem definition	Make me a website	Make a personal tool site with entry cards for 3 small AI tools
Expected result	Make it look nice	Responsive layout, dark theme, hover animation on cards, works on mobile
Constraints	(Not stated)	No frameworks, pure HTML+CSS+JS, single-file deploy, no backend

Vague vs. clear input for the same request

Note: Think first ≠ write a perfect spec first

"Think first" doesn't mean "write a perfect PRD before starting." It's a quick but structured thinking pass —30 seconds on those three questions beats 30 minutes of back-and-forth in chat. Small tasks need short answers; big projects can expand them into full requirement docs (see Chapter 40).

Principle 2: Simplicity First

The biggest trap in AI coding isn't "can't build it" —it's "builds too much." Ask AI for a "user management system" and it may generate auth, permissions, roles, audit logs, data export —when all you need is a simple login page. Overbuilding is the main cause of bloated AI code.

"Simplicity First" means following the "minimum viable" rule every time you prompt: implement the core with the least code, solve the problem with the simplest design, validate the idea with the fewest features. Karpathy put it well: "If you need a discounted price, you don't need an e-commerce engine."

In practice, Simplicity First means a few concrete habits:

- Have AI implement one feature at a time, not the entire system in one prompt;
- Prefer standard libraries and simple designs over heavy dependencies;
- If 50 lines can solve it, don't let AI write 500;
- When AI's output goes beyond scope, boldly tell it to cut the extras.

Scenario	Overbuilt (avoid)	Simple (recommended)
Need to store data	Set up MySQL + ORM	Start with JSON files or SQLite
Need user login	OAuth2.0 + JWT + role permissions	Start with simple password auth
Need a frontend framework	React + Redux + Router	Try plain HTML + vanilla JS first
Need to deploy	Docker + CI/CD + K8s	Get it running locally first

Overbuilt vs. simple approaches for common scenarios

Principle 3: Surgical Changes

When iterating an existing project, every change should be surgical —touch only what must change. This matters especially in AI coding because AI loves to "tidy up while it's in there": you ask it to fix one bug and it refactors half a file.

Surgical Changes means every prompt explicitly states the scope and boundaries: "Only modify the calculate_discount function," "don't touch anything else," "keep the existing code style." If AI goes beyond scope, immediately tell it to revert the extras.

Karpathy illustrated this with a diff example. Say you ask AI to add "member pricing" to a cart discount function. Good output: only that function changes, a few lines added. Bad output: the whole file is refactored —the new code might look "cleaner," but you can't be sure it didn't introduce bugs. In AI coding, "looks better" is never a reason to ignore instructions.

Bad prompt (vague scope)

```
Help me optimize this project's discount logic
```

Good prompt (precise scope)

```
Only modify the calculate\_discount function in cart.py: 1. Add member pricing on
    top of existing discounts 2. Members get an extra 10\% off 3. Do not change the
     function signature or other functions 4. Keep the existing variable naming
    style
```

Principle 4: Goal-Driven Execution

This principle answers a key question: should you tell AI "how to do it" or "what to do"? Karpathy's answer is the latter. Instead of "create a class, then add methods, then write tests," say "I need a tool that parses a CSV and counts unique values per column." Let AI pick the implementation path; you accept the result.

Goal-driven execution has a double payoff. AI gets room to choose the best path instead

of being boxed in by your middle steps. You shift attention from "how" to "is the result right" —exactly what a conductor should do.

Note: Goal-driven ≠ hands-off

You still need strict acceptance after AI runs: Does the result meet the need? Are edge cases handled? Any obvious overengineering? If it's off, give specific feedback —not "redo it," but "this raises IndexError on empty list input; please add a null/empty check."

Instruction style	Example	Outcome
Imperative (how)	"First create a Python class called DataParser, add a parse method that takes filename, use the csv module to read⋯"	AI becomes a typist; your middle steps may not be optimal
Goal-based (what)	"I need a tool: read a CSV, count unique values per column, output JSON. Please implement it fully."	AI picks the best path; you accept the result

Imperative vs. goal-based instructions

Principle 5: Iterate and Verify

This is the fifth principle this book adds to Karpathy's four. Core belief: AI's first output is almost never final. Effective AI coding is a generate verify correct loop, not one-shot prompt output.

"Iterate and Verify" isn't passive "try again if wrong" —it's a structured validation flow. After each AI output, verify on three levels:

- Level 1 • Functional: Does it run? Are inputs and outputs right?
- Level 2 • Boundary: What about empty, oversized, or invalid inputs?
- Level 3 • Architectural: Is the structure sound? Hardcoded values? Easy to extend later?

Define the goal

AI generates code

Feature check

Pass

Edge-case check

Pass

Architecture review

Passed —go to the next step

Fail

Give specific feedback

Regenerate

Figure 2: The iterative verification loop in AI coding

The key step is "give specific feedback." When verification fails, don't say "wrong, redo." Say "the function raises IndexError on empty list input; please add an empty check on line 15." Specific feedback lets AI fix precisely; vague feedback just makes it re-guess your intent.

Harness Engineering: Agent = Model + Harness

In mid-2026, Harness Engineering started trending in the AI coding community. The core formula is simple:

An agent's capability = model capability × harness quality

Model capability is the brain's intelligence ceiling; the Harness is the "work environment" around it —context management, tool integration, workflow orchestration, memory systems, and more.

This explains a common puzzle: with the same GPT-4 or Claude, why do some people ship stunning projects while others get mediocre code? The difference isn't the model —it's the harness. A good harness includes clear project spec files, sensible tool integration, organized conversation context, and solid verification flow. The model sets the ceiling; the harness decides how much of it you can use.

The genius intern analogy

Think of the model as a genius intern —sky-high IQ, but without project docs, tool guidance, deadlines, and acceptance criteria, they won't ship great work. Harness Engineering is the craft of "setting up a genius intern's workspace."

The rest of this book is essentially about building a good harness: Chapter 39 teaches context management (project memory), Chapter 40 requirements (PRD), Chapter 43 conversation technique (prompt layering), Chapter 45 quality assurance (verification flow). Together, that's your harness.

Hands-on: Break down your next idea with the "thinking triangle"

Pick a small thing you've wanted to build but never started (a countdown page, a drink-water reminder script). Spend 3 minutes answering: (1) What problem does it solve? (2) What should it look like done? (3) What constraints (time, tech, environment)? Write the answers as one paragraph. Congratulations —you just did your first Think Before Coding.

That paragraph is a solid high-quality prompt.

Chapter recap

- Five principles from Karpathy's 70-line CLAUDE.md (70K+ Stars); this book adds "Iterate and Verify"
- Think Before Coding: Answer three questions first —what, what result, what constraints?
- Simplicity First: AI's biggest trap is building too much —a discount doesn't need an e-commerce engine
- Surgical Changes: Change with surgical precision; refuse AI's "while I'm in here" refactors
- Goal-Driven Execution: Say what, not how —let AI choose the path
- Iterate and Verify: Three-level checks (functional/boundary/architecture) + specific feedback, loop
- Harness Engineering: Ability = Model × Harness —manage AI like a genius intern's workspace

Chapter 39: Context Engineering — Make AI Understand You

Ever open a new chat and watch AI forget everything about your project? It's not messing with you —it's just forgetful by design. This chapter teaches you to install a "long-term memory system" for AI, and introduces one of 2026's hottest development paradigms: Spec-Driven Development.

Project memory files: AI's long-term memory

AI models have a fundamental limit: their memory is short-term. At the start of every conversation, AI knows nothing about your project. You'd have to keep saying "we use React," "backend is Node.js," "database is PostgreSQL," "code style uses double quotes." That's inefficient and easy to miss —like onboarding a new colleague every single day.

The fix is a project memory file —a plain text file in the project root that AI tools auto-read at the start of each conversation. Different tools use different names:

Tool	File name
Cursor	`.cursorrules`
Claude Code	`CLAUDE.md`
GitHub Copilot	`copilot-instructions.md`
Generic option	`AGENTS.md`

Project memory file naming conventions by tool

Names differ, but the idea is the same: write down what you want AI to always remember. A good project memory file includes: project overview (what this is), tech stack (what you're using), code conventions (naming, indentation, comments), directory layout (how code is organized), and development practices (branch strategy, commit style, testing). Karpathy's CLAUDE.md is only 70 lines but covers all key info —concise and complete.

```
# Project: personal finance tracker
## Overview
A full-stack personal finance tracker that helps users record daily income and
    expenses.
```

```
## Tech stack
- Frontend: React + TypeScript + Tailwind CSS
- Backend: Node.js + Express + TypeScript
- Database: SQLite (dev) / PostgreSQL (production)
- Testing: Vitest
## Code style
- Use double quotes
- 2-space indentation
- Function names in camelCase
- Type definitions in PascalCase
- Every function must have a JSDoc comment
## Directory structure
/src
  /frontend  - frontend code
  /backend   - backend code
  /shared    - types shared by frontend and backend
  /tests     - test files
## Development conventions
- Commit messages in English
- New features must have tests written first
- APIs always return { success, data, error }
```

Note: Don't turn it into a second requirements doc

Watch the length —don't write a multi-page requirements document. The purpose is to help AI build background context quickly, not to describe every feature in detail. Keep it concise, structured, and limited to what AI needs to know.

Spec-Driven Development (SDD)

Spec-Driven Development (SDD) is a 2026-era paradigm with one core idea: write the spec first, then the code. Unlike classic TDD (test-driven development), SDD "specs" aren't just test cases —they include requirement descriptions, interface definitions, data models, and more. AI generates code from those specs; humans write and maintain the specs.

SDD workflow in five steps:

1. Describe the requirement spec in natural language (in a .md file);
2. Define interfaces and data structures (in the spec file);
3. Have AI generate code from the spec;
4. Run tests to verify the output matches the spec;
5. If it doesn't match, fix the spec or have AI fix the code.

In this flow, the spec file is the single source of truth —code can be regenerated, but the spec must be right first.

Write the spec file

Is the spec complete?

Complete

AI generates code from the spec

Run tests to verify the output

Incomplete / tests fail

Code problem fix the code; spec problem fix the spec

Back to verification

All pass deliver

Figure 3: Spec-Driven Development (SDD) workflow

SDD's advantage: specs are more stable than code. When models or tools change, your spec file stays valid —just have the new AI tool regenerate the code. That's why this book insists methodology beats tools: the spec file is the concrete carrier of your methodology.

Context window management

Even with a project memory file, you'll still hit "not enough context." AI context windows (how much info they can handle at once) are limited. When a chat gets too long or pulls in too many files, AI "forgets" early info and quality drops —like chatting with a friend for six hours and them forgetting the first thing you said.

The core strategy for context windows is layered referencing:

Context layer	**Content**	**How to manage**
Layer 1: Project memory	Tech stack, conventions, directory structure	Written in CLAUDE.md etc., auto-loaded —no need to repeat
Layer 2: File references	Code files for the current task	Reference precisely with @; don't paste entire files into chat
Layer 3: Conversation context	Chat history for the current task	Start a new chat after finishing a task to avoid context bloat
Layer 4: Immediate instructions	Specific requirements for the current step	Clear, concise, targeted —avoid long-winded descriptions

Layered context window management

Practical tips for context engineering

Context engineering isn't a one-time setup —it's ongoing tuning. Here are tips proven over and over in practice:

1. Keep the memory file in sync with the project. When you add a new technology or change the architecture, update the memory file right away instead of explaining it ad hoc in chat.
2. Prefer @ references over pasting whole files. Most AI coding tools support @file or @folder syntax and inject content in the background —more efficient and cleaner than manual paste.
3. Keep chats short and focused. One conversation handles one task or one feature. Start a new chat for the next one. Long chats burn context and make AI "distracted."
4. Show AI the result, don't just describe it. When you hit a bug, @-reference the raw error log instead of saying "there's some error." AI works better with original information than your paraphrase.
5. Clean up and archive regularly. Finished chats and temp files clutter the workspace. Cleaning them keeps the project tidy and helps AI understand the structure faster.

Hands-on: Write a 70-line memory file for your project

Pick a project you're doing (or planning) and write a CLAUDE.md: project overview (2–3 sentences), tech stack (list it), code conventions (3–5 rules), directory structure (draw a tree), dev practices (2–3 items). Keep it under 70 lines —you'll find your own understanding of the project sharpens too.

Chapter recap

- AI memory is short-term: every new chat starts at zero project knowledge
- Project memory files (CLAUDE.md etc.) are AI's long-term memory, auto-loaded each chat
- Good memory file = overview + stack + conventions + layout + practices —70 lines is plenty
- SDD: spec first, then code. The spec is the single source of truth and more stable than code
- Four context layers: project memory @file references chat history immediate instructions
- Five practical tips: keep in sync, use @, keep chats short, show raw info, clean up regularly

Chapter 40: Requirements Engineering —Turn Ideas into AI-Ready Instructions

If context engineering helps AI understand the background, requirements engineering helps AI know what to do this time. It's where people differ most in AI coding —same tools, same models, yet some ship stunning projects while others only make mediocre toys. The gap is rarely raw technical skill; it's the ability to express requirements.

From fuzzy idea to structured requirements

Every project starts as a fuzzy idea — "I want a budgeting app," "a personal website," "a to-do list." Nothing wrong with that, but toss it straight to AI and quality is pure luck. Requirements engineering turns that fuzzy idea into something AI can execute precisely.

Good news: structured requirements don't need a multi-page PRD. Just answer five questions —the "five requirement elements":

Element	**Question**	**Example: budgeting app**
Who	Who is it for?	Personal use, tracking daily income and expenses
What	What does it do?	An app that records transactions, summarizes monthly, and visualizes spending
Why	Why build it?	To understand my spending structure and cut unnecessary expenses
How	Roughly how?	React frontend, data in browser localStorage, no backend or login yet
Constraint	Any constraints?	Solo developer, 2 weeks, no need to deploy publicly

The five requirement elements with example

You don't need a formal document for these five —even a quick mental pass is enough. The key is building the habit of "think clearly before you speak." Once you can answer these five clearly, your requirement descriptions naturally become structured.

The art of layered requirement breakdown

For bigger projects, one requirement description isn't enough. Break the big one into small, independently verifiable sub-requirements. This is "requirement breakdown." Core rule: each sub-requirement should be completable, verifiable, and deliverable on its own.

For "build a personal budgeting app," a sensible breakdown:

1. Implement basic transaction recording (add, view, delete entries);
2. Add categories (tag entries as dining, transport, etc.);
3. Implement monthly summaries (income and expenses by month);
4. Add data visualization (charts for spending structure).

Each step is a feature increment that runs and verifies on its own.

The golden rule of breaking down requirements: after every step, the project should still be a "thing that works" —even if crudely. Never let the project sit in a state where "everything is half done but nothing works."

Two benefits. First, you always see progress —every step adds a feature, and that positive feedback matters for long projects (humans need wins). Second, each AI task stays small, lowering the risk of misunderstanding —smaller tasks, clearer understanding, better output.

PRD writing template and practice

For larger projects (full-stack apps with frontend, backend, and multiple modules), the five elements may not be enough. You'll need a more formal PRD. One common misconception: a PRD is not for AI to read —it's for you, to clarify your thinking so you can give AI more precise instructions.

A good PRD doesn't need to be long, but should cover six parts: project overview (one paragraph on what it is), feature list (each feature defined), technical plan (stack and rough architecture), data model (core structures), interface description (what main screens look like), and development plan (steps to finish).

```
# Personal finance tracker PRD
## Project overview
A lightweight personal finance tracker supporting income/expense records,
    categories, monthly summaries and data visualization.
```

```
## Feature list
1. Record income/expenses: amount, type (income/expense), category, note, date
2. Category management: presets for food, transport, shopping, entertainment, other
3. Monthly summary: total income, total expenses and balance by month
4. Spending visualization: a pie chart of each category's share
5. Data export: export as CSV
## Technical approach
- Frontend: React + TypeScript + Tailwind CSS
- Data storage: localStorage (no backend needed)
- Charts: Chart.js
- Build: Vite
## Data model
Record {
  id: string
  amount: number
  type: 'income' | 'expense'
  category: string
  note: string
  date: string  // YYYY-MM-DD
}
## Development plan
Step 1: basic record features (CRUD)
Step 2: category management
Step 3: monthly summaries
Step 4: data visualization
Step 5: data export
```

Note the writing style: concise lists and structured formatting instead of long essays. Each feature has a clear definition; data models use simple type descriptions. This style is easy for you to read and easy for AI to understand precisely when you hand it over.

Hands-on: full walkthrough from idea to plan

Theory is boring —let's walk through a complete requirements engineering example. Suppose your idea is: "I want a study tracker that records what I learned each day."

Step 1: Run through the five requirement elements.

- Who: Personal use.
- What: A tool that records daily study content, tracks study time, and reviews progress.
- Why: Build a consistent study habit and see the accumulation.
- How: Start with a web version, data in browser localStorage.
- Constraint: Beginner developer, core features done in 1 week.

Step 2: Break down the requirements.

1. Step 1: Implement ”record study entries” (date, content, duration);
2. Step 2: Add ”study time stats” (today, this week total);
3. Step 3: Implement a ”study calendar” view (like a GitHub contribution graph);
4. Step 4: Add ”study review” (randomly show a past day’s entries for revision).

Step 3: Write a short PRD. It doesn’t need to be long —the breakdown above is already the PRD body. Organize it into a .md file in the project root as the shared reference for you and AI.

Step 4: Start developing the first step. A prompt to AI could be:

```
I need the first feature of a learning tracker — logging learning entries.
The user can add a learning record with the date, what was studied and the duration
    in minutes.
Data is stored in localStorage.
The frontend uses React + TypeScript.
Please give a complete implementation.
```

Notice the prompt’s traits: it describes only the current step’s goal (recording), without other features mixed in; it states technical constraints (React, TypeScript, localStorage); it gives enough info for AI to know “what” and “how roughly,” while leaving implementation details to AI. That’s a high-quality AI coding prompt after requirements engineering.

Hands-on: Write a five-element card for your pet project

Pick a small project you’ve been brewing and spend 5 minutes filling a card: Who / What / Why / How / Constraint, one sentence each. Then break it into 3–5 steps where each step is “usable on its own.” You’ll find that once written down, the idea turns from a daydream into a to-do list.

Chapter recap

- Five requirement elements: Who / What / Why / How / Constraint —think before you speak
- Golden rule of breakdown: after every step the project is ”something that works”
- A PRD is for you: overview, features, technical plan, data model, UI, plan
- Good prompt = only the current step + clear constraints + what, not how
- Full walkthrough template: five elements breakdown PRD prompt —four steps

Chapter 41: The 2026 AI Coding Tools Landscape

A heads-up first: this chapter is a snapshot of the August 2026 tool landscape. It'll be outdated in six months—and that's exactly the first lesson. The goal isn't "which tool to use," but a judgment framework for how to choose tools. Understand each tool's design philosophy and you can pick what fits anytime.

Three schools and a choice framework

In 2026, AI coding tools roughly fall into three schools, each with a different design philosophy. First, the panorama:

School	Representative tools	Design philosophy	Best scenarios
IDE-native AI	Cursor 3, Trae, Windsurf, CodeBuddy	Deeply integrate AI into the editor	Daily development, code review, incremental enhancement
Terminal agents	Claude Code, Codex CLI, Aider	Terminal-first; AI operates files and commands directly	Project-wide operations, batch edits, CI/CD integration
Cloud builders	v0, Bolt, Replit Agent	Create complete projects from scratch in the browser	Rapid prototypes, zero setup, instant preview

Three schools of AI coding tools

Choosing isn't about "which is best" but "which fits your current situation." Three practical dimensions:

- How's your coding foundation? Beginners cloud builders. Some experience IDE-native AI. Advanced terminal agents.
- What stage is your project at? From scratch cloud builders. Iterating IDE-native AI. Maintenance/refactor terminal agents.

- What's your deployment need? Quick validation cloud builders. Serious projects IDE-native AI. Automated pipelines terminal agents.

IDE-native AI school

IDE-native AI is the mainstream school: AI capabilities embedded in your code editor, keeping the classic coding experience while adding AI boosts. In 2026 the leaders are Cursor 3, Trae, Windsurf, and CodeBuddy.

Cursor 3

Cursor is the world's most widely used AI coding IDE. Cursor 3, released April 2026, marked a turning point: Agent mode usage overtook Tab completion —developers shifted from "AI completes my code" to "AI finishes my task." Key Cursor 3 features include multi-file editing (AI changes several files to complete a task), context awareness (understands project structure and dependencies), and deep terminal integration (AI runs commands and sees output right in the editor).

Trae

Trae is ByteDance's AI coding IDE with a strong user base in China. Its SOLO mode (also called Builder mode) is the classic "create a project from one sentence" —describe the need and AI handles init to code generation. Trae shines in Chinese support and domestic network adaptation; great for Chinese users to start fast.

Other tools

Windsurf (formerly Codeium) is known for smooth completions —great for developers who love "typing at the speed of thought." Tencent Cloud CodeBuddy, as a VS Code plugin, integrates well with Tencent Cloud and WeChat Mini Programs. Alibaba Tongyi Lingma and Baidu Comate offer similar IDE integrations, each tuned for Chinese scenarios.

Terminal agent school

Terminal agents are the fastest-growing school of 2026. Core idea: AI shouldn't be just an editor plugin —it should operate the whole development environment like a developer: read/write files, run commands, check output, install dependencies. Claude Code leads this school.

Claude Code

Claude Code is Anthropic's terminal AI coding tool, reaching $2.5 billion annualized revenue by mid-2026. It works completely differently from a classic IDE: describe the task in natural language in the terminal, and Claude Code reads files, edits code, runs tests, checks results, and iterates —without opening an editor.

Claude Code's edge is full-project awareness: it isn't limited to editor windows and can browse the whole project tree and understand file dependencies. It also supports CLAUDE.md project memory files so AI knows the background at session start. For large refactors, batch edits, and test writing, terminal agents beat IDE plugins.

Codex CLI and Aider

OpenAI's Codex CLI offers a similar terminal experience and supports /import to bring in existing codebases for AI to understand. Aider is an open-source terminal AI coding tool supporting many models (GPT, Claude, DeepSeek, and more) —great if you like switching models. Different flavors, same idea: give AI full development-environment control in the terminal.

Cloud builder school

Cloud builders are the most beginner-friendly school. No installs —open a browser, describe the need, and AI creates a full project in the cloud with instant preview.

- v0 (by Vercel): Great at frontend UI components and pages —perfect for quick web prototypes.
- Bolt: Creates full-stack apps from one sentence, including frontend, backend, and database.
- Replit Agent: Works in Replit's cloud IDE —creates projects and can run and deploy in the cloud.

Shared advantage: zero setup —open the browser and go. Limitation: when projects grow complex, cloud editing is less flexible than a local IDE. Best for the 0-to-1 prototype stage: validate ideas quickly in a cloud builder, then move to a local IDE for deeper development.

Tools change; methodology doesn't

If every AI coding tool disappeared tomorrow, three habits should remain: think it through before you act, take one small step at a time, and verify when done. Tools are amplifiers; methodology is the foundation. Amplifiers can be swapped out —the foundation must not shake.

Look at 2024–2026: GitHub Copilot ruled 2024, Cursor took over in 2025, Claude Code surged in 2026. Every new tool brings "this time is different" claims, yet the people who use new tools well are the ones with solid methodology. They adapt in days; people who only memorized buttons start from scratch each time.

Suggested path: start with any one tool, practice the five thinking principles, and once your methodology is solid, try tools from different schools to feel their design philosophies. Eventually you'll form your own combo —say Cursor for daily work, Claude Code for refactors and tests, v0 for quick prototypes. Combos vary by person; methodology is shared.

Hands-on: Assemble your own tool kit

Locate yourself on the three dimensions (coding foundation, project stage, deployment need). Beginner try Bolt or v0 for a page. Some experience use Cursor or Trae for an iteration task. Feeling ambitious run Claude Code in the terminal. Then answer: which school fits your current stage best?

Chapter recap

- Three schools: IDE-native AI (daily work), terminal agents (project-scale ops), cloud builders (beginner prototypes)
- Three questions when choosing: coding foundation? project stage? deployment need?
- Cursor 3 Agent mode overtook Tab completion; Claude Code hit $2.5B annualized revenue
- Cloud builders fit 0-to-1; after validation, move to a local IDE for depth
- Tools iterate every three months: people with methodology adapt in days

Chapter 42: LLM Landscape and API Practice

Previous chapters covered how to work with AI. This one answers a more basic question: which AI, and how? By August 2026 the LLM landscape is rich —globally GPT, Claude, and Gemini compete; in China DeepSeek, Tongyi Qianwen, GLM, Kimi, and Doubao bloom. Each model has its own personality and strengths: pick right and you fly; pick wrong and you drag. This chapter introduces the models, shows how to wire them into your apps via API, and how to choose a Token Plan that fits your usage.

Global major LLMs: the big three

As of August 2026, the international LLM market is a three-way race: OpenAI (GPT), Anthropic (Claude), and Google (Gemini). Each has a distinct personality —let's meet them one by one.

OpenAI GPT-5.6 series: the all-rounder

OpenAI kicked off the LLM wave, and GPT remains the world's most widely used model family. The GPT-5.6 series, released July 2026, comes in three tiers: flagship GPT-5.6 Sol for high-end reasoning, balanced GPT-5.6 Terra for daily development, and lightweight GPT-5.6 Luna for high-throughput, low-cost scenarios. All three share a 1.05 million-token context window and support image input, function calling, and structured output.

Model	**Positioning**	**Input (/1M tokens)**	**Output (/1M tokens)**	**Context window**
GPT-5.6 Sol	Flagship reasoning	US$5.00 (about ¥36)	US$30.00 (about ¥217)	1.05M tokens
GPT-5.6 Terra	Balanced daily	US$2.50 (about ¥18)	US$15.00 (about ¥109)	1.05M tokens
GPT-5.6 Luna	Lightweight, low cost	US$1.00 (about ¥7)	US$6.00 (about ¥44)	1.05M tokens

OpenAI GPT-5.6 series pricing (August 2026)

GPT's core strength is balance —reasoning, coding, writing, multimodal, no glaring weaknesses. Plus OpenAI's huge plugin ecosystem and developer community, with the most complete docs and tooling. When you don't know what to pick, GPT is the safe default. Weak spots: needs special network access in China, occasional translation-flavored Chinese, and mid-to-high API prices.

Good news

In August 2026 OpenAI upgraded free users' default model to GPT-5.6 Luna and opened unlimited text chat. That means you can try GPT's basics at zero cost and only pay when you need stronger reasoning or API calls.

Anthropic Claude series: coding and rigor at the top

If GPT is the all-rounder, Claude is the coding ceiling. Anthropic's Claude series has long led SWE-bench (software engineering benchmark) by a wide margin, with top code quality and real-world engineering delivery. Even better, Claude's hallucination rate (chance of making things up) is the lowest online at about 1% —ideal for academic, legal, and other high-precision work.

Claude's other killer feature is long context: up to 1 million tokens, able to "read" hundreds of pages of contracts or a whole codebase without missing details. Nearly irreplaceable for refactors and long-document analysis. Claude Code, the terminal AI coding tool, hit $2.5 billion annualized revenue by mid-2026 —strong evidence of Claude's coding reputation.

Model	Positioning	Input (/1M tokens)	Output (/1M tokens)	Context window
Claude Fable 5	Flagship	US$10.00 (about ¥72)	US$50.00 (about ¥362)	1M tokens
Claude Opus 4.8	Professional	US$5.00 (about ¥36)	US$25.00 (about ¥181)	1M tokens
Claude Sonnet 5	Value	US$2.00 (about ¥14)	US$10.00 (about ¥72)	1M tokens

Anthropic Claude series pricing (August 2026; Sonnet 5 at promotional rate)

Claude's downside is price —Fable 5 output is the most expensive of all models at $50 per million tokens. Multimodal (image/video understanding) is also relatively weak. For individual developers on a budget, Sonnet 5's promotional price is excellent value.

Google Gemini series: the multimodal king

Google's Gemini series reached 3.1 Pro in 2026. Its biggest edge is native multimodal — precise image-text parsing, accurate OCR for tables, efficient video frame extraction, and deep audio/video analysis. Deeply tied to Google's ecosystem (Search, Gmail, cloud), it shines at

real-time information retrieval and cross-modal understanding. Gemini also supports up to 2 million tokens of context —the largest window today.

Gemini positions itself as the first choice for scientific reasoning and multimodal work. If your project involves lots of image analysis, video understanding, or scientific computing, Gemini has few rivals. But pure-text code generation is slightly behind Claude, and its Chinese is less natural than domestic models.

Comparison item	GPT-5.6	Claude Fable 5	Gemini 3.1 Pro
Overall score	94.8	95.0	92.1
Coding (SWE-bench)	74.9%	64.3% (Pro)	Medium
Context window	1.05M tokens	1M tokens	2M tokens
Multimodal	Balanced	Weaker	Strongest
Hallucination rate	Relatively low	Lowest (~1%)	Relatively low
Availability in China	Needs special network	Needs special network	Needs special network
API ecosystem	Most complete	Growing	Tied to Google ecosystem

Side-by-Side Comparison of the Big Three International Models (August 2026)

Note: Real-World Hurdles for Users in China

For developers in China, the core obstacle to GPT, Claude, and Gemini is network access. You can reach them indirectly through Azure proxies, API aggregation platforms, and the like, but you'll need to weigh stability and compliance yourself. If your project has strict data security requirements, we recommend looking at Chinese models first.

China's Major Large Models: A Hundred Flowers in Bloom

By 2026, China's large models have gone from 'chasers' to running neck and neck with the leaders, and in some areas they have even pulled ahead. Unlike the 'wrapper' controversies of 2023, the flagship Chinese models of 2026 are on par with top international models in logical reasoning, code generation, and Chinese-language understanding, with the gap narrowed to within 5%. More importantly, Chinese models have irreplaceable advantages in Chinese-language scenarios, cost, and compliant deployment.

DeepSeek V4: The Best Bang for Your Buck

DeepSeek is a phenomenon among Chinese large models. In early 2025, DeepSeek-V3 shook the market with its positioning of 'one-tenth the price of GPT with comparable capability,' and the industry dubbed it the 'kill line' for general-purpose AI models —nothing cheaper works as well, and nothing more expensive gives you more. By August 2026, DeepSeek V4-Pro

was officially released: a 1.6-trillion-parameter MoE architecture with about 49B parameters active, support for 1 million tokens of lossless context, and coding value widely seen as the best in the world.

DeepSeek V4 made a huge leap in agent capability —its DeepSWE benchmark score jumped from 12.8 to 62.7 (+390%), and NL2Repo rose from 38.5 to 61.5. The API now supports both the Responses API and the Anthropic API, so it works with mainstream coding tools like Codex and Claude Code. That means you can swap DeepSeek's API in for Claude's and use it inside Claude Code —cutting your costs several times over with a nearly identical experience.

Model	Positioning	Input (per 1M tokens)	Output (per 1M tokens)	Context Window
DeepSeek V4 Pro	Flagship reasoning	3 yuan (approx. US$0.43)	6 yuan (approx. US$0.87)	1M tokens
DeepSeek V4 Flash	Lightweight and fast	1 yuan (approx. US$0.14)	2 yuan (approx. US$0.28)	1M tokens

DeepSeek V4 series pricing (August 2026, promotional prices)

Price comparison: for the same 1 million input tokens plus 1 million output tokens, DeepSeek V4 Flash costs about $0.42, GPT-5.6 Sol costs about $35, and Claude Fable 5 costs about $60. That is a gap of 80 to 140 times.

Watch Out: DeepSeek Is About to Raise Its Prices

On August 6, 2026, DeepSeek officially announced that it 'plans to significantly raise API prices in the near future,' so the promotional prices may not last long. Some media reports speculate the increase could be as much as 30 times, and as of August 14 the company had not yet published its final pricing plan. On top of that, the official DeepSeek V4 release introduced peak/off-peak pricing —calls during peak hours cost exactly double. Keep an eye on the official announcements for the latest prices.

Tongyi Qianwen Qwen3.7 Max: Alibaba Cloud's Enterprise Choice

Alibaba Cloud's Qwen (Tongyi Qianwen) series leans on the Alibaba Cloud ecosystem, giving it a natural edge in enterprise deployment. Qwen3.7 Max, released in July 2026, is the current flagship: a dense 235B-parameter model with 256K of native context, expandable to 1M. Qwen's core strengths are a mature open-source ecosystem (the entire lineup from 7B to 235B is open source), precise understanding of Chinese-language scenarios, and deep integration with Alibaba Cloud infrastructure.

Model	Output Price (per 1M tokens)	Use Case
qwen-turbo	0.6 yuan	Simple Q&A, text classification, high concurrency
qwen-long	2.0 yuan	10M ultra-long context processing
qwen3.7-flash	7.2 yuan	General conversation, content generation
qwen3.7-plus	12.0 yuan	Multimodal (images + video), Chinese content creation
qwen3.7-max	36.0 yuan	Flagship Chinese capability, complex tasks

Qwen (Tongyi Qianwen) series tiered pricing (August 2026)

Qwen's pricing is tiered —from the ultra-low-cost qwen-turbo (0.6 yuan per 1M output tokens) up to the flagship qwen3.7-max (36 yuan per 1M output tokens), covering everything from lightweight Q&A to complex reasoning. For companies already on Alibaba Cloud, Qwen is the most convenient integration choice.

Zhipu GLM-5.1: Top Chinese Model on SWE-bench

Zhipu AI's GLM series shone in 2026. GLM-5.1 took first place among Chinese models on SWE-bench: a 744B hybrid MoE-dense architecture with support for 1 million tokens of context. Its hallmarks are precise Chinese semantic understanding and support for compliant private deployment in China —which matters a great deal for finance and government scenarios with strict data security requirements. GLM-5.1 is priced at 6 yuan for input and 24 yuan for output per 1M tokens, in the middle of the pack among Chinese models.

Kimi K3: The King of Ultra-Long Text

Moonshot AI's Kimi series is known for its ultra-long text handling. Kimi K3, released in July 2026, has 2.8 trillion parameters and lands in the global top tier across many complex-task benchmarks. Kimi's core use cases are large codebase analysis and long-document retrieval augmentation —it can chew through the text of an entire book or the code of an entire project, and it excels at 'needle in a haystack' retrieval tasks. Kimi is relatively pricey (about 12-20 yuan per 1M output tokens), but in long-text scenarios nothing else matches its value for money.

Doubao: ByteDance's All-Rounder

ByteDance's Doubao series has been steady in 2026. Doubao Seed 2.1 Pro is the flagship model on Volcano Engine, with 256K context and consistently solid results in real-world testing, standing out in Chinese-language scenarios. Doubao's advantages: a big bundle of free tokens the moment you register, well-polished integration with the Volcano Engine ecosystem, and pricing among the lowest tiers in China. For beginners who want a quick taste of AI coding, Doubao is a zero-barrier way in.

Comparison Item	**DeepSeek V4**	**Qwen3.7 Max**	**GLM-5.1**	**Kimi K3**	**Doubao 2.1 Pro**
Overall Score	87.5	89.7	90.5	88+	85+
Coding Ability	Extremely strong	Strong	No. 1 in China on SWE	Strong	Above average
Chinese Understanding	Excellent	Excellent	Precise	Excellent	Excellent
Context Window	1M	256K-1M	1M	Ultra-long	256K
Output Price (yuan per 1M)	6	36	24	12-20	30
Open Source	MIT license	Entire lineup open source	Partially open source	Closed source	Closed source
Private Deployment	Yes	Yes	Yes	No	No

Side-by-Side Comparison of China's Top Five Models (August 2026)

One important trend: the sense of a 'generation gap' between Chinese models and GPT/Claude has disappeared. In logical reasoning and math competitions, Chinese flagships trail GPT-5 by single digits. Claude still leads in systematic architectural thinking and complex engineering reasoning —it produces architecture plans with more 'flair,' while Chinese models can sometimes come across as 'by the book.' But in Chinese-language scenarios, cost control, and compliant deployment, Chinese models hold an overwhelming advantage.

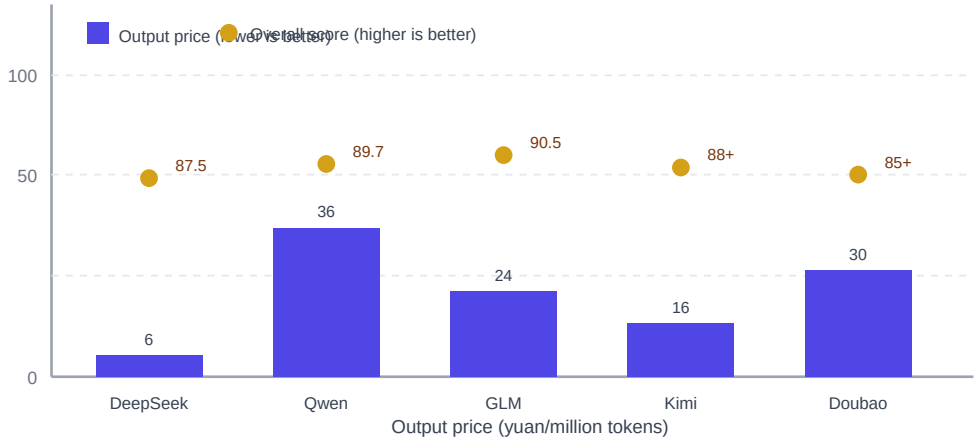


Figure 4: Price vs. Performance of the Five Models (lower price is better, higher score is better)

A Decision Framework for Choosing a Model

Now that you know each model's strengths, how do you pick one for your project? Here is a 'four-question decision method' —answer four questions, and you'll lock onto the model that fits you best.

Is your project deployed in China or internationally?

China choose a Chinese model

|

International overseas models are an option

Is the core task coding or something else?

Coding Claude or DeepSeek

|

Other keep narrowing down

Need extra-long context (>500K tokens)?

Yes Gemini or Kimi

|

No keep narrowing down

Is budget a concern?

Yes DeepSeek or Qwen-turbo

|

No GPT-5.6 or Claude

Figure 5: The Four-Question Decision Method for Choosing a Large Model

Tip: The Combination Strategy

This decision method isn't dogma —many situations call for combining several models. One common strategy: use DeepSeek or Qwen for everyday development (cheap, good with Chinese), Claude for critical architecture design (strong reasoning, low hallucination), and Gemini for multimodal analysis (image and video processing). Switch models as needed instead of riding 'one model to the grave.'

Getting Started with API Calls: Connecting to a Large Model from Scratch

Many AI coding tutorials only teach you how to use the chat interface, but real 'AI programming' —having your application call a large model —goes through an API. An API (Application Programming Interface) is a communication protocol between programs. Through an API, your code can call on a large model's abilities as easily as calling a function.

What Is a Token

Before APIs, you must first understand the concept of a 'token.' A token is the basic billing unit for large models —think of it as the AI's unit of reading and writing. One token is roughly 0.75 English words (4 characters), or 0.5-1 Chinese characters. A 1,000-character Chinese passage consumes about 1,500-2,000 tokens.

A token is not a word count, nor a character count. Large models use a dedicated tokenizer to split text into tokens. The same word can cost a different number of tokens in different models —'artificial intelligence,' for instance, might be 3 tokens in GPT but 2 in DeepSeek. Most models' APIs return the actual number of tokens consumed, and you can check it in the API response.

Quick Token Cost Estimates

One million Chinese characters is roughly 1.5 million tokens. Processing 1 million characters of input plus 1 million characters of output with DeepSeek V4 Flash costs about $0.42 (around 3 yuan); the same volume with Claude Fable 5 costs about $60 (around 434 yuan). That is why your choice of model directly shapes your project's cost.

Getting an API Key

The first step in using a large model API is getting an API Key —a string used to verify your identity. Every model provider has its own API platform, where you register an account and create an API Key. Here are the API platform entry points for the mainstream models and the steps to get a key:

Model	API Platform	How to Get a Key	Free Allowance
DeepSeek	platform.deepseek.com	Sign up identity verification create API Key	Free allowance for new users
Qwen (Tongyi Qianwen)	bailian.aliyun.com (Bailian platform)	Log in with an Alibaba Cloud account activate the service create a Key	Free trial for new users
Zhipu GLM	open.bigmodel.cn	Sign up identity verification create API Key	Free allowance for new users
Kimi	platform.moonshot.cn	Sign up identity verification create API Key	Limited free allowance
Doubao	volcengine.com (Volcano Engine)	Sign up activate Volcano Ark create API Key	Large free token grant just for signing up
OpenAI GPT	platform.openai.com	Sign up link a credit card create API Key	No free allowance (paid)
Anthropic Claude	console.anthropic.com	Sign up link a card create API Key	Small allowance for new users

API platforms for mainstream models and how to get a key (August 2026)

Security Note: Your API Key Is Your Wallet Password

An API Key is the equivalent of your wallet password —anyone who gets hold of it can spend your quota calling the model. So: never write an API Key into your code (manage it with environment variables); never commit code containing a Key to a public Git repository; never share screenshots containing a Key on social media. If a Key leaks, delete it on the platform and create a new one immediately.

Your First API Call

Once you have an API Key, you can call a large model API from any programming language. Here is the simplest possible call, shown in Python —send one message to DeepSeek and get the reply back.

```
# Install the SDK: pip install openai
# DeepSeek is compatible with the OpenAI SDK format — use it directly
from openai import OpenAI
import os

# 1. Create the client (the API key is read from an environment variable)
client = OpenAI(
```

```
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 2. Send the request
response = client.chat.completions.create(
    model="deepseek-chat",  # pick the model
    messages=[
        {"role": "system", "content": "You are a Python coding assistant."},
        {"role": "user", "content": "Write a quicksort function."}
    ]
)

# 3. Get the reply
print(response.choices[0].message.content)

# 4. Check token usage
print(f"Input tokens: {response.usage.prompt_tokens}")
print(f"Output tokens: {response.usage.completion_tokens}")
print(f"Total tokens: {response.usage.total_tokens}")
```

The core logic of this code: create a client (carrying the API Key and the model address) build the message list (system sets the role, user is the human's input) send the request get the reply. This pattern works for nearly every OpenAI-compatible model —just swap the api_key and base_url to change models. That is why so many Chinese models choose to be OpenAI SDK compatible: developers can switch models by changing two lines of code, so the cost of migrating is almost nil.

Here is what changes when you switch to another model —note that only two lines need to change:

```
# DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)
# model="deepseek-chat"

# Qwen (Tongyi Qianwen, Bailian platform)
client = OpenAI(
    api_key=os.environ.get("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
# model="qwen-plus"

# Zhipu GLM
client = OpenAI(
```

```
    api_key=os.environ.get("GLM_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4"
)
# model="glm-4"

# OpenAI GPT
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY"),
    # the default base_url is fine
)
# model="gpt-5.6-terra"
```

Streaming Responses

In a real application, you usually don't want to wait for the AI to finish everything before displaying it in one lump —that makes users wait far too long. Streaming responses let the text appear word by word as the AI generates it, which vastly improves the user experience. Like a typewriter with letters popping out one at a time —it feels completely different to the user.

```
# Streaming: output one character at a time
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "Write a tutorial on bubble sort in Python"}
    ],
    stream=True  # enable streaming
)

# Receive chunk by chunk and print
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
# The output appears character by character, like a typewriter
```

A Buyer's Guide to Token Plans

Most model providers offer two billing methods: pay-as-you-go and subscription plans (Token Plans). Understanding the difference between the two can cut your costs dramatically.

Pay-As-You-Go vs. Subscription Plans

Aspect	Pay-as-you-go	Subscription plan
Billing Method	Pay only for the tokens you use	Pay a fixed amount up front and receive an allowance
Best For	Unsteady usage, occasional calls	Steady usage, frequent calls
Price Advantage	Standard price	20%-50% cheaper than pay-as-you-go
Flexibility	Available anytime, no commitment	Requires prepayment; unused allowance may expire
Going Over Quota	Automatically billed as you go	Switches to pay-as-you-go once the allowance runs out
Typical Platforms	Each model's official API platform	OpenAI Plus/Pro, Claude Pro, Cursor Pro

Comparing the Two Billing Methods

For beginners, start with pay-as-you-go —no upfront cost, and you pay for exactly what you use. Once your monthly usage settles at a steady level (say, above 50 yuan a month), consider switching to a subscription plan for a lower unit price.

Mainstream Subscription Plans at a Glance

Plan	Monthly Fee	What's Included	Best For
ChatGPT Free	Free	Unlimited text chat with GPT-5.6 Luna	Trying it out as a total beginner
ChatGPT Plus	US$20/month	GPT-5.6 Sol, image generation, data analysis, plugin access	Moderate users
ChatGPT Pro	US$200/month	Unlimited use of all models, priority access to new features	Heavy professional users
Claude Free	Free	A limited number of Claude chats	Trying it out as a total beginner
Claude Pro	US$20/month	Unlimited Claude usage, priority access to new models	Coders
Cursor Free	Free	A limited number of AI completions and chats	Getting started as a beginner
Cursor Pro	US$20/month	Unlimited AI completions, 500 premium requests per month	Everyday developers
Cursor Business	US$40/month per person	Team collaboration, admin console, privacy mode	Teams

Mainstream AI tool subscription plans (August 2026)

Chinese models handle subscriptions a little differently. DeepSeek is mainly pay-as-you-go, with no monthly subscription plan —you top up and get charged based on actual usage. Qwen offers prepaid resource packages on the Bailian platform (buy a fixed amount of tokens at a discounted price). Doubao has similar prepaid options on Volcano Engine. For developers in China, the most economical combination is usually: DeepSeek pay-as-you-go (daily development) plus Doubao's free allowance (trying things out).

API Aggregation Platforms

If you need to use several models at once, an API aggregation platform is a hassle-free option. An aggregator gives you one unified API Key that can call dozens of models worldwide, with no need to register separately on each platform.

Platform	Features	Price Advantage	Best For
OpenRouter (openrouter.ai)	Overseas aggregator: one Key for models worldwide	Transparent pricing with real-time comparison	Flexibly switching among many overseas models
SiliconFlow (siliconflow.cn)	China-based aggregator offering APIs for many Chinese models	20%-30% below official prices	Developers in China comparing model prices
Azure OpenAI (Microsoft's proxy)	The OpenAI API proxied through Microsoft Azure	50% off with the Batch API	Enterprise-grade OpenAI calls; large asynchronous batches
Baidu Qianfan (qianfan.baidubce.com)	Baidu's AI cloud platform, offering ERNIE and other models	Free allowance for new users	Users in Baidu's ecosystem

Mainstream API Aggregation Platforms (August 2026)

Watch Out: Don't Buy 'Shared Keys'

Do not buy so-called 'shared API Keys' on Taobao, Xianyu, or similar platforms —these Keys usually come from unclear sources, can be banned at any moment, and carry the risk of data leaks. Always get API Keys through official platforms or reputable aggregators.

Cost Optimization Strategies

Picking the right model is only the first step —knowing how to 'use it frugally' matters just as much. Here are a few cost optimization strategies proven in practice:

1. Make the most of cache hits. Many models offer 'caching' —when your request includes content that has been processed before (such as a system prompt), that portion is billed at a tiny fraction of the normal price. DeepSeek charges just 1/60 of the regular price for cache hits. Putting your fixed system prompt at the start of the message list maximizes cache hits.

2. Tiered routing —use a cheap model for simple tasks and an expensive one for complex tasks. For example, DeepSeek V4 Flash for text classification (2 yuan per 1M output tokens) and Claude for architecture design (181 yuan per 1M output tokens). In a typical project, 80% of calls can go to the cheap model.

3. Cap your output length. APIs bill by output tokens, so longer output costs more. In scenarios that don't call for long-winded answers, set the max_tokens parameter to limit output length and keep the AI from 'rambling on.'

4. Use the Batch API for non-real-time tasks. If you have lots of tasks that don't need instant responses (such as bulk data labeling or document summarization), the Batch API earns you a 50% discount. Both OpenAI and Azure offer this feature.

5. Monitor your token usage. Most API platforms provide a usage dashboard. Check the usage breakdown regularly, find the calls that burn the most tokens, and optimize them specifically. One small prompt tweak (say, changing 'please explain in detail' to 'please explain in 3 sentences') can save 50% of your tokens.

Strategy	**Savings**	**Difficulty**	**Notes**
Cache hits	Up to 98%	Low	Put fixed content at the start of your messages
Tiered routing	50%-80%	Mid	Use a cheap model for simple tasks
Cap output length	30%-60%	Low	Set the max_tokens parameter
Batch API	50%	Low	Use batch processing for non-real-time tasks
Prompt optimization	20%-50%	Mid	Streamline prompts and trim redundancy

Cost optimization strategies and their impact (August 2026)

Hands-On: Wiring a Large Model API into a Project

Let's wrap up the chapter with a complete hands-on exercise —building an 'AI Summarizer': the user pastes in a long piece of text, and the app calls a large model API to produce a summary. This exercise walks through the full API workflow: environment setup, the API call, error handling, and cost control.

```
# ai_summarizer.py - AI summarizer
import os
from openai import OpenAI

# 1. Configure (the API key is read from an environment variable — never hard-code
    it)
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def summarize(text: str, max_length: int = 200) -> dict:
```

```
    """Summarize a text

    Args:
        text: the original text to summarize
        max_length: maximum summary length (in tokens)

    Returns:
        dict: the summary plus token-usage info
    """
    try:
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=[
                {
                    "role": "system",
                    "content": "You are a summarizer."
                               "Summarize the user's text concisely."
                               "Keep the summary to 3 sentences at most."
                },
                {
                    "role": "user",
                    "content": f"Summarize the following text:\n\n{text}"
                }
            ],
            max_tokens=max_length,  # cap the output length
            temperature=0.3  # low temperature = more stable
        )

        return {
            "success": True,
            "summary": response.choices[0].message.content,
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
            "total_tokens": response.usage.total_tokens
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 2. Usage example
if __name__ == "__main__":
    long_text = """
    (paste a long text here...)
    """
```

```
result = summarize(long_text)

if result["success"]:
    print(f"Summary: {result['summary']}")
    print(f"Tokens used: {result['total_tokens']}")
    # estimate the cost: DeepSeek V4 Flash
    cost = result['total_tokens'] * 0.000004  # about 0.4 fen per 1,000 tokens
    print(f"Estimated cost: {cost:.4f} yuan")
else:
    print(f"Something went wrong: {result['error']}")
```

This code embodies several API best practices: the API Key is read from an environment variable and never hardcoded; the function has clear documentation and type hints; max_tokens is set to cap the output length; temperature is set low (0.3) for more stable output; error handling is complete; and token consumption is returned for cost tracking. When you have the AI write this code, you can simply tell it these requirements —it is fully capable of implementing them.

The final lesson of this chapter: models are the "ammunition," methodology is the "tactics." The right ammunition makes the battle smoother, but without good tactics even the best ammo loses the war. Now that you know each model's strengths and how to use the APIs, you have a well-stocked arsenal —but how you fight the battle still depends on the thinking methods from the earlier chapters.

Try It Yourself: Get Your First API Call Running

Register on a Chinese model platform (we recommend DeepSeek or Zhipu —both give new users a free allowance), create an API Key, put it into your system environment variables (not into your code!), and then run the 'first API call' code above. The moment you see the AI's reply print out, you have crossed the line between 'using AI' and 'building AI applications.'

Chapter Recap

- The big three international models: GPT-5.6 (a balanced all-rounder), Claude (top-tier coding with a hallucination rate of just 1%), Gemini (king of multimodal with 2M context)
- China's top five: DeepSeek V4 (best value for money —watch for the price-increase announcement), Qwen3.7 (enterprise-grade), GLM-5.1 (No. 1 in China on SWE), Kimi K3 (ultra-long text), Doubao (a free way to start)
- The four-question decision method: Where will it be deployed? Coding or something else? Ultra-long context? Tight budget?
- Tokens are the billing unit: 1 Chinese character is about 1.5-2 tokens; 1 million

characters is about 1.5 million tokens

- An API Key is your wallet password: manage it with environment variables, never hardcode it, never upload it to Git
- Chinese models are generally OpenAI SDK compatible: switching models means changing just two lines —api_key and base_url
- Five ways to save: cache hits, tiered routing, capping output, the Batch API, and monitoring usage

Chapter 43: Conversation Engineering —How to Talk with AI Effectively

Conversation is the core mode of interaction in AI coding. But note: 'conversation' is not the same as 'chatting'—an effective AI coding conversation is a structured communication technique, which we call 'conversation engineering.' It studies how to get the highest-quality code output in the fewest rounds of dialogue. Plainly put, this chapter teaches you how to 'make yourself clear' to the AI.

Prompt Layering

A conversation with the AI can be split into three layers, each with its own job. Once you understand these three layers, you can stack them like building blocks to assemble highly effective prompts.

Layer	Purpose	Example	Frequency
System layer	Sets the AI's role and the global constraints	"You are a React expert; all code must be written in TypeScript"	Set once at the start of the project
Task layer	Describes the specific task at hand	"Build a form component with username and password fields"	Given at the start of each task
Feedback layer	Evaluates and corrects the AI's output	"The password field is missing a show/hide toggle — please add it"	Given based on the quality of the output

The Three Layers of a Conversation Prompt

The system-layer prompt usually lives in the project memory file (such as CLAUDE.md) and doesn't need to be repeated in every conversation. The task layer is what you'll use most in conversation —it should contain three elements: 'what to do,' 'what technology to use,' and 'what constraints apply.' The feedback layer is where real skill shows —good feedback lets the AI make precise corrections, while bad feedback only makes it guess again.

Tip: The Three-Part Feedback Formula

A practical feedback template is 'point out the problem + state the expectation + provide context.' For example: 'The error handling on line 25 only uses console.log (point out the problem); production should use the unified error handler and return a user-friendly message (state the expectation); there is a handleError function in the project you can use (provide context).' Feedback like this is ten times more effective than 'fix the error handling.'

Debug Conversations: Describing Errors So the AI Can Pinpoint Them Fast

Debugging is one of the most frequent scenarios in AI coding. When you hit a bug, how you describe the problem to the AI directly determines how quickly it can locate and fix it. Here are the four keys to a good debugging conversation:

1. Let the AI 'see' the raw information, not your paraphrase of it. Reference the error logs, exception stack traces, and relevant code directly with @, rather than describing it as 'there seems to be some kind of error.' The AI reading the raw information is more accurate than the AI reading your interpretation.

2. Describe the gap between 'expected behavior' and 'actual behavior.' 'I expect clicking the button to take me to the home page (expected), but when I actually click it, nothing happens (actual)' —that is far more useful than 'the button is broken.'

3. Provide reproduction steps. 'First I enter the username, then I click the login button, and this error appears' —reproduction steps let the AI retrace your path, which makes the problem easier to pinpoint.

4. Be clear about what you have already tried. 'I already checked the network request — it returns 200 and the data is fine' —this helps the AI rule out the directions you have already verified, instead of wasting time rechecking them.

A Poor Debugging Description

```
The button doesn't do anything when I click it, take a look
```

A Good Debugging Description

```
Clicking the login button does nothing. Expected behavior: clicking it should
    navigate to the /home page Actual behavior: nothing happens at all, and the
    console shows no errors Reproduction steps: 1. On the login page, enter
    username admin and password 123456 2. Click the "Log In" button 3. Nothing
    happens on the page Already ruled out: - Network requests are fine, returning
    200 - The onClick event is bound - Reviewed the LoginButton component code
    Related files: @src/components/LoginButton.tsx @src/pages/Login.tsx
```

Iteration Strategy: Small Quick Steps vs. Big Refactors

Iteration in AI coding has two basic strategies, each suited to different situations. Understanding the difference helps you make the right choice in each case.

The 'small quick steps' strategy suits feature development and bug fixes. Its core: have the AI make one small change at a time, and only move on to the next after verifying it. The advantage is that every step can be verified and rolled back, so risk stays under control; the drawback is that it is slower and takes frequent back-and-forth. Good for: developing new features, exploring when you are not sure of the best approach, and careful iteration whenever you are not fully confident in the AI's output.

The 'big refactor' strategy suits large-scale changes where the goal is clear. Its core: describe the complete goal in one go and let the AI plan and carry out the entire change on its own. The advantage is speed —the AI sees the whole picture and can make better architectural decisions; the drawback is high risk —if the AI misunderstands, correcting it is costly. Good for: clear-cut architecture upgrades, large code migrations, and refactors protected by thorough tests.

Aspect	**Small quick steps**	**Big refactor**
Change Size	One feature or one bug	One module or one architectural layer
Verification Frequency	Verify right after every change	Verify everything together once all changes are done
Conversation Rounds	Many (5-10 rounds per feature)	Few (1-3 rounds per module)
Risk	Low —every step can be rolled back	High —fixing a misunderstanding is costly
Best For	New features, exploratory development	Refactors and migrations with a clear goal
Prerequisites	None in particular	Needs test coverage

Comparing the Two Iteration Strategies

In practice, the two strategies are often combined. For example, when refactoring a module, first use the 'big refactor' approach to have the AI produce an overall plan; after you review

the plan, execute it step by step with 'small quick steps.' That way you keep the big-picture view while keeping execution risk under control.

Context Management for Multi-Turn Conversations

As the number of turns grows, you need to actively manage the conversation's context to avoid the 'the AI forgot what we said earlier' problem. Here are a few practical context management techniques.

First, 'anchor' key information. At the start of a conversation, spell out the key constraints, and restate them when the time comes. For example, 'Remember, our data lives in localStorage —we can't use a database.' Repeat that constraint in the relevant change requests so the AI doesn't forget.

Second, use the 'summarize and continue' technique. When a conversation gets very long, ask the AI to 'summarize the progress so far and what is left to do,' then open a new conversation and use that summary as the starting point. That preserves the key context while sidestepping the decay of a long conversation.

Third, 'one conversation, one task.' This is the simplest and most effective context management strategy. Each conversation handles just one task or one feature; when it is done, open a new conversation for the next one. This naturally caps the conversation length and keeps the context clean.

Fourth, use your tool's context management features. Most AI coding tools have a 'clear context' or 'new session' button —when you sense the quality of the AI's answers slipping, don't hesitate: open a new session. Losing a bit of conversation history beats struggling on inside a degraded context.

Try It Yourself: Play 'Patient' for Your AI

Deliberately build a small page in the browser with a planted bug (say, a button that does nothing). Then, following the 'good debugging description' format from this chapter, give the AI one complete description: raw information, expected vs. actual, reproduction steps, and what you have already ruled out. Feel the difference in how fast the AI pins down the problem —you will understand why 'describing the problem' is a craft of its own.

Chapter Recap

- Three prompt layers: system layer (kept in the memory file), task layer (what to do + technology + constraints), feedback layer (problem + expectation + context)
- Four debugging essentials: give the raw information, expected vs. actual, reproduction steps, and what you have already ruled out
- Small quick steps: low risk, many rounds, good for new features; big refactor: high risk, few rounds, needs test coverage

- Four context management moves: anchor key information, summarize-and-continue, one conversation per task, and open a new session without hesitation

Chapter 44: From 0 to 1 —A Complete Project, End to End

The theory is done; now it is time to get your hands dirty. This chapter is not a simple 'type along' tutorial but a practical methodology covering how to think, how to ask, and how to verify. Using three projects of rising difficulty, we will show the complete thought process from requirements to finished product. No matter which tool you use, this workflow applies.

Project 1: A Minimal Personal Tools Site (Beginner)

Thinking Through the Requirements

Goal: build a personal tools page that shows entry points to 3-4 handy little tools. This is a pure front-end project —no back end and no database needed. The point is not how fancy the tools are but experiencing the full 'from requirements to finished product' process.

Applying the five requirement elements: Who —personal use. What —a web page showing tool entry points. Why —gather your frequently used tools on one page for easy access. How —a single-file HTML+CSS+JS build with no frameworks. Constraint —finish within 30 minutes, no deployment needed.

Giving the Instruction

Following the Think Before Coding principle, your instruction should carry enough information for the AI to understand the goal, without specifying every implementation detail:

```
Please create a personal toolbox web page with these requirements:
1. Single-file HTML (inline CSS and JS), no frameworks
2. Dark theme, modern minimalist style
3. Show 4 tool entry cards:
   - Text summarizer
   - Word counter
   - Color picker
   - QR code generator
4. Each card has an icon, a title and a short description
5. Cards have a hover animation
```

```
6. Responsive layout, usable on phones
7. A title bar at the top
Please give the complete HTML file directly.
```

Acceptance and Iteration

Once the AI hands over code, accept it using the three-layer check from iterative verification. First layer, functional check: does the file open in the browser? Do all 4 cards display? Do the hover animations work? Second layer, edge-case check: does the layout hold up at phone screen sizes? Do the cards wrap when the window gets narrow? Third layer, architecture check: is the code readable? Is the CSS inlined in a single style tag? Is there obvious redundancy?

If you want one of the tool cards to actually work (rather than just be an entry point), add it gradually —implement one tool's functionality, verify it, then add the next. That is the 'small quick steps' strategy in action.

Project 2: A Full-Stack Expense Tracker (Intermediate)

Requirements Thinking and the PRD

Goal: a full-stack expense-tracking app with a front end, a back end, and a database. This project is far more complex than the first and calls for the requirements engineering method from Chapter 40.

Start by writing the PRD. Following the template from Section 3 of Chapter 40, lay out the project overview, feature list, technical plan, data model, and development plan. When choosing the tech stack, apply the Simplicity First principle —React on the front end but no complex state management like Redux; Node.js on the back end but no microservices; SQLite instead of PostgreSQL for the database (during development).

Break the development plan into 5 steps: Step 1, the back-end API skeleton (an Express server + SQLite + CRUD endpoints); Step 2, the basic front-end pages (a records list + an add form); Step 3, hooking the front end to the back end (API calls + data display); Step 4, categories and filtering; Step 5, monthly summaries and simple charts. After every step you have 'something that works.'

Setting Up the Project Memory File

Before any coding begins, create the project memory file. This is a core context engineering practice —it gives the AI the project background at the start of every conversation.

```
# Project: full-stack finance tracker
## Overview
```

```
A personal finance tracker with income/expense records, categories and monthly
    summaries.
## Tech stack
- Frontend: React + TypeScript + Vite
- Backend: Node.js + Express + TypeScript
- Database: SQLite (via better-sqlite3)
- Shared type definitions for frontend and backend
## API conventions
- Base path: /api
- Response format:{ success: boolean, data: any, error: string }
- RESTful style
## Data model
Record { id, amount, type, category, note, date }
## Code style
- TypeScript strict mode
- 2-space indentation
- Functions have JSDoc comments
- Comments and commit messages in English
```

Building It Step by Step

Follow the development plan, completing each step in its own separate conversation. Here is a sample instruction for the first step:

```
I need a backend API skeleton.
Requirements:
1. Express server, port 3001
2. SQLite database, table name records
3. Fields of the records table:
   id (TEXT PRIMARY KEY)
   amount (REAL)
   type (TEXT: 'income'|'expense')
   category (TEXT)
   note (TEXT)
   date (TEXT: YYYY-MM-DD)
4. CRUD endpoints:
   GET /api/records - get all records
   POST /api/records - add a record
   PUT /api/records/:id - update a record
   DELETE /api/records/:id - delete a record
5. Error-handling middleware
Please give the complete backend code structure.
```

Notice what makes this instruction work: it describes only the current step (the back-end skeleton) without mixing in front-end requirements; it explicitly lists the fields and endpoints (giving the AI firm constraints) while leaving the concrete implementation to the AI; and it

follows the Goal-Driven Execution principle —saying 'what to do' rather than 'how to do it.'

As soon as the AI hands over code, verify it immediately: run the server and test each endpoint with curl or Postman. Once everything checks out, open a new conversation for Step 2 (the basic front-end pages). Every step follows the 'instruct - verify - new conversation' loop.

Project 3: An AI-Powered App (Advanced)

Conceiving the Project

Goal: an app that connects to a large model API —say, an 'AI Study Assistant' where the user enters study material and the AI generates a summary, links related concepts, and recommends learning resources. What makes this project special: your application itself is calling the AI, so you have to handle new problems like API calls, prompt engineering, and error handling.

The most important lesson this project teaches: when you 'use AI to write code that calls AI,' you must pay extra attention to prompt engineering. The prompt your app sends to the large model API also needs to follow the principles discussed in this guide —clear, concise, and goal-driven.

Key Technical Points

This project involves a few technical points you did not meet in the first two projects:

- API key management: keys must never be hardcoded in the code; manage them with environment variables;
- Streaming response handling: large model replies usually arrive as a stream and need to be displayed bit by bit;
- Token usage control: large models bill by the token, so usage must be controlled at the application level;
- Prompt template management: different features use different prompt templates, which need to be managed in a structured way.

The AI can help you implement all of these technical points, but you need to understand the 'why' behind each one —why can't keys be hardcoded? Why are streaming responses needed? Why must token usage be controlled? Only once you understand the 'why' can you judge whether the AI's solution makes sense.

The Universal Hands-On Checklist

Whatever the size of the project, here is a universal AI coding workflow checklist. Run through it every time you start a new project to make sure no key step slips through.

Stage	What to Do	Principle Applied
1. Requirements thinking	Go through the five requirement elements and get clear on what to build	Think Before Coding
2. Breaking down requirements	Split big requirements into small, independently verifiable steps	Simplicity First
3. Writing the PRD	Organize it into a structured document for reference	Goal-Driven
4. Setting up the memory file	Create CLAUDE.md or a similar file	Harness Engineering
5. Building step by step	One conversation per step: instruct - verify - next	Surgical Changes
6. Testing and verifying	Test that each step works correctly right after finishing it	Iterate & Verify
7. Reviewing the code	Check architecture soundness, security, and maintainability	Iterate & Verify
8. Iterating and improving	Iterate based on the test and review results	Iterate & Verify

The Universal AI Coding Workflow Checklist

Try It Yourself: Finish 'Project 1' This Week

Treat the 'personal tools site' from Section 1 of Chapter 44 as your first AI coding assignment: spend 5 minutes on the five elements, send the instruction to the AI (any tool will do), then accept the result with the three-layer verification. Finish it within 30 minutes —and your first AI coding creation is born. When you are done, save it somewhere safe; it is the keepsake of your move from 'reader' to 'practitioner.'

Chapter Recap

- Project 1 (beginner): a pure front-end tools site —experience the full workflow, done in 30 minutes
- Project 2 (intermediate): a full-stack expense tracker —PRD + memory file + 5 steps, with something that 'works' at every step
- Project 3 (advanced): an AI study assistant —use AI to write code that calls AI, with attention to prompt engineering
- The universal eight-step workflow: requirements thinking break down PRD memory file build step by step test review iterate

- Every step follows the 'instruct - verify - new conversation' loop

Chapter 45: Quality Assurance — Making AI-Written Code Reliable

AI-written code has a telltale trait: it usually 'looks right' —tidy variable names, thorough comments, clear logic. But 'looking right' is not the same as 'being right.' Hidden problems in AI code are often harder to catch than in hand-written code, precisely because its surface quality is so high that it lulls you into lowering your guard. This chapter shows you how to build an effective quality assurance system for AI code —three lines of defense (testing, review, and version control) to turn 'looks right' into 'is right.'

Test-Driven: Have the AI Write the Tests

Testing is the first line of defense for quality in AI coding. The good news: having the AI write tests is easier and more reliable than having it write feature code —because a test's expected behavior is well defined, it rarely produces code that 'looks right but isn't.'

The test-driven AI coding flow: first have the AI write the feature code; once the basic functionality passes, immediately have it write test cases. The test cases should cover the normal path, boundary cases, and error cases. Then have the AI fix the feature code based on the test feedback. The beauty of this flow: once the tests are written, they become a standing verification tool —any later change can be checked by running the tests to confirm no regressions slipped in.

```
Write unit tests for the calculate_discount function.
The test cases should cover:
1. Happy path:
   - 100-yuan item at 20% off -> 80
   - 200-yuan item at 50% off -> 100
2. Edge cases:
   - 0-yuan item at 20% off -> 0
   - 100-yuan item at 0% off -> 100 (no discount)
   - 100-yuan item at 10% off -> 100 (full price)
3. Error cases:
   - negative price -> should throw an error
   - discount rate > 1 -> should throw an error
```

```
   - non-numeric input -> should throw an error
Use the Vitest framework and give the complete test file.
```

Notice what makes this instruction work: it explicitly lists the three categories of test cases (normal, boundary, and error), leaving the AI no room to 'test selectively.' Sometimes an AI will write only happy-path tests to 'get it done fast' —spelling out every category prevents that.

Code Review Techniques

Even with test coverage, you still need to review the AI's code yourself. Code review does not mean reading line by line —that is too slow and too dependent on your own coding skill. Effective AI code review focuses on these key dimensions:

Review Dimension	What to Look For	Common Problems
Security	Input validation, key handling, SQL injection protection	The AI often forgets to validate input and hardcodes keys in the code
Error Handling	Are exceptions caught, and are the error messages sensible	The AI loves empty catch blocks, or just a console.log
Performance	Any obvious performance problems, like N+1 queries or infinite loops	The AI sometimes writes $O(n^2)$ algorithms
Maintainability	Is the code structure clear, and do the names make sense	AI occasionally generates overly abstract code
Dependency management	Whether the dependencies brought in are actually needed and their versions are explicit	AI tends to pull in unnecessary libraries

Key dimensions for reviewing AI code

When reviewing, you don't have to check everything one by one yourself —you can have the AI do the first pass. An instruction like: "Please review this code for security, error handling, and performance, and point out potential issues." AI's self-review ability keeps improving; it can't fully replace human review, but it helps you filter out most of the obvious problems.

III. Version Control: Git's Role in AI Coding

Git plays an even bigger role in AI coding than in traditional programming. Because every change the AI makes can introduce unpredictable shifts, you need a reliable way to undo mistakes —and version control is exactly that.

The best Git practice in AI coding is “commit often, move in small steps.” Every time you finish one verifiable piece of functionality, make a commit. That way, when one of the AI's changes goes wrong, you can roll back precisely to the last working state instead of throwing away all your work.

Another handy trick is “branch isolation.” For AI changes with uncertain results (like a big refactor), create a new branch first and do the work there. If you like the result, merge it back into the main branch; if not, just delete the branch and the main branch stays untouched. This is far safer than making repeated changes on the main branch.

1. Before each AI change, make sure the working area is clean (no uncommitted changes). That way, if you don't like what the AI did, you can quickly roll back with git checkout .

2. After each AI change, once it passes verification, commit right away. The commit message should describe what the change does, like ”feat: add user registration”.

3. For big refactors, create a branch first: git checkout -b refactor/auth-system. Finish the changes on the branch, and merge only after the tests pass.

4. Build the habit of looking at the diff. After every AI change, use git diff to see exactly what changed. This not only helps you spot problems, it also helps you learn the code.

IV. Common Pitfalls and How to Dodge Them

A few pitfalls show up again and again in AI coding, and almost every beginner steps into them. Knowing them in advance saves you a lot of detours.

Pitfall One: The Trust Trap

“Code written by AI looks so professional —it must be fine, right?” This is the most dangerous trap. AI-generated code is very tidy on the surface —proper naming, complete comments, clear structure —and that “surface polish” makes people lower their guard. But a tidy surface doesn't mean the logic is correct. The antidote: always verify through testing, never judge by “does it look right.”

Pitfall Two: The Bloat Trap

“Since AI can write my code, why not make the features more complete?” That leads to code bloat. AI is great at “giving extra” : ask it for a login page and it hands you a full user management system. Extra features don't just add complexity —they raise the odds of bugs too. The antidote: strictly follow the Simplicity First principle and tell the AI explicitly to “build only this feature, add nothing extra.”

Pitfall Three: The Understanding Trap

"The AI should understand what I mean, right?" Not necessarily. AI's comprehension is strong, but for vague requirements it still has to "guess." When it guesses wrong, the code it produces is wrong. Worse, code built on a wrong guess usually "looks reasonable," making you believe it understood. The antidote: put enough specific detail in your instructions to shrink the space the AI has to "guess" in.

Pitfall Four: The Context Trap

"I said this earlier in the conversation —the AI should still remember, right?" It very likely doesn't. As conversations get longer or you start new sessions, the AI loses earlier context. If a key constraint isn't written into the project memory file, it will very likely be ignored in later changes. The antidote: write key constraints into the project memory file instead of relying on conversational memory.

The Four Traps at a Glance

Trust trap —don't trust "it looks right"; Bloat trap —AI loves to "give extra"; Understanding trap —vague requirements make it "guess"; Context trap —conversational memory "forgets." The matching antidotes: verify with tests, cut the extras, give full details, write to the memory file.

Hands-On: Build Three Lines of Defense for Your Project

If you've already started Project One (the tool site): (1) have the AI write the three types of test cases for your core functions; (2) review the code against all five dimensions (security / error handling / performance / maintainability / dependencies); (3) initialize the project with git and make your first commit. With all three lines of defense in place, your project truly belongs to you.

Chapter Summary

- AI code that "looks right" ≠ "is right" —when the surface looks too good, it's easy to lower your guard
- Have the AI write tests: explicitly list the three case types (normal / boundary / error) to prevent "selective testing"
- Five review dimensions: security, error handling, performance, maintainability, dependency management
- Git is your "undo button": commit often, move in small steps, isolate with branches, check the diff often
- The four traps: Trust (verify with tests), Bloat (cut the extras), Understanding (give full details), Context (write to the memory file)

Chapter 46: The Road Ahead — From User to Architect

Once you've mastered the methodology of Chapters 37 through 44 and can use AI to finish projects with confidence, what comes next? This chapter explores the path from "AI tool user" to "AI system architect." At this stage the focus is no longer "how do I get the AI to write good code" but "how do I fit AI better into the whole development process."

I. The MCP Protocol and Tool Extensions

MCP (Model Context Protocol) is an open protocol that appeared in 2025 and spread gradually through 2026. It defines a communication standard between AI models and external tools. Simply put, MCP lets an AI "use tools" —not just generate code, but also run commands, query databases, call APIs, and work with the file system.

Before MCP, every AI tool had its own way of integrating tools, and none were compatible. The arrival of MCP means: a tool that follows the MCP standard can be used by any AI tool that supports MCP. That sharply lowers the cost of tool integration, and it stretches AI's capability from "generating text" to "taking actions."

For advanced users, MCP means you can build custom tool capabilities for your AI. For example, if your team has an internal deployment system, you can write an MCP tool that lets the AI call the deployment system directly, with no human switching to the terminal. This "AI calling tools" capability is the infrastructure of Agentic Engineering.

II. Custom Workflows and Skills

When you keep repeating certain fixed AI coding routines, you can solidify them into "custom workflows" or "Skills." For example, your team may have a standard code review process —check security first, then performance, then maintainability. You can write that process as a Skill so the AI automatically follows it every time it reviews code.

The core value of custom workflows is "consistency" —they make sure every run follows the same standard, without drifting because the conversation ran long or the context differs.

This matters especially for team collaboration: when every member uses the same workflow, the style and quality standards of the code they produce converge.

How you build a custom workflow varies by tool. Cursor supports Rules and Custom Instructions, Claude Code supports custom commands, and Replit supports Runme. But the core idea is the same: "encode" your best practices into instruction templates the AI can execute.

III. Multi-Agent Collaboration Patterns

By 2026, multi-agent collaboration has become a hot topic. The core idea: instead of using one AI to finish a task, use several AIs with different roles working together. For example, one AI writes the code, one writes the tests, one reviews the code —each does its own job and checks the others.

The strengths of multi-agent collaboration are "division of labor" and "cross-validation": each agent focuses on its own domain and produces higher-quality output, and agents cross-checking one another catch problems any single one might miss. But the drawbacks are just as clear: coordination is costly, communication overhead is large, and debugging gets more complicated.

Tip: You Can Play "Multi-Agent" Solo

As an individual developer, you don't need to set up a complicated multi-agent system —you can "play" the different roles yourself to simulate multi-agent collaboration. For example: first have the AI write feature code in one conversation; then open a new conversation (with the CLAUDE.md context provided) and ask the AI to "review this code as a code reviewer"; finally open another conversation and ask the AI to "revise the code based on the review comments." This "one person, many roles" approach captures most of the benefits of multi-agent collaboration without any complicated infrastructure.

IV. The Continuous Learning Path

AI coding is a fast-moving field. Today's best practices may need updating tomorrow. Here are a few suggestions for continuous learning:

First, follow the source, not secondhand takes. Andrej Karpathy's original tweets and CLAUDE.md, the official documentation from Anthropic and OpenAI, the changelogs of the major tools —these are first-hand information. Community articles and tutorials are interpretations and summaries of that information; they're valuable but can be distorted. Build the habit of reading primary sources.

Second, let practice drive learning. Don't "learn first, then do" — "do and learn along the way." Every time you meet a new kind of problem (say, handling file uploads for the first time, or connecting to a third-party API for the first time), take it as a chance to learn the

relevant knowledge. Learning in practice is far more efficient than book theory detached from real scenarios.

Third, stay aware of tools without chasing every new release. Every month or two, spend half a day catching up on the latest tools and trends —but don't switch every time something new appears. The test is simple: does the new tool meaningfully relieve one of your bottlenecks? If not, your current setup is enough.

Fourth, build a personal knowledge base. Write down every conversation where you solved a hard problem, every lesson from a stumble, every method that worked. You don't have to publish it —it's your own "methodology notebook." As it accumulates, it becomes your most valuable asset.

Hands-On: Start a "Lessons Learned" Notebook

Create a document today (a notes app or a .md file both work) and write down the three AI coding lessons you most want to remember so far —even if it's just "think it through before asking." From now on, every time you hit a pitfall or learn a new trick, add an entry. Half a year from now, you'll thank today's you.

Chapter Summary

- MCP is the communication standard between AI and tools: it lets AI run commands, query databases, and call APIs
- The core value of custom workflows/Skills is "consistency": encode best practices into instruction templates
- Multi-agent = division of labor + cross-validation; solo developers can simulate it with "one person, many roles"
- Four habits for continuous learning: read the source, learn by doing, don't chase every new thing, build a personal knowledge base

Chapter 47: Quick Reference and Resources

This chapter is a quick-reference manual that works on its own. When you can't quite remember a principle or need to make a fast decision, flip straight here. Treat it as a "pocket card" : the five principles at a glance, a tool decision tree, common questions, and learning resources —every page included.

I. Mental Model Cheat Sheet

Principle	**In One Sentence**	**Key Actions**
Think Before Coding	Think It Through Before Asking	Answer three questions: What to build? What should the result look like? What are the constraints?
Simplicity First	Solve Problems with the Least Code	One feature at a time, and cut unnecessary complexity
Surgical Changes	Precise Changes, No Overstepping	Specify the change scope explicitly and refuse the AI's "let me fix that too" edits
Goal-Driven Execution	Say What, Not How	Give the goal, not the steps, and let the AI pick its own path
Iterate & Verify	Generate-Verify-Fix Loop	Three-layer verification: functionality, boundaries, architecture

Cheat Sheet: The Five Thinking Principles

II. The Tool-Choice Decision Tree

When you need to pick an AI coding tool, follow the decision path below. Note: this isn't a ranking of "which is best" —it's a framework for judging "which is best for you."

What do you want to do?

Total beginner, want a quick taste pick a cloud Builder (v0/Bolt/Replit)

Everyday development, need code completion pick an IDE with built-in AI (Cursor/Trae)

Large projects, need bulk operations pick a terminal Agent (Claude Code)

Validate an idea with a quick prototype pick a cloud Builder or Trae SOLO

Figure 6: Decision tree for choosing an AI coding tool

III. FAQ: Common Questions

Q1: I don't know how to program at all —can I still do AI coding?

Yes. AI coding tools in 2026 are strong enough that you can describe what you want in plain language and have the AI generate the code. Two suggestions: first, start with cloud Builder tools (like v0 and Bolt), which need no development environment setup; second, learn basic concepts as you go —you don't need to master the syntax, but you do need to "read" the general meaning of the code the AI writes, so you can check its work effectively.

Q2: Which programming language should I learn?

It depends on what you want to build. Website front end —JavaScript/TypeScript; data processing or AI —Python; enterprise back end —Java or Go. But more importantly: in the AI coding era, the importance of "which language" is falling. AI can handle the syntax details for you; what you need to understand is "the structure of programs" —what a function is, what a loop is, what a data structure is. These concepts carry across languages, and once you understand them, picking up any language is fast.

Q3: Is there a copyright issue with AI-written code?

As of August 2026 this is still a contested legal question. Generally speaking, AI-generated code isn't considered to have a clear copyright owner in most jurisdictions. But if you're using it in a commercial project, it's wise to: (1) check the intellectual property terms in the terms of service of the AI tool you use; (2) review and modify key code yourself so it carries your own creative input; (3) avoid having the AI directly copy open-source code that may be copyrighted.

Q4: Will AI replace programmers?

Not in the short term. What AI replaces isn't the "programmer" as a profession but the task of "coding line by line." Just as the calculator didn't replace mathematicians —it freed them from tedious hand computation. AI coding tools shift programmers from "writing code" to "thinking through problems" —which actually raises the ceiling on a programmer's value.

Future competitiveness won't lie in "how many lines of code you can write" but in "how complex a problem you can solve."

Q5: How do I judge whether the AI's output is correct?

Use the three-layer verification method. Layer one: functional verification —does it run? Are the inputs and outputs right? This is the most basic. Layer two: boundary verification — what happens with empty input, extra-long input, weird input? This surfaces hidden problems. Layer three: architecture verification —is the code structure sound? Any hard-coded values? Is it easy to extend? This takes some technical judgment of your own. If you can't judge layer three, at least do the first two —running fine with sane boundaries already puts you ahead of most beginners.

IV. Recommended Learning Resources

Below are the recommended learning resources as of August 2026. Keep in mind that internet links change; if a link is dead, searching its title usually turns up the latest version.

Type	**Resource**	**Why It's Recommended**
Primary Source	Andrej Karpathy's CLAUDE.md	The source of the five principles —70 lines where every word counts
Primary Source	Anthropic official docs (Claude Code)	The best-practices guide for terminal agents
Tutorial	A Skills collection curated by the Vibe Coding community	Battle-tested tips compiled by community practitioners
Tutorial	Cursor official docs and YouTube channel	The authoritative guide to AI built into an IDE
Community	awesome-claude-code on GitHub	Best practices and plugin collections for Claude Code
Community	The r/cursor community on Reddit	Experience sharing and discussion from Cursor users worldwide
Basics	MDN Web Docs (developer.mozilla.org)	The most authoritative reference for front-end development
Basics	The official Python tutorial (docs.python.org)	The official starting point for learning Python

V. Closing Thoughts: Tools in Hand, Methodology in Mind

Let's come back to this book's core belief: tools iterate every three months; methodology evolves only every three years. Today, in August 2026, AI coding tools are astonishingly powerful —you can create a website with one sentence, generate a full-stack app from a short description, finish a complex refactor with a single instruction. But the more powerful the tools, the more methodology stands out —because powerful tools amplify every one of your decisions.

If your requirements are clear, AI helps you build them efficiently; if they're vague, AI efficiently builds the wrong thing. If you follow Simplicity First, AI helps you validate ideas quickly; if you want everything at once, AI quickly piles up technical debt for you. If you make precise changes, AI optimizes your code with surgical precision; if your instructions are fuzzy, AI boldly introduces bugs for you.

That's why this book spent most of its pages on methodology rather than tool operations. Not because tools don't matter, but because methodology is the tool's "multiplier" —the better your methodology, the more your tools are worth; with zero methodology, even the strongest tool is just zero times anything.

You're no longer coding alone. You are the commander and AI is your army; the quality of your thinking determines your army's fighting power. From today, arm your mind with methodology and amplify your ability with tools —that's the whole secret of AI coding.

References

1. Andrej Karpathy. (2025). Vibe Coding. Social media post introducing the concept of vibe-driven programming with AI.

2. Andrej Karpathy. (2025). CLAUDE.md - Project guidelines for AI-assisted development. GitHub repository, 70+K stars.

3. Anthropic. (2026). Claude Code: Terminal-native AI programming. Official documentation and best practices.

4. Cursor Team. (2026). Cursor 3: The next generation of AI-first IDE. Release notes and documentation.

5. OpenAI. (2026). Codex CLI: Bringing AI to the terminal. Technical documentation.

6. Model Context Protocol (MCP) Specification. (2025-2026). Open protocol for AI-tool integration.

7. Vibe Coding Community. (2026). Community-curated skills and best practices for AI-assisted programming.

8. Spec-Driven Development (SDD) methodology. (2026). Emerging practices in specification-first AI programming.

Part VIII: Android Development in Practice (Chapters 48-58)

Chapter 48: Getting to Know Android —From Operating System to Development Ecosystem

Congratulations on making this decision —going from "someone who plays with phones" to "someone who makes phone apps." Before we start, let's spread out the map: where did Android come from? Why are developers all over the world writing apps for it? And from the moment your fingers hit the keyboard to when an app lands on a user's phone, what happens in between?

I. Android's Origin Story: From "Little Robot" to "The World's Biggest System"

Android's story begins in 2003 —an engineer named Andy Rubin founded Android Inc., with a modest goal: build a smart operating system that "understood cameras better." Google acquired it in 2005, and in 2007 joined dozens of manufacturers to form the Open Handset Alliance (OHA). In September 2008, the first Android phone, the HTC Dream (G1), went on sale.

You know the rest of the story: by 2026, Android is the world's largest mobile operating system by market share, with active devices in the billions. Your phone, TV box, car dashboard screen, and smartwatch are very likely all running Android. What does that mean? It means every line of code you write has a chance to be used by the largest number of people on Earth.

Tip: Android's Open-Source DNA

Android's core code (AOSP, the Android Open Source Project) is open source —any manufacturer can take it and customize it. That's why Xiaomi has MIUI (now called HyperOS), Huawei has its HarmonyOS compatibility layer, and Samsung has One UI. That's good news for developers: what you learn is "Android development," not "development for one brand" —one skill set that works on every device.

The version numbers are fun too: starting with 1.5 Cupcake, every major release was

named after a dessert —Donut, Eclair, Froyo, Gingerbread... until Android 10 in 2019 switched to plain numbers. As of August 2026, the latest official release is Android 17, and the "minimum supported version" you actually need to care about in development is usually set by mainstream apps somewhere between Android 8 (API 26) and Android 10 (API 29) —support devices from a few years back and your potential user base is big enough.

Version	Codename / Year	What Matters for Developers
Android 8 (API 26)	Oreo • 2017	Notification channels and background limits —the minimum line for many older devices
Android 10 (API 29)	2019	Scoped storage arrives; the rules for file access get a major overhaul
Android 12 (API 31)	2021	Material You dynamic color; a big upgrade in look and feel
Android 13 (API 33)	2022	Runtime notification permission —a popup asks whether to allow notifications
Android 14-15 (API 34-35)	2023-2024	Predictive back, 16KB pages, privacy tightened further
Android 16-17 (API 36-37)	2025-2026	Faster minor-release cadence, AI capability integration (like Gemini extensions)

A Brief History of Android Versions (as of August 2026 —key items for development)

II. Which Language to Write In: Java or Kotlin?

Two mainstream languages have appeared over Android's history. Java is the "founding elder" —Android used it from day one. Kotlin was named a "first-class citizen" by Google in 2017, and from 2019 the official stance has been "Kotlin-first." By 2026, nearly all new projects use Kotlin, and most older projects are gradually migrating too.

Let's be honest: if you're a complete beginner, learn Kotlin directly and don't look back at Java. Three reasons: first, Kotlin takes 30%-40% less code than Java (same features, shorter and clearer); second, it's natively protected against "null pointers," a beginner's nightmare; third, all modern tutorials, official docs, and AI coding tools (remember our AI coding book?) are Kotlin-based. You'll only run into Java when maintaining old projects or reading old code.

Java style (the wordy version)

```
public class MainActivity extends Activity \{
@Override
protected void onCreate(Bundle b) \{
super.onCreate(b);
setContentView(R.layout.activity);
\}
\}
```

Kotlin style (the clean version)

```
class MainActivity : ComponentActivity() \{
override fun onCreate(b: Bundle?) \{
super.onCreate(b)
setContent \{ App() \}
\}
\}
```

Same task, but Kotlin drops a pile of curly braces and boilerplate. We'll learn it properly in Chapter 50; for now you only need to remember: the "common language" of Android development is called Kotlin.

III. How Is an App Born?

Many people think "developing an app" means opening the computer and furiously writing code. In reality, an app goes through roughly this pipeline from idea to a user's phone:

Idea

Design (UI + features)

Write code

Compile and package (APK/AAB)

Test (emulator + real device)

Release (store/installer)

Figure 1: The full journey of an Android app from idea to the store

Two concepts from this pipeline you need to know right now:

- APK: the installation package format for Android apps (.apk). Like a Windows .exe, it installs onto a phone with a tap. Use it for your own testing or for sharing with friends to try.
- AAB: Android App Bundle (.aab), the "advanced packaging format" used to publish on Google Play. The store dynamically trims the package to match each user's phone model, so everyone downloads a smaller file. You can't install it yourself —it only goes through the store.

Another term you'll hear constantly: SDK (Software Development Kit). It's the full set of "development gear" provided officially by Android —the API libraries for each version, debugging tools, emulator images, and more. Classes like Activity and Button that you call in your code all come from the SDK.

IV. The Android Tech Stack at a Glance: Your "Weapon List"

Before we formally start learning, meet the standard configuration of a mainstream Android project in 2026 —it's fine if this list makes no sense yet; come back after you finish the book and it will all click:

Layer	**Mainstream Choice (2026)**	**What It's For**
Language	Kotlin 2.x	The language for writing logic
UI	Jetpack Compose + Material 3	Draws the UI in code (the main thread of this book)
Architecture	MVVM + ViewModel	Keeps UI and logic separate so the code stays tidy
Data Storage	Room (database) + DataStore (settings)	Saves data on the phone
Networking	Retrofit + OkHttp + Kotlin coroutines	Talks to the server
Build Tool	Gradle + Kotlin DSL	Turns code into an installable package
Development Environment	Android Studio	Google's official all-in-one IDE

The Mainstream Android Tech Stack of 2026 at a Glance

Don't be intimidated by all that English. These are just the pots and pans in the kitchen —right now you only need to know that "cooking means entering the kitchen"; how each tool works will be unpacked for you chapter by chapter. And here's the good news: when Android Studio creates a new project, it sets up this whole stack for you automatically. You just get cooking.

V. Android vs. iOS: A Developer's First Lesson in Choosing Sides

You may be holding an Android device and an Apple device at the same time. As a developer, the differences in experience between the two sides are worth knowing up front —

this isn't about taking sides on "which is better," it's the practical math of "which side suits your start" :

Dimension	Android Development	iOS Development
Development Computer	Windows / Mac / Linux all work	Must be a Mac (Xcode is exclusive)
Development Tool	Android Studio (free)	Xcode (free) + $99/year to publish
Store Review	Lenient; a few hours to a few days	Strict; 1-7 days is the norm
On-Device Testing	APKs can be shared and installed freely	Distribution is restricted; testing requires signing/TestFlight
Revenue Environment	Mainly ads and domestic channels	App Store users have better in-app purchase habits
Learning Cost	More concepts to learn (the four major components, etc.)	Closed ecosystem, but unified and superbly documented

Android vs. iOS Development Paths (August 2026)

Let's be honest: if your computer isn't a Mac, this question answers itself —Android is the only option. If you do have a Mac, you can read Parts Eight and Nine of this book side by side; the programming mindset transfers, and learning the second side after the first is much faster.

Hands-On: Look at the "Android" in Your Phone

Open [Settings] > [About phone] (or "About device") on your phone and find the "Android version" row —tap it several times in a row and you'll trigger the system's hidden Easter-egg game. Note your version number and check it against the version table above to see which generation it belongs to. Your phone is your very first "development test device."

Chapter Summary

- Android was born in 2008; by 2026 it's the world's largest mobile system by market share, with billions of active devices
- The latest official release is Android 17; mainstream apps support Android 8-10 at minimum, so the user base stays big
- New projects all use Kotlin: 30%+ less code, null-pointer protection, officially endorsed
- APK for installing yourself, AAB for the store; the SDK is the official development gear kit

- The standard 2026 stack: Kotlin + Compose + MVVM + Room + Retrofit + Gradle + Android Studio

Chapter 49: Setting Up Your Development Environment — Android Studio and Your First Project

To do a good job, first sharpen your tools. In this chapter we set up the "kitchen" : install Android Studio, get the emulator running, create the first Android project of your life, then take the project structure apart until it makes sense. Once the environment is set up, you'll see your own code turn into a tappable button for the first time —a moment many people remember for the rest of their lives.

I. Installing Android Studio

Android Studio is Google's official integrated development environment (IDE). It's free and a full all-in-one package: writing code, previewing UI, debugging, and packaging all happen inside it. Four steps to install:

1. Open the official site developer.android.com/studio and download the latest stable version (as of August 2026, the release after Narwhal; Windows/Mac/Linux builds are all available);

2. Run the installer and click "Next" all the way through —the setup wizard automatically includes the Android SDK and emulator components;

3. On first launch, download the SDK components as prompted (the default selections are fine; it may take a few minutes and a few GB of space);

4. When it's done you'll see the welcome screen —congratulations, the "kitchen" is set up.

Note: Know the Hardware Requirements

Android Studio is a memory hog: 16GB of RAM is recommended (8GB will run, but the emulator can lag), reserve at least 20GB of disk, and your CPU needs virtualization support (enable VT-x/AMD-V in the BIOS). If your computer really is old, use "real device debugging" instead of the emulator (covered in Section 4 of this chapter) —the experience is actually smoother.

II. Creating Your First Project

On the welcome screen click New Project and pick the Empty Activity template (the cleanest beginner template), then fill in three things:

Setting	What to Enter	Notes
Name	My First App	The app's name, shown on the phone's home screen
Package name	com.example.myfirstapp	A globally unique identifier; the convention is a reversed domain name —keep the default for now
Minimum SDK	API 26(Android 8.0)	Minimum supported version; covers 95%+ of devices

The three things to fill in when creating a project

After you click Finish, Android Studio uses Gradle to automatically download dependencies and build the project (the first time takes a few minutes; every run after that is much faster). When the build finishes, you have a runnable app —even if it can only display one line: "Hello Android!".

Tip: The Ins and Outs of Package Names

The package name is like the app's "ID number" —it must be globally unique and can't be changed after publishing. For personal practice, com.example.xxx is fine; for a real product, switch to your own domain in reverse (like com.yourname.appname). Never publish with com.example —the store will reject it.

III. Running It: Emulator and Real Device

The code is written —now you need a "phone" to run it on. You have two choices:

The Emulator

An emulator is a virtual Android phone inside your computer. In Android Studio, open Device Manager > Create Device, pick a device (the Pixel series is recommended) > choose a system image (one with Google APIs) > download > launch. Then click the green triangle Run in the toolbar, and a few seconds later your app appears on the "phone" screen.

Real Device Debugging (More Recommended)

A real phone is always the truest test environment —swipe feel, real network, real sensors; the emulator can't match any of it. Debugging on a real Android phone is simple:

1. On the phone, open [Settings] > [About phone] and tap "Build number" 7 times in a row to unlock "Developer options";

2. In [Developer options], turn on USB debugging;

3. Connect the phone to the computer with a data cable and choose "Allow USB debugging" in the popup;

4. Back in Android Studio, your phone appears in the device dropdown in the toolbar — click Run.

Watch Out: Don't Leave USB Debugging On

USB debugging is like opening a "back door" to your phone for computers. When charging in public or connecting to a computer you don't trust, be sure to turn USB debugging off so no one can read your data through it. Turn it off after each development session —make it a habit.

IV. Project Structure Explained: What Each File Does

The file tree on the left can make your head spin the first time you open a project. Don't panic —in daily development, 90% of your time is spent with just these few:

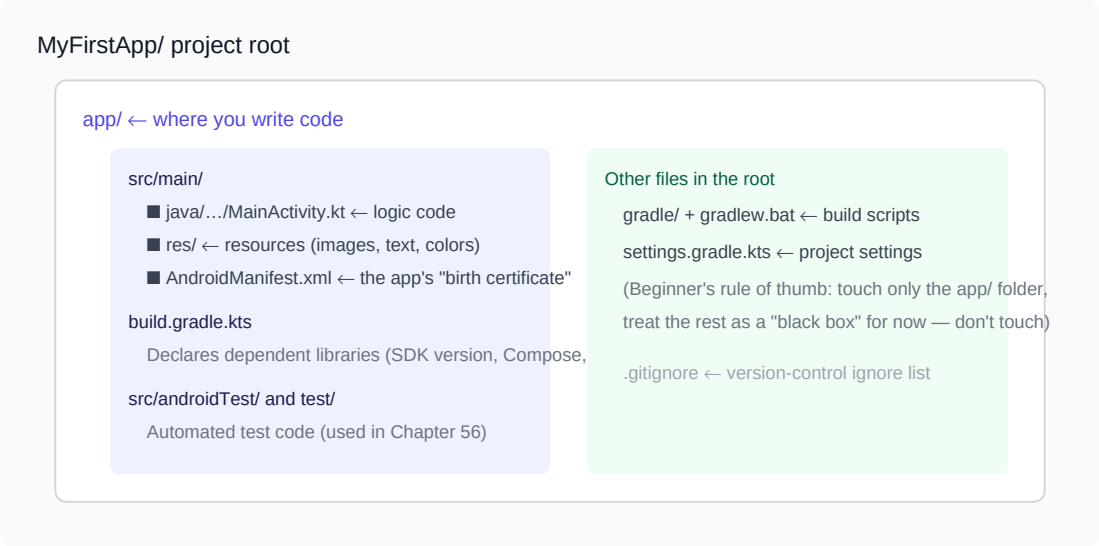


Figure 2: Project structure every beginner must know (the mantra: only touch the app/ folder)

- MainActivity.kt: the app's "front door." When a user taps the icon, this is the first thing to wake up. In the current template it's responsible for displaying "Hello Android!".
- AndroidManifest.xml: the app's "household registry" —it declares what this app is called, which icon it uses, which screens it has, and which permissions it needs (camera, location, etc.).
- The res/ folder: the resource warehouse, storing images, icons, strings, and colors. Keeping text in resource files instead of hard-coding it into code is a good habit in Android development (it makes switching languages or themes easy).
- build.gradle.kts: the build recipe —it declares the minimum supported version and which libraries to depend on (like Compose).

V. A First Try: Change "Hello" to Your Name

Open MainActivity.kt and you'll see the code the Compose template generated. Find the line Text("Hello Android!") and replace the text inside the quotes with your name, e.g. Text("Hi, Zhang San! This is my first app"), then press Ctrl+R (Cmd+R on Mac) to run again. A few seconds later, your name appears on the emulator.

```
setContent {
    MyFirstAppTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            // v change this to your own text
            Text("Hello, Alex! This is my first app")
        }
    }
}
```

Congratulations —you've just completed a full "edit code compile install run" cycle. This cycle is your daily development routine from now on. Strike while the iron is hot and change something else: make the font bigger, try another color —the next chapter shows where these "magic powers" come from.

VI. First Aid for Common Errors: The Beginner's "ER" for Your First Two Weeks

During environment setup and your first build, you'll most likely hit the three errors below. Don't panic —all of them have antidotes:

Error	Common Cause	Fix
Gradle sync failed / slow dependency downloads	Poor network access to the repositories	Configure a mirror repository for Gradle (Alibaba Cloud / Tencent Cloud Maven mirrors); search online for "gradle China mirror" and follow the instructions
Missing SDK components	The SDK download was skipped during installation	Settings > SDK Manager: install the missing platforms and tools
Emulator fails to start / black screen	Virtualization not enabled, or a corrupted image	Enable VT-x/AMD-V in the BIOS; delete and recreate the emulator; or just debug on a real device

First-Aid Table for Beginner Environment Errors

Tip: How to Search Error Messages

The first line of an error message is the most valuable —copy it verbatim into a search engine and add the keyword "android studio". Don't search vague questions like "why can't I set up my environment"; search specific errors like "Gradle sync failed: Could not resolve com.android.tools.build". Precise questions get precise answers.

Hands-On: Complete Your "Developer Ritual" Checklist

(1) Install Android Studio and get the emulator running; (2) run your app on a real phone once and take a screenshot to keep; (3) change the Hello text to your name; (4) zip up the project as a backup. Once these four steps are done, you can announce to your friends: I'm an Android developer now (screenshot as proof).

Chapter Summary

- Android Studio = the official free IDE; recommended 16GB RAM + 20GB disk + virtualization enabled
- Three essentials for a new project: name, package name (globally unique, never publish com.example), minimum SDK (API 26 or higher)
- Two ways to run: the emulator (a virtual phone on your computer) and a real device (enable USB debugging, remember to turn it off after)
- Project mantra: only touch the app/ folder —MainActivity.kt is the front door, the Manifest is the household registry, res/ is the resource warehouse
- The developer's daily loop = edit code Run check the result, over and over

Chapter 50: Kotlin Quick Start — Android's First Language

This chapter is where we officially learn the language. Don't be nervous —Kotlin is famous for being beginner-friendly, and you don't need to memorize a syntax book: master 20% of the syntax and you can read and write 80% of everyday code. Look up the rest as you work on projects. Remember our learning principle: understanding beats memorizing, using beats mastering.

I. Variables and Types: Giving Data a Name

A program is "data + logic." A variable is just a name you give data so you can store it. Kotlin has two ways to declare one:

```
val name = "Alex"        // val: read-only variable (the recommended default)
var age = 18             // var: mutable variable (can change later)

age = 19                 // <i class="fas fa-check-circle mr-1"></i> var can change
// name = "Sam"         // <i class="fas fa-times-circle mr-1"></i> val can't be
    changed — compile error

val score: Int = 100     // you can also declare the type explicitly
val pi = 3.14            // omitted, it's still inferred automatically
```

There are only a few common types —remember them with one table:

Type	What It Stores	Example
Int	Whole numbers	42
Double	Decimal numbers	3.14
String	Text	"Hi"
Boolean	True/False	true / false
List< T>	A group of data	listOf(1, 2, 3)

Kotlin's Common Basic Types

Tip: The val-First Rule

An iron rule of the Kotlin community: use val by default and only use var when truly necessary. If you use var where val would work, the IDE will warn you with yellow text. This habit saves you a whole class of "variable got changed unexpectedly" bugs —the less your code changes, the easier it is to reason about.

II. Functions: Packaging Logic into "Buttons"

A function is a named piece of logic, like a button on a remote control —press it (call it) and a sequence of actions runs. In Kotlin you declare one with fun:

```
fun greet(name: String): String {
    return "Hello, $name!"     // the $ sign inserts the variable into the string
}

println(greet(Alex))        // prints: Hello, Alex!

// single-line functions can be written as expressions
fun add(a: Int, b: Int) = a + b

println(add(3, 5))            // prints: 8
```

A few key points: parameters go in the parentheses (name first, type after, separated by a colon); the return type goes after the parentheses; and string templates like $variableName are Kotlin's signature convenience —no more gluing "string + variable" together the way Java does.

III. Conditions and Loops: Teaching the Program to "Think"

A program's intelligence comes from two things: knowing how to judge (if/when) and knowing how to repeat (for/while).

```
// Decisions: if / else
val age = 17
if (age >= 18) {
    println("adult")
} else {
    println("minor")
}

// Multi-branch: when (a hundred times handier than switch)
val score = 85
val level = when {
    score >= 90 -> "excellent"
    score >= 60 -> "pass"
```

```
    else -> "Keep it up!"
}

// Repetition: a for loop
val fruits = listOf("apple", "banana", "orange")
for (fruit in fruits) {
    println("I love $fruit")
}
```

when is Kotlin's most common way to write multi-branch logic, and you'll use it every day in Android development (like showing different screens based on state).

IV. Null Safety: Kotlin's "Life Saver"

The most famous bug of the Java era is the NullPointerException —the program touches something that "isn't there" and crashes on the spot. Statistics say it's the perennial number one on Android's crash leaderboard. Kotlin solved it at the language level: null values are not allowed by default.

```
var name: String = Alex   // a regular type: never allowed to be null
// name = null                // <i class="fas fa-times-circle mr-1"></i> compile
    error — it stops you right there

var nickname: String? = null // add a question mark: null is allowed
nickname = "Al"

// When using nullable types, you must handle them safely:
println(nickname?.length)    // safe call: returns null instead of crashing
val len = nickname?.length ?: 0   // Elvis operator: default value when null
println(len)
```

Note: Don't Confuse ? and ?

String? means "null is allowed," while a ?: b means "if a is null, use b" (it looks like Elvis's hairstyle, hence the name Elvis operator). When you're starting out and the compiler complains about a "null-related error," odds are you forgot to handle a nullable type —that's the language protecting you.

V. Classes and Objects: Molds for Making "Parts"

A class is the "mold" for objects; objects are the "finished products" of a class. Everything in Android —screens, buttons, data —is an object. Kotlin has a magic tool called the data class, built specifically for "holding data," where one line does what takes dozens in Java:

```
// Regular class: a mold
```

```
class Dog(val name: String) {
    fun bark() = println("$name says woof!")
}

val dog = Dog("Rex")      // create an instance
dog.bark()                 // Rex says woof!

// data class: a box for data (with equals/copy built in)
data class User(val id: Int, val name: String, val email: String)

val me = User(1, Alex, "zhangsan@example.com")
val twin = me.copy(name = "Alexandra")   // copy and change one field
```

Later, when you build an expense tracker, each entry is a data class: data class Record(val amount: Double, val category: String, val date: String). See —you can already write the code structure of a real project.

VI. Lambdas and Higher-Order Functions: Kotlin's "Syntax Candy"

In Android code you'll see { ... } every day —a "function without a name" called a lambda (anonymous function). Button's onClick = { count++ } is a lambda —it bundles "what to do when clicked" into one package and hands it to the button.

```
// The regular way: define a named function and pass it in
fun double(x: Int) = x * 2
val result = listOf(1, 2, 3).map(::double)

// The lambda way: define it inline, quick and direct
val result2 = listOf(1, 2, 3).map { it * 2 }   // it = the current element

// Filter + transform combo (extremely common in real projects)
val bills = listOf(12.5, 88.0, 5.0, 200.0)
val bigBills = bills.filter { it > 50 }.sorted()
// [88.0, 200.0] — "bills over 50, sorted"
```

Memorize a few high-frequency "collection combos" : filter, map, sorted, and sum. 90% of the list handling in your future is a combination of these four moves. Kotlin's philosophy: if logic can be said in one sentence, never write five.

VII. A Syntax Cheat Sheet for Absolute Beginners

Don't try to memorize every bit of syntax in one day. Save this card, refer to it while you code, and within two weeks it will feel natural:

What You Want to Do	How to Write It
Define a read-only variable	`val x = 1`
Define a mutable variable	`var x = 1`
Define a function	fun name(params): ReturnType { ... }
Decision	if (condition) {...} else {...}
Multi-way branch	when { condition -> result }
Loop	for (x in list) { ... }
String interpolation	"Hi, $name"
Handle null	`x?.length ?: 0`
Define a data class	`data class X(val a: Int, val b: String)`

Kotlin Syntax Cheat Sheet (Beginner Edition)

Hands-On: Play with Kotlin for 10 Minutes, No Install Needed

Open play.kotlinlang.org in your browser (Kotlin's official online playground), paste in every code snippet from this chapter and run it, tweak the numbers and text, and watch how the output changes. These 10 minutes of "play before learn" will make Chapter 51 twice as easy.

Chapter Summary

- Prefer val, use var sparingly; types can be inferred automatically, or declared explicitly after a colon
- A function = packaged logic; the $ string template is the signature convenience
- if/when to decide, for to repeat; when is Android's all-rounder
- Null safety is Kotlin's life saver: normal types reject null, nullable types must be handled with ? / ?:
- A data class is a box for data, one field per line, used daily in real projects
- 20% of the syntax does 80% of the work: learn to use it first, master it later

Chapter 51: Building the UI — Getting Started with Jetpack Compose

The UI is everything the user sees. Before Compose came along, Android UIs were written in XML —as tedious as fill-in-the-blank exercises. In 2021 Google introduced Jetpack Compose: you "draw" the UI directly in Kotlin code, what you see is what you get, and the preview refreshes the moment you change a line of code. By 2026, Compose is the clear mainstream for new projects. This chapter is where we learn to "draw."

I. Compose's Big Idea: The UI Is a Function

Compose's core idea in one sentence: UI = function. Everything that can appear on screen (text, buttons, images) is a "composable function," marked with @Composable. Nest them like building blocks, and that's your UI.

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")     // a "text building block"
}

// Nested composition: big blocks around small ones
@Composable
fun HomeScreen() {
    Column {                         // vertical stack
        Greeting(name = "Alex")
        Greeting(name = "Sam")
        Button(onClick = { /* what happens on tap */ }) {
            Text("Tap me")
        }
    }
}
```

Note the @Composable "hat" —it tells the compiler "this is a UI building block." Blocks

can nest infinitely; that's the whole secret of Compose.

II. Basic Components: Text, Button, Input Field, Image

Compose's components are "building blocks." The four most common:

Component	What It Does	Required Parameter
Text	Displays text	text (the text to show)
Button	Button	onClick (the click callback)
TextField	TextField	value and onValueChange (the callback for input changes)
Image	Displays images	painter (the image resource)

Compose's Four Basic Components

```
@Composable
fun BasicWidgets() {
    Column {
        Text(text = "This is some text")
        Button(onClick = { println("clicked") }) {
            Text("Button")
        }
        var input by remember { mutableStateOf("") }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("Type something") }
        )
    }
}
```

In those two lines for the input field you'll spot remember { mutableStateOf(" ") } — that's Compose's "state" magic, covered in detail in Section 5 of this chapter. For now, just get acquainted with it.

III. The Layout Trio: Column, Row, Box

You have the components —how do you arrange them? Remember three "boxes" :

- Column: stacks things vertically (like a line at the cafeteria counter);
- Row: lines things up horizontally (like sitting side by side);

- Box: stacks things on top of each other (like a pile of playing cards —great for badges and overlays).

```
Column {
    Text("First row")
    Row {
        Text("Left")
        Text("Right")
    }
    Box {
        Text("Bottom layer")
        Text("Corner badge stacked on top")
    }
}
```

Want the layout to look better? Add a Modifier to components —it's like putting makeup on your building blocks:

```
Text(
    text = "Pretty text",
    modifier = Modifier
        .padding(16.dp)          // padding on all sides
        .fillMaxWidth()          // fill the width
        .background(Color.Blue)  // background color
        .clickable { /* tap */ }// clickable
)
// Chained calls: one after another, order matters (padding first, then background
    — the background excludes the padding)
```

Note: Modifier Order Matters

padding(16.dp).background(Blue) and background(Blue).padding(16.dp) give different results —the first is "spacing first, then paint" (the background excludes the padding), the second is "paint first, then spacing" (the background includes the padding). This is where beginners most often paint themselves a "mysterious white border." When unsure, swap the order —the preview instantly tells you the answer.

IV. State: Bringing the UI to Life

A static UI is boring. To make a number go up by 1 when a button is clicked, or let an input field accept typing, you need state. Compose's rule: the UI always faithfully reflects the state —when the state changes, the UI redraws automatically.

```
@Composable
fun Counter() {
    // remember + mutableStateOf: remember this changing value
```

```
    var count by remember { mutableStateOf(0) }

    Column {
        Text(text = "You tapped $count times")
        Button(onClick = { count++ }) {   // change the state
            Text("Tap me +1")                // the UI refreshes automatically
        }
    }
}
```

Three lines of code, one live counter. This is where Compose feels better than the old way: no manually "refreshing the UI" —just change the data. Remember the two key functions: remember (keeps a value from being wiped when the UI recomposes) and mutableStateOf (tells Compose "redraw when this value changes").

User taps

State changes (count++)

Compose notices the change

UI redraws automatically (recomposition)

Figure 3: Compose's one-way data flow from state to UI

V. Theming and Dark Mode: Configure Once, Apply Everywhere

The MyFirstAppTheme { ... } in the template project is the theme. Based on the Material 3 design spec, it manages colors, fonts, and shapes in one place. Supporting dark mode takes only three steps:

1. Define both a light and a dark color scheme in the theme (the template already generates the ColorScheme);

2. When the system switches to dark mode, isSystemInDarkTheme() detects it automatically;

3. Use the colors the theme provides (like MaterialTheme.colorScheme.primary) instead of hard-coded color values.

Tip: Material 3 Dynamic Color

Starting with Android 12, "Material You" lets an app pull colors from your wallpaper. Compose templates support it by default (dynamicColor). Your app will "change outfits" on different users' phones —that's the romance of modern Android.

VI. Preview, Animation, and Live Edit: Compose's "Cheat Codes"

Three "cheat codes" for Compose's developer experience —once you use them, there's no going back:

- @Preview: add the @Preview annotation to a Composable and Android Studio instantly renders the UI on the right side —what you see is what you get as you edit, no need to install on a phone every time;
- Live Edit: the 2026 Android Studio supports "hot updating" certain changes on the emulator or a real device —small tweaks like changing text color appear on the device in seconds;
- Animation APIs: animateFloatAsState and AnimatedVisibility let elements transition smoothly in a few lines of code —that "premium feel" is often just these few dozen lines away.

```
@Composable
fun AnimatedBox() {
    var big by remember { mutableStateOf(false) }
    // when the value changes, it smoothly animates to the target
    val size by animateDpAsState(if (big) 120.dp else 60.dp)

    Box(Modifier
        .size(size)
        .background(MaterialTheme.colorScheme.primary)
        .clickable { big = !big }
    )
}
```

Tip: Function First, Animation Later

A common beginner mistake is fussing over animations before the UI even works. Get the feature running first, then add motion at the end —that's the Simplicity First principle too: animations are the icing on the cake, features are the cake itself.

Hands-On: Build a "Mood Button"

Turn the template project into a mood switcher: one button that, when tapped, cycles through happy sad angry, with the text and background color changing along. Hint: use when to pick the wording and color based on the state. Send it to friends when it's done and collect your first round of "whoa, you can make apps now".

Chapter Summary

- Compose's idea: the UI = @Composable functions, nested like building blocks
- Four basic components: Text / Button / TextField / Image
- The layout trio: Column stacks vertically, Row lines up horizontally, Box stacks on top
- Modifier chains apply "makeup" in sequence: padding, fillMaxWidth, background, clickable —order matters
- State is the soul: remember + mutableStateOf —change the data and the UI redraws itself
- Theming with Material 3 + dynamic color: configure dark mode once and it applies everywhere

Chapter 52: The Four Components and the Activity Lifecycle

If Compose is an app's "face," then this chapter is about the app's "skeleton." Android has four basic components, and every app is assembled from them. The most important one, the Activity, has a "lifecycle diagram" that every Android developer must burn into memory. Once you finish this chapter, you'll finally understand age-old mysteries like "why did my app lose all its state after I switched away and came back?"

The Four Components: Android's Pillars

Android officially defines four basic components, each responsible for one kind of job:

Component	Job	Think of it as
Activity	A visible screen	The storefront —what users see and interact with
Service	The invisible workhorse doing jobs in the background	A warehouse worker —playing music, downloading files; no screen needed
BroadcastReceiver	Receives system broadcasts	The gatehouse —the loudspeaker for "battery low" and "network lost"
ContentProvider	Shares data across apps	A public filing cabinet —for example, sharing contacts with other apps

Android's four components at a glance

As a beginner you'll spend 95% of your time working with the Activity. The Service and BroadcastReceiver come into play for things like music playback and push notifications, and the ContentProvider is used the least (the system has already wrapped the common data for you). One more thing: all four components must "register their household" in AndroidManifest.xml —if they're not registered, the system can't find them.

The Activity Lifecycle: Your First Android "Treasure Map"

Apps on a phone can be interrupted at any moment: a call comes in, the user hops over to WeChat, the system runs short on memory and kills you... An Activity's whole life is a loop around this diagram:

onCreate (birth: initialization)

onStart (visible)

onResume (in the foreground, interacting)

onPause (partially covered)

onStop (fully hidden)

onDestroy (destroyed)

onRestart / onStart / onResume: the same route in reverse when returning to the foreground

Figure 4: The Activity lifecycle (going forward: the top row is birth the bottom row is exit; coming back retraces the same path)

Callback	When it fires	Typical use
onCreate	When the Activity is created (once)	Load the UI, initialize data
onStart	The screen becomes visible	Start some lightweight logic
onResume	Has focus and can be interacted with	Start animations, resume playback
onPause	Partly covered, or the moment the user switches away	Pause animations, save drafts
onStop	Fully invisible	Release heavy resources, pause downloads
onDestroy	Before being destroyed	Final cleanup

Typical uses of the six lifecycle callbacks

Watch out: rotating the screen "kills" the Activity

The most classic beginner fail: you rotate the screen and everything typed into the input fields is gone. Why? By default, rotating the screen destroys and recreates the Activity (running through onDestroy onCreate). The fix: save state with rememberSaveable (the Compose replacement for remember); it automatically recovers the values when the Activity is rebuilt. You'll use it again and again in Chapters 6 and 7.

Intents: The "Tickets" for Traveling Inside and Outside the App

An Intent is the "ticket" components use to talk to each other. Whether you want to jump from one screen to another, open the browser, or launch the camera, you have to "buy a ticket and hop on":

```
// Method 1: explicit Intent — you name the exact destination screen
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("name", Alex)     // carrying "luggage" (data) along for the ride
startActivity(intent)

// Method 2: implicit Intent — you say what to do, the system finds who does it
val web = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
startActivity(web)   // the system opens a browser / or lets you choose
```

Each kind has its own job: explicit intents are for jumping between screens inside your own app; implicit intents are for using the system's or other apps' abilities (opening a web page, making a call, sharing). In a Compose project, get the Context with LocalContext.current inside a Composable and you can call startActivity.

Multiple Screens and Navigation

Real apps have more than one screen. The navigation solution officially recommended for Compose is Navigation Compose (the androidx.navigation library):

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{name}") { entry ->
        val name = entry.arguments?.getString("name")
        DetailScreen(name ?: "Unknown")
    }
}

// Navigating from a screen:
navController.navigate("detail/Alex")     // with arguments
navController.popBackStack()              // go back
```

Tip: What if you have lots of screens?

As the project grows, keep all the screen routes together in one file (say, AppNavGraph.kt) and manage them in one place —don't let the "where do I go next" logic scatter across every screen. It's the first step from "it runs" to "it's maintainable."

A Quick Look: What Is a Fragment (and Why This Book Doesn't Teach It)

Digging through older tutorials, you'll definitely bump into Fragments —reusable pieces of UI embedded in an Activity, the absolute star of the old multi-screen architecture. But in the Compose era, Composable functions plus Navigation routes are all you need, and the official docs have downplayed Fragments. That's why this book leaves them off the main path.

My advice: don't reach for Fragments in new projects; just know they're "UI pieces embedded in an Activity" for when you read old code or get asked in an interview. For learning materials, always prefer anything from 2023 onward —Android's stack of "old almanacs" goes stale faster than you'd think.

Hands-on: Give the Counter a "Memory"

Upgrade the counter from Chapter 51: (1) swap remember for rememberSaveable and rotate the screen to confirm the number survives; (2) add an "About" button that uses Navigation to jump to a second screen showing the version number; (3) print logs (Log.d) in onCreate/onPause on both screens and watch the lifecycle call order in Logcat. Finish these three steps and the lifecycle stops being just theory.

Chapter Summary

- The four components: Activity (the storefront), Service (the background), BroadcastReceiver (the loudspeaker), ContentProvider (the filing cabinet)
- The six lifecycle callbacks: onCreate onStart onResume onPause onStop onDestroy
- Rotating the screen destroys and recreates the Activity: use rememberSaveable to keep your state
- An Intent is a "ticket": explicit for your own screens, implicit for system abilities
- Multiple screens use Navigation Compose: NavHost + composable routes

Chapter 53: Data Storage and Network Requests

An app you can only look at is dead; an app that can save data and reach the network is alive. This chapter introduces the "three-piece set" of Android's data layer: DataStore (for small settings), Room (for structured data), and Retrofit (for talking to servers), plus their "head dispatcher" —coroutines. After this chapter you'll be able to write real apps that remember user preferences and update their content in real time.

Choosing a Storage Option: Look at This Table First

Option	Good for storing	Traits	Where this book uses it
DataStore	Scattered settings (theme, toggles)	Key-value pairs; simple and safe	Chapter 54, expense tracker settings
Room	Structured data (transactions, users)	A real wrapper over the SQLite database; queryable	Chapter 54, transaction records
Retrofit	(Not stored; fetched online) server data	The de facto standard HTTP request library	Chapter 55, weather data

How the data trio divides the work

Memorize this mantra: "small settings go in DataStore, batches of data go in Room, outside data travels via Retrofit." SharedPreferences, common in older tutorials, has been replaced by DataStore, and hand-written SQL for SQLite has been replaced by Room —learn the new way and skip the old road.

DataStore: Saving Small Settings

To remember scattered values like "the currency the user picked last time" or "whether dark mode is on," use DataStore (the Preferences flavor):

```
val Context.dataStore by preferencesDataStore(name = "settings")

// Write: store a value
suspend fun saveTheme(isDark: Boolean) {
    context.dataStore.edit { prefs ->
        prefs[THEME_KEY] = isDark
    }
}

// Read: fetch it (a Flow notifies about changes automatically)
val themeFlow: Flow<Boolean> = context.dataStore.data
    .map { it[THEME_KEY] ?: false }
```

Note that what comes back is a Flow —a data stream that "flows." The moment a value changes, everywhere that observes it automatically receives the new value. This concept (reactivity) is a cornerstone of modern Android; you'll put it into practice in Chapter 54.

Room: Giving Your Data a "Real Home"

An expense tracker holds hundreds or thousands of entries, and you'll want to query and summarize by category and by month —that takes a database. Room is the official wrapper around SQLite that makes "creating tables and querying data" feel as natural as writing Kotlin. Three pieces:

```
// 1. Entity: the table structure (one table = one data class)
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val amount: Double,
    val category: String,
    val note: String,
    val date: String
)

// 2. Dao: the operations interface (CRUD)
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)
```

```
    @Query("SELECT SUM(amount) FROM records WHERE category = :c")
    suspend fun sumByCategory(c: String): Double
}

// 3. Database: final assembly
@Database(entities = [Record::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun recordDao(): RecordDao
}
```

You can lift this "trio" template straight into any of your projects —just change the field names. Queries return a Flow, so the UI refreshes automatically as the data changes. Simply delightful.

Retrofit: Talking to Servers

Weather, news, exchange rates... that data isn't on the phone; it's on someone else's server. Retrofit makes "asking a server for data" as easy as calling a function. Three steps:

```
// 1. Define the interface: describe HTTP requests as functions
interface WeatherApi {
    @GET("v1/weather")
    suspend fun getWeather(@Query("city") city: String): WeatherResponse
}

// 2. Create the instance
val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApi::class.java)

// 3. Call it like a function (inside a coroutine)
val result = api.getWeather("Beijing")
```

Watch out: two iron rules of networking

(1) Never make network requests on the main thread —it will freeze the UI or even crash the app (NetworkOnMainThreadException). All network calls must go inside a coroutine (viewModelScope.launch). (2) Plain-text HTTP is blocked by default: Android 9 and up requires HTTPS. If you need an http:// local address while testing, you must explicitly declare usesCleartextTraffic in the Manifest —but a real product must use HTTPS everywhere.

JSON Parsing: Translating Network Data

Most data a server returns is JSON —text that looks like a "nested dictionary." Retrofit needs a "translator" to turn it into Kotlin objects. The two mainstream options in 2026:

Option	Traits	Best for
Gson / Moshi	The veteran reflection-based option; zero configuration	Quick starts and small projects without many fields
kotlinx.serialization	The official compile-time option; faster and safer	The first choice for new projects (mainstream in 2026)

The two major JSON parsing options

```
// Add the @Serializable annotation to the data class
@Serializable
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
)

// Retrofit gets the matching Converter (official support since Retrofit 2.11+):
// .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

Whichever option you pick, the iron rule is: look at the real JSON response first, then define your fields. Many "parsing failed" bugs come from misspelled field names or wrongly guessed nesting —print out the raw JSON and compare, and you'll pinpoint the issue in ten seconds.

Coroutines: The "Common Language" of Android Concurrency

Coroutines: The "Common Language" of Android Concurrency

The suspend and launch you keep seeing above come from Kotlin coroutines —Android's official concurrency solution. A rough way to think of it: operations that "need to wait a while" (network, database) don't block the UI.

```
// Launch a coroutine in the ViewModel
viewModelScope.launch {
    val records = dao.getAll().first()  // suspend: pauses and waits automatically
    // update the UI state...
}
```

```
// Keyword cheat sheet:
// suspend   — "I need to wait, don't block the others"
// launch    — "Get to work!"
// viewModelScope — the "workshop" tied to the screen; it wraps up automatically
    when the screen is destroyed
```

If you're starting from zero, just memorize the usage for now; you can dig into the "suspend and resume" mechanics later. In everyday development, 90% of what you write is this one template.

Hands-on: Build a "Quote of the Day" Mini App

Find a free quotes API (something like hitokoto), use Retrofit to fetch a "quote of the day," show it in the center of the screen, and add a "New quote" button (re-request inside a coroutine). Then use Room to save favorite quotes into a "collection." This one exercise uses DataStore/Room/Retrofit/coroutines all together —finish it, and Chapters 7 and 8 will be a piece of cake.

Chapter Summary

- The mantra: small settings in DataStore, batches of data in Room, outside data via Retrofit
- DataStore returns a Flow: when a value changes, observers get the new one automatically
- Room's trio: @Entity (table) + @Dao (operations) + @Database (final assembly) — the template can be reused as-is
- Retrofit in three steps: define the interface create the instance call it inside a coroutine
- Two iron rules: no networking on the main thread; real products must use HTTPS
- Coroutines = waiting without blocking the UI; viewModelScope is the screen's own workshop

Chapter 54: Project 1 —My Expense Tracker (Local Version)

Welcome to your first "real project." This expense tracker never goes online, but it has everything it needs: record a transaction, view the ledger, see monthly summaries, delete and edit. Finishing it means you've walked the full "data logic UI" pipeline from start to end. Don't rush to copy code —think it through first, then build. That's our standing principle.

Thinking Through Requirements: Ask Five Questions First

Before building, sort out the idea with the "five elements of a requirement":

Element	Answer
Who (who's it for)	Me; family and friends are fine too
What (what it does)	Record income and spending, view the ledger, monthly summaries, category breakdowns
Why (the reason)	Practice + figuring out where the money goes
How (the approach)	Kotlin + Compose + Room + MVVM, fully local, no backend
Constraint (the limits)	Done within a week; supports Android 8 at minimum; written to store standards even if never published

The five requirement elements of the expense tracker

Then break it into small steps where "every step is usable": (1) build the Room data layer (can save one transaction); (2) the list screen (can view the ledger); (3) the entry screen (can add a transaction); (4) delete and edit; (5) a monthly summary card. After each step, the app is something "that works."

The Data Layer: Giving Each Transaction a Home

First define what one transaction looks like —enough fields to be useful, but don't get greedy:

```
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String,        // "income" = money in | "expense" = money out
    val amount: Double,      // amount (in yuan)
    val category: String,    // category: food, transport, salary...
    val note: String = "",   // note (optional)
    val date: String         // "2026-08-15"
)
```

Add two "summary" queries to the Dao —these are the soul of the tracker:

```
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC, id DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='income' AND
    date LIKE :month || '%'")
    suspend fun monthIncome(month: String): Double   // month looks like "2026-08"

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='expense' AND
    date LIKE :month || '%'")
    suspend fun monthExpense(month: String): Double
}
```

Tip: why use Double for amounts?

Rigorous finance apps use BigDecimal or "an Int counted in cents" to avoid floating-point errors (the old $0.1 + 0.2 \neq 0.3$ problem). For a practice project, Double is plenty; if you ever build a real financial product, remember to switch to cents as the unit —it's a pitfall countless developers before you have stumbled into.

MVVM: UI and Logic “Live Apart”

Let the UI talk to the database directly? It runs, but the UI code turns long and ugly. The MVVM architecture gives each part its own job: the View (UI) only draws, the ViewModel (the logic hub) handles data, and the Model (Room) handles storage.

View (the Compose UI)

ViewModel (state and logic)

Model (the Room data layer)

Figure 5: The three layers of MVVM —data flows in one direction, and no layer crosses into another

```
class RecordViewModel(private val dao: RecordDao) : ViewModel() {

    // Turn the Flow into observable UI state
    val records = dao.getAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), emptyList())

    // User actions: call suspend functions
    fun addRecord(type: String, amount: Double, category: String, note: String,
    date: String) {
        viewModelScope.launch {
            dao.insert(Record(type = type, amount = amount,
                category = category, note = note, date = date))
        }
    }

    fun deleteRecord(record: Record) {
        viewModelScope.launch { dao.delete(record) }
    }
}
```

In the UI (Compose) you'd write: val vm: RecordViewModel = viewModel(), then val list by vm.records.collectAsState() —the moment the data changes, the list refreshes by itself, and you do nothing. That's the power of MVVM + Flow + Compose working as one.

The UI: List Screen + Entry Screen

The list screen uses LazyColumn (a lazy-loading list) —smooth no matter how much data:

```
@Composable
fun RecordListScreen(vm: RecordViewModel) {
    val records by vm.records.collectAsState()

    Column {
        // summary card at the top (income - expenses = balance)
```

```
        SummaryCard(records)

        // the transactions list
        LazyColumn {
            items(records, key = { it.id }) { record ->
                RecordRow(record) { vm.deleteRecord(record) }
            }
        }

        // floating "add entry" button  opens the entry dialog
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Text("+")
        }
    }
}

@Composable
fun RecordRow(record: Record, onDelete: () -> Unit) {
    Row(Modifier.fillMaxWidth().padding(12.dp)) {
        Text(record.category, fontWeight = FontWeight.Bold)
        Spacer(Modifier.weight(1f))
        Text(if (record.type == "income") "+${record.amount}" else "-${record.
    amount}",
             color = if (record.type == "income") Color(0xFF27AE60) else Color(0
    xFFC0392B))
    }
}
```

An AlertDialog (dialog) is the easiest way to enter data: a few input fields + an income/expense toggle + a category dropdown; tap OK and it calls vm.addRecord(...).

Note: don't skimp on input validation

Always validate the amount field: empty values, negative numbers, and illegal input like "abc" must all be blocked (check with toDoubleOrNull()). One of the most common reasons apps crash is "trusting the user's input like a well-behaved child." Remember: treat everything the user types as hostile until proven otherwise.

Edit and Delete: Let Data "Come and Go"

An expense tracker that can only add and never delete is useless within three days. Add two more operations:

```
// Add two more to the Dao:
@Query("UPDATE records SET amount=:amount, category=:category, note=:note WHERE id
    =:id")
```

```
suspend fun update(id: Long, amount: Double, category: String, note: String)

@Query("DELETE FROM records WHERE id=:id")
suspend fun deleteById(id: Long)

// In the UI: long-press a list item to open an "edit/delete" menu (use
    DropdownMenu or AlertDialog in Compose)
// Best to confirm before deleting — accidental deletion is what expense-app users
     hate most
```

Two bits of interaction wisdom: deleting always needs a second confirmation (to prevent fat-finger slips), and editing should pre-fill the original values (fixing one character beats re-typing everything). Don't underestimate these details —that "polished product" feel is built exactly from details like these.

Acceptance: a three-layer check + two advanced challenges

Acceptance: a three-layer check + two advanced challenges

When the project is done, accept it across three layers: "features edge cases architecture." Can you record and delete, is the data still there after reopening the app (features); does entering 0 or a negative amount crash it, does rotating the screen lose data (edge cases); are UI and data tangled together, are any fields hard-coded (architecture).

Once the basic version passes acceptance, serve yourself two "advanced dishes":

1. Monthly filter: add a month switcher at the top (e.g., 2026-08 v) so the list shows only that month's transactions;
2. Category pie chart: use Compose's Canvas or add a lightweight charting library to draw "where the money went" as a pie chart —finish this and the tracker starts to look like a real product.

Hands-on: finish it this week

Build in this order: "data layer ViewModel list screen entry screen summary card," one step a day. When stuck, paste the error as-is into a search engine or an AI (remember Part 7, "AI Coding in Practice"? When describing a problem, spell out the "expected behavior, actual behavior, and steps to reproduce"). Once it's done, track a week of spending with it —using an app you built yourself gives you a sense of achievement that pushes you forward.

Chapter Summary

- Five requirement elements break into "every step usable" small steps, then start building

- Data layer: the Record entity + Dao (including monthly-summary SQL) + Database
- MVVM division of labor: the View only draws, the ViewModel runs the logic, the Model handles storage
- Flow + stateIn + collectAsState: the UI refreshes automatically when data changes
- LazyColumn for lazy-loading lists; AlertDialog for data entry
- Three-layer acceptance (features/edge cases/architecture) + advanced challenges (monthly filter, category pie chart)

Chapter 55: Project 2 —Weather App (Networked Version)

The expense tracker was "internal training" ; the weather app is "seeing the world" —it has to deal with the internet. In this project you'll learn to: apply for a free API, parse JSON, handle the three UI states of "loading/success/failure," and keep your API key well hidden. Finish it, and you can build any "connected app" —news, exchange rates, translation; the recipe is exactly the same.

Apply for a Free Weather API

For personal practice, OpenWeatherMap or China's QWeather are good picks —both have free tiers that work right after you sign up. Taking OpenWeatherMap as an example:

1. Sign up for an account generate a key (apikey) on the API keys page;
2. The free tier can call the Current Weather endpoint: api.openweathermap.org/data/2.5/weather?q=beijing&appid=YOUR_KEY&units=metric;
3. Try it in your browser first to see what the returned JSON looks like —this "preview" is a must before wiring up any API.

Watch out: never hard-code your key into the code

An API key is like your "wallet password" —the moment it's committed to GitHub, bots will scrape and abuse it within hours. The right way: put it in local.properties (a file already ignored by gitignore), then read it into your code through Gradle's buildConfig. Search for "best practices for hiding API keys on Android" and follow along.

The Network Layer: Retrofit + JSON Parsing

First define your "data model" based on the JSON the API returns; ignore any fields that don't match:

```
// JSON: {"weather":[{"description":"Sunny"}],"main":{"temp":26.5,"humidity":40},"
    name":"Beijing"}
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
)
data class Weather(val description: String)
data class Main(val temp: Double, val humidity: Int)

interface WeatherApi {
    @GET("data/2.5/weather")
    suspend fun getWeather(
        @Query("q") city: String,
        @Query("appid") key: String,
        @Query("units") metric: String = "metric",
        @Query("lang") lang: String = "en"
    ): WeatherResponse
}
```

Gson automatically stuffs the JSON into these data classes —the field names line up, and the magic happens on its own.

Three UI States: Loading / Success / Failure

The soul of a connected app is "state management." Slow network, no connection, a mistyped city name... the UI needs an answer for the user in every case. Express it elegantly with a sealed class:

```
sealed class UiState {
    object Loading : UiState()                    // spinner showing
    data class Success(val data: WeatherResponse) : UiState()  // success
    data class Error(val message: String) : UiState()          // failure
}

// In the ViewModel:
fun search(city: String) {
    viewModelScope.launch {
        _state.value = UiState.Loading
        _state.value = try {
            UiState.Success(api.getWeather(city))
        } catch (e: Exception) {
            UiState.Error("Can't find that city — try another name?")
        }
    }
```

```
}

// In the UI:
when (val s = state) {
    is UiState.Loading -> CircularProgressIndicator()   // spinner
    is UiState.Success -> WeatherContent(s.data)        // show
    is UiState.Error -> Text(s.message)                 // error
}
```

This Loading/Success/Error three-part pattern is the "all-purpose template" for connected screens. Memorize it now, and every connected app you build later just follows the mold.

Real UI Code: Drawing the Weather

With the states in place, the UI looks like this (key excerpts):

```
@Composable
fun WeatherContent(data: WeatherResponse) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(data.name, fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(
            "${data.main.temp}°C",
            fontSize = 72.sp,
            fontWeight = FontWeight.Light
        )
        Text(
            data.weather.firstOrNull()?.description ?: "Unknown",
            fontSize = 20.sp,
            color = Color.Gray
        )
        Spacer(Modifier.height(16.dp))
        Row {
            Text(" Humidity ${data.main.humidity}%")
            Spacer(Modifier.width(24.dp))
            Text(" Feels like ${data.main.temp}°C")
        }
    }
}
```

A few small details worth learning: firstOrNull() safely grabs the first element (the list may be empty), and ?: "Unknown" provides fallback text —the null-safety principle applies in UI code too. The temperature uses a big 72.sp font to anchor the screen, and the Light weight makes the number look elegant —a classic recipe for weather apps.

UI and Interaction Details

UI and Interaction Details

- Search box: a TextField at the top; type a city name and tap search (set the keyboard's "Search" action so Enter searches);
- Weather card: a large-font temperature (26.5°), the weather description (Sunny), the humidity, and the city name;
- Favorite cities: save the cities you check often into DataStore and query them automatically on next launch —a handy review of Chapter 6;
- Pull to refresh: use PullToRefreshBox in Compose (provided by Material 3) and the feel instantly turns "professional."

Tip: images bring the UI to life

Give each kind of weather an emoji or icon (), mapped with when(description) —three lines of code, and the UI feels twice as warm. You could also load a third-party weather icon library, but for practice, emoji is plenty.

Hands-on: weather app acceptance checklist

(1) A normal search returns the weather; (2) with the network cut off, the app reports the error gracefully instead of crashing (test with airplane mode!); (3) entering a nonexistent city shows a friendly message; (4) two favorite cities survive an app restart; (5) rotating doesn't lose state. Only when all five pass does this project graduate.

Chapter Summary

- Test the API in the browser and understand the JSON before writing code —"previewing" never hurts
- Hide the API key in local.properties; never let it into the codebase
- Retrofit interface + data classes: fields line up, and the JSON parses itself
- The Loading/Success/Error three-part state pattern is the all-purpose tool for connected screens
- No connection, empty input, unknown city —error handling is the passing grade for connected apps

Chapter 56: Polish and Release — Testing, Debugging, and Publishing on Google Play

The app is built —now what? This chapter covers the three things that take you from "it runs" to "it ships" : how to debug efficiently to find bugs, how to write tests to prevent regressions, and how to put the app on Google Play for the whole world to download. Publishing isn't just ceremony —it's the dividing line between a finished work and a product.

The Three Debugging Tools: Logcat, Breakpoints, and the UI Inspector

Tool	What it does	How to use it
Logcat	View system and your own logs	In code: Log.d("TAG","value=$x"); filter by searching TAG in the panel
Breakpoint debugging	Pause the program at a line and inspect variable values	Click next to the line number to drop a red dot, run in Debug mode, and step through
Layout Inspector	Inspect the live UI structure	Menu Tools Layout Inspector; watch the component tree in real time

The three debugging tools

Tip: how do you read an error?

After a crash, Logcat shows a red stack trace. Scan from the bottom up for the first line that starts with your own package name (e.g., com.example.yourapp) —that line is basically the crime scene. First check the "Exception type" (a null pointer? a bad cast?), then the "at ... yourfile: line" . This one trick solves 80% of crashes.

Testing: Insuring Your Code

Manual testing can't be trusted —fix one feature and another quietly breaks. Tests are both "regret medicine" and a "bulletproof vest." Android testing comes in three layers; as a beginner, master the first one first:

Layer	What it tests	Where it lives	Speed
Unit tests	Pure logic (calculations, validation)	app/src/test/	Seconds
Instrumented tests	UI behavior on a real device	app/src/androidTest/	Minutes
Manual testing	A real human feeling out the experience	——	Always needed

The three testing layers

```
// Extract logic like "amount string to number" into a pure function
fun parseAmount(input: String): Double? {
    val v = input.trim().toDoubleOrNull()
    return if (v != null && v > 0) v else null
}

// Testing it (JUnit4)
@Test
fun emptyInputReturnsNull() {
    assertEquals(null, parseAmount(""))
}
@Test
fun negativeReturnsNull() {
    assertEquals(null, parseAmount("-5"))
}
@Test
fun normalValueParses() {
    assertEquals(12.5, parseAmount("12.5"))
}
```

The amount validation and summary calculations in the tracker should all be covered by unit tests. To run them: right-click the test directory Run. Green across the board means you can change code with confidence.

Signing and Packaging: Stamping Your App

Android requires every installable package to be signed —like an official seal proving "this package came from me and hasn't been tampered with."While debugging, Android Studio signs automatically with the debug key; for release you use your own release signature. The process:

1. Menu Build Generate Signed Bundle / APK;

2. Create the keystore file —set a password and keep it safe;

3. Choose release + the AAB format, and the build is done.

Watch out: losing the keystore = losing the app

Updates after publishing must use the same signature, or the store will reject the update. If the keystore file and password are lost, your only option is to publish as a brand-new app —none of your existing users can come along. Back up the keystore and password somewhere secure (encrypted cloud storage + a notebook). It's one of the most expensive lessons an Android developer can learn.

The Full Google Play Publishing Process

Register a developer account

Prepare store assets

Upload the AAB

Fill in the data-safety form

Review (hours to days)

Published

Figure 6: The six steps to publishing on Google Play

- Register: a one-time $25 registration fee; you need a working Google account;

- Assets: an app icon (512×512), screenshots (at least 2), a feature description, and a privacy policy link (required if you collect data);

- Data safety form: honestly declare what data you collect —an offline expense tracker can declare "zero collection";

- Review: a new account's first submission is often reviewed by a human, taking anywhere from hours to days; if it's rejected, fix things per the feedback.

A note for users in mainland China: Google Play requires special network access, and distribution in mainland China usually goes through domestic stores (Huawei, Xiaomi, Tencent MyApp, etc.). Publishing to domestic stores is a similar process, but each store has its own review quirks —the same AAB/APK can be distributed to several stores. As practice, getting through just one store is already a win.

Making Store Assets: First Impressions Are Downloads

The store page is the shop window users see while browsing. Three assets decide the first impression:

- Icon: 512×512; don't just make do with a screenshot. If design isn't your thing, use an AI tool to generate a clean geometric shape —it only counts if it's still recognizable at home-screen size;
- Screenshots: at least 2, ideally 4-6, showing your most core feature scenes; pair each with one line of plain speech ("Log an expense in a second" is ten times better than "An efficient expense-tracking solution");
- Description: the first sentence should make clear "what this is + who it's for"; don't pile up "most professional, most powerful" —store search ranking depends mainly on keywords and download conversion, and honesty works better than exaggeration.

Tip: A/B against your peers

Before publishing, browse the top ten apps in your category and screenshot them for comparison: what style are their icons? What selling points do their screenshots push? How are their titles written? Imitating isn't copying —learn the playbook and swap in your own content. It's the fastest cold-start lesson in every product's life.

A quick checklist for performance and crash reporting

Before publishing, these two things are each worth half an hour:

Four performance self-checks

1. Use LazyColumn for lists: never lay out long lists with a plain Column —lazy loading is Android performance lesson one;
2. Don't shove full-size images in: use the Coil image library to auto-scale and cache; a single 5MB original is enough to choke a low-end phone;
3. Keep heavy work off the main thread: database, network, and large files all belong in coroutines;
4. Measure with the Profiler: Android Studio's built-in profiler shows memory, CPU, and battery use —measure first, then optimize; don't go by feel.

Crash reporting

Not crashing during your tests doesn't mean it won't crash on users' phones. Before publishing, add Firebase Crashlytics (Google's official, free tool): a few lines of config, and

when a user hits a crash it automatically reports the stack trace, so from the console you can see "which device model, which version, and which line it crashed on." Publishing without crash monitoring is driving with your eyes closed.

Hands-on: "deliver" the tracker once

(1) Add 3 unit tests for logic like parseAmount; (2) use Logcat to catch a crash you created on purpose and fix it; (3) generate a release-signed APK and send it to a friend to install (that's the real "finished work" ceremony); (4) if you have energy to spare, register a developer account and actually submit the tracker for publication. The first three can be done tonight.

Chapter Summary

- The three debugging tools: Logcat for logs, breakpoints for variables, Layout Inspector for the UI
- Reading crashes: scan the stack from the bottom up for the first line with your package name
- Three testing layers: unit (seconds —learn this first) / instrumented / manual
- Publishing requires signing: back up the keystore and password; lose them and the app "changes identity"
- Six publishing steps: register assets upload the AAB data safety form review live

Chapter 57: The Road Ahead — Permissions, Notifications, and Modern Architecture

Being able to build an app is only the start. This chapter covers the hurdles between "can build" and "builds well": how to ask for permissions gracefully, how to send notifications without annoying people, how to lift your architecture another level, and —in 2026 —how to pair with AI to write Android code. Every one of these will show up in your real projects right away.

Runtime Permissions: Knock First, Then Come In

Camera, location, contacts, notifications... for these "sensitive zones," Android requires the app to ask the user at runtime (Android 6 and up). The rules: declare first (the Manifest), then request (in code), and degrade gracefully if the user refuses. In Compose, the easiest path is rememberPermissionState from the official Accompanist library:

```
// First declare it in the Manifest:
// <uses-permission android:name="android.permission.CAMERA" />

@Composable
fun CameraButton() {
    val permission = rememberPermissionState(
        permission = android.Manifest.permission.CAMERA
    )
    Button(onClick = {
        when {
            permission.status.isGranted -> openCamera()
            permission.status.shouldShowRationale -> showExplainDialog()
            else -> permission.launchPermissionRequest()
        }
    }) { Text("Take photo") }
}
```

Note: what if you're permanently denied?

Once the user checks "don't ask again," launchPermissionRequest won't pop up anymore. At that point, guide them to turn it on manually in system settings (jump to the app's settings page with an Intent), and explain in the UI "what this permission is for." A kinder tone makes users far more willing to grant it —the words you use in a permission request are worth writing carefully.

Notifications: Saying Useful Things at the Right Time

Notifications are the channel apps use to reach users —and the feature users block first. Three disciplines for notifications: only send useful things, let users turn them off, and never pester at midnight. The basic usage:

```
// On Android 13+ you must request POST_NOTIFICATIONS first (see the previous
    section)

// 1. Create the notification channel (required on Android 8+)
val channel = NotificationChannel(
    "reminder", "Reminder", NotificationManager.IMPORTANCE_DEFAULT
)
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(channel)

// 2. Send the notification
val notification = NotificationCompat.Builder(this, "reminder")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Expense reminder")
    .setContentText("You haven't recorded today's expenses yet")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS) == PackageManager.
    PERMISSION_GRANTED
) {
    manager.notify(1, notification)
}
```

For scheduled tasks (like "remind me to log expenses at 20:00 every day"), use WorkManager —the officially recommended "background scheduling butler" that guarantees tasks run at the right moment, with no need to fuss over battery optimization yourself. For alarm-type needs, it's always the right choice.

Architecture, Next Level: Dependency Injection and Modularity

As the project grows, two "sweet troubles" come knocking:

- Messy object creation: every screen builds its own database, its own API —use Hilt (a dependency injection framework) to manage them centrally, so "whoever needs something just asks and it's handed over";
- Code piling into a mountain: single-module projects get slower and slower to change — split into multiple modules (app / core / feature-xxx) so each feature is developed and tested independently.

Tip: don't over-engineer

Hilt and modularity are both good tools, but don't rush them into a solo project. For the tracker, a single module plus manual dependency creation is entirely enough. Bring them in when the code passes tens of thousands of lines, compilation starts to slow down, or several people begin collaborating —and only as needed. Tools serve goals; they're not for showing off —that's also the "simplicity first" principle from our AI coding book.

The 2026 Way: AI-Assisted Android Development

In 2026, AI is deeply embedded in the Android development workflow. If you want to speed up, this combination is well proven:

Stage	How AI helps	Your responsibility
Writing code	Gemini built into Android Studio / third-party AI plugins for completion and generation	Describe requirements clearly; review the output
Looking things up	Feed official docs and error messages to AI for interpretation	Verify key conclusions against the official docs
Writing tests	Have AI generate cases in the three categories "normal/edge/exception"	Confirm the cases match the intended behavior
Fixing bugs	Paste the stack trace + relevant code; AI locates and fixes	Understand the change; run tests to verify

Four scenarios for AI-assisted Android development

The full methodology —how to give AI instructions, how to review its work, how to avoid pitfalls —spans 11 complete chapters in Part 7 of this book, "AI Coding in Practice" (Chapters 37-47), which I strongly recommend reading alongside: leave the Android syntax to this part, and the AI collaboration know-how to Part 7.

Compose Performance Tuning and Debugging Tools

Writing Compose is a joy, but "too much recomposition" can also lag. Three practical techniques:

- Stabilize parameters: add @Immutable to data classes or use derivedStateOf to keep unrelated state changes from triggering large recompositions;
- Remember to use keys: items(list, key = { it.id }) makes list additions and deletions touch only the items that should change —without a key, deleting one item may redraw the whole list;
- Layout Inspector's Compose mode: watch each Composable's recomposition count in real time —you can see at a glance who recomposes too often, and optimize those hotspots.

Also turn on the Compose compiler reports (metrics) to generate a report of "which components are unstable" —on the road to mastery, data is always more reliable than gut feeling.

Hands-on: add the "trio" to the tracker

(1) Add a "remind me to log expenses at 20:00 every day" feature to the tracker (WorkManager + a notification channel + the Android 13 notification permission); (2) switch the database and API creation to Hilt injection; (3) try using an AI tool to speed-build the "monthly filter" feature, then check its output with the three-layer acceptance method. Finish all three and your project starts to look like a proper, professional operation.

Chapter Summary

- Sensitive permissions: declare in the Manifest + request at runtime + guide gracefully after a refusal
- Three notification disciplines: only useful things, can be turned off, no pestering; channel + permission, both required
- Use WorkManager for scheduled tasks; don't reinvent the wheel
- Hilt manages objects, modularity manages code —hold off on both for small solo projects
- Four AI-assisted scenarios: writing code, research, writing tests, fixing bugs; the methodology is in Part 7 (Chapters 37-47)

Chapter 58: Quick Reference and Resources

This chapter is a quick-reference handbook you can use entirely on its own. When you can't quite remember a bit of syntax or need to make a fast decision, flip straight here. Treat it as "pocket cards" : common code snippets, frequently asked questions, and learning resources, page by page.

Common Code Cheat Sheet

What you want to do	Key code
Hold a mutable state	var x by remember { mutableStateOf(initialValue) }
Keep state across rotation	rememberSaveable { mutableStateOf(initialValue) }
Vertical/horizontal layouts	`Column { }/Row { }`
Lazy-loading list	LazyColumn { items(list) { ... } }
Jump between screens	navController.navigate("route")
Do waiting work in a coroutine	`viewModelScope.launch { ...}`
Observe a data stream	`val list by flow.collectAsState()`
Save small settings	context.dataStore.edit { it[KEY] = value }
Save structured data	Room's trio: @Entity / @Dao / @Database
Fetch data over the network	Retrofit interface + suspend function
Send a notification	NotificationChannel + NotificationCompat.Builder
Log messages	`Log.d("TAG", "message")`

Android development cheat sheet (August 2026 edition)

FAQ

Q1: I can't code at all —how long does learning Android development take?

Follow this book's path: weeks 1-2, set up the environment and learn Kotlin basics; weeks 3-4, learn Compose and build screens; weeks 5-6, finish the expense tracker project; weeks 7-8, complete the weather project. Two months from zero to independently building small apps — provided you put in 1-2 hours a day and actually type code instead of only reading.

Q2: Do I still need to learn Java?

No. New projects, new tutorials, and new jobs are all Kotlin-first. You only run into lots of Java when "maintaining a ten-year-old project." After learning Kotlin, picking up Java for work goes fast —the two share the same underlying virtual machine ecosystem.

Q3: Emulator or real device?

For development and debugging, the emulator is convenient (snapshots, many device types), but acceptance must pass on a real device —touch feel, performance, real networks, and sensors can't be replaced by an emulator. If your budget allows, buying a mid-range phone as a test device is the best-value investment.

Q4: Are Android development jobs easy to find?

The 2026 market: entry-level jobs that only involve "writing UI" are fiercely competitive, but developers who "know Kotlin + understand architecture + have a shipped app" are still in demand. While you're learning, the best resume is that app you built sitting in the store.

Q5: What do I do when an error has me stuck?

The golden three steps: (1) search the first line of the error, word for word, on StackOverflow or a search engine; (2) check the corresponding page in the official docs; (3) if that still fails, package up "expected behavior + what actually happens + the full error + relevant code" and ask AI or the community. The vast majority of problems are solved at step one.

Recommended Learning Resources

Type	Resource	Why it's recommended
Official	developer.android.com	The official Android documentation; the Compose/Kotlin tutorials all live here —the top authority
Official	The Kotlin website (kotlinlang.org)	Official language docs + an online playground
Official	Google Codelabs	Hands-on, step-by-step codelabs; maximum beginner-friendliness
Community	Stack Overflow (the android tag)	The first stop when errors strike
Community	Juejin / CSDN Android sections	Lots of practical articles in Chinese; check freshness (look at the date first)
Video	The Google Developers official channel	New-technology talks from each year's developer conference
Companion	Part 7, "AI Coding in Practice" (Chapters 37-47)	The complete know-how for AI-assisted Android development

Android development learning resource list (August 2026)

Common Gradle Commands Cheat Sheet

Clicking buttons is usually enough, but the command line is sometimes more efficient (especially when pairing with AI tools or CI pipelines):

Command	What it does
`./gradlew assembleDebug`	Build a debug APK (on Windows, drop the ./)
`./gradlew assembleRelease`	Build a release (signing must be set up first)
`./gradlew test`	Run all unit tests
`./gradlew lint`	Static analysis: find potential bugs and sloppy code
`./gradlew clean`	Clean out build outputs (the compile-time version of "turn it off and on again")
`./gradlew :app: dependencies`	View the dependency tree (the magic tool for hunting down version conflicts)

Common Gradle commands cheat sheet

Tip: when builds misbehave, clean first

For "it worked yesterday but not today" hauntings, six or seven times out of ten it's a build cache in disarray. Rebuild after ./gradlew clean and most of the voodoo disappears. If it still misbehaves, restart Android Studio —and pair that move with File Invalidate Caches.

Closing: From Your First App to Your Nth App

Closing: From Your First App to Your Nth App

Looking back: you went from "not knowing what an Activity is" to setting up the environment, learning Kotlin, drawing screens with Compose, connecting your app to a database and the network, and on to testing, packaging, and publishing —you've already walked the hardest zero-to-one stretch of this road.

The tech world changes fast: frameworks upgrade, tools get replaced, and AI takes over more and more of the "typing" work. But one thing never goes out of date: your ability to take a fuzzy idea, break it into clear steps, and build it one step at a time. Tools are amplifiers; thinking is the foundation. Amplifiers can be swapped out; the foundation must not shake.

You're no longer just "someone who knows how to use a phone." You can turn the ideas in your head into real apps running on 300 million Android devices. From today, create with your keyboard —that's the romance of being an Android developer.

References

1. Android Developers. (2026). Android Developer Guides and Jetpack Compose Documentation. developer.android.com.

2. Kotlin Foundation. (2026). Kotlin Language Documentation. kotlinlang.org.

3. Android Open Source Project. (2026). Android Platform Version History and API Levels.

4. Google Play. (2026). Play Console Help: Publishing and App Signing.

5. AndroidX Release Notes. (2026). Room / DataStore / Navigation / WorkManager releases.

6. Square. (2026). Retrofit Documentation. github.com/square/retrofit.

Part IX: iOS Development in Practice (Chapters 59-69)

Chapter 59: Getting to Know iOS —Apple's Ecosystem and Developer Ecosystem

Congratulations on this decision —going from "someone who uses an iPhone" to "someone who writes apps for the iPhone." Before you start, let's spread the map out flat: where did iOS come from? What does Apple's "walled garden" mean for developers? What happens between your fingers hitting the keyboard and an app landing on a user's phone?

iOS's Origin Story: One Phone Changed an Industry

In January 2007, Jobs pulled the first-generation iPhone out on stage. At the time it didn't even support third-party apps —not until 2008, when Apple launched the App Store and the iPhone SDK, letting developers worldwide write software for the iPhone. That was the moment the "app economy" was born: today, the App Store hosts millions of apps and has paid developers a cumulative total of hundreds of billions of dollars.

The system itself has evolved too: it was first called iPhone OS and renamed iOS in 2010; the version number marched up to iOS 17 (2023), and then in 2025 Apple switched to year-based naming —iOS 26 shipped in 2025, and as of August 2026 the current version is iOS 27. The naming changes, but one thing never has: iOS users' willingness to pay and the platform's reputation for software quality are widely considered the best in mobile —the same app usually earns noticeably more on iOS than on Android.

Milestone	Year	What mattered to developers
App Store launches	2008	Third-party development opens up
Swift is released	2014	The modern language for iOS development arrives
SwiftUI is released	2019	Declarative UI framework; the main line of this book
iOS 26 (the year of the renaming)	2025	Naming follows the year; Apple Intelligence fully integrated
iOS 27	2026	Current latest version (this book's baseline)

A timeline of iOS development (as of August 2026)

The Walled Garden: The Apple Ecosystem's "Rules" and "Rewards"

The Apple ecosystem is often described as a "walled garden"—lots of rules, tightly enforced: apps must pass review before they can be listed, development requires a Mac and Xcode, and distribution goes mainly through the App Store. Sounds stifling? But for developers, those rules buy rewards worth real money:

- Almost no device fragmentation: there are only a handful of iPhone models, and screen sizes and system versions are highly uniform—adapt to just 5 models and you cover 95% of users (on Android you might have to cover dozens);
- Extremely high system adoption: within a few weeks of a new release, 70–80% of users have upgraded, so you can confidently use new APIs without worrying about "systems from five years ago";
- Users are happy to pay: App Store users are more willing to spend money on apps and in-app purchases;
- Strict review, but a clean store: knockoffs and junk apps are rare, so good products are easier to get seen.

Note: Let's Be Clear About the Cost First

iOS development has hard requirements: you must have a Mac (Xcode runs only on macOS), and publishing requires a $99/year developer account (a free personal developer account lets you test on a real device locally, but you can't publish to the store). Think

through that investment before you set off—the good news is that a free account is completely enough for the learning stage, as Chapter 60 will cover.

Which Language to Write In: Objective-C or Swift?

iOS development history has two "lead actors" as well. Objective-C was the founding veteran—used from 2008 to 2014; Swift is the new language Apple released in 2014, and it has evolved all the way to Swift 6 (released in 2024, bringing strict concurrency checking). By 2026, nearly all new projects use Swift.

A bit of honest advice for absolute beginners: go straight to Swift and don't even look at Objective-C—unless your future job means maintaining legacy code from ten years ago. Swift's advantages: clean syntax (none of the C-era baggage of semicolons and header files), an Optional mechanism that wipes out "null pointer crashes" at the language level, seamless integration with SwiftUI, and Swift Playgrounds on the iPad for learning while you play.

Objective-C style (ancient relic)

```
NSString *name = @"Tom"; [name stringByAppendingString:@"Hello"]; // The classic
    trio: square brackets + pointers + semicolons
```

Swift style (modern and clean)

```
var name = "Tom" name += "Hello" // As plain as everyday language
```

How Is an App Born?

Like every platform, iOS apps come off an assembly line. But the Apple version adds two extra "security checks":

Idea

Design (UI + features)

Write code (Swift + SwiftUI)

Compile and sign

Test (emulator + real device)

Apple review

Published on the App Store

Figure 1: The full journey of an iOS app from idea to the App Store (note the extra "review" step)

Key concepts in the pipeline:

- Signing: every iOS app must be "signed" before it can be installed—proof that "this package came from you and hasn't been tampered with." During the learning stage,

Xcode signs automatically with your Apple ID; publishing uses a developer certificate. Chapter 67 covers the details.

- Simulator: an iPhone virtualized on your Mac—the main arena for development and debugging. It runs on the Mac's CPU, so it's remarkably smooth—a big step up from Android emulators.
- TestFlight: Apple's official "beta distribution" tool—before release, you send test builds to up to 10,000 testers, who install and try them with the TestFlight app. It's an elegance unique to the iOS ecosystem.

The Full iOS Tech Stack: Your "Weapon List"

Layer	**Mainstream choice (2026)**	**What it's for**
Language	Swift 6	The language for writing logic
Interface	SwiftUI + Apple's design guidelines (HIG)	Drawing interfaces in code (the main line of this book)
Architecture	MVVM + @Observable	Keeping interface and logic apart
Data storage	SwiftData (database) + UserDefaults (settings)	Saving data on the phone
Networking	URLSession + async/await + Codable	Talking to servers
Build tools	Xcode Build System / xcodebuild	Turning code into an installable package
Development environment	Xcode (macOS only)	Apple's official IDE

The mainstream iOS tech stack at a glance (2026)

Notice anything? This list is quite a bit shorter than Android's—because Apple "takes care of" a lot of things for you. The closed ecosystem costs you freedom and pays you back in peace of mind: no agonizing over which networking library or which database to pick—the official option is usually the best answer.

Hands-on: Look Up Your Phone's Apple Profile

On your iPhone, open [Settings] > [General] > [About] and jot down three things: the iOS version, the model name, and the available storage. Then wander around the "Today" tab of the App Store and pay attention to the screenshots and copy of the featured apps—starting today, you're browsing the store with a "developer's" eyes instead of a "user's".

Chapter summary

- 2008: the App Store launches and the "app economy" is born; as of 2026 the current version is iOS 27
- The walled garden's ledger: plenty of rules, but fewer devices, fresher systems, paying users, and a clean store
- Hard requirements: a Mac is a must; publishing needs a $99/year developer account; a free account is enough for the learning stage
- Go straight to Swift for the language; Objective-C belongs to legacy code only
- The pipeline has one extra stop compared to Android: Apple's review; TestFlight is the go-to tool for beta distribution
- The standard 2026 stack: Swift 6 + SwiftUI + MVVM + SwiftData + URLSession + Xcode

Chapter 60: Setting Up Your Development Environment—Xcode and Your First Project

A craftsman is only as good as their tools. In this chapter we'll set up the "kitchen": install Xcode, get the simulator running, create the first iOS project of your life, and then take the project structure apart so you understand every piece. Once the environment is ready, you'll see your own code turn into a button you can tap for the very first time—a moment many people remember for the rest of their lives.

Getting a Mac Ready

First, the entry requirement: iOS development must happen on macOS. If you don't have a Mac, you have three options: buy one (even a used Mac mini or MacBook Air is enough), rent a cloud Mac for the short term, or read Part 8, "Hands-On Android Development," first (you can do that on Windows).

Spec	**Minimum**	**Comfort zone (recommended)**
Chip	Apple Silicon (M1 or later)	M2 or newer
Memory	8GB	16GB
Disk	Reserve 40GB+ (Xcode itself is chunky)	256GB to start
System	The latest macOS	Keep auto-updates on

Reference specs for a Mac development machine (August 2026)

Tip: You Can Start Learning the Language Without a Mac

The free Swift Playgrounds app on the iPad teaches the basics of the Swift language (no Mac needed); you can even install it on the iPhone. Get comfortable with the Chapter 61 language material in Playgrounds first, and you'll take off the moment you get a Mac. This

is the classic "sharpen the axe before you chop" principle.

Installing Xcode

Xcode is Apple's official integrated development environment, and it's free. Three steps to install:

1. On your Mac, open the App Store, search for "Xcode," and click Install—heads up, it's several GB, so pick a time when your connection is fast;
2. On first launch after installing, accept the agreement and let the components install (it happens automatically);
3. In the menu bar, go to Xcode > Settings > Locations and make sure Command Line Tools is selected—the command line tools are a prerequisite for many scripts and third-party tools.

Note: The Version and System "Matching Game"

A new Xcode usually demands the latest macOS, while older Macs get stuck on older versions. When you publish to the App Store, Apple requires building with a relatively recent Xcode (usually with a grace period). So don't put off system updates too long—iOS developers are among the busiest "new version chasers" out there.

Creating Your First Project

Open Xcode > Create New Project > choose iOS > the App template, then fill in:

Setting	What to enter	Description
Product Name	MyFirstApp	App name
Team	Your Apple ID (free)	Used for signing; a personal account is fine
Organization Identifier	com.example	Combined with the Product Name to form the bundle identifier
Interface	SwiftUI	The main framework used in this book
Language	Swift	No explanation needed

The five things to fill in when creating a project

After you click Create, Xcode generates a project that runs right away. Pick a simulator in the top-left corner (say, iPhone 17) and click > to run—a few seconds later, a window saying "Hello, world!" appears on the simulator's screen. Your first iOS app has been born.

Project Structure, Dissected: What Each File Does

On the left is the project navigator—as a beginner, you only need to know these:

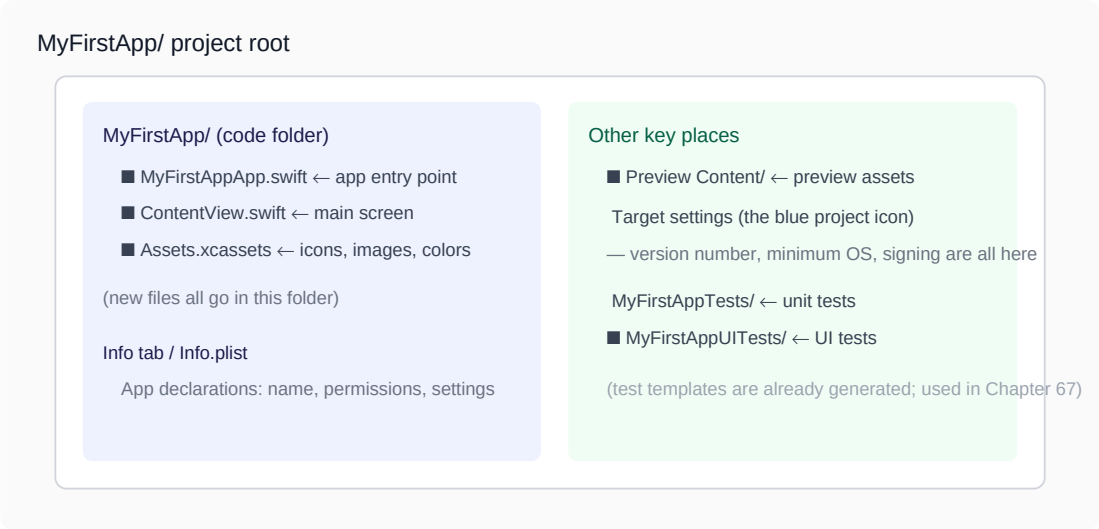


Figure 2: The Xcode project structure every beginner must know

- MyFirstAppApp.swift: the program's entry point. In SwiftUI it's only a few lines—"launch, then show the main screen";

- ContentView.swift: the main screen view—everything you're looking at right now lives here;

- Assets.xcassets: the resource warehouse—icons, images, and colors all go here;

- Target settings: click the blue project icon on the left to get in. The version number, the minimum iOS version, and signing are all in there. The app's display name and icon are configured here too.

Testing on a Real Device: Install the App on Your iPhone

Xcode Efficiency Tricks: Five "Shortcut Power-Ups" for Beginners

Get fluent with these and you'll look "very professional":

Action	Shortcut	What it does
Run	CmdR	Build and run
Open/toggle the canvas preview	OptionCmd	The what-you-see-is-what-you-get switch
Re-indent	CtrlI	Automatically tidy up the indentation of the selected code
Quick Open	CmdO	Type a file name and it opens instantly (the most used!)
Comment/uncomment	Cmd/	Switch code on and off while debugging

High-frequency Xcode shortcuts (the beginner's five)

Two more hidden perks: multiple simulators at once (run an iPhone and an iPad side by side to compare layouts) and the snippet library (drag a few lines of frequently used code into the Snippets panel on the right, then insert them with one click next time)—use these efficiency tools early, and the fluency will follow naturally.

Testing on a Real Device: Install the App on Your iPhone

The simulator is nice, but a real device is the destination. You can test on a real device even with a free Apple ID:

1. Connect your iPhone with a cable (or debug wirelessly over the same Wi-Fi);

2. Pick your phone from the device list at the top of Xcode; the first time, you'll be asked to sign in with your Apple ID and signing gets configured automatically;

3. On the iPhone, go to [Settings] > [General] > [VPN & Device Management] and trust your developer certificate;

4. Click >, and the app lands on your real device.

Note: The Free Account's "Seven-Day Limit"

An app signed with a free Apple ID stops working after 7 days (just run it again to renew). This is Apple's anti-abuse mechanism, and it's completely fine for the learning stage; a paid developer account ($99/year) has no such limit—Chapter 67 has the details.

Try It Yourself + First Aid for Common Errors

Try It Yourself + First Aid for Common Errors

Open ContentView.swift, change the text inside Text(“Hello, world!”) to your name, and press CmdR to run again—a few seconds later, your name shows up on the simulator. Congratulations: you’ve completed your first full “edit code build install run” loop.

Symptom	Common cause	The fix
Signing error	No Team selected, or an expired certificate	In Target > Signing, check your account; with a free account, just run again
Simulator shows a black screen / won’t open	Corrupted simulator image	Menu: Device > Erase All Content; or delete the simulator and add it again
Xcode downloads slowly / won’t install	App Store network hiccup	Try a different time of day; or download the .xip offline installer from the Apple developer website

First-aid table for beginner environment errors

Hands-on: Finish Your ”Developer Initiation” Checklist

1) Install Xcode and get the simulator running; 2) run your app on a real device once and take a screenshot to mark the occasion; 3) change the Hello text to your name; 4) go to the Target settings and change the app’s display name to “My First App.” Once these four steps are done, you’re officially someone with a creation of your own.

Chapter summary

- Hard requirements: a Mac + Xcode; no Mac yet? Learn the language first with Swift Playgrounds on the iPad
- Install Xcode from the App Store; new versions demand new systems, so don’t put off updates
- The five essentials of a new project: name, Team, organization identifier, SwiftUI, Swift
- The project mantra: all your code lives in the MyFirstApp/ folder; Target settings manage the version and signing
- Real-device testing with a free Apple ID has a 7-day signing limit; a paid account has no such limit
- The developer’s daily routine = edit code CmdR see the result, over and over

Chapter 61: Swift Quick Start—Apple's Top Development Language

In this chapter we officially start learning the language. Don't worry—Swift is famous for being "beginner-friendly," and you don't need to memorize a grammar book: once you've mastered 20% of the syntax, you can read and write 80% of everyday code. Look up the rest as you go, while working on projects. Remember our learning principle: understanding beats memorizing, and being able to use it beats being an expert.

Variables and Types: Giving Data a Name

A program is just "data + logic." A variable is simply a name you give to a piece of data so you can store it. Swift declares them with let and var:

```
let name = "Alex"        // let: constant (the recommended default)
var age = 18             // var: variable (can change later)

age = 19                 // <i class="fas fa-check-circle mr-1"></i> var can change
// name = "Sam"         // <i class="fas fa-times-circle mr-1"></i> let can't be
    changed — compile error

let score: Int = 100     // you can also declare the type explicitly
let pi = 3.14            // omitted, it's still inferred automatically

print("Hello, \(name)")   // string interpolation: backslash + parentheses
```

Memorize the common types with one table:

Type	What it holds	Example
Int	Whole numbers	42
Double	Decimals	3.14
String	Text	"Hello"
Bool	True/false	true / false
[Int] (an array)	A group of values	[1, 2, 3]

Common basic types in Swift

Tip: The let-First Principle

An iron rule of the Swift community: use let by default, and only use var when it's truly necessary. If you use var where let would do, Xcode will warn you with yellow text. This habit saves you from a whole family of "variable got changed by accident" bugs—the less your code changes, the easier it is to reason about.

Optionals: Swift's "Anti-Crash Charm"

This is the most important concept in Swift, bar none. In one sentence: when a value might "not exist," Swift forces you to deal with it—wiping out the industry's nightmare of "null pointer crashes" at the language level.

```
var name: String = Alex   // a regular type: never allowed to be nil
// name = nil                // <i class="fas fa-times-circle mr-1"></i> compile
    error — it stops you outright

var nickname: String? = nil  // add a question mark: nil is allowed (an optional)
nickname = "Al"

// To use an optional, you must "unwrap" it:
if let n = nickname {        // if let: safe unwrapping
    print("Hello, \(n)")
} else {
    print("No nickname yet")
}

let len = nickname?.count ?? 0   // ?? provides a default (the nil-coalescing
    operator)
print(len)
```

Note: ! Is a "High-Risk Operation"

Writing nickname! in code (force unwrapping) is like shouting at the compiler, "I guarantee it's not nil" —guess wrong once, and the app crashes on the spot. Beginner rule: use if let / ?? wherever possible, and never use !. When Xcode autocompletes your code it occasionally slips in a !—remember to swap it out.

Functions and Closures: Packing Logic into "Buttons"

A function is a named piece of logic, like a button on a remote control. A closure is a "function with no name," one you can hand to someone else on the spot:

```
// Functions: name first, then parameters and return type
func greet(name: String) -> String {
```

```
    return "Hello, \(name)!"
}
print(greet(name: "Alex"))     // the argument label is required at the call site

// Closures: defined inline, quick and direct
let numbers = [1, 2, 3, 4]
let doubled = numbers.map { $0 * 2 }   // $0 = the current element
// [2, 4, 6, 8]

let big = numbers.filter { $0 > 2 }    // filter
// [3, 4]
```

Closures are everywhere in SwiftUI: the braces after Button("Tap me") { ... } are a closure. Please commit these "collection combo moves" to memory—map, filter, and sorted —because 90% of the list handling you'll ever do is just combinations of them.

Structs and Protocols: Swift's "LEGO" System

Swift's official preference is clear: prefer structs, and use classes only when you need inheritance. A struct is a "value type" —each copy lives its own separate life, which naturally cuts out a whole family of bugs:

```
// A struct: a box for data
struct Record {
    var id = UUID()
    var amount: Double
    var category: String
    var date: Date

    // Structs come with a built-in "memberwise initializer": you can create one
    directly
}
let r = Record(amount: 25.5, category: "Food", date: Date())

// A protocol: a contract for "which abilities are required"
protocol Identifiable {
    var id: UUID { get }
}
struct Record: Identifiable { ... }   // conforming to a protocol = promising to
    fulfill its requirements
```

Almost everything in SwiftUI is a struct (View is a protocol). It's fine if that sentence doesn't click yet—you'll run into it every day in Chapter 62. In the world of Swift, embrace the struct first.

A Syntax Cheat Sheet for Absolute Beginners

What you want to do	How to write it
Define a constant/variable	`let x = 1`/`var x = 1`
String interpolation	”Hello, \(name)”
Define a function	func name(parameter: Type) -> ReturnType { ... }
Making a decision	if condition { ... } else { ... }
Looping through items	for x in array { ... }
Safe unwrapping	if let x = optionalValue { ... }
Default value	x ?? defaultValue
Define a struct	`struct X { var a: Int; var b: String }`
Closures	`{ $0 * 2 }`

Swift syntax cheat sheet (beginner edition)

Hands-on: Play with Swift in Playgrounds for 15 Minutes

If you have a Mac, open Xcode > Playground; if not, use Swift Playgrounds on the iPad or a web-based playground. Run every piece of code in this chapter, fiddle with the numbers and text, and deliberately create a few errors to see how Xcode reacts. These 15 minutes of “play first, learn after” will make starting Chapter 62 twice as easy.

Chapter summary

- let first, var with care; types are inferred automatically, but you can also declare them explicitly after a colon
- Optionals are the soul of Swift: if a value might be missing, you must handle it—if let / ?? are your protective charm, and ! is a high-risk move
- Function calls carry argument names; a closure = a function without a name, and they’re everywhere in SwiftUI
- Prefer structs and conform to protocols; almost everything in SwiftUI is a struct
- 20% of the syntax does 80% of the work: learn to use it first, master it later

Chapter 62: Building Interfaces—Getting Started with SwiftUI

The interface is everything the user sees. SwiftUI is the declarative interface framework Apple launched in 2019—you only need to "declare" what the interface looks like, and the system automatically takes care of rendering, dark mode, and every kind of screen. By 2026, SwiftUI is the clear mainstream for new projects. In this chapter, we learn to "paint."

The SwiftUI Idea: Declarative, Not Imperative

Old-style interface development is "imperative": you give the system orders one step at a time—"create a button place it at (20,30) set the color." SwiftUI is "declarative": you simply describe "there's a red button on the screen that says 'Tap me'," and the system takes care of how to draw it and when to redraw it.

```
import SwiftUI

struct ContentView: View {
    var body: some View {      // body: describes "what the screen looks like"
        VStack {               // vertical stack
            Text("Hello, world!")
                .font(.largeTitle)      // modifier: large title
            Button("Tap me") {
                print("clicked")
            }
            .buttonStyle(.borderedProminent)  // solid blue button
        }
        .padding()
    }
}
```

Key point: a view = a struct that conforms to the View protocol, the body property describes the interface, and .modifier() chains "apply the makeup."That's the entire foundation of SwiftUI.

The Basic Views: Text, Buttons, Text Fields, Images

View	What it does	Example
Text	Displays text	Text("Hello")
Button	Button	Button("Tap me") { action }
TextField	Text field	TextField("Placeholder", text: $text)
Image	Displays an image	`Image(systemName: "star.fill")`

SwiftUI's four basic views

Notice Image(systemName:)—Apple ships a built-in icon library called SF Symbols (thousands of free icons); a name like "star.fill" instantly shows a solid star. This is a hidden perk of iOS development: you almost never have to hunt for icon assets.

The Layout Trio: VStack, HStack, ZStack

- VStack: lines things up vertically (like queuing for lunch);
- HStack: lines things up horizontally (like sitting side by side);
- ZStack: stacks things on top of each other (like a pile of playing cards—perfect for badges and overlays).

```
VStack(alignment: .leading, spacing: 12) {
    HStack {
        Text("Bill")
        Spacer()                 // flexible space: pushes the right side to the
    edge
        Text("-25.5 yuan").foregroundColor(.red)
    }
    Divider()                    // divider
    Text("Note: lunch")
}
.padding(16)                     // padding on all sides
.background(Color(.systemGray6)) // light gray background
.cornerRadius(12)                // rounded corners

// Modifier order matters: padding first, then background — the background
    excludes the padding
```

Note: Modifier Order Matters

padding(16).background(gray) and background(gray).padding(16) produce different results —the former "spaces first, then paints" (the background excludes the padding), the latter "paints first, then spaces" (the background includes the padding). This is the source of beginners' most common "mystery white edge." When in doubt, swap the order and the preview will tell you the answer instantly.

State and Binding: Bringing the Interface to Life

A static interface is dull. SwiftUI's rule: the interface always faithfully reflects the state —when the state changes, the interface redraws automatically.

```
struct CounterView: View {
    @State private var count = 0   // @State: when this value changes, redraw

    var body: some View {
        VStack {
            Text("You tapped \(count) times")
            Button("Tap me +1") {
                count += 1          // change the state; the UI refreshes
    automatically
            }
        }
    }
}
```

A text field needs "two-way binding" —the $ symbol hands over both the value and the right to change it:

```
struct InputView: View {
    @State private var text = ""

    var body: some View {
        TextField("Type something", text: $text)  // $ = two-way binding
            .textFieldStyle(.roundedBorder)
        Text("You typed: \(text)")   // updates live as you type
    }
}
```

User action

State changes (count += 1)

SwiftUI notices the change

UI redraws automatically (body recomputed)

Figure 3: SwiftUI's one-way "state interface" data flow

Previews, Theming, and Animation: SwiftUI's "Power-Ups"

- The #Preview macro: add #Preview { ContentView() } at the bottom of a file, and the canvas on the right renders in real time—change the code and see the result immediately;

- Dark mode for free: use system colors (Color(.systemGray6), .primary) instead of hard-coded ones, and dark mode adapts automatically without a single extra line;

- Animation in one line: withAnimation { count += 1 } or add .animation(.default, value: count) to a view—buttery-smooth transitions.

```
struct AnimatedHeartView: View {
    @State private var liked = false

    var body: some View {
        Button {
            withAnimation(.spring(duration: 0.4)) {
                liked.toggle()
            }
        } label: {
            Image(systemName: liked ? "heart.fill" : "heart")
                .font(.system(size: 48))
                .foregroundColor(liked ? .red : .gray)
                .scaleEffect(liked ? 1.3 : 1.0)
        }
    }
}
```

Tip: Function First, Animation Later

A classic beginner mistake is "polishing animations before the interface even works." Get the functionality running first, then add motion at the end—animation is the icing, function is the cake. That's also our standing "simplicity first" principle.

Hands-on: Build a "Mood Button"

Common Modifiers Cheat Sheet: One Page in Hand, No Interface Worries

What you want to do	Modifier
Spacing	`.padding(16)`
Rounded corners	`.cornerRadius(12)`
Background color	`.background(Color(.systemGray6))`
Font and size	`.font(.title.bold())/.font(.system(size: 20))`
Foreground color (text/icons)	`.foregroundColor(.red)`
Fill the width	`.frame(maxWidth: .infinity)`
Fixed size	`.frame(width: 100, height: 44)`
Alignment	`VStack(alignment: .leading)`
Opacity	`.opacity(0.6)`
Shadow	`.shadow(radius: 5)`
Tap gesture	`.onTapGesture { ...}`
Keyboard type	`.keyboardType(.decimalPad)`

Common SwiftUI modifiers cheat sheet

No need to memorize any of this—this page is for looking things up. The ones you'll use most are actually just the first four: padding, rounded corners, background, and font.
Turn the template project into a mood switcher: one button that cycles through happy sad angry with each tap, while the text and background color change along with it. Hint: store the three pairs (text + color) in an array, keep the index in @State, and on each tap use (index + 1) % 3. Send it to a friend when you're done and collect your first round of "wow, you can make apps now."

Chapter summary

- SwiftUI is declarative: you describe "what it looks like," and the system handles drawing and redrawing
- A view = a struct conforming to the View protocol; body describes the interface; modifiers chain on like "makeup"
- The four basic views: Text / Button / TextField / Image; the SF Symbols icon library is free and all-you-can-eat
- The layout trio: VStack stacks vertically, HStack horizontally, ZStack in layers; Spacer is flexible space
- @State brings the interface to life; $ feeds two-way bindings into text fields

- #Preview is WYSIWYG; system colors adapt to dark mode automatically; withAnimation adds motion in one line

Chapter 63: Views, State, and Navigation

You already know how to build a single-screen toy. This chapter tackles two "growing up" problems: how to hop between multiple screens (navigation) and how data flows between screens (state management). After this chapter, you'll be able to build the classic multi-screen app skeleton: "list detail edit."

The Big Picture of State Management and Data Flow

@State isn't the only state tool in SwiftUI. Start with this overview so you understand "which one to use when" :

Tool	What it manages	Who uses it	Typical scenario
@State	Private, temporary state for one screen	The current view	Counters, text field content
@Binding	Borrowing someone else's state	A child view changing a parent view	Custom components
@StateObject / @Observable	Logic shared across screens	The view model (ViewModel)	Data loading, business logic
@EnvironmentObject / .environment	State shared across the whole app	The entire app	Login state, theme settings

The SwiftUI state toolbox at a glance (2026, @Observable is now mainstream)

```
// The modern way on iOS 17+: the @Observable macro (recommended for new projects)
import Observation

@Observable
class CounterModel {                    // the view model
    var count = 0
    func increment() { count += 1 }
}
```

```
struct CounterView: View {
    @State private var model = CounterModel()   // holds the model

    var body: some View {
        VStack {
            Text("You tapped \(model.count) times")
            Button("Tap me") { model.increment() }
        }
    }
}
```

Blunt advice for beginners: use @State for state private to one screen, and @Observable classes for anything shared across screens or involving logic. The @StateObject/@ObservableObject in older tutorials are the previous generation—being able to read them is enough; write new code the way described above.

NavigationStack: The “Stack” Behind Screen Jumps

Since iOS 16, multi-screen navigation uses NavigationStack—like a stack of plates: push one on, and pop them off one by one as you go back:

```
struct HomeView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("Alex's ledger") {
                    DetailView(name: "Alex's ledger")   // destination screen
                }
                NavigationLink("Sam's ledger") {
                    DetailView(name: "Sam's ledger")
                }
            }
            .navigationTitle("Ledger list")   // large title at the top
        }
    }
}

struct DetailView: View {
    let name: String
    var body: some View {
        Text("\(name) details")
            .navigationTitle(name)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

NavigationLink is a "tappable link to another screen"; navigationTitle automatically generates the navigation bar title and the back button. When you need programmatic navigation (say, returning automatically after tapping "Save"), use @Environment(\.dismiss) to close the current screen—together, these two tricks cover 90% of your navigation needs.

Pop-Ups and Bottom Tabs: sheet and TabView

Besides "pushing screens," there are two more classic interactions:

```
// 1. Slides up half/full screen (the favorite for entry forms)
struct HomeView: View {
    @State private var showAdd = false
    var body: some View {
        Button("Add entry") { showAdd = true }
            .sheet(isPresented: $showAdd) {
                AddRecordView()     // the entry sheet
            }
    }
}

// 2. The bottom tab bar (the app's "skeleton")
struct MainTabView: View {
    var body: some View {
        TabView {
            RecordListView()
                .tabItem { Label("Ledger", systemImage: "book") }
            StatsView()
                .tabItem { Label("Stats", systemImage: "chart.pie") }
            SettingsView()
                .tabItem { Label("Settings", systemImage: "gear") }
        }
    }
}
```

Tip: When Do You Use Which?

Rules of thumb: use NavigationStack to push screens for the main flow (list detail), use a sheet pop-up for temporary actions (adding, editing, confirming), and use TabView for several side-by-side sections (bottom tabs). Combine the three, and you have the skeleton of the vast majority of apps.

List: iOS's Go-To List

The most used—and most "Apple-flavored"—component on iOS is List: it comes with dividers, swipe-to-delete, and tap highlighting out of the box:

```
struct RecordListView: View {
    @State private var records = ["Lunch 25.5", "Metro 6.0", "Salary +8000"]

    var body: some View {
        List {
            ForEach(records, id: \.self) { r in
                Text(r)
            }
            .onDelete { indexSet in          // swipe left to delete
                records.remove(atOffsets: indexSet)
            }
        }
    }
}
```

List + NavigationLink + sheet + TabView—put these four together, and you can already build a complete app skeleton: "ledger list tap for details tap + to open the entry sheet." The expense tracker project in the next chapter is built exactly this way.

Hands-on: Build a "Three-Screen Skeleton"

The App Lifecycle: Foreground and Background

Much like Android's Activity lifecycle, iOS apps also move through "born, foreground, background, dead" state changes. In SwiftUI, watch them with the scenePhase environment value:

```
@main
struct MyApp: App {
    @Environment(\.scenePhase) private var phase

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .onChange(of: phase) { _, newPhase in
            switch newPhase {
            case .active:    print("Back in the foreground, resuming refresh")
            case .inactive:  print("Transitioning (call / just backgrounded)")
            case .background: print("Went to background, saving draft")
            @unknown default: break
            }
        }
    }
}
```

Typical uses: refresh data when coming back to the foreground, save drafts when heading to the background. iOS runs a tight ship on background activity—an ordinary app gets suspended (frozen) shortly after going to the background, so don't expect to secretly do heavy work back there; if you truly need scheduled tasks, use the local notifications from Chapter 68 instead of "staying alive in the background."

Build a fake-data skeleton of the expense tracker: a TabView with three tabs (Ledger / Stats / Settings) the Ledger screen shows fake data in a List, and tapping a row jumps to the details (NavigationStack) the "+" button in the top-right opens the entry sheet. No saving, no internet—pure skeleton practice. Get this skeleton right, and filling it with real data in Chapter 65 will be a smooth ride.

Chapter summary

- The four state tools: @State (screen-private), @Binding (borrowed), @Observable (logic), .environment (global)
- Use the @Observable macro for view models in new projects; being able to read the older @ObservableObject is enough
- NavigationStack pushes screens; @Environment(\.dismiss) navigates back programmatically
- Push screens for the main flow, pop up a sheet for temporary actions, and use TabView for side-by-side sections
- List is the go-to list: dividers and swipe-to-delete work out of the box

Chapter 64: Data Storage and Network Requests

An app you can only "look at" is dead; an app that can store data and reach the network is alive. This chapter introduces the "three essentials" of the iOS data layer: UserDefaults (for small settings), SwiftData (for structured data), and URLSession (for talking to servers), plus Codable for parsing JSON and the modern async/await. After this chapter, you'll be able to write real apps that remember user preferences and update their content live.

Choosing a Storage Option: Look at This Table First

Option	What it's good for	Key traits	Where this book uses it
UserDefaults	Loose settings (toggles, preferences)	Key-value pairs, simple and direct	Chapter 65 settings, Chapter 66 favorite cities
SwiftData	Structured data (transactions, records)	The official database wrapper, declared in one line with @Model	Chapter 65 transaction records
URLSession	Server data (not stored, fetched online)	The built-in networking framework, no third-party dependencies	Chapter 66 weather data

The division of labor among the data trio

Memorize this mantra: "small settings go into UserDefaults, batches of data go into SwiftData, and data from the outside travels over URLSession." Core Data from old tutorials still works (SwiftData is its modern incarnation), while hand-written SQLite is essentially a thing of the past—learn the new way directly and don't take the old road.

UserDefaults: Storing a Few Small Settings

```
// Write: store a value
UserDefaults.standard.set(true, forKey: "isDarkMode")
UserDefaults.standard.set("Beijing", forKey: "favoriteCity")

// Read: fetch it (falls back to the default)
let isDark = UserDefaults.standard.bool(forKey: "isDarkMode")
let city = UserDefaults.standard.string(forKey: "favoriteCity") ?? "Beijing"

// Delete: remove a key
UserDefaults.standard.removeObject(forKey: "isDarkMode")
```

Keep in mind that it's only right for loose little settings—hand dozens or hundreds of records over to SwiftData, and don't use UserDefaults as a database (it slows down launch time and makes searching a pain).

SwiftData: Giving Data a "Real Home"

SwiftData is the official database solution starting with iOS 17, and it boils "creating tables, adding, deleting, updating, querying" down to the bare minimum—one @Model annotation turns a data class into a database table:

```
import SwiftData

// 1. @Model: this class automatically becomes a "table"
@Model
class Record {
    var amount: Double
    var category: String
    var note: String
    var date: Date

    init(amount: Double, category: String, note: String, date: Date) {
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}

// 2. Attach the container at the app's entry point
@main
struct MyApp: App {
    var body: some Scene {
```

```
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Record.self)   // database ready
    }
}

// 3. In views, read with @Query and write via @Environment
struct RecordListView: View {
    @Query(sort: \Record.date, order: .reverse) var records: [Record]
    @Environment(\.modelContext) private var context

    var body: some View {
        List(records) { r in
            Text("\(r.category) \(r.amount) yuan")
        }
    }

    func add(amount: Double, category: String) {
        let r = Record(amount: amount, category: category, note: "", date: Date())
        context.insert(r)     // store
        try? context.save()   // persist to disk
    }
}
```

This “@Model + modelContainer + @Query” template can be dropped into any of your projects as-is—just swap in your own field names. The moment the data changes, @Query refreshes the interface automatically. Simply delightful.

Codable: A “Translator” for JSON

Most of what servers send is JSON text. Swift’s Codable protocol makes the “JSON struct” conversion happen automatically—line up the field names, and the magic works:

```
// JSON: {"name":"Beijing","main":{"temp":26.5},"weather":[{"description":"Sunny
    "}]}
struct WeatherResponse: Codable {
    let name: String
    let main: Main
    let weather: [Weather]
}
struct Main: Codable { let temp: Double }
struct Weather: Codable { let description: String }

// JSON string  struct
let decoder = JSONDecoder()
```

```
let data = jsonString.data(using: .utf8)!
let result = try decoder.decode(WeatherResponse.self, from: data)
print(result.weather[0].description)   // "Sunny"
```

Iron rule: look at the JSON that actually comes back before you define your fields. Many "parse failed" bugs come from a misspelled field name or a wrongly guessed type (the server sends the string "26.5" and you declared a Double)—print out the raw JSON and compare, and you'll locate the problem in ten seconds.

URLSession + async/await: Talking to Servers

iOS networking uses the built-in URLSession, paired with Swift's async/await (asynchronous functions), so "waiting for the network" feels as natural as "waiting for a delivery":

```
struct WeatherService {
    // Async functions: async marks "I need to wait a moment," throws marks "this
    can fail"
    func fetchWeather(city: String) async throws -> WeatherResponse {
        let url = URL(string:
            "https://api.example.com/weather?city=\(city)")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}

// At the call site (inside .task):
.task {
    do {
        let weather = try await service.fetchWeather(city: "Beijing")
        // update the UI...
    } catch {
        print("Something went wrong: \(error)")
    }
}
```

Pitfall Alert: Two Iron Rules of Networking

1) Networking must use async/await and must never block the main thread—a frozen interface is a crash-level experience. Swift's structured concurrency makes this natural, but don't forget try await and do/catch. 2) ATS blocks plain-text HTTP by default: iOS requires HTTPS. While testing a local http:// service, you can temporarily allow an exception in Info.plist (NSAppTransportSecurity), but a production app must use HTTPS throughout—App Review keeps an eye on this too.

Hands-on: Build a "Quote of the Day" Mini App

"Grade" Your Network Errors Too: Don't Throw Every Error at the User

No connection, timeout, 404, server 500... different errors deserve different messages for the user:

```
func friendlyMessage(_ error: Error) -> String {
    if let urlError = error as? URLError {
        switch urlError.code {
        case .notConnectedToInternet: return "Network is down — check your
    connection?"
        case .timedOut:               return "Request timed out — try again in
     a bit?"
        case .cannotFindHost:         return "Server not found"
        default:                      return "Network problem: \(urlError.code)
    "
        }
    }
    if let d = error as? DecodingError {
        return "Unexpected data format (the API may have changed)"
    }
    return "Something went wrong: \(error.localizedDescription)"
}
```

Show the user "plain words," and save the "raw words" for the logs (print the full error). Don't fling the gibberish English of error.localizedDescription straight at the user—that feeling of a polished product hides in details like these.

Find a free quotes API (something like hitokoto), use URLSession + async/await to fetch a "quote of the day," show it in the middle of the screen, and add a "New quote" button. Then use SwiftData to save your favorite quotes into a "favorites" list. This one exercise uses UserDefaults/SwiftData/Codable/URLSession all at once—finish it, and Chapters 7 and 8 will be a piece of cake.

Chapter summary

- The mantra: small settings into UserDefaults, batches of data into SwiftData, outside data over URLSession
- SwiftData's three moves: @Model declares the table, modelContainer attaches the container, @Query reads and context.insert writes
- Codable "translates" JSON automatically: line up the fields and the magic works
- URLSession + async/await: networking feels like waiting for a delivery; do/catch handles failures

- Two iron rules: never block the main thread; production apps must use HTTPS (ATS)

Chapter 65: Project One—My Expense Tracker (Local Version)

Welcome to your first "serious project." This expense tracker never touches the internet, but it's complete in every part: you can record a transaction, view the running history, see monthly totals, and edit or delete entries. Finishing it means you've walked the whole "data logic interface" path end to end. Don't rush to copy code—think it through first, then build. That's our standing principle.

Thinking About Requirements: Ask Five Questions First

Element	Answer
Who (who's it for)	For yourself; family and friends too
What (what it does)	Record income and spending, view the history, monthly summaries, statistics by category
Why (the reason)	Practice + finding out where the money goes
How (the approach)	Swift + SwiftUI + SwiftData + MVVM, purely local, no backend
Constraint (the limits)	Done within one week; supports iOS 17 at minimum (the SwiftData requirement)

The five requirement elements of the expense tracker

Then break it into small steps where "every step is usable": 1) build the SwiftData data layer (can save a transaction); 2) the list screen (can view the history); 3) the entry sheet (can add a transaction); 4) deleting and editing; 5) the monthly summary card. After every step, the app is a "usable thing."

The Data Layer: Giving Transactions a Home

First, define what a transaction looks like—enough fields to do the job, but don't get greedy:

```
import SwiftData

@Model
class Record {
    var type: String      // "income" = money in | "expense" = money out
    var amount: Double    // amount (in yuan)
    var category: String  // category: food, transport, salary...
    var note: String      // note (optional)
    var date: Date        // expense date

    init(type: String, amount: Double, category: String, note: String = "", date:
    Date = Date()) {
        self.type = type
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}
```

Tip: Why Use Double for Amounts?

Rigorous financial apps use Decimal (exact decimal arithmetic) to avoid floating-point errors (the old $0.1 + 0.2 \neq 0.3$ problem). Double is plenty for a practice project; if you ever build a real financial product, remember to switch to Decimal or "an Int counted in cents"—a pitfall countless developers before you have stumbled into.

MVVM: When Interface and Logic "Move Apart"

Let the interface talk to the database directly? It runs, but the interface code becomes bloated and ugly. The MVVM architecture gives each part its own job: the View (interface) only draws, the ViewModel (logic hub) manages the data, and the Model (SwiftData) handles storage.

View (the SwiftUI interface)

ViewModel (the @Observable logic hub)

Model (the SwiftData data layer)

Figure 4: MVVM's three layers—data flows in one direction, and no layer oversteps its bounds

```
import Observation

@Observable
class RecordViewModel {
    var records: [Record] = []
```

```
    private let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func load() {
        // filtering + sorting with SwiftData's #Predicate
        let descriptor = FetchDescriptor<Record>(
            sortBy: [SortDescriptor(\.date, order: .reverse)]
        )
        records = (try? context.fetch(descriptor)) ?? []
    }

    func add(type: String, amount: Double, category: String, note: String) {
        let r = Record(type: type, amount: amount, category: category, note: note)
        context.insert(r)
        try? context.save()
        load()                          // reload; the UI refreshes
    }

    func delete(_ record: Record) {
        context.delete(record)
        try? context.save()
        load()
    }
}
```

In the interface, write @State private var vm = RecordViewModel(...), then render the list straight from vm.records—the moment the data changes, the interface refreshes automatically.

The Interface: List Screen + Entry Sheet

```
struct RecordListView: View {
    @State private var vm: RecordViewModel
    @State private var showAdd = false

    var body: some View {
        NavigationStack {
            List {
                Section("This month's balance") {          // summary card
                    SummaryRow(records: vm.records)
                }
                Section("Transactions") {
                    ForEach(vm.records) { r in
                        RecordRow(record: r)
                    }
                    .onDelete { idx in
```

```
                        idx.forEach { vm.delete(vm.records[$0]) }
                    }
                }
            }
            .navigationTitle("My expense tracker")
            .toolbar {
                Button { showAdd = true } label: {
                    Image(systemName: "plus")
                }
            }
            .sheet(isPresented: $showAdd) {
                AddRecordView(vm: vm)          // the entry dialog
            }
        }
    }
}
```

Entry sheet essentials: use .keyboardType(.decimalPad) for the amount field (a number pad), a segmented control (Picker) for income/expense, a menu picker for the category, and on "Save," call vm.add(...) and dismiss() to close the sheet.

Note: Don't Get Lazy with Input Validation

Always validate the amount field: block empty values, negative numbers, and invalid input like "abc" (check with Double(text), which returns nil on failure). One of the most common reasons apps crash is "treating the user's input like a well-behaved child." Remember: treat everything the user types as hostile until proven otherwise.

Editing, Deleting, and Summaries

- Delete: List's .onDelete swipe-to-delete plus a confirmation dialog (accidental deletion is what users of expense apps hate most);
- Edit: tapping a row opens an edit sheet pre-filled with the original values (fixing one word beats retyping everything);
- Summaries: use filter + reduce to compute "this month's income / spending / balance," shown in a card at the top:

```
func monthSummary(_ records: [Record]) -> (income: Double, expense: Double) {
    let cal = Calendar.current
    let monthRecords = records.filter {
        cal.isDate($0.date, equalTo: Date(), toGranularity: .month)
    }
    let income = monthRecords
        .filter { $0.type == "income" }
```

```
        .reduce(0) { $0 + $1.amount }
    let expense = monthRecords
        .filter { $0.type == "expense" }
        .reduce(0) { $0 + $1.amount }
    return (income, expense)
}
```

Acceptance: A Three-Layer Check + Two Advanced Challenges

When the project is done, run acceptance across three layers— "function edge cases architecture" : can you record and delete, and does the data survive restarting the app (function); does entering 0 or a negative amount crash it (edge cases); are interface and data mixed together, and are any fields hard-coded (architecture)?

Once the basic version passes acceptance, serve yourself two "advanced dishes" :

1. Category statistics: add a "Stats" tab and use Swift Charts to draw category spending as a pie chart or bar chart—Apple's official charting library, charts in just a few lines of code;
2. iCloud sync: With SwiftData, turning on iCloud sync only takes a few settings—the ledger syncs automatically between iPhone and iPad, and instantly feels "pro."

Get hands-on: finish it this week

Work in the order "data layer ViewModel list page entry sheet summary card," one step a day. If you get stuck, paste the error message straight into a search engine or an AI (remember Part 7, "Hands-On AI Coding" ? When describing a problem, clearly state the "expected behavior, actual behavior, and steps to reproduce"). Once it's done, use it to track a week of expenses—using an app you built yourself gives you a sense of accomplishment that will push you forward.

Chapter summary

- Five elements of a requirement break it into small, "usable at every step" steps, then start building
- Data layer: @Model models + the modelContainer container + FetchDescriptor queries
- MVVM division of labor: the View only draws, the @Observable ViewModel handles logic, SwiftData handles storage
- UI: List + Section + toolbar + a sheet for the entry dialog
- Three-layer acceptance (features/edge cases/architecture) + advanced challenges (Swift Charts, iCloud sync)

Chapter 66: Project 2—Weather App (the Networked Version)

The expense tracker was "internal training"; the weather app is "seeing the world"—it has to deal with the internet. In this project you'll learn: how to apply for a free API, parse JSON, handle the three UI states of "loading/success/failure," and how to keep an API key safe. Once you finish it, you'll be able to build any "networked app"—news, exchange rates, translation—the pattern is exactly the same.

Apply for a free weather API

For personal practice, we recommend OpenWeatherMap or, in China, QWeather (HeFeng Weather)—both have free tiers you can use right after registering. Taking OpenWeatherMap as an example:

1. Register an account generate a Key (apikey) on the API keys page;
2. The free tier can call the Current Weather endpoint: api.openweathermap.org/data/2.5/weather?q=beijing&appid=YOUR_KEY&units=metric;
3. Try it in a browser first and see what the returned JSON looks like—this "preview" is a must before hooking up any API.

Pitfall warning: never hard-code the key into your code

An API key is like your "wallet password." Put it in code, commit it to GitHub, and crawlers will scan it up and abuse it within hours. The right way on iOS: put it in a Config.xcconfig (a config file that's already gitignored) and read it into your code via Info.plist. Search "iOS xcconfig manage keys" and follow along.

The network layer: URLSession + Codable

First define the model based on the JSON the API returns, then write the service:

```
// JSON: {"weather":[{"description":"Sunny"}],"main":{"temp":26.5,"humidity":40},"
    name":"Beijing"}
struct WeatherResponse: Codable {
    let weather: [Weather]
    let main: Main
    let name: String
}
struct Weather: Codable { let description: String }
struct Main: Codable { let temp: Double; let humidity: Int }

struct WeatherService {
    let key: String   // read in from xcconfig

    func fetchWeather(city: String) async throws -> WeatherResponse {
        var comps = URLComponents(string: "https://api.openweathermap.org/data/2.5/
    weather")!
        comps.queryItems = [
            URLQueryItem(name: "q", value: city),
            URLQueryItem(name: "appid", value: key),
            URLQueryItem(name: "units", value: "metric"),
            URLQueryItem(name: "lang", value: "en")
        ]
        let (data, _) = try await URLSession.shared.data(from: comps.url!)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}
```

Three UI states: loading / success / failure

The soul of a networked app is "state management." Slow network, no connection, a misspelled city name... the UI needs to give the user an answer for every case. Express it elegantly with an enum + switch:

```
enum LoadState {
    case loading                  // spinner showing
    case success(WeatherResponse) // success
    case error(String)            // failure
}

@Observable
class WeatherViewModel {
    var state: LoadState = .loading
    private let service = WeatherService(key: Config.apiKey)

    func search(city: String) async {
```

```
        state = .loading
        do {
            let data = try await service.fetchWeather(city: city)
            state = .success(data)
        } catch {
            state = .error("Can't find that city — try another name?")
        }
    }
}

// In the UI:
switch vm.state {
case .loading: ProgressView("Loading…")
case .success(let d): WeatherContent(data: d)
case .error(let msg): Text(msg).foregroundColor(.secondary)
}
```

This loading/success/error three-part pattern is the "all-purpose template" for networked interfaces—memorize it first, and every networked app you build later can follow the same recipe.

Hands-on UI code: drawing the weather

```
struct WeatherContent: View {
    let data: WeatherResponse

    var body: some View {
        VStack(spacing: 12) {
            Text(data.name)
                .font(.title.bold())
            Text("\(Int(data.main.temp))°C")
                .font(.system(size: 76, weight: .thin))
            Text(data.weather.first?.description ?? "Unknown")
                .font(.title3)
                .foregroundColor(.secondary)
            HStack(spacing: 24) {
                Label("Humidity \(data.main.humidity)%", systemImage: "humidity")
                Label("Feels like \(Int(data.main.temp))°C", systemImage: "
    thermometer")
            }
        }
        .padding()
    }
}
```

A few details worth learning: first? safely grabs the first element (the list may be empty), and ?? “Unknown” provides fallback text; a large temperature number in a thin font weight is a classic recipe for weather apps; Label is the official “icon + text” component, cleaner than hand-assembling an HStack.

Interaction details and favorite cities

- Search box: the .searchable modifier at the top gives you system-native search interaction; call vm.search on submit;
- Favorite cities: store frequently viewed cities in UserDefaults, and they’re looked up automatically the next time you open the app;
- Pull to refresh: one line with .refreshable { await vm.search(...) } does it;
- Weather icons: map descriptions with Image(systemName:) (sun.max / cloud.rain / snowflake) and the UI comes alive instantly.

Tip: the quirk with Chinese city names

OpenWeatherMap’s support for Chinese city names is spotty—searching a Chinese name may return no results while “beijing” works fine. The production-grade approach is to bundle a “city name English/coordinates” mapping table, or hook up a domestic weather service. At the practice stage, just support pinyin/English city names—don’t fight it.

Get hands-on: the weather app acceptance checklist

How to debug network requests: printing, packet capture, and simulating a slow network

When a networked app misbehaves, nine times out of ten it’s “the request wasn’t sent right” or “the response wasn’t parsed right.” Three debugging tools:

1. Print the raw data: print String(data: data, encoding: .utf8) to see the raw JSON—field names and types at a glance, ten times faster than guessing;
2. Packet capture tools: the free Proxyman on Mac, or the open-source Charles, can intercept all of the app’s requests and responses, letting you see the network layer like an X-ray;
3. Simulate a weak network: the Simulator menu’s Features Network Link Conditioner (download the tool from the developer website) can simulate 3G and high latency—surfacing those ”spinner spins forever” experience problems.

Tip: request blocked by ATS?

If a request fails the moment it's sent with an "App Transport Security" error, you're probably using a plain-text http:// address. Production endpoints must use HTTPS; for local debugging you can temporarily add an exception (NSAllowsLocalNetworking in Info.plist), but remember to clean it up before release.

(1) Search for weather normally; (2) disconnect the network and the app fails gracefully without crashing (turn off your Mac's network and test in the Simulator!); (3) enter a nonexistent city and get a friendly message; (4) favorite two cities and they're still there after restarting the app; (5) the UI looks good in Dark Mode. Only when all five pass does this project graduate.

Chapter summary

- Try the API in a browser and understand the JSON before writing code—"previewing" never hurts
- Hide the API key in an xcconfig—never in the codebase
- Codable models + URLSession async/await: match the fields and the JSON parses automatically
- The loading/success/error three-state pattern is the Swiss Army knife of networked UIs
- No connection, empty input, nonexistent city—error handling is the passing grade for a networked app

Chapter 67: Polish and Ship—Testing, Debugging, and the App Store

The app works—now what? This chapter covers the three things on the road from "it runs" to "it's deliverable": how to debug efficiently to find bugs, how to write tests to prevent regressions, and how to get your app onto the App Store for the whole world to download. Shipping isn't just ceremony—it's the dividing line between a project and a product.

The three debugging essentials: print, breakpoints, and previews

Tool	What it does	How to use it
print + console	View run logs	In code: print("value = \(x)"), and watch the output in the Xcode console at the bottom
Breakpoint debugging	Pause the program at a given line and inspect variable values	Click next to a line number to place a blue marker; execution pauses there and you can step through
SwiftUI previews + debugger	See the UI and view hierarchy in real time	#Preview canvas + the runtime View Hierarchy inspector

The three debugging essentials

Tip: how do you read a crash?

When Swift crashes, the console prints "Fatal error" plus the reason, and Xcode stops on the offending line and highlights it. Historical crashes on real devices can be viewed in Xcode Window Organizer Crashes (paid account required). When searching, prioritize

"the first sentence of the error message + swiftui" —precise questions get precise answers.

Testing: insurance for your code

Manual testing can't be trusted—fix one feature and another quietly breaks. Tests are both the "undo button" and the "bulletproof vest." iOS uses the XCTest framework, in two flavors:

Level	What it tests	Target	Speed
Unit tests	Pure logic (calculations, validation)	MyFirstAppTests	Seconds
UI tests	Real interface behavior	MyFirstAppUITests	Minutes
Manual testing	Human feel and experience	——	Always needed

iOS testing levels (the template project already includes two Test Targets)

```
import XCTest

// Extract logic like "amount string to number" into a pure function
func parseAmount(_ input: String) -> Double? {
    guard let v = Double(input.trimmingCharacters(in: .whitespaces)),
          v > 0 else { return nil }
    return v
}

final class AmountTests: XCTestCase {
    func testEmptyInputReturnsNil() {
        XCTAssertNil(parseAmount(""))
    }
    func testNegativeReturnsNil() {
        XCTAssertNil(parseAmount("-5"))
    }
    func testNormalValueParses() {
        XCTAssertEqual(parseAmount("12.5"), 12.5)
    }
}
```

Run tests with CmdU. Change code only once everything is green, and you'll feel confident. The amount validation and summary calculations in the expense tracker should all be covered by unit tests.

Signing and packaging: stamping your app

Every iOS app must be signed before it can be installed—proof that "this package came from you and hasn't been tampered with." The hierarchy:

- Learning stage: a free Apple ID + Xcode automatic signing (Personal Team) runs on a real device, renewed every 7 days;
- Distribution stage: a paid developer account (99 USD/year) issues the certificates and provisioning profiles needed to ship and install long-term.

Packaging: in Xcode, Product Archive; once the build finishes, in Organizer choose Distribute App App Store Connect upload. Archiving must target the Generic iOS Device (not the Simulator)—a step beginners often forget.

Pitfall warning: lose the certificate = updates cut off

Once a signing certificate and its private key are lost, you can no longer push updates to your old app—users can only install it as a "brand-new" app. Back up your certificates carefully (export a .p12 from Keychain) and keep the password safe. Fortunately, Apple offers "automatic signing" and cloud certificate management (check Automatically manage signing in Xcode), which spares you most of the manual pitfalls.

The full App Store submission process

Register a developer account

Prepare store assets

Upload the build

Fill in privacy and review info

Apple review (1-7 days)

Published

Figure 5: Six steps to the App Store

- Register: developer.apple.com, personal accounts 99 USD/year;
- Assets: icon (1024×1024, no rounded corners—Apple does the rounding itself), screenshots (each iPhone size), description, privacy policy link;
- Privacy questionnaire: honestly declare what data you collect—an offline expense tracker can declare "no data collected";
- Review: the first submission often waits 1–7 days; if rejected, fix things one by one per the "review notes"—Apple's rejection reasons are usually written very specifically, so just follow them;

- TestFlight: before release, send a beta build to friends for internal testing (up to 10,000 people), gather a round of feedback, then submit officially—very professional.

Store assets: first impressions drive downloads

The store page is the shop window users see while "browsing." Three assets decide the first impression:

- Icon: 1024×1024, don't just make do with a screenshot. If you have no design skills, use an AI tool to generate a simple graphic; it only passes if it's still recognizable when shrunk down on a home screen;
- Screenshots: at least 2 sets (mainstream iPhone sizes), showing your most core feature scenarios; give each one a plain-language caption ("Log an expense in one second" beats "An efficient expense-tracking solution" ten times over);
- Description: the first sentence should make clear "what this is + who it's for." Don't pile on "the most professional, the most powerful"—sincerity works better than hype.

Get hands-on: "ship" the expense tracker once

A mini checklist for crash reporting and performance

Before release, these two things are each worth half an hour:

Crash monitoring

Just because it didn't crash in your testing doesn't mean it won't on users' phones. Apple provides free crash reports in the Xcode Organizer (paid accounts can see real-device crash logs); an even more hassle-free option is a third-party SDK (such as Firebase Crashlytics). Shipping without crash monitoring is like driving with your eyes closed.

Four performance self-checks

1. Use LazyVStack / List for lists: don't lay out long lists with a plain VStack—lazy loading is lesson one of performance;
2. Don't force in full-size images: use AsyncImage (for remote images) and the system cache; a 5MB original can bog down older phones badly;
3. Don't do heavy work on the main thread: route network calls and large files through async/await;
4. Measure with Instruments: Xcode's built-in profiler (Time Profiler, memory leak checks)—measure first, then optimize; don't go by feel.

(1) Add 3 unit tests for logic like parseAmount (green with CmdU); (2) use a breakpoint to catch a crash you created yourself and fix it; (3) build a package with Product Archive and invite friends to test via TestFlight; (4) if you have the bandwidth, register a developer

account and actually submit the expense tracker to the store. The first three steps can be done tonight.

Chapter summary

- The three debugging essentials: print logs, breakpoints, and SwiftUI previews with the view inspector
- XCTest: unit tests (CmdU) + UI tests + manual testing, prebuilt in the template
- Signing: free accounts get automatic signing (renewed every 7 days); distribution requires a paid account
- Packaging: Generic iOS Device + Archive + Distribute; always back up your certificates
- Six steps to release: register assets upload privacy questionnaire review go live; beta test with TestFlight first

Chapter 68: The Road Ahead—Permissions, Notifications, and Modern Architecture

Being able to build an app is only the start. This chapter is about the hurdles between "can build" and "builds well": how to ask for permissions gracefully, how to send notifications without being annoying, how to lift your architecture another level, and—here in 2026—how to team up with AI to write iOS code. Every item is something you'll run into right away in a real project.

Permissions: knock before you enter

Camera, location, contacts, notifications... for these "sensitive zones," iOS requires the app to show a prompt asking the user at the moment of use. The rule is three steps: "declare + ask + degrade gracefully":

```
// 1. Declare the usage text in Info.plist (without it, instant crash!)
// Privacy - Location When In Use Usage Description: "Used to look up the weather
    in your city"

// 2. Request it when you need it
import CoreLocation

@Observable
class LocationModel {
    private let manager = CLLocationManager()

    func request() {
        switch manager.authorizationStatus {
        case .notDetermined: manager.requestWhenInUseAuthorization()
        case .denied, .restricted: /* guide the user to Settings */
        default: /* already granted, go ahead */
        }
    }
```

```
}

// 3. Degrade gracefully after denial: show a hint + open Settings
// UIApplication.shared.open(URL(string: UIApplication.openSettingsURLString)!)
```

Note: iOS permissions are "one-shot"

The iOS permission prompt appears only once—once the user declines, it never pops up again. So: time the request for "the exact moment the user is about to use the feature" (not a pile of prompts at launch), and write the copy so it explains the purpose. Post-denial guidance ("go to Settings and turn on location") is standard equipment—don't leave users stuck in a dead end.

Notifications: say something useful at the right time

Notifications are the channel apps use to reach users, and also the feature most easily "blocked" by them. iOS uses the UserNotifications framework. Three rules for notifications: only send what's useful, let users turn them off, and never pester people in the middle of the night. Basic usage:

```
import UserNotifications

// 1. Request notification permission (one-time dialog)
UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge]) { granted, _ in
        print(granted ? "granted" : "denied")
    }

// 2. Schedule a "daily 20:00 expense reminder"
func scheduleDailyReminder() {
    let content = UNMutableNotificationContent()
    content.title = "Expense reminder"
    content.body = "You haven't recorded today's expenses yet"
    content.sound = .default

    var comps = DateComponents()
    comps.hour = 20
    let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)

    let request = UNNotificationRequest(
        identifier: "daily-reminder",
        content: content,
        trigger: trigger
    )
    UNUserNotificationCenter.current().add(request)
}
```

Note: the iOS notification permission is also "one prompt only," and users can switch it off anytime in system settings—so the sense of value in each notification matters a lot. One spam blast can lose a user forever.

Architecture, going deeper: MVVM and testability

Chapter 65 used MVVM; here are three advanced points to add:

- Protocol abstraction: make the ViewModel depend on a protocol rather than a concrete class—then tests can inject a "fake data source." This is the cornerstone of testability;
- Single responsibility: one ViewModel handles one thing only (the expense-tracking one doesn't handle weather); when features grow, split it up;
- Don't over-engineer: for a solo small project, "an @Observable class + protocol abstraction" is plenty. Heavy architectures like VIPER or Clean Architecture can wait until team size and complexity come knocking—tools serve the goal; they're not for showing off.

Tip: SwiftUI's "ecosystem dividend"

The iOS ecosystem has two more "only-here" advanced tricks: widgets (WidgetKit) let users see information without opening the app (put the expense tracker's "spending this month" on the home screen), and Shortcuts (App Intents) let users invoke your features through Siri or automations. Both have official frameworks and are cheap to learn, yet they massively boost the user experience—worth unlocking early on the road to mastery.

The 2026 twist: AI-assisted iOS development

By 2026, AI is deeply embedded in Apple's development workflow. If you want to speed up, this combo is mature:

Stage	How AI helps	Your responsibility
Writing code	Xcode's built-in code completion + third-party AI plugins generate it	Describe requirements clearly, review the output
Looking things up	Hand official docs and error messages to AI to interpret	Verify key conclusions against the official docs
Writing tests	Have AI generate XCTest cases in the "normal/boundary/exception" categories	Confirm the cases match the business meaning
Fixing bugs	Paste the crash log + relevant code, and AI locates and fixes it	Understand the changes, run tests to verify

Four scenarios for AI-assisted iOS development

For the full methodology—how to give AI instructions, how to review its work, how to avoid pitfalls—the complete 11 chapters are in Part 7 of this book, "Hands-On AI Coding" (Chapters 37 - 47), which I strongly recommend reading alongside this one: leave the Swift syntax to this part, and the art of working with AI to Part 7.

Get hands-on: add the "power trio" to the expense tracker

Swift concurrency essentials: @MainActor and Task

Swift 6's concurrency system has two rules every beginner must understand:

- The UI can only be updated on the main thread: SwiftUI views are @MainActor by default, so bring network callbacks back to the main thread before changing state (await MainActor.run { ... }, or mark the ViewModel @MainActor);
- Manage async work with Task: the .task { } modifier starts automatically when the view appears and cancels automatically when it disappears—that's the standard way to kick off async work in SwiftUI; don't scatter raw Tasks in body.

```
struct WeatherScreen: View {
    @State private var vm = WeatherViewModel()

    var body: some View {
        Group {
            switch vm.state {
            case .loading: ProgressView()
            case .success(let d): WeatherContent(data: d)
            case .error(let m): Text(m)
            }
        }
```

```
        .task {                          // starts when it appears, cancels when
    it disappears
            await vm.search(city: "Beijing")
        }
    }
}
```

Behind these two rules is Swift 6's "data-race safety" design—the compiler errors out the moment you get it wrong, instead of waiting for the app to crash to find out. The compiler's errors aren't nitpicking; they're taking a bullet for you.

(1) Add a "daily 8:00 PM expense reminder" to the tracker (notification permission + local notifications); (2) switch the ViewModel's data source to a protocol abstraction and write a unit test that injects fake data; (3) add a "spending this month" home-screen widget. Finish the trio and your project starts to look like a regular army.

Chapter summary

- Permissions in three steps: declare in Info.plist (miss it and it crashes), ask at the moment of use, guide to Settings when denied
- iOS permission prompts appear only once: pick the timing of "the moment they're about to use it," and write the purpose clearly
- Three rules for notifications: only send useful ones, let users turn them off, don't pester; UserNotifications framework
- MVVM advanced: protocol abstraction (testable), single responsibility, no over-engineering
- Ecosystem dividends: WidgetKit widgets + App Intents Shortcuts
- Four AI-assisted scenarios: writing code, research, writing tests, fixing bugs; methodology in Part 7 (Chapters 37–47)

Chapter 69: Quick Reference and Resources

This chapter is a quick-reference handbook that works on its own. When you can't remember a bit of syntax or need to make a fast decision, flip straight here. Treat it as a set of "pocket cards" : common code, common questions, and learning resources—nothing left out.

Common code cheat sheet

What you want to do	Key code
Page-private state	@State private var x = initialValue
Logic shared across pages	`@Observable class VM { var x = 0 }`
Vertical/horizontal layout	`VStack { }/HStack { }`
Flexible space	`Spacer()`
Lists	List { ForEach(array) { ... } }
Navigating between pages	`NavigationStack + NavigationLink`
Bottom sheet	.sheet(isPresented: $show) { content }
Dismiss the current page	`@Environment(\.dismiss) var dismiss`
Fetch data over the network	`try await URLSession.shared.data(from: url)`
JSON parsing	`struct X: Codable+JSONDecoder().decode`
Store small settings	UserDefaults.standard.set(value, forKey: "key")
Store structured data	`@Model+modelContainer+@Query`
Dark mode support	Use system colors (.primary / Color(.systemGray6)); never hard-code
Run tests	CmdU

Frequently asked questions (FAQ)

Q1: Can I learn iOS development without a Mac?

You can learn the language, but not do the full development. Swift Playgrounds on iPad/iPhone can cover everything in Chapter 61; but Xcode is macOS-only, so in the end you'll need a Mac. On a tight budget, you can rent a cloud Mac or buy a used Mac mini as a stopgap, then decide how much to invest after learning the language.

Q2: How much does learning iOS development cost?

Learning stage: 0 dollars (Xcode is free + a free Apple ID for on-device debugging). Release stage: a developer account at 99 USD/year (about 700 RMB). Add a Mac, and that's the entire cost. Compared with the payoff—even a small utility on the App Store has paying users—the math works out.

Q3: SwiftUI or UIKit—which one first?

SwiftUI. In 2026, new projects, new tutorials, and new job listings all center on SwiftUI, and this book uses it throughout. UIKit is the previous-generation framework, still used in many older projects—once you're comfortable with SwiftUI, pick up UIKit as needed (Apple has interop tutorials), and it will come quickly. Just don't reverse the order.

Q4: Is iOS development easy to find work in?

The 2026 job market: entry-level roles that amount to "can draw interfaces" are fiercely competitive, but developers who "know Swift + SwiftUI and have a shipped app" are still in demand. Median pay for iOS roles has ranked near the top among tech stacks for years. While you're learning, the best line on your resume is that app you made, sitting in the store.

Q5: What if my app gets rejected in review?

Don't panic—it's normal; first submissions get rejected at a decent rate. Apple's rejection reasons are usually written very specifically (which page, which guideline). Fix things one by one, reply with an explanation in App Store Connect, and resubmit. The three most common rejection reasons: permission purposes not clearly explained, buggy interfaces, and metadata (screenshots/description) that doesn't match the real thing. Self-check those three ahead of time and you'll save yourself a round of waiting.

High-frequency Xcode shortcuts

Action	Shortcut
Run / test	CmdR / CmdU
Open the canvas preview	OptionCmd
Quickly open a file	CmdO
Search everywhere	CmdF
Fix indentation	CtrlI
Comment / uncomment	Cmd/
Clean the build cache	CmdK
Jump to definition	Cmd-click
Rename (safe renaming across the project)	Right-click Refactor Rename

High-frequency Xcode shortcuts

Tip: when builds act haunted, clean the cache first

That spooky “it ran yesterday but not today” phenomenon is, six or seven times out of ten, a scrambled build cache. Clean with CmdK and rebuild, and it cures the majority of these hauntings. Still broken? Restart Xcode (and the Mac along with it), and if that fails, “delete the DerivedData folder” —the legendary ultimate trio, passed down among iOS developers.

Recommended learning resources

Recommended learning resources

Type	Resource	Why it's recommended
Official	developer.apple.com	Apple's official documentation, with the SwiftUI/Swift tutorials inside—the first authority
Official	Apple's SwiftUI Tutorials	Apple's official hands-on course—maximum beginner-friendliness
Official	The Swift website (swift.org)	Official language docs and an online playground
Community	Hacking with Swift	Paul Hudson's free tutorial library, widely recognized as top-tier in quality
Community	Stack Overflow (the swiftui tag)	The first stop when an error has you stumped
Video	The official Apple Developer channel (WWDC videos)	Authoritative coverage of each year's new technologies
Design	Apple's Human Interface Guidelines (HIG)	The official bible for building interfaces with that "Apple feel"
Companion	Part 7, "Hands-On AI Coding" (Chapters 37–47)	The complete playbook for AI-assisted iOS development

iOS development learning resources (August 2026)

Closing: from your first app to your Nth app

Looking back, you went from "having no idea what SwiftUI was" to installing Xcode, learning Swift, drawing interfaces with SwiftUI, hooking your app up to a database and the network, and on to testing, packaging, and shipping—on this road, you've already walked the hardest stretch of the journey from zero to one.

The world of technology changes fast: frameworks upgrade, tools get replaced, and AI takes over more and more of the "typing" work. But one thing never goes out of date: your ability to take a fuzzy idea, break it into clear steps, and then build it one step at a time. Tools are the amplifier; the way you think is the foundation. The amplifier can be swapped out; the foundation must not shake.

You're no longer just "someone who knows how to use a phone." You can turn the ideas in your head into real apps running on hundreds of millions of iPhones. From today, create with your keyboard —that's the romance of being an iOS developer.

References

1. Apple Inc. (2026). SwiftUI Framework Documentation. developer.apple.com.
2. Apple Inc. (2026). Swift Programming Language Guide. swift.org.
3. Apple Inc. (2026). SwiftData Framework Documentation. developer.apple.com.
4. Apple Inc. (2026). App Store Review Guidelines. developer.apple.com/app-store/review.
5. Apple Inc. (2026). Human Interface Guidelines (HIG). developer.apple.com/design.
6. Apple Inc. (2026). Xcode 27 Release Notes and App Distribution Guide.

Part 10: Appendices

Appendix A: New Phone Unboxing "Step-by-Step" Guide

Unboxing Inspection "Three-Step Process" :

- **: Make a test call to check the earpiece and microphone, take a photo to check the camera.**2. Activation & Initial Setup "Key Reminders":
- **Network**: Connect to a stable Wi-Fi first.
- **Account**: Log in to your Apple ID, Huawei account, Xiaomi account, etc.

Activation & Initial Setup "Key Reminders" :

- **: Turn on "Find My Phone" and set a reliable lock screen password.**eSIM Setup
- **: If your phone supports eSIM (e.g., iPhone, some Android flagships), you can add an eSIM plan in Settings without inserting a physical SIM card. eSIM is especially convenient when traveling abroad—you can directly purchase a local carrier's eSIM plan.**Data Migration
- **: When migrating data from an old phone, ensure both phones have sufficient battery. iPhone uses "Quick Start", Android uses the official device transfer app.**3. Installing Apps "Safety First":
- **Use Official Stores First**: Apple uses App Store, Android uses the phone's built-in app store.
- **Don't Grant Permissions Carelessly**: When permission requests pop up during installation, think carefully: "Does this app really need this?"

Installing Apps "Safety First" :

- **Basic Practice**: Clean up phone storage—the goal is to find and delete those old videos you "forgot what they were", duplicate screenshots, and apps you haven't used in a long

time.

- **Software Practice**: Set up WeChat storage cleanup. The path is usually [Me] - [Settings] - [General] - [Storage].

Appendix B: Hands-On Practice Reference

- **: Check battery health. On iPhone, it's in [Settings] - [Battery] - [Battery Health].**Universal Tip
- **: When you encounter a problem, restart first! This is the universal key that solves more than half of all minor phone issues.**Appendix C: Minimalist Tech Jargon Glossary
- **Term You May Hear**It Actually Means...
- **One-Line Explanation**SoC / Chip

Appendix C: Minimalist Tech Jargon Glossary

The core component that determines how fast the phone is and whether games lag.	**RAM (Memory)**	**Phone's "short-term work desk"**
The bigger the desk, the more apps can run simultaneously, and switching between them is smoother.	ROM (Storage)	Phone's "large warehouse"
Where photos, videos, and apps are stored. 128GB, 256GB refers to this.	OLED / LCD	Two types of screen materials
OLED has richer colors and saves more power; LCD is easier on the eyes and costs less.	NFC	Near-field wireless communication
The "tap" feature that lets your phone transform into a transit card, access card, or bank card.	OTA Update	Over-the-air system upgrade
That "System Update" button you tap in [Settings]—that's it.	Root	Android super administrator privileges
Once obtained, you can modify system files, but there are risks involved.	BL Lock	Bootloader Lock
A security lock added by manufacturers; it must be unlocked before flashing custom firmware.	NPU	Neural Processing Unit
A chip dedicated to handling AI tasks—dozens of times faster than CPU with lower power consumption.	On-Device AI	AI running locally on the phone
AI runs directly on the	Qi2	Next-generation wireless

Appendix D: Where to Find Reliable Resources?

Official Support Is Your First Choice:

- **: Apple uses iTunes, Huawei uses Huawei Phone Assistant, Xiaomi uses Xiaomi Device Transfer.**Useful Websites & Bloggers:
- **Look Up Tips**: Search "[Your Phone Model] tips" on Bilibili.
- **Watch Reviews**: Watch review videos from several different bloggers.

Useful Websites & Bloggers:

- **: Follow bloggers who teach mobile photography tutorials.**Security Self-Check One-Sentence Checklist:
- **Have you set a lock screen password?**Is "Find My Phone" turned on?
- **Have you enabled two-factor authentication for important accounts?**Do you regularly back up photos and WeChat chat history?

Security Self-Check One-Sentence Checklist:

1. Appendix E: Acknowledgments & Notes
2. Thank you for sticking with this "phone course" to the end! During the writing process, we referenced extensive manufacturer official documentation, user manuals, and publicly available tech education articles to ensure accuracy of information.
3. Technology evolves rapidly. The content in this book is based on mainstream technology and products at the time of writing. By the time you read this, there may already be new features and products available. We hope the thinking patterns and methods taught in this book will help you confidently face even more digital innovations in the future.
4. Version 1.6 Update Highlights

5. Expanded ”Digital Asset Value & Management Philosophy”, added new ”Image & File System Management” chapter (time+event naming, categorization & organization, 72-hour auto-cleanup, EXIF sanitization).

Appendix E: Thanks and Notes

Thank you for sticking with this "phone class" to the end! While writing it, we drew on a large number of manufacturers' official guides, user manuals, and publicly available tech explainers to make sure the information is accurate.

Technology advances at a dizzying pace, and the book's content is based on the mainstream technology and products at the time of writing. By the time you read it, new features and products may well have appeared. We hope the ideas and methods this book teaches will help you face plenty more digital novelties in the future with calm confidence.

What's new in version 1.6

- Expanded "the value of digital assets and how to manage them"; added a new "photo and file system management" chapter (naming by time + event, sorting into categories, 72-hour auto-cleanup, EXIF scrubbing).
- Improved the "clean/protection mode" and "digital parasite" protection system.
- Added common AI-era scam tactics (face-swapping and voice cloning, impersonating authority, fake investments) and the countermeasures that beat them.
- Rewrote the "unboxing" and social-engineering attack protection.
- Expanded the accessibility chapter (full VoiceOver/TalkBack operation, magnification and color filters, AI image descriptions, Live Captions, RTT/TTY dual-channel calls).
- Rewrote the prompt framework using the "role-background-task-constraints" approach, with 3 complete scenario examples.
- Added three advanced technical appendices: Appendix to Chapter 1: how Android memory extension works (what it actually is, zRAM/swap principles, speed bottlenecks, where it helps and where it doesn't, and each manufacturer's naming and how to turn it on).
- Appendix to Chapter 20: Termux—a portable Linux terminal environment on Android devices (core features, the plugin ecosystem, installation and setup, and typical scenarios like mobile development, server administration, and security testing).

- Appendix to Chapter 23: a beginner's guide to GitHub—from code hosting to a collaboration platform (Git vs. GitHub, core concepts, signing up and getting started, the Pull Request collaboration flow, security reminders, and access options within China).

What's new in version 1.7

1. Four new chapters

- Chapter 29: Digital Wellbeing—screen time analysis, app limits, Do Not Disturb, the 20-20-20 eye rule, saving your neck, sleep management, information anxiety and "selective blindness," and a one-week digital wellbeing challenge.
- Chapter 30: Screen Casting and Cars—a comparison of the AirPlay / Miracast / DLNA three camps, wired casting options, Samsung DeX desktop mode, the four car systems CarPlay / Android Auto / HiCar / CarLife, and casting in meeting rooms in practice.
- Chapter 31: Short-Video Creation—the three questions for picking a topic, 5 setup steps before shooting, five camera moves (push, pull, pan, shift, follow), the six-step CapCut (JianYing) workflow, cover-image essentials, and the "first 3 seconds decide everything" content principle.
- Chapter 32: Safety First-Aid Handbook—the golden 30 minutes after losing your phone (SOP), the 72-hour response process after being scammed, safe-mode checks for malware, six steps to take back a hijacked account, how to handle ransom messages, and five daily prevention habits.

2. Two new chapters

- Chapter 27: Seniors and Family—an easy-mode comparison table across brands, six common scams and three lines of defense, remote assistance (ChangLian / Sunflower), WeChat Care Mode and voice input, plus SOS / fall detection / medication reminders.
- Chapter 28: Buying Guide—the three-step method for choosing a phone, a "only look at these five" spec sheet, recommendations across five budget tiers, a seven-scenario chooser table, five buying traps, and an analysis of when and where to buy.

3. New read-aloud and interaction features

- Floating read-aloud control bar: supports play/pause, previous/next paragraph, 0.5x–2.5x speed adjustment, and stop.
- Quick earphone button on the right: start read-aloud with one press, automatically resuming from your current reading position.
- Keyboard shortcuts: R toggles read-aloud, ESC stops it.

- Read-aloud improvements: automatic continuous chapter turning, and read-aloud controls hide automatically when printing.

4. Structure updates

Table of contents, learning path (four steps five steps), chapter map, version number (1.6 1.7), appendix numbering (five six), and metadata all refreshed in sync.

5. Content freshness updates

- Processor standards: the Snapdragon 8 Elite Gen5 and Dimensity 9500 now count as top tier (2026 baseline).
- Flagship reference models: iPhone 17 Pro, Samsung S26 Ultra, Xiaomi 17 Ultra, Huawei Pura 80 Ultra, vivo X300 Pro.
- AI chapters: updated to DeepSeek R1, ChatGPT-5, and an on-device NPU standard of 80 TOPS.
- Foldables: the price range now reaches down to the 4,000–5,000 RMB tier.
- Buying guide: all budget tiers and scenario recommendations refreshed to the 2026 market.
- Network and ecosystem: Wi-Fi 7 / 5G-Advanced / the HarmonyOS ecosystem / LE Audio all updated to the latest standards.

6. New feature module: the Learning Dashboard (below the Hero area)

- Reading progress ring: shows the percentage of chapters read in real time, with an animated SVG ring.
- Phone health self-check: an 8-item safety checklist (password, two-factor authentication, Find My Phone, cloud backup, WeChat backup, payment security, anti-fraud app, and password-free payment cleanup).
- Persistent state: checkmarks are written to localStorage, with the health score calculated dynamically and shown as a gradient progress bar.
- Tiered messages: different feedback like "Perfect / Pretty good / Passing / At risk" depending on the score.

What's new in version 1.8

Added Part 7, "Hands-On AI Coding" (Chapters 37 – 47, 11 chapters in total): the paradigm revolution and Agentic Engineering, the five thinking principles of AI coding, context engineering and spec-driven development (SDD), requirements engineering, the 2026 landscape

of AI coding tools, the large-model landscape and hands-on API work, conversation engineering, a complete from-zero-to-one project workflow, quality assurance, the road ahead (MCP / multi-agent collaboration), and a quick-reference handbook. The table of contents, learning path (five steps six steps), chapter map, and quick-reference index were updated in sync. The content comes from the August 2026 edition of "AI Coding Mindset: A Methodology Guide from Zero Basics to Full-Stack Practice."

Added Part 8, "Hands-On Android Development" (Chapters 48–58, 11 chapters) and Part 9, "Hands-On iOS Development" (Chapters 59–69, 11 chapters): both platforms cover environment setup (Android Studio / Xcode), a language crash course (Kotlin / Swift), UI frameworks (Jetpack Compose / SwiftUI), data and networking, two complete hands-on projects (an expense tracker + a weather app), testing and store release (Google Play / App Store), and quick-reference handbooks. The table of contents, learning path (six steps eight steps), chapter map, and quick-reference index were updated in sync. The content comes from the August 2026 editions of "Android Development from Scratch" and "iOS Development from Scratch."

Content additions: practical apps and harassment blocking

- Chapter 13 adds a new Section 6, "Practical System Tools": cleanup tools like Qingzhuo, the built-in storage cleaner, the National Anti-Fraud Center app, brand phone managers, carrier anti-harassment services, and AdGuard, along with the selection principle of "system first / official supplements / third-party apps in moderation."

- A brand-by-brand table for built-in harassment blocking: covering iOS, Xiaomi/Redmi, Huawei/Honor, OPPO/realme/OnePlus, vivo/iQOO, and Samsung—entry paths and recommended settings for each.

- Chapter 8's harassment-blocking coverage expanded: added hands-on instructions for caller ID, blacklists and whitelists, SMS keyword filtering, iOS Silence Unknown Callers, and Do Not Disturb integration, with cross-links to Chapter 13.

- A three-layer protection recommendation: system blocking + carrier anti-harassment + the National Anti-Fraud Center app, with emphasis on whitelists to prevent false positives.

Content additions: weather alerts / natural disaster alerts / earthquake early warning

- Chapter 13 adds a new Section 7: weather alerts (official meteorological channels + the system weather app + color-coded severity levels), natural disaster alerts (multiple weather and emergency channels), and how earthquake early warning works plus common misconceptions (an early warning is not a forecast).

- A table for turning on earthquake early warning by brand: common entry points and permission notes for Xiaomi, Huawei/Honor, the OPPO family, vivo, iPhone, Samsung, and more.
- Chapter 32 adds a new ”Scenario Zero”: the evacuation order after an earthquake alert, how to handle torrential rain and typhoons, and how this ties into satellite SOS and no-network communication; the first-aid handbook and Chapter 13’s alert settings cross-link.
- Weather section linkage: the weather app entry in the travel section now mentions alert notifications and the reminder that ”you have to turn on notifications to receive them.”

Content additions: cross-device teamwork and productivity tools

- Chapter 13, Section 8: controlling your PC from your phone (Sunflower, ToDesk) and the anti-fraud red lines; AI office work (Trae / WorkBuddy / Dumate) and phone-remote PC workflows; miHoYo’s Artificial Desktop dynamic wallpaper.
- LAN file sharing and clipboard: LocalSend, Feige Chuan Shu, AirDrop; KDE Connect, Universal Clipboard, and brand-specific phone-to-PC transfer.
- Dynamic 2FA codes: Aegis, Microsoft Authenticator, plus backup codes and things to watch out for when switching phones.
- TickTick and paper scanning: expanded uses for schedules/tasks; more on scanning tools and photo tricks.

May you master your phone, enjoy the convenience and fun that technology brings, and become a clear-headed, confident digital citizen.

—Your guide to digital life

Appendix F: Foldable Phone Purchasing and Usage Special Topic

Currently, there are two mainstream categories:

I. How Far Have Foldable Screens Developed?

: When closed, it's the size of a regular phone; when opened, it's nearly square like a small tablet. Representatives: Huawei Mate X series, Samsung Galaxy Z Fold series, Xiaomi MIX Fold series, vivo X Fold series. Main selling point: "one phone = phone + tablet" .**Vertical Folding (Clamshell-style)**: When closed, it looks like a compact powder case; when opened, it's a normal-sized phone. Representatives: Huawei Pocket series, Samsung Galaxy Z Flip series, OPPO Find N Flip, Xiaomi MIX Flip. Main selling point: "compact portability + cover screen quick view" .

Key advancements in 2024-2025: Hinge lifespan is generally rated at 500,000 folds (opening/closing 100 times daily lasts 13 years), UTG ultra-thin glass has replaced early plastic films, creases are visible to the naked eye but barely perceptible by touch, prices have dropped from the 10,000+ yuan range down to the 5,000-6,000 yuan range.

- **One-Sentence Verdict**If your current slab phone fully meets your needs and you have no pain point of "screen not being big enough," don't buy a foldable. It's a "nice-to-have" device, not a "must-have" device.

- **II. Who Should Buy a Foldable Phone?**Based on real user feedback, these groups have a high probability of finding foldables "worth it":

Key advances in 2024-2026: hinge life generally rated at 500,000 folds (100 folds/day for 13 years), UTG ultra-thin glass replacing early plastic films, creases visible but barely felt by touch, prices dropping from the 10,000 yuan tier to 4,000-5,000 yuan.

: Frequently view PDFs, annotate documents, and read long articles on their phone. When unfolded, the ~7.5-inch inner screen lets you view A4 PDFs without zooming—a transformative experience.

Frequent Travelers / Business Users

II. Who Should Buy a Foldable Phone?

Creatives Who Love “Flex Mode”

1. **: Fold to 90° and place on a desk—upper screen for framing, lower screen for shutter (selfies/timelapse); upper screen video, lower screen notes; upper screen livestream, lower screen comments—a form factor slab phones simply cannot achieve.**Female Users Who Want Portability + Big Screen
2. **: Vertical foldables fit into small bags when closed, unfold to normal phone size, and the cover screen can serve as a makeup mirror or notification viewer.**These People Should NOT Buy One
3. **If you only use your phone for short videos, WeChat, and calling—foldables offer zero benefit and are heavier and thicker instead.**If you have small hands or wrist tendonitis—horizontal foldables weigh 230g+ when unfolded, making single-handed use tiring.
4. **If you’re on a tight budget and plan to keep a phone for 4+ years—foldables have lower resale value than slab phones, and repair costs are high.**III. Buying Guide: Check These 6 Metrics

Don’t get dazzled by the word ”folding”—focus on these key points when shopping

- Hinge Type
- : Waterdrop hinge (shallower crease, longer lifespan, currently mainstream) is better than U-type hinge (deeper crease, early solution). Ask sales staff or check reviews to confirm which type.
- Inner Screen Material

III. Buying Guide: Check These 6 Metrics

Crease Performance

- **: Go to a store and unfold it yourself. The 2025 qualified standard: almost invisible head-on with screen on, visible as a shallow groove under angled light, slight depression when finger runs across but not sharp.**Cover Screen Size

- **: For horizontal foldables, the larger the outer screen the better (ideally 6.4"+) so it works as a normal phone when closed; for vertical foldables, larger is more practical (4"+ allows replying to messages and viewing navigation).**Weight & Thickness

- **: Horizontal foldables at 230-260g unfolded is normal; under 230g is excellent. Closed thickness of 12-14mm; under 12mm is thin. Vertical foldables must fit comfortably in a shirt pocket when closed to qualify.**: Go to a physical store and unfold it yourself. The 2026 acceptable standard is: almost invisible when screen is on, a shallow groove visible under angled light, slight depression when fingers pass over but not sharp.

- **: Focus on inner screen crack repair cost. Some brands offer free inner screen replacement once in the first year; others charge 2,000-4,000 yuan out-of-pocket. This is the biggest hidden cost of owning a foldable.**In-Store Experience Checklist

- **Open and close it 10 times yourself—feel if the hinge damping is smooth and if there are any unusual sounds.**Unfold it and look at the crease under angled light, then look at it head-on with the screen on.

- **Hold it single-handed in unfolded state for 30 seconds—feel if the weight is acceptable.**Try Flex Mode: fold to 90°, place on desk, try taking photos and watching videos.

Ask clearly about the standalone inner screen repair price and warranty policy.

1. IV. Unique Foldable Usage Tips
2. After buying one, here are experiences you can't get with a slab phone:
3. 1. Split-Screen & Multitasking
4. When a horizontal foldable is unfolded, most systems support top/bottom or left/right split-screen. Typical scenarios: upper screen for online course video, lower screen for notes; left side WeChat, right side documents; upper screen map navigation, lower screen music. Much more practical than slab phone split-screens because the screen is large enough that neither half feels cramped.
5. 2. Flex Mode (90° Fold)

IV. Unique Foldable Usage Tips

Selfies / Group Photos

: Place on desk folded to 90°, upper screen for framing, lower screen for shutter—no need to ask someone else to take the photo.

Timelapse / Night Shots

: Same stable placement on desk—saves you from needing a tripod.

Video Calls

- **: Frees both hands; the other person sees you from a more natural angle too.**Watching Videos
- **: Use as a mini TV on a desk—especially practical on airplane/high-speed rail tray tables.**Cover Screen Preview + Inner Screen Shooting
- **: When using the rear camera to shoot yourself, the cover screen displays the live preview for more accurate composition.**3. Cover Screen Quick View
- **When a vertical foldable is closed, the cover screen can: show time/notifications/weather, reply to WeChat (voice-to-text or quick replies), tap transit cards, view navigation, act as a makeup mirror (using front camera). No need to unfold every time—saves power and effort.**4. Desktop Mode
- **Some horizontal foldables (Huawei, Samsung) can switch to "Desktop Mode" when unfolded. After connecting an external display, keyboard, and mouse, it works like a basic PC—adequate for temporary document processing and email replies. Not a primary productivity tool, but great for emergencies.**V. Maintenance Tips: Help Your Foldable Last Longer

The most delicate parts of a foldable are the inner screen and hinge. These habits can significantly extend its lifespan:

Don't Force Open/Close in Low Temperatures

: Below 0°C, UTG glass becomes brittle. Warm it in your pocket for a few minutes before opening/closing. This is something written in every official manual but nobody reads.

Avoid Nails/Hard Objects Touching the Inner Screen

V. Maintenance Tips: Help Your Foldable Last Longer

Check for Foreign Objects Before Closing

- **: A single grain of sand caught inside when closing could directly crush the inner screen. Glance over it before closing.**Don’t Apply Regular Tempered Glass Film
- **: The inner screen can only use official specialized film or UTG protective film. Regular tempered glass film affects folding and touch functionality. Recommend going to official after-sales for screen protector application.**Keep Dust & Liquids Away from the Hinge
- **: Although most claim IPX8 water resistance, dust resistance ratings are generally low. Be careful at beaches, construction sites, etc.**Apply Even Force When Opening/Closing
- **: Don’t violently flick it open with one hand. Opening smoothly with both hands from the center is most stable.**High-Risk Scenarios for Inner Screen Damage
- **Catching headphone/charging cable when closing—directly cracks the inner screen.**Mixing with keys in a bag—outer screen fine, but keys press creases into the inner screen.
- **Immediately opening/closing when moving from cold outdoors to warm indoors in winter—temperature difference causes glass stress changes.**After a year of use, feeling ”the crease got deeper” and trying to peel off the factory film—peeling the film will damage the inner screen along with it.

VI. Foldable “Pitfalls” & Realistic Expectations

- Before buying, keep these things in mind:
- The Crease Will Always Be There
- : 2025 technology cannot achieve ”completely invisible”—only ”almost invisible head-on with screen on”. If you’re a perfectionist who would stare at the crease uncomfortably every day, don’t buy.
- Heavier & Thicker

VI. Foldable “Pitfalls” & Realistic Expectations

Battery Life Is Generally Average

- **: Because space must accommodate the hinge mechanism, battery capacity is compressed. At the same price point, battery life typically falls short of slab phones.**: 2026 technology can’t achieve ”completely crease-free” —only ”almost invisible from the front when screen is on.” If you’re a perfectionist who’ll be bothered by the crease daily, don’t buy.

- **: After one year, the second-hand price is usually 50-60% of the new phone price—drops faster than slab phones.**Expensive Repairs
- **: Inner screen assembly repair costs 2,000-4,000 yuan; hinge repair costs 500-1,500 yuan. Check warranty policies clearly before buying.**Inconsistent App Adaptation
- **: Mainstream apps have adapted to large screens, but niche apps may just be "stretched/enlarged" with mediocre experience.**A foldable is not a "better phone"—it's "a phone in a different form factor." Buy it because you need its unique form, not because it's "more advanced."
- **——On the Philosophy of Choosing a Foldable Phone**VII. How to Choose Among Mainstream Brands (2025)
- **Not listing specific models (they'll become outdated)—just brand personalities to help you quickly orient yourself:**Huawei Mate X Series

: The benchmark for horizontal folding. Strong HarmonyOS ecosystem synergy. Cover screen ratio close to a normal phone—works as a primary device when closed. High price but relatively best resale value.

Samsung Galaxy Z Fold/Flip

VII. How to Choose Among Mainstream Brands (2026)

Xiaomi MIX Fold/Flip

- **: Value-for-money approach. Packed with specs at relatively affordable prices. Suitable for users who want to try foldables without spending too much.**vivo X Fold Series
- **: Among horizontal foldables, the cover-to-inner screen ratio comes closest to a normal phone—great experience using it as a slab phone when closed.**OPPO Find N Series
- **: Among horizontal foldables, the size leans compact—single-hand holdable, suitable for users who find large foldables too heavy.**Huawei Pocket Series
- **: Among vertical foldables, the largest and most practical cover screen. Makeup mirror, UV detection, and other women-oriented features are well-implemented.**Buying advice: If budget allows and you want ecosystem integration, choose Huawei Mate X; for the most mature hinge, choose Samsung Z Fold; for best value, choose Xiaomi/vivo; for vertical folding, look at Huawei Pocket and Samsung Z Flip.

- **Final Reminder**Foldable screen technology iterates every year. This appendix was written in 2025. Before buying, be sure to check the latest reviews and user feedback, focusing especially on: actual hinge lifespan testing, crease performance, and inner screen repair policy. Hands-on experience in-store always matters more than reading reviews.
- **Last read at: ...**Continue Reading

Dismiss

Final Reminder

Foldable screen technology iterates every year. This appendix was written in 2026. Before buying, be sure to check the latest reviews and user feedback, focusing on: hinge life tests, crease performance, and inner screen repair policies. Touching it in person at a store is always more important than reading reviews.

Conclusion: Stay Curious, Stay Clear-Headed

Congratulations on making it this far.

This book covers everything from phone hardware to software ecosystem, from daily settings to developer mode, from security protection to digital legacy. If it helped you solve one practical problem —even just learning to turn off an unnecessary permission or forming a backup habit —then these words weren't written in vain.

But more important than any specific technique are two things:

- **Stay Curious**: Phones change every year, AI is changing, foldable screens are changing, satellite communication is changing. The operation steps you learn today may be outdated tomorrow, but the curiosity of "wanting to know why" never goes out of date. Asking one more question —"What does this permission do?" "Where is this data stored?" "Could this feature leak privacy?" —is more useful than memorizing a hundred operation steps.
- **Stay Clear-Headed**: The phone is a tool, not a master. It's designed to capture your attention, occupy your time, and guide your consumption. Using it is your choice; being used by it is your loss of control. Periodically check your screen time, and think "Am I using my phone to get things done, or is my phone consuming me?" —this isn't melodrama, it's a survival instinct in the digital age.

Technology changes, but human needs don't —we need security, connection, and respect. The phone is just a medium. The real "phone class" isn't in a book —it's in the moment you unlock your screen every day.

One Last Exercise

Close this book, pick up your phone, and do one thing:**Take 30 seconds to think: what three things in your phone would you "truly regret losing" ?**Then, make sure they're backed up. This book was worth it.

Appendix G: Quick-Reference Index for the Whole Book

Sorted by pinyin so you can quickly locate the corresponding chapter.

A-B

- BL lock Chapter 24
- ADB Chapter 20
- Installing and managing apps Chapter 4
- AI assistants Chapter 22
- AI phones Chapter 26
- AI face-swap scams Chapter 32
- AI coding Chapters 37–47
- The five principles of AI coding Chapter 38
- API key Chapter 42
- AI agents Chapter 37
- CLAUDE.md Chapter 39
- Android development Chapters 48–58
- Android Studio Chapter 49
- Activity lifecycle Chapter 52
- App Store release Chapter 67

C-D

- Car connectivity Chapter 34
- Battery and battery life Chapter 15
- Password management Chapter 14
- Passkey Chapter 14
- Developer mode Chapter 20
- Kids and phones Chapter 35
- Claude Code / Cursor Chapter 41
- DeepSeek Chapter 42
- Compose Chapter 51
- DataStore Chapter 53

F-H

- Foldable phones Appendix F
- Troubleshooting Chapter 19
- Google Play release Chapter 56
- GaN chargers Chapter 15
- iCloud Chapter 16
- Smart home Chapter 36
- Computational photography Chapter 11 appendix

K-R

- Photo tips Chapter 11
- Permission management Chapter 4
- SafetyNet attestation Chapter 20
- SIM swap attacks Chapter 32
- Root access Chapter 20 / Chapter 24
- Termux Chapter 20 appendix
- MCP protocol Chapter 46

- PRD requirements documents Chapter 40
- Prompt engineering Chapter 43
- Kotlin Chapter 50
- MVVM architecture Chapter 57 • Chapter 68
- Room database Chapter 53
- Retrofit Chapter 53

S-Z

- Digital legacy Chapter 25
- Data backup Chapter 18
- Harassment blocking Chapter 8 • Chapter 13
- Weather alerts Chapter 13
- Natural disaster alerts Chapter 13 • Chapter 32
- Earthquake early warning Chapter 13 • Chapter 32
- Practical apps and cleanup tools Chapter 13
- Qingzhuo (cleaner app) Chapter 13
- UWB (ultra-wideband) Chapter 17
- Network security Chapter 21
- Buying guide Chapter 28
- Cloud services Chapter 16
- SDD (spec-driven development) Chapter 39
- Token Chapter 42
- Vibe Coding Chapter 37
- Swift Chapter 61
- SwiftUI Chapter 62
- SwiftData Chapter 64
- URLSession Chapter 64
- Xcode Chapter 60
- iOS development Chapters 59–69

Others

- File management Chapter 2
- Senior-friendly features Chapter 27
- Recommended utility apps Chapter 13
- Anti-harassment / National Anti-Fraud Center Chapter 13 • Chapter 27
- Remote PC control (Sunflower/ToDesk) Chapter 13
- AI office tools (Trae etc.) Chapter 13 • Chapter 41
- LocalSend / LAN file sharing Chapter 13
- KDE Connect / clipboard sync Chapter 13 • Chapter 17
- 2FA one-time codes (Aegis etc.) Chapter 13 • Chapter 14
- TickTick Chapter 13
- Scanning paper documents Chapter 13
- Artificial Desktop (dynamic wallpaper) Chapter 13
- Earthquake / disaster phone emergencies Chapter 32
- Screen casting and cars Chapter 30
- Short-video creation Chapter 31
- Safety first aid Chapter 32
- Git/GitHub Chapter 23 appendix
- Git version control Chapter 45
- Context Engineering Chapter 39
- Requirements Engineering Chapter 40
- Expense Tracker App Chapter 54 • Chapter 65
- Weather App Chapter 55 • Chapter 66
- WidgetKit Widgets Chapter 68

Appendix H: Version Roadmap

This book is still being updated. Below are the planned directions for future versions — and feel free to tell us in your feedback what you'd most like to see.

v1.8 has landed: three brand-new parts — "AI Coding in Practice," "Android Development in Practice," and "iOS Development in Practice" (Chapters 37-69, 33 chapters in total)

From the paradigm shift of AI coding, the five thinking principles, context and requirements engineering, the 2026 landscape of tools and models, hands-on API practice, conversation engineering, three complete projects, quality assurance, and the road to more advanced skills —taking you from "knowing how to use AI" up to "directing AI to write code"; to the complete development pipeline for the two big platforms, Android and iOS: environment setup (Android Studio / Xcode), language crash courses (Kotlin / Swift), UI frameworks (Jetpack Compose / SwiftUI), data and networking, hands-on projects (an expense tracker + a weather app), testing and publishing (Google Play / App Store), and a quick-reference handbook — taking you all the way from everyday digital user to app developer.

What's planned for the next version (v1.9)

- Phones and wearables: an in-depth guide to deep integration with Apple Watch, Huawei watches, and smart rings (Oura/Galaxy Ring)
- Travel and your phone: hands-on eSIM, international data plans, offline maps, paying overseas, and translation app comparisons
- Phones and health management: fitness tracking, sleep monitoring, heart rate and blood oxygen, women's health, and mental health apps
- AI short-video creation: hands-on tutorials for CapCut AI, Kling AI, digital avatars, and AI voiceovers
- Getting started with vertical mini-dramas: script structure, filming techniques, and platform policies

A note on timeliness

Phone technology moves extremely fast, and this book was written based on the technology and market conditions of mid-2026. AI capabilities, chip models, system versions, and price ranges may all have changed by the time you read this. For specific step-by-step instructions, go by the actual system version on your device; for purchasing decisions, check the latest reviews and user feedback.